DRIVING OPTIMISATION RACING AI (DORA)

Joan Mathew Kurian

Kennet School

Table of Contents

1 Analysis 3

1.1 The project identification and background 3

1.2 Objectives 5

1.3 Research 11

1.3.1 Interviews/Surveys 11

1.3.2 Technical/Skills research 15

1.3.3 Research Table 31

1.4 Prototypes 36

1.5 Justification of chosen solution 42

1.6 Initial Modelling 44

1.6.1 Screen Layout 44

1.6.2 Training loop 45

1.6.3 Story Board 46

1.6.4 Sketches and process flow of proposed output 47

1.7 Analysis Data Dictionary 48

2 Design 54

2.1 Design Introduction 54

2.2 Hierarchy chart: 56

2.3 Complex Code 57

2.3.1 Game core and main 57

2.3.1 Game Modes 61

2.3.1 Spatial Hash Grid, Ray casting and Collision Detection: 64

2.3.2 Track Maker: 72

2.3.3 Track 101

2.3.4 Car 108

2.3.5 Lap Timer 116

2.3.6 Car Simulation 117

2.3.7 Neural Network 120

2.3.8 Layers 121

2.3.9 Neural Network Class: 124

2.3.10 Adaptive Moment Estimation (Adam) 125

2.3.11 Stochastic Policy Gradient Reinforcement Learning Model 128

2.3.12 Monte Carlo Returns 134

2.3.13 REINFORCE Model: 135

2.3.14 Monte Carlo Actor Critic (MCAC): 135

2.3.15 Reward Engineering: 137

2.3.16 AI Training Environment 146

2.3.17 Model Training optimisation 153

2.4 Object diagrams and class definitions 155

2.5 Design data dictionary and data structures 156

2.6 File organisation 161

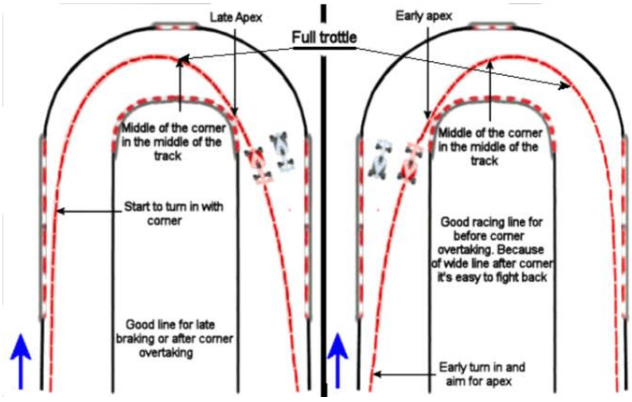
2.7 Human–Computer-interface design 161

2.7.1 UI Theme 161
2.7.2 Configuration Screens 162
2.7.3 Use of AI 168
3 Testing 172
3.1 Test Plan - 172
3.2 Test Evidence 184
4 Evaluation 188
4.1 Evaluating Objectives 188
4.2 End user feedback 195
4.3 Possible Improvements 197

1 Analysis

1.1 The project identification and background

Formula One is the highest class of worldwide racing for open-wheel single-seater motorsport. Teams first design their cars in accordance with a set of technical regulations and compete by racing these cars racing tracks worldwide. To maximize performance in races, teams conduct extensive research and data analysis to develop the most optimal strategy for the races.



One important aspect of a race is the "Qualifying session", during which drivers attempt to set the fastest lap time without directly racing other drivers. The results determine where they start on the grid for the main race. To develop an ideal strategy for this, teams must identify the fastest racing line, along with precise braking points and an appropriate driving style tailored to their car's performance characteristics on that specific track. The determination of racing lines includes the consideration of a variety of factors. The geometric line is the shallowest arc possible through a corner. However, this is not always the most ideal line when all variables are taken into consideration. Each corner will have its own unique ideal line given the track conditions, car, and nature of the corner (long corners, hairpins, chicanes etc.).

My project is an investigation that explores the feasibility of implement reinforcement learning (RL) models to optimise racing lines in a track in a simulation environment. I also wish to implement features that allow the creation of unique tracks to experimentally test the AI. The program will allow the user to train an agentic AI to optimally drive around any track using reinforcement learning. I wish to also add the ability to race around the track in an identical car simulation environment used by the RL model and race the RL model on the same track to compare the efficacy of the model to a real life human.

The data used as input to the reinforcement leaning model is "sensors" that provide distances to the nearest walls in different directions. I intend to use neural networks in my model since I want to be able to model very complex environments. This also aids in scalability, since if this project scope was scaled to a hyper realistic simulation, then the size of the neural network can just be increased by adding more layers. The larger a neural network is, the more complex the environment can be.

The goal of the model is to minimise the time taken to complete a full lap around the track (by crossing all turns and without exiting the track limits) with every episode (i.e. with every repeated attempt). I also aim to compare the efficacy of different Reinforcement learning algorithms for this investigation.

My limitations in this investigation lie in 2 main areas: The simulation design and the RL model design. Since I intend to program the simulation environment independently, I am limited in my ability to emulate real life physics of car driving by including internal engine forces, gear shifts, tire and surface grip, oversteer and understeer and so on. Instead, I will implement a simplified 2D environment that roughly emulates driving a car. I also will not be able to model too much complexity due to limitations in compute, since a high-resolution simulation also needs high

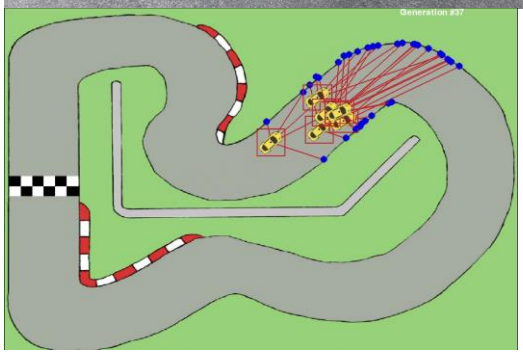
compute. Modelling higher resolution environment also needs larger models, which once again increase compute. With the RL model design, I am mainly limited in my ability to design an efficient “reward function”, since this is a complicated sub domain of AI. Due to my lack of specialist expertise, I am limited in how optimal I can make this function and as a result how optimal the RL model can be.

Project inspiration:

rFpro:

F1 teams use software like rFpro to simulate vehicles and track conditions to a very high degree of accuracy. It is currently the “world’s most realistic, engineering-grade digital environment, featuring accurate track surface modelling, low system latency, realistic visual imagery, high frame rate and high dynamic range rendering”. However, due to lack of resources like software, data, time and knowledge, I will not cover the scope of such advanced software. I aim to model cars in a 2D birds eye view simulation. This would remove the need to model forces in the vertical plane, and I can assume no change in elevation of the track. Additionally, due to lack of accessible data of the real life F1 car, characteristics of the engine, brake, steering etc. will be based on arbitrary values. I will also not model gear shifts. All these simplifications will compromise on the accuracy of the simulation; however, they make it more feasible to design from scratch given my limited expertise.

<u>rFpro</u> <u>What I aim for</u>



AlphaGo:

The Google DeepMind project AlphaGo was an artificial intelligence system that was developed to play the game of Go and famously defeated Lee Seudol, the most proficient Go player in the world. AlphaGo used Reinforcement learning with deep neural networks, which allowed it to learn optimal strategies by repeatedly playing games against players and itself. This idea of learning strategies instead of explicitly computing the best actions is crucial when it comes to modelling environments with very large number of possible permutations. This approach is one that I have adopted into my own investigation, since the task of driving a car around a track can

become immensely complicated when the simulation modelling increases in resolution and accuracy. So, although alternative methods of explicitly computing the best driving strategy explicitly may be better for modelling a low-resolution environment like the one in my project, for it to be scalable, reinforcement learning with neural networks would be the best approach

End user and specialist feedback:
Since this investigation includes highly technical aspects of AI, I also refer to specialists for advice on my project design. D. Davis, ex manufacturing engineer and C. McCormack, Machine Learning Engineer were the specialists that I consulted for feedback on my project. Conversations with these specialists were greatly insightful and helped me to develop a sense of direction in how to approach this project effectively while being realistic in the goals I set for myself.

1.2 Objectives

1. Main Menu screen
   1.1. Program starts with a title screen graphic and displays buttons to start different subprograms
   1.2. “Create new track” button starts a new track maker which includes a canvas to draw track and various creation tool options in a sidebar
   1.3. “Edit a track” button starts a track maker interface, but with a previously saved track loaded and rendered onto the canvas ready to be edited
   1.4. “Train a model” button starts a training an AI reinforcement learning model through repeated batch simulations.
   1.5. “Race AI driver” allows user to pick a track and race against a trained AI model to get a faster lap time
   1.6. “Exit button” should close the program
2. User Input
   2.1. User keystrokes and button presses are stored in respective hash sets for event handling.
   2.2. User can choose to use mouse or keyboard for controlling the car
3. Design a simulated car
   3.1. Car movement
      3.1.1. Use vector-based representations of acceleration, velocity and displacement to update the car parameters.
      3.1.2. Calculate the acceleration of the car from the throttle input. Apply braking/ negative acceleration for negative throttle input
      3.1.3. Use trigonometry to calculate vector of velocity and displacement based on scalar speed and angle orientation
      3.1.4. Simulate friction by adding grip to the movement and slowing down when no throttle is applied
   3.2. Car Turning
      3.2.1. Car turns w.r.t the current steering input
      3.2.2. Apply basic version of kinematic bicycle model to handle car rotation:

3.2.2.1. Ensure rate of turning is related to the speed, steering and wheelbase
3.2.2.2. Compute the angular velocity and update the orientation of the car w.r.t this angular velocity

3.3. Car Rendering
3.3.1. Rotate the image of the car with respect to its original image and the current orientation.
3.3.2. Rotated image of the car and the relative position of the image is stored to be rendered.

3.4. Controlling car
3.4.1. Input sets a target parameter value, and the values are smoothly incremented to approach this target value
3.4.2. Design throttle and braking input mechanism
3.4.2.1. Throttle and braking are incremented by separate factors where braking increments slower than throttle
3.4.2.2. Positive throttle values lead to increasing acceleration and negative braking values lead to deceleration.
3.4.3. Design steering input mechanism
3.4.3.1. Steering is updated by a constant factor each time step until it reaches the target steer value.

4. Player-controlled car
4.1. User input
4.1.1. User can press the 'W' key to increment the throttle and 'S' to apply braking
4.1.2. User can use 'A' and 'D' keys for steering
4.1.3. Alternatively, the user has the option to use an on-screen slider which they can control with the cursor to steer the car.

5. Track
5.1. Use data of user defined points and widths to generate a track with inner and outer walls.
5.2. Render track with a checkered line at the start/finish. Render walls as pattern of white and red
5.3. Implement optimised collision detection methods
5.4. Implement ray casting for AI car sensor data.

6. Agentic AI cars
6.1. Input
6.1.1. Input values are set directly from the output of the reinforcement learning model
6.2. Create sensors for AI car to observe the environment
6.2.1. Start point of ray is centre of car and a max ray length is set
6.2.2. Ray is set to an angle from the direction the car is facing
6.2.3. First intersection with a wall along the ray is set to the hit point
6.2.4. Measure distance along the ray up till hit point to get sensor value

- 6.3. Design reward function
  - 6.3.1. Reward efficiency (calculated from the speed and distance moved)
  - 6.3.2. Reward steering in the direction of more open space
  - 6.3.3. Reward steering if approaching corner
  - 6.3.4. Reward survival
  - 6.3.5. Reward lap completion
  - 6.3.6. Punish crashing
  - 6.3.7. Punish not steering when moving into a wall
  - 6.3.8. Punish driving very close to walls
  - 6.3.9. Punish staying static/steering without moving
- 7. Creating Racing Environment
  - 7.1. Start and finish line
    - 7.1.1. Car(s) starts at a designated start line being oriented in the correct direction to drive
    - 7.1.2. Timer records the time taken for car to finish lap
  - 7.2. Crash reset
    - 7.2.1. When a car crashes, it is reset to the start line
    - 7.2.2. The progress of the car is reset to the start of the running lap. Completed laps are not reset
  - 7.3. End of Simulation
    - 7.3.1. When cars in the racing environment complete the number of laps set, simulation ends
    - 7.3.2. Ends screen displays lap time statistics. The car with the best average lap time/fastest lap time is shown as winner
- 8. Track maker interface
  - 8.1. Curve tool for make track shape
    - 8.1.1. Create an anchor point of when left mouse button is clicked and connect all anchor points with a smooth continuous curve.
    - 8.1.2. Anchor points can be dragged to different positions using the cursor
    - 8.1.3. Clicking on the track spine curve inserts a new anchor in between
    - 8.1.4. Curve can be made into a smooth closed loop
    - 8.1.5. Anchors can be deleted
  - 8.2. Design tools to allow the user to modify features of the track.
    - 8.2.1. User can clear interface and start over.
    - 8.2.2. Create curvature handles that the user can edit to modify the curvature of the track
    - 8.2.3. User can click at a point on the track and use a slider to modify width at the point
  - 8.3. Enforcing track criteria
    - 8.3.1. Check track forms a full complete loop
    - 8.3.2. Track must have a minimum length
    - 8.3.3. Check there are no overlapping sections
    - 8.3.4. Ensure there are no kinks in the shape of the track

8.4. Save track
  8.4.1. User is prompted to enter a unique track name that names the save file
  8.4.2. All track properties are stored into a json file
  8.4.3. Saving the track updates the previously saved values to the new values
8.5. Load track
  8.5.1. User is prompted to enter the name of saved track
  8.5.2. Track data is loaded in from the selected file
  8.5.3. Implement algorithm to regenerate smooth curves for track walls from the track data loaded from file

9. Multi-Layer Perceptron (MLP) Neural network
9.1. Create “layers” of neurons
  9.1.1. Each layer has a vector of weights and biases where each weight and biases is associated with a single neuron
  9.1.2. Every neuron of one layer is connected to all the neurons in the next layer
  9.1.3. Includes an ‘input layer’, an ‘output layer’, and a certain number of ‘hidden layers.
  9.1.4. The weight and biases parameters can be updated during training
9.2. Get actions from network from feedforward
  9.2.1. Input is passed into the input layer
  9.2.2. Data is passed from one layer to the next through the network and the final output is taken from the output layer
  9.2.3. Implement options for non-linear activation functions that wrap the output of each layer
    9.2.3.1. Tanh
    9.2.3.2. Elu
    9.2.3.3. ReLu
9.3. Update network parameters with backpropagation
  9.3.1. Compute gradients of objective function w.r.t weights and biases
  9.3.2. Implement Adam optimiser to optimally update weights and biases to converge on global minimum
    9.3.2.1. Compute Adam moments of Momentum and uncentered Variance
    9.3.2.2. Update parameters by factors of the Adam moments
9.4. Design network to compute batch inputs of sensor data to produce batch output where each output corresponds to a single simulated agent

10. Create base Policy Gradient Reinforcement Learning model
10.1. Plugs the input vector into the neural network to get the output
10.2. Computes actions stochastically from neural network output
10.3. Stores trajectory of recent episodes
10.4. Computes total return at each time step
10.5. Implements gradient ascent to update neural network

11. Creating Specific RL models
    - 11.1. REINFORCE model
        - 11.1.1. Implement a single neural network that outputs the actions each cycle
        - 11.1.2. Computes Monte-Carlo returns
        - 11.1.3. Uses returns to compute advantage value for network parameter updates
    - 11.2. Monte Carlo Actor Critic (MCAC) model
        - 11.2.1. Implement an actor network that outputs action each cycle
        - 11.2.2. Implement a critic network that computes a Value estimate at end of each episode
        - 11.2.3. Use predicted critic value and returns to compute the advantage value for network parameter updates.
12. Model training environment
    - 12.1. Batch simulation management
        - 12.1.1. Allow user to configure number of cars to train simultaneously
        - 12.1.2. Spawn all configured cars at track start line with independent state tracking
        - 12.1.3. Process all cars in parallel each timestep, collecting individual trajectories
        - 12.1.4. Handle individual car crashes and resets without affecting other cars in batch
    - 12.2. Performance tracking and metrics
        - 12.2.1. Track average and maximum progress across all cars in recent
        - 12.2.2. Track average and fastest lap times when cars complete laps
        - 12.2.3. Update performance metrics in real-time
        - 12.2.4. Log episode number, duration, average progress, and average reward for each episode
    - 12.3. Model rollback system
        - 12.3.1. Save best model parameters when average performance improves
        - 12.3.2. Detect performance degradation when recent average drops significantly below best historical average
        - 12.3.3. Revert to best parameters if performance degradation is detected.
    - 12.4. Training speed controls
        - 12.4.1. Implement tick speedup
        - 12.4.2. Process multiple simulation frames per render frame based on speedup value
    - 12.5. Model configuration interface
        - 12.5.1. Allow user to configure neural network architecture (hidden layer sizes and activation functions.

12.5.2. Allow user to configure training hyperparameters (learning rate, entropy bonus, Adam betas, L2 regularisation)
12.5.3. Allow user to configure critic network architecture for MCAC model.
12.5.4. Display appropriate error messages for invalid configurations.

13. Racing Modes
13.1. Initialise Cars at start line
13.2. Create control panel for user to drive car, along with HUD for speedometer and throttle meter
13.3. Track Completion of laps and record individual lap times
13.4. Reset crashed car to start of current lap
13.5. Implement solo racing and racing against AI models
13.5.1. Solo racing
13.5.1.1. Only player car drives around track and player lap times are recorded
13.5.1.2. End screen displays individual lap times and total race time
13.5.2. Race AI
13.5.2.1. 1 Player and 1 AI car race on the track
13.5.2.2. Lap times for each car recorded separately
13.5.2.3. Car with the lowest total race time is displayed as the winner.

14. File selection
14.1. Create a global file selection system for saving and loading tracks and AI models
14.1.1. Displays all relevant existing files within the required directory in a selection list
14.1.2. Allows user to enter a file name into a text box
14.1.3. Loading
14.1.3.1. User must select a relevant existing file in the directory.
14.1.3.2. User is displayed an appropriate error message if try to select a non-existing file
14.1.3.3. User is displayed an appropriate error message if there is an error in loading the selected file
14.1.4. Saving
14.1.4.1. User is allowed to name their file a unique or existing name
14.1.4.2. If file name already exists, the user is displayed a warning message and asked to confirm their choice of overwriting the file
14.1.4.3. User is displayed an appropriate error message if the file name they have entered is invalid.

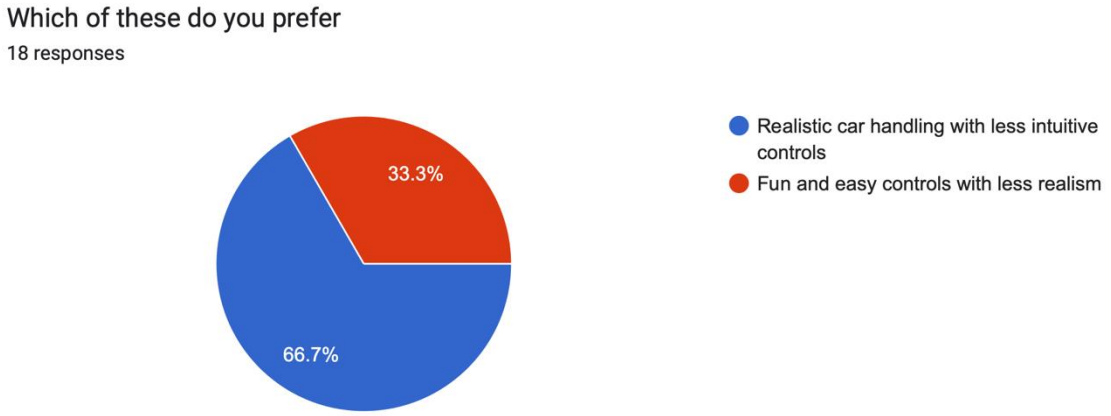
1.3 Research

1.3.1 Interviews/Surveys

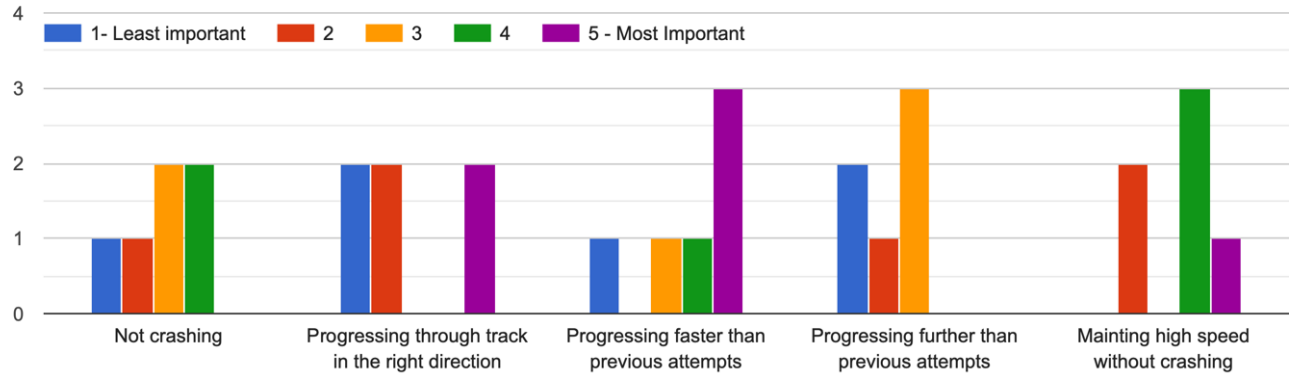
I conducted surveys of my test users as well as other peers in my class to gain a better understanding of some of the priorities I must have in the design of my simulation.

From this survey, I identified that I must prioritise accuracy in simulation over a fun stimulating game experience. This means I will not focus on adding features that add game like features like levels, objectives and score, but rather allow user to accurately simulate driving. This also translates to controls, where I will focus on realism over intuitive and easy to use controls. In terms of graphics and modelling, I will stick to a bird's eye view 2D simulation, and therefore it will be a simplified model of real-world driving physics. I also gained insight on the most important features of a custom track creation.

I also gained some generic insight on how to design the more technical aspects of my simulation. I concluded that I must model the throttle and braking pedals are gradually being pressed, with the brake pedal taking longer to press down. This emulates how in real cars the brake pedal is harder and therefore takes more effort to fully hold down. I also gained some initial insight into how to design and weight a reward function for the agent to learn from

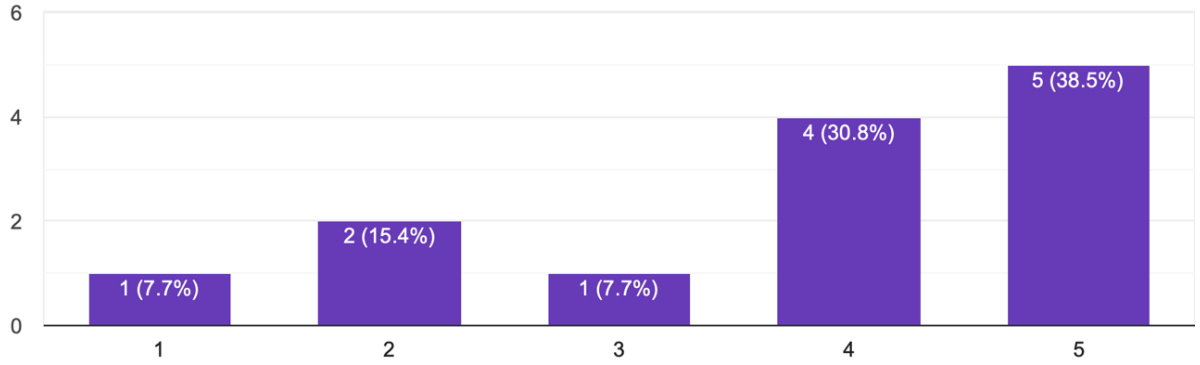


Rank these in order of how much the AI should be awarded



How important is it to model pressing throttle and braking pedals realistically?

13 responses



Summary of conversations with technical specialists

Conversation with D. Davis

Through my conversation with D. Davis, I was able to develop a better understand of how to define the scope of the simulation. We first discussed the process of modelling engine forces and gear shifts and concluded the level of complexity of such modelling exceeds the scope of this investigation. However, he agreed that the modelling of pedals realistically is an important part of the simulation and is important to model
We then discussed drifting mechanics, and the practical aspects of how to model oversteer, understeer and realistic turning. Having discussed concepts of slip angles and understeer gradient, I decided although it would be possible for me to implement this feature, I was not confident as to if a smaller RL model would be able to model it effectively, and so I decided against it. This is because when the car starts drifting, the mechanics of acceleration and turning work very differently, and this may confuse the model.

Finally, we covered the methods of modelling turning mechanics. I was aware about the idea of Ackermann steering geometry where both wheels rotate about a common centre point, but the inner wheel turns at a sharper angle than the outer wheel. However, when we discussed this, Mr. Davis suggested me an alternative method of modelling steering which was based on the kinematic bicycle model. This simplifies the modelling of a 4 wheeled car to a 2 wheeled modelled where the wheels are separated by the wheelbase by assuming an instantaneous centre of rotation. The primary assumption of this model is that there is no wheel slip, and since I already decided earlier that I would not model oversteer and understeer, my simulation would meet this criterion. And so, the kinematic bicycle model would help simplify the model further without catastrophically losing on realism.

Conversation with C. McCormack

Through my conversations with C. McCormack, I managed to outline how to design an AI solution to my problem statement. Since I was new to understanding the incredibly fast field of reinforcement learning (RL), I wasn't sure on which form of RL would be appropriate for this investigation and so we spent our initial discussion going over the various possible RL algorithms to narrow down which ones I should consider. We started simpler methods like *Q-Learning*, which I was able to determine were not appropriate. We also covered several deep learning methods such as *Deep Q-Learning*, and various stochastic policy gradient methods like *REINFORCE, Proximal Policy Optimisation (PPO), Advantage Actor-Critic (A2C), Soft Actor-Critic (SAC)* and so on. And finally, some deterministic policy gradient methods like *Deep Deterministic Policy Gradient (DDPG) and Twin Delayed Deep Deterministic Policy Gradient (TD3)*. Through this, we deduced that a stochastic policy gradient method may be the most appropriate since they allow for sufficient complexity while still being simple enough to implement from scratch. Mr. McCormack also suggested that my environment is complex enough to require neural networks for the model to learn to navigate the track. Having then discussed specific algorithms in more detail, I decided that REINFORCE would be the best algorithm for me to start with since it's

the simplest form of stochastic policy gradient that directly updates all parameters at the end of each episode.

In our next discussion, we discussed the possibility of expanding the scope by implementing A2C, since this only needs the implementation of a second critic neural network. Through our discussion I also understood that the type of returns computed and update mechanism by A2C differs from that of REINFORCE. REINFORCE computes Monte Carlo returns and updates parameters at the end of each episode, while A2C computed bootstrapped returns and updates at every 'N' time steps during each episode. However, I was conscious of the fact that my manual implementation of this algorithm would not be as optimised as mainstream libraries, and so the task of updating the parameters of the model would be computationally intensive. I was concerned that carrying out updates during the episode could lead to performance issues with the simulation caused by bottleneck. In response to this, Mr. McCormack suggested the idea of diverging from the traditional implementation of A2C and just using the same update mechanism of REINFORCE by computing just carrying out updates at the end of each episode with Monte Carlo returns. While this does compromise the integrity of the model by introducing higher variance, it seems worth the benefit of preventing bottleneck. This implementation is now a Monte Carlo Actor Critic algorithm, and this is what I decided to implement

In our final conversation, I gained insight into reward engineering. We discussed on the idea of how the function is meant to create a net signal for different behaviours and through this I was able to better understand the importance of scaling rewards. I was initially under the impression that I could just set arbitrary values for different events, like going fast has reward of 10 and crashing has reward of -100, but Mr. McCormack was able to explain the idea of dominating rewards, and how large spikes in reward signals could have catastrophic effects on gradient updates. So, I would have to very carefully design signals that don't overwhelm each other. He also cautioned me that for MCAC, large values for rewards could lead to very slow learning, since the critic network updates very slowly. And so, for MCAC (and possibly even for REINFORCE), I may have to apply a scaling factor to shrink the signals to smaller values.

1.3.2 Technical/Skills research

Creating smooth curves:

Bezier Curves:

A bezier curve is defined by some number of points which influence the shape of the curve, allowing for complex smooth curves to be rendered by only specifying a few points. The degree of a bezier curve is determined by the number of control points that are set. This is done by applying repeated parametric interpolation. Firstly imaginary lines are drawn between the 'n' defined points. Then n-1 more points are defined where each point lies some distance along each of these imaginary lines. This distance is defined by a parameter 't' where $t \in [0,1]$. So when t = 0, all these n-1 points lie at the start of the lines, when t = 0.5, all points lie half way along the lines and when t = 1, all points lie at the end of the line. The process is repeated to then define n-2 more points along the lines connecting the points defined in the previous step until only 1 point remains. All these points are defined by the same parameter 't'. This last remaining point is used to define the points along the bezier curve by varying the continous parameter t.

https://www.jasondavies.com/animated-bezier/

https://javascript.info/bezier-curve

For the purpose of rendering curves from user defined points, it is also essential to have a mathematical formulation of this parametric stepping. De Casteljau algorithm mathematically defines points on the curve by slowly incremeting the paramter t by small values. In its full formulation, it carries out the repeated interpolation mentioned above reccursively.

https://www2.cs.sfu.ca/CourseCentral/361/inkpen/Notes/361_deCasteljau_john.pdf

de Casteljau's labeling scheme

This formalizes the method:

- Each level of recursion denoted by a superscript.
- control points are the zeroth level and do not need to be superscripted.
- Each successive level of recursion has one less point than the previous level.
- The final level $P_0^n(t) = P(t)$ is the point on the curve
- For any point, it can be shown that

$$P_i^j(t) = (1-t)P_i^{j-1} + tP_{i+1}^{j-1}(t) \text{ for } i = 0,..,n;\ j = 0,1,..,i$$

Bezier curves always pass through start and end points, but don't pass through any other control points. So using a very high degree bezier curve to specify a comple curve would be highly ineffiicient, since a change in any one control point changes the shape of the whole curve (i.e there is no local control of the curve). It is also hard to predict the shape of the curve will take for higher degree curves since it only passes thorugh 2 of the specified points. So creating more complex track shapes with sharp bends would need a large number of points, and altering the shape of the curve would be very inconvinent since any minor movement changes the full track shape.

Bezier Splines:

Composite Bezier Curves or Bezier Splines are created by connecting Bezier curves end to end to create a single continuous path. Since each individual bezier curve passes through 2 of the specified control points, a bezier spline passes through more control points, which means the curve takes a more predictable shape. Additionally,

since each bezier curve segment is only specified by the conrol points near it, altering one control points only changes the shape of the bezier curves near it, not the whole spline (i.e, there is local control). The most commonly used degree of a bezier spline is the cubic bezier spline (uses cubic bezier curve). Each bezier curve is defined by 4 control points, and passes through 2 of them. Therefore cubic bezier splines offer an ideal balance between predictible controlled curves, and flexibility in the shape of the curve. The 2 end points determine where the curve passes, and the 2 control points determine the direction and curvature between them.
The individual bezier curve segments are generating the same as before. Since the degree of bezier curves being used is always 3, It is possible to bypass the recursive step of de casteljau's algorithm and directly formulate the parametric equation for points of the curve.
https://en.wikipedia.org/wiki/Bézier_curve

The explicit form of the curve is:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3,\ 0 \le t \le 1.$$

Where B(t) is a point on the cubic bezier curve.

Spline Continuity

Continuity of splines is

Catmull-Rom Spline

A Hermite spline is defined by 2 end points, and 2 tangent vectors, one at each end point. These splines don't use control points, but rather define the magnitude and direction of the curve at the end points directly using the tangent vectors.
A Catmull-Rom spline is a special type of hermite spline where these tangent vectors are calculated implictly using the neighbouring points and the control points are infered by these tangent vectors. The user therefore only specifies the anchor points of the curve, and the curve is rendered based on the calculated tangents.
The Catmull-Rom spline is ideal for allowing the user to draw curved tracks since it passes thorugh every point defined by the user. All control points are interpolated based on adjacent points, which allows intuitive and predictable curves to be generated.

Reinforcement Learning:

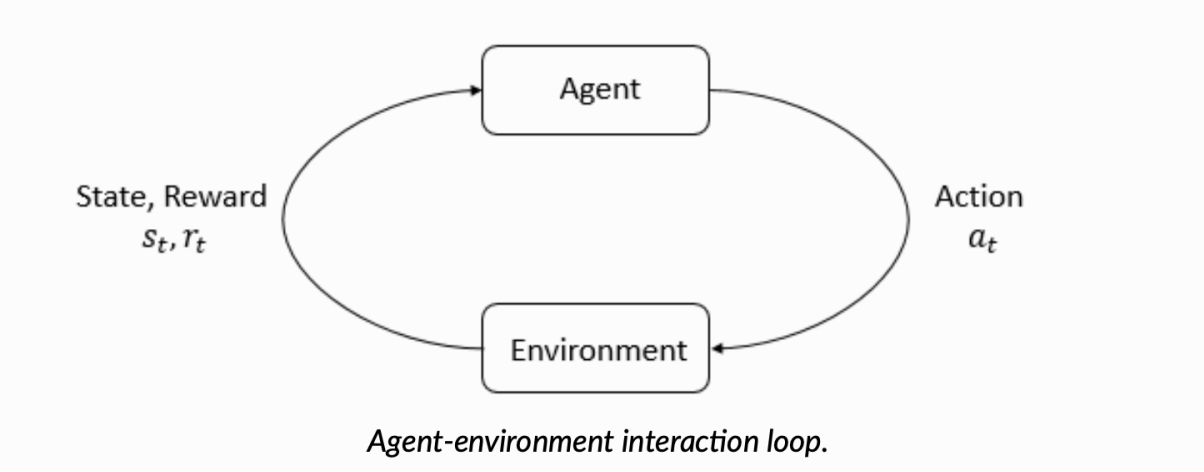
Why Reinforcement Learning (RL):

There are many different types of machine learning, each suited for a specific type of problem. For my program to learn the fastest path around a track, the most suited form of ML is reinforcement learning (RL), since supervised and unsupervised learning methods require a predefined dataset to learn from, while RL learns from interacting with the testing environment. RL solves a particular kind of problem where decision making is sequential, and the goal is long term. The task of driving a racing car satisfies this requisite. In this project, RL aims to train up an Artificial Intelligence (AI) model that can optimally race a car on a track.

How RL works:

RL works by using some sort of "agent" to live in the given test environment. This agent must be capable of perceiving the "state" of the environment as a set of "features" and then execute an "action" in that state to bring it to the next state. Different actions bring different magnitudes of "rewards", which are indications of how good the action taken was. The "policy" is the strategy that the agent uses to decide what action to take in each state. The goal of an RL algorithm is to learn the most optimal policy to maximise the total expected reward over the time that the model is run.

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html
https://www.youtube.com/watch?v=NFo9v_yKQXA&list=PLzvYlJMoZ02Dxtwe-MmH4nOB5jYlMGBjr



Agent-environment interaction loop.

Key Terminology:

Agent: The entity which is meant to be controlled. In this project, the agent is the car that is being controlled by the AI

Environment: The environment is defined by space where the agent acts, and the interactions the agent has with its surroundings. The environment in this project is the track, and the physics associated with driving the car on the track

State: The characteristics of the environment that can be observed or experienced by the agent at a given time.

Features: The numerical representation of the state the agent is observing. This allows the agent to mathematically make decisions following a policy

Actions: The decisions taken by the agent at a given time step to move from one state to another state of the environment.

Rewards: A numerical value that describes how good or bad a given action taken by the agent was. The reward value for each action-state pair is defined by a reward function, which aims to describe the desired behaviour. For example, a reward function for this project may include, driving fast: +5, finishing lap: +10, crashing: -50.
Policy: The rules/values that define how the agent chooses actions for a given set of features.

Trajectory: The record maintained of all state-action pairs that are experienced by the agent each time step during an episode.

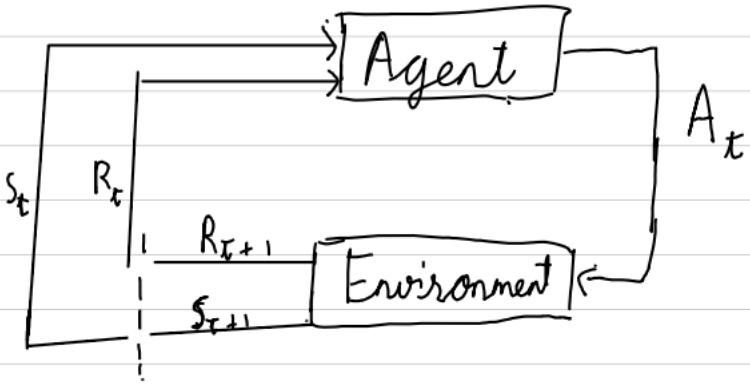
Returns: This is the total accumulated reward from a given time step onward, often with future rewards discounted, and it represents how good a state–action choice is in the long term, not just immediately.

Markov Decision Process (MDP) formulation:
MDP is a mathematical model for sequential decision making where outcomes are uncertain and is solved using dynamic programming methods. The process involves evaluating what actions should be taken, what happens after action is taken and how good is the outcome from an action. MDP almost works like a sort of stochastic path finding algorithm, where instead of determining the most optimal path to achieve a goal, it determines the most optimal policy on how to select a particular action given the current state that it is in.

A key property of a MDP problem is that the value of any future actions and states on depends on the current state and not the history of states. Put into practice, this means that the future of driving the car only depends on its current state of where it is on the track, how fast its going, where it's steering etc. Since this property has been fulfilled, this problem can be classed as an MDP problem.

Reinforcement Learning

Finite Markov Decision Process (MDP):



$t \in \{0, 1, 2 \ldots\}$
$s \in S$
$a \in A(s)$
$r \in R \subset \mathbb{R}$

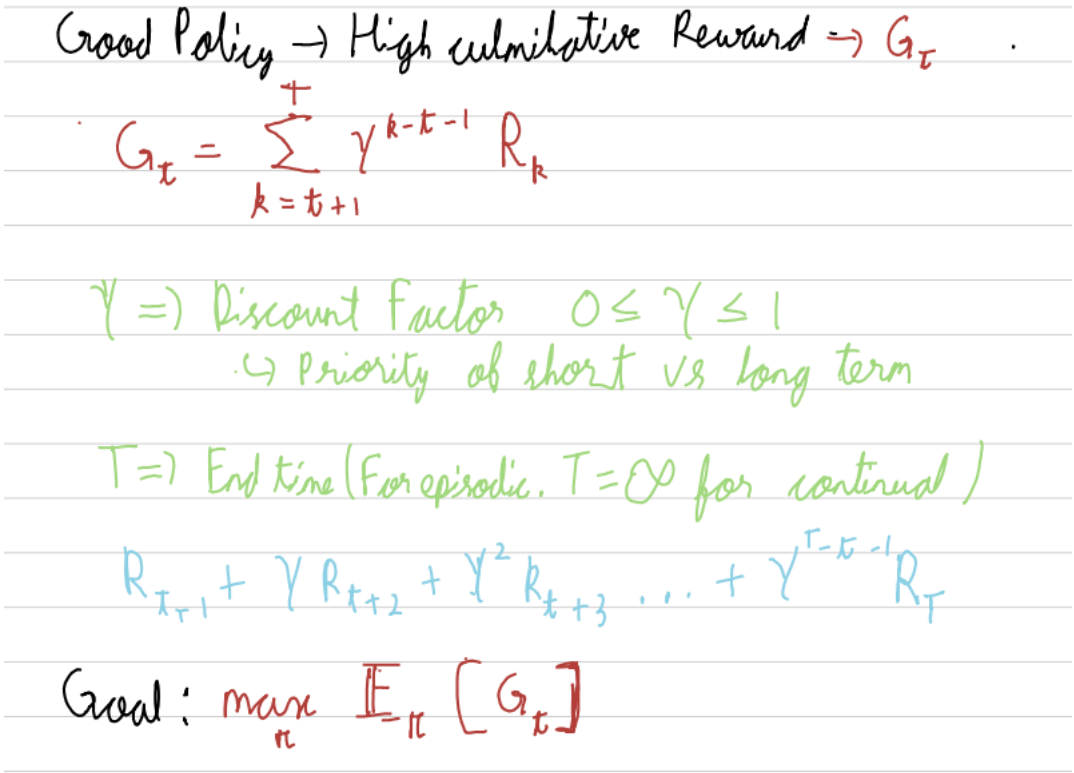
$$P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$$
$$\Rightarrow p(s', r \mid s, a)$$

Markov Property :

$$P(X_{t+n} = x \mid X_t, X_{t-1} \ldots, X_{t-k}) = P(X_{t+n} \mid x \mid X_t)$$

Policy : $\pi(a \mid s)$ or $a = \pi(s)$

↓ Probabilistic ↓ Deterministic



https://www.youtube.com/watch?v=2iF9PRriA7w&t=693s

In Reinforcement Learning, MDP provides formal structure that defines the environment and guides how decisions are taken over time.
https://www.geeksforgeeks.org/machine-learning/what-is-markov-decision-process-mdp-and-its-relevance-to-reinforcement-learning/

There are 5 main components to an MDP problem:

Components of MDP

- **Agent:** The decision-maker that learns by interacting with the environment to achieve a goal.
- **Environment:** Everything the agent interacts with, which responds to the agent's actions by changing states and giving rewards.
- **State (S):** A representation of the current situation of the environment at a given time.
- **Action (A):** A choice made by the agent that influences the next state of the environment.
- **Reward (R):** A numerical feedback signal that evaluates the quality of an action taken by the agent.

For RL, the way the actions are determined is determined by the output of the model, and the way the agent transitions from one state to the next is dependent on the environment. The model trains the parameters of a neural network to define the policy (how actions are generated from a given state to maximise reward).

Value Functions:
Value functions determine how good the current situation the agent is in. There are 2 types of value functions

1) State Value function $V^{\pi}(s)$: Expected return if starting from a state 's' the agent always acts according to the policy 'π'

$$\mathbf{V^{\pi}(s)} = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = \mathrm{s}]$$

2) Action Value Function Q^{π} (s,a): Expected if starting from state 's' the agent takes the action 'a' and goes on to act according to policy 'π'

$$\mathbf{Q^{\pi}(s,a)} = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = \mathrm{s}, a_0 = \mathrm{a}]$$

The primary connection between the state and action value functions is that the state value is the expected (or average) value of the action value functions of all possible actions in the given state under the policy. Formally this is written as:

$$V^{\pi}(\mathrm{s}) = \mathbb{E}[\, \mathrm{Q}^{\pi}(\mathrm{s},\mathrm{a})]$$

These value functions provide numerical values for the environment. However, in this implementation of RL, these value functions are unknown and cannot be explicitly calculated, since the size and dynamics of the environment are largely variable and continuous (meaning they cannot be iterated through). To solve this problem, stochastic policy gradient implements heuristics to estimate these value functions and carry out policy optimisation updates by following gradients.

Reinforcement Learning objective:
The goal of a reinforcement learning problem is to maximise the expected reward when following the policy across all possible trajectories.

$$J(\theta) = E_{\tau\sim\pi}[R(\tau)]$$

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$

Return: cumulative reward — Gamma: discount rate

Trajectory (read Tau)
Sequence of states and actions

Where J(θ) is the objective function to be optimised. This function can be written in terms of value functions, where the objective can be expressed as maximising the value of starting states.

When updating parameters, each parameter is updated by the gradient of the objective function with respect to that parameter. In other words, each parameter is

updated to maximise an increase in the objective function by calculating the rate of change of the objective function as the parameter changes.

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Where θ is the set of parameters and $\nabla_{\theta} J(\theta)$ is the gradient of the objective function with respect to each parameter θ

Policy Gradient Theorem:

The gradient of the objective function is directly related to the action value functions of the environment

The policy gradient theorem allows us to compute the gradient of the objective function, without needing to explictly know the environments transition probabilities or dynamics. This is useful since in most reinforcement learning problems, environment dynamics are unknown and the transition function is not differentiable. SO directly computing the gradient of the objective function is not possible.

Derivation of the Policy Gradient Theorem:

Key:

θ ← Parameters (weights and biases) of the neural network.

$\pi_\theta(a \mid s)$ ← Stochastic policy parameterised by θ. (Probability of taking action **a** in a given state **s**.)

τ ← Trajectory of the sequence of states and actions: (s_0, a_0, s_1, a_1, …, s_t)

$p(\tau \mid \theta)$ ← Probability of generating trajectory τ when following policy π_θ.

$p(s_0)$ ← Probability distribution over initial states.

$P(s_{t+1} \mid s_t, a_t)$ ← State transition probability of the environment.

R(τ) ←Total Monte Carlo returns obtained over trajectory τ. This term is the culmilation of all G_t terms used in the neural netowork update derivation discussed earlier.

$J(\pi_\theta)$ ← Expected return of policy π_θ.

$\mathbb{E}[\cdot]$ ← Expectation operator:

$$E(x) = \sum_{i=1}^{n} \left(x_i\right) P\left(x_i\right)$$

Where x_i is a possible value of X and $P(x_i)$ is the probability that X takes this value.

∇_θ ← Gradient with respect to the policy parameters θ.

$\log P(\tau \mid \theta)$ ← Log-probability of trajectory τ under policy π_θ.

$\log \pi_\theta(a_t \mid s_t)$ ← Log-probability of selecting action a_t in state s_t under policy π_θ.

$\sum_{t=0}^{T}$ ← Summation over all time steps in an episode.

T ← Final time step (episode length).

$\nabla_\theta J(\pi_\theta)$ ← Policy gradient. i.e. direction of steepest increase in expected return. This term dictates the updates.

α ← Learning rate (step size for parameter updates).

θ_k ← Policy parameters at iteration k.

Let π_θ be stochastic Policy (θ is the weights of the neural network)

Goal :- Maximise Expected return over an episode (or trajectory τ)

Let $J(\pi_\theta)$ be the objective function to maximise

$$\therefore J(\pi_\theta) = \underset{\tau \sim \pi_\theta}{\mathbb{E}}[R(\tau)] = \int_\tau P(\tau|\theta) \cdot R(\tau)$$

↳ Sum over all possible τ

i) $$P(\tau|\theta) = \rho_0(s_0) \prod_{t=0}^{T} P(s_{t+1}|a_t, s_t)\, \pi_\theta(a_t|s_t)$$

↳ Probability of a given trajectory

$$\Rightarrow \log P(\tau|\theta) = \log\left(\rho_0(s_0) \prod_{t=0}^{T} P(s_{t+1}|a_t, s_t)\, \pi_\theta(a_t|s_t)\right)$$

$$= \log(\rho_0(s_0)) + \sum_{t=0}^{T} \log(s_{t+1}|a_t, s_t) + \log \pi_\theta(a_t|s_t)$$

$$\therefore \nabla_\theta \log P(\tau|\theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

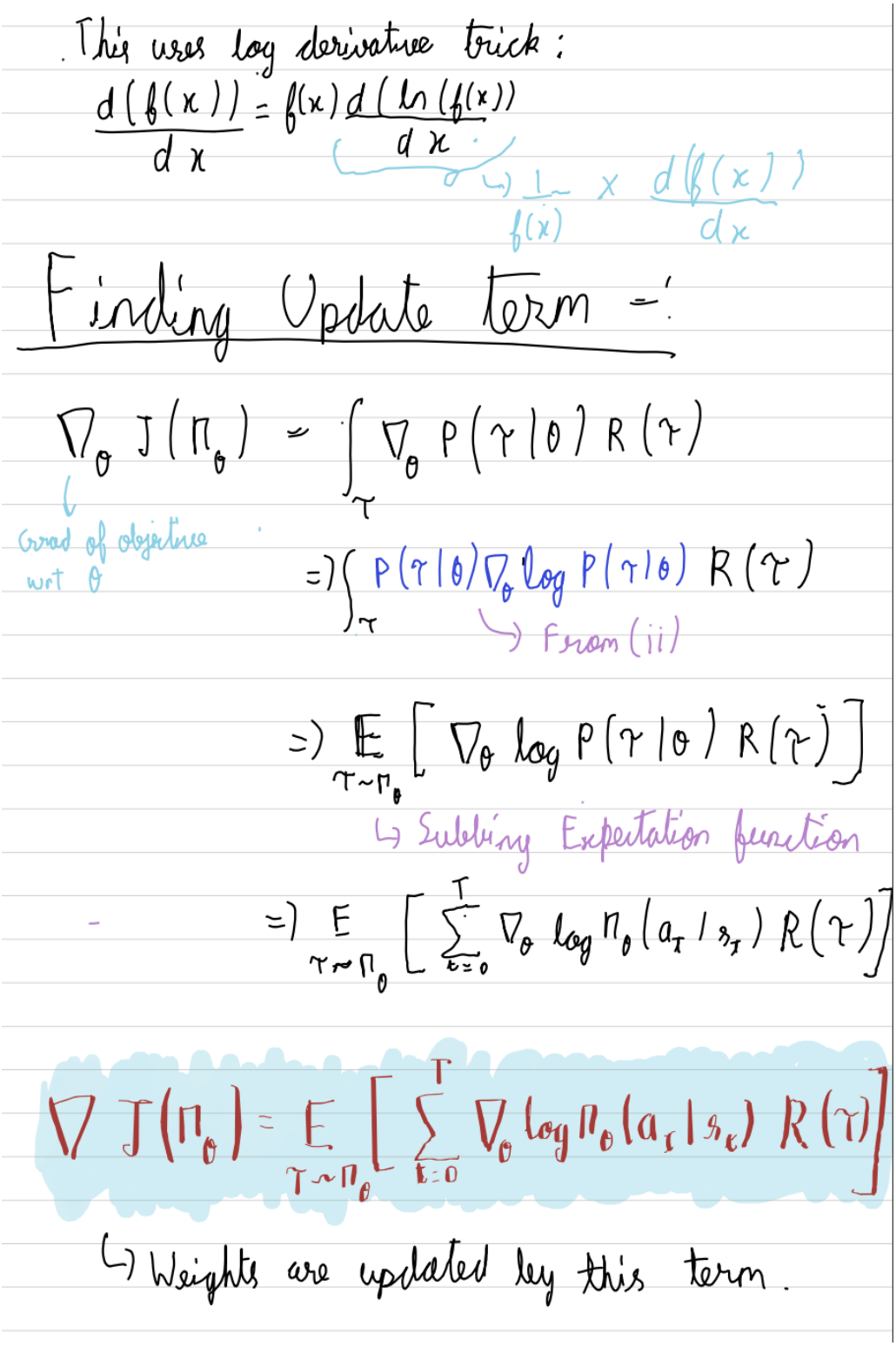
↳ $\nabla_\theta \log \rho_0(s_0) = 0$

$\nabla_\theta \log(s_{t+1}|a_t, s_t) = 0$ → No θ Term

ii) $$\nabla_\theta P(\tau|\theta) = P(\tau|\theta)\, \nabla_\theta \log P(\tau|\theta)$$

↳ Differentiate w.r.t θ.

↳ In RL, $\log(x)$ is $\log_e(x)$



So finally the update term:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\cdot R(\tau)\right]$$

This formulation uses Monte Carlo returns $R(\tau)$ of each taken action to approximate the action value function $\mathbf{Q^{\pi}(s, a)}$**.** This way, updates are directly related to how good each action was.

Advantage function:
The advantage function $\mathbf{A^{\pi}(s,a)}$ Corresponding to the policy π describes how much better it is to take a specific action 'a' in state 's' over any other action that could be selecting. Mathematically, this is defined by

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

Since the state value $\mathbf{V^{\pi}(s)}$ is the expected value of all possible actions in the state.

This advantage function term can be used instead of the Monte Carlo return R(τ) term in the policy gradient equation. Updating parameters directly scaled by R(τ) can have high variance, since it assumes that the single sequence of actions is the best estimate for the overall action value function. Subtracting the state value function acts as a bassline which helps to stabilise updates and learning.

Stochastic Policy Gradient methods:

https://opencourse.inf.ed.ac.uk/sites/default/files/https/opencourse.inf.ed.ac.uk/rl/2024/rl11policygradientmethods_0.pdf

There are 2 main approaches to obtaining optimal policy through RL. Value based methods (Q-Learning, Deep Q-Learning) calculate state and action value functions and use this to iterate a policy with the biggest action value (Useful when a finite set of actions and it is possible to know action values). Policy based methods (REINFORCE, TRPO) directly optimise policy without using a value function and instead use total rewards over a trajectory to make updates to the policy (Useful for continuous or stochastic action spaces).

For this project, I have chosen to investigate 2 stochastic policy gradient methods: REINFORCE and Monte Carlo Actor Critic (MCAC). Both these algorithms stochastically sample actions and then use Monte Carlo returns at the end of each episode to carry out parameter updates. I implement these algorithms to train neural networks to produce an AI model.

REINFORCE
The REINFORCE algorithm is a policy gradient method that is used to learn in continuous action spaces. The method implements a stochastic action sampling and uses policy gradient to update neural network parameters to increase probabilities of actions that had a greater return and decrease probability of actions that led to lower return. By iteratively conducting several episodes, the algorithm aims to maximise reward to obtain the optimal policy. As more episodes are carried out, more trajectories are sampled and the neural network parameters converge towards the optimal policy.

Monte Carlo Actor Critic (MCAC)

Monte Carlo Actor Critic is a variation of a traditional Actor-Critic algorithm, that implements Monte Carlo returns instead of TD-bootstrapping. These means updates are carried out at the end of each episode instead of every few time steps like traditional actor critic. The method implements 2 separate neural networks, the 'actor' and the 'critic'. The 'actor' network works and trains just like the main neural network of the REINFORCE algorithm, where actions are determined stochastically

and the parameter updates are carried out w.r.t the gradient of the objective function. The 'critic' network is a secondary network that works to predict the state value function $\mathbf{V}^{\boldsymbol{\pi}}(\boldsymbol{s})$ at each state. This value estimate is used to compute the advantage function $\mathbf{A}^{\boldsymbol{\pi}}(\boldsymbol{s}, \boldsymbol{a})$ using:

$$\mathbf{A}^{\boldsymbol{\pi}}(\boldsymbol{s}, \boldsymbol{a}) \approx \mathrm{R}(\tau) - \mathbf{V}^{\boldsymbol{\pi}}(\boldsymbol{s})$$

This advantage value is then used in the update gradients to stabilise learning. The value network is trained simultaneously by taking the root mean square error of the value predicted and the actual returns obtained.

Neural Network:

Deep learning is a class of machine learning that involves training neural networks with many layers. The purpose of deep is to be able to solve more complex problem by modelling higher dimensional patterns.
https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning
https://hediyetapan.medium.com/understanding-the-mathematics-behind-multilayer-perceptrons-mlps-0de46159fe5a

Neural networks play a central role in Deep RL. At its core, neural networks are computational systems that model complex patterns and tasks by taking inspiration from how the human brain works to do the same. Like a biological brain, a neural network consists of interconnected nodes or neurons, and these neurons pass data from one to another. The connections, or synapses, have weights associated with them and determine the strength of the connections between the neurons and thereby how much of each signal passes through. Neural networks act as universal function approximators. In other words, they model complex patterns and very highly dimensional environments as mathematical functions where a given input 'state' maps to a particular output 'actions'. They are also black boxes, which means that the mathematical functions it models are not explicitly defined. There are many different neural network architectures, each suitable for different tasks.

Feedforward neural network:

The neural network architecture that is suitable for this project is the Multi-Layer Perceptron (MLP) architecture. This is a type of feedforward neural network that is fully connected, i.e., every neuron of one layer passes data to every neuron of the next layer. A feedforward networks passes data only one way through the network from one layer to the next, without any convolutions.

Weights and biases

Each layer in the network performs a linear transformation followed by a non-linear activation function. Each layer has its set of weights (matrix with shape [input size, layer size]) and biases (vector of shape [layer size]). The linear transformation is applied to an input vector 'x' to produce a new vector 'z' of same size as 'x'. this is written as

$$z = W.x + b$$

Activation function

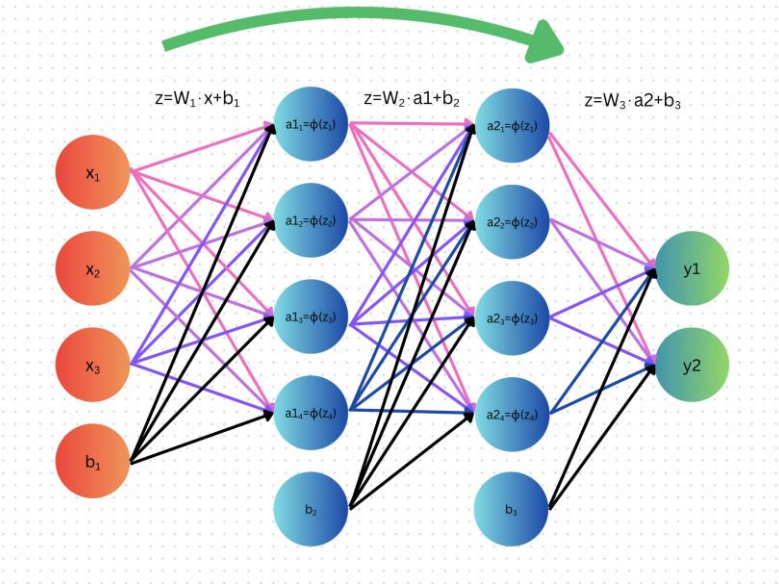
If just z was the output of each layer, the stacked linear layers would still be a linear function, and so can only model linear patterns. To model more complex non-linear patterns, we must introduce some non-linearity to the network. This is done by

applying a non-linear activation function to the ‘z’ vector to produce the output vector ‘a’.

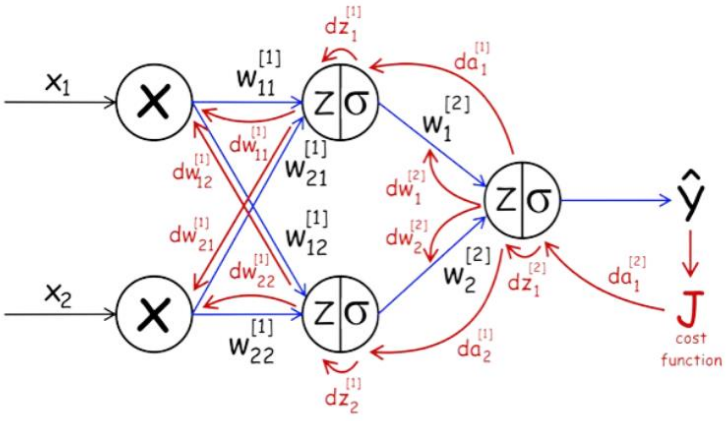
$$A = \phi(z)$$

Where ϕ is some non-linear activation function (like ReLu, Tanh, SoftMax etc.)

To program my own neural network, I learnt about the forward and backpropagation algorithms needed to make a feedforward network.



Forward Propagation



Backpropagation

Forward Propagation:

Forward propagation is the process of passing an input vector into a neural network, and passing it through all the layers to get the output. Each layer passes its output as the input to the next layer, and the final output layer outputs the actions. For a neural network with N layers:

Layer 1:

$$z^{(1)} = W^{(1)}.x + b^{(1)}$$
$$a^{(1)} = \phi(z^{(1)})$$

Layer 2:

$$z^{(2)} = W^{(2)}.a^{(1)} + b^{(2)}$$
$$a^{(2)} = \phi(z^{(2)})$$
$$\dots$$

Layer N:

$$z^{(N)} = W^{(N)}.a^{(N-1)} + b^{(N)}$$
$$\text{Output} = \phi(z^{(N)})$$

Backpropagation:

Backpropagation is the procress of passing gradients backwards from the output layer to the first layer of the network in order to carry out parameter updates. This is because the gradient of the objective function can only be directly related to the output of the network and no other hidden layers, since the input each layer recieves (and therefore the way it influences the objective function) is dependent upon all the previous layers. So, gradients need to flow backwards through the network and "reverse engineer" the gradients by applying chain rule. So the gradient of the objective function with respect to the output is found $\frac{\partial J}{\partial\ output}$ (using policy gradient theorem) and then this gradient passes through the layers to carry out updates:

For a neural network with N layers:

Layer N:

$$\frac{\partial J}{\partial W^{(N)}} = \frac{\partial J}{\partial\ \text{output}} \cdot \frac{\partial\ output}{\partial z^{(N)}} \cdot \frac{\partial z^{(N)}}{\partial W^{(N)}}$$

Layer N-1:

$$\frac{\partial J}{\partial W^{(N-1)}} = \frac{\partial J}{\partial W^{(N)}} \cdot \frac{\partial W^{(N)}}{\partial z^{(N-1)}} \cdot \frac{\partial z^{(N-1)}}{\partial W^{(N-1)}}$$
$$\dots$$

Layer 1:

$$\frac{\partial J}{\partial W^{(1)}} = \frac{\partial J}{\partial W^{(2)}} \cdot \frac{\partial W^{(2)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial W^{(1)}}$$

Same equations apply for finding gradients and updating biases of each layer.

Guessed values for hyperparameters:
L ← number of layers
N ← num of neurons per layer
X ← input vector
Y ← output vector
η← learning rate
γ← discount factor

Linear vectorised operation at each layer

Linear outputs wrapped in activation function to model non-linear patterns.

Output of each layer is passed as input to next layer

The actions are randomly sampled from gaussian distribution with the mean as the output of the neural network. (Here the standard deviation is a constant, however I have designed it to be a learnable parameter

Action is a vector of size 2 (steer and throttle)

This is the explicit formula of the gaussian distribution

π_θ ← Policy π (The agent's strategy for picking an action 'a' in each state 's') parameterised by θ (The weights and biases of the neural network).
$\pi_\theta(a|s)$ ← The probability of picking action 'a' in a state 's' under the policy π_θ. Therefore, this is same as the equation for the gaussian distribution
$log\pi_\theta(a|s)$ ← natural log of the gaussian distribution

G_t ← Monte Carlo returns which is discounted sum of all subsequent rewards (discount factor γ)

L ← Loss function to be minimised (I decided to implement this as an objective function $J(\pi)$ to be maximised. This is the negative of Loss)

∇θL ← Gradient of the parameter weights and biases w.r.t the Loss function (I implement ∇θJ instead)

This means each individual weight and bias value is updated based on its gradient w,r,t the Loss (or objective)

L ← Loss function to be minimised (I decided to implement this as an objective function $J(\pi)$ to be maximised. This is the negative of Loss)

$Grad_{wi}$ / $Grad_{bi}$ ← Gradient w.r.t each individual weight/bias parameter

$Grad_{wi}$ / $Grad_{bi}$ ← Gradient w.r.t each individual weight/bias parameter
Parameters are updated by the discounted gradient.

1.3.3 Research Table

Research Type	Date	Description	Main actionable points
Car movement and physics	June 2023	To simulate driving a car, I will create a Car object that is controlled using throttle and steering, similar to driving a car in real life. These parameters can be controlled by a user or an AI agent. Simulating the physics of driving a real life car is highly complex and impractical, and so I will model an abstracted version of the physics. The car position and velocity is updated directly using rates of change and vectors, which omits the need to model complex forces. For steering, I will also model changes in direcctions through vectors and vector roration. To simplify the modelling, I will omit modelling the 4 wheels of the car individually as well as slip and drifting.	• Throttle directly updates the acceleration by being multiplied by an arbitrary constant. Velocity and displacement are then calculated using rates of change calculations. • Implement the kinematics bicycle model of car physics that models the wheels of the car as 2 points separated by the wheelbase of the car. This allows for a more simplified calculation of the angular velocity to model turning • Steering is modelling by updating the direction the car is facing based on its current angular velocity. Angular velocity is calculated from the steer input, the current speed and the wheelbase of the car.
Collision detection optimisation	July 2023	In training an AI model, I will be simulating many cars simultaneously and each car will have multiple “sensors” to measure distances in different directions. This necessitates optimised collision detection methods to reduce computation load and avoid bottleneck. Many of these methods use data structures that divide the environment into a grid and only	• I can use Spatial Hash Grid to divide screen into cells and hash all track wall segments to their respective cells • To get sensor data of distances in different directions, I can use ray casting. I can use the Digital Differential Analyser (DDA) algorithm to query cells that the rays pass through. • To identify collisions between rays/hitbox and

		check collisions between objects in the same cell. There are many methods of create a grid structure (Spatial Hash Grid, Quadtree etc) and traversing them (DDA, Bresenham line drawing).	track wall segments, I can use a parametric line intersection algorithm that returns the point of collision.
Neural Network	September 2023	To make an AI to effectively model my problem statement, neural networks are essential. A neural network is essential a universal function approximator, and deeper networks can model more complex problems. I investigated the various types of networks like RNNs, CNNs, feedforward networks etc, to determine which one suits my problem best. I researched into the structuring a network using neurons (perform linear operations and wrapped in an activation function) organised into layers (pass data through the network).	• Create “Layers” of nodes with their respective “weights” and “biases” to pass data both forward and backward through the network • Get actions from network by feedforward through the layers using by computing dot product of input with weights and adding the bias vector. This value is wrapped in an activation function to compute the output vector with is passed as the input into the next layer. • The parameters (weights and biases) of the network are updated by their gradients w.r.t an “advantage”. These gradients determine the direction in which the parameters are meant to be updated to maximise reward. • To optimise the updates of parameters, I will implement “Adaptive Moment Estimation (Adam)” optimiser. This optimisation algorithm computes the “momentum” and “velocity” of previous updates to determine updates

Stochastic Policy gradient	September 2023	This is a type of reinforcement learning where actions are chosen probabilistically rather than deterministically. The policy (neural network) outputs parameters for probability distribution. The learning process follows that the probability of actions leading to greater rewards is increased and those leading to lesser rewards is decreased proportional to their probability and the scale or reward. This allows policy to learn even when environment is noisy or uncertain.	• Manage instance of neural network (this is the policy that is parameterised by the weights and biases of the nodes) • Produce a state vector from the position of the car on the track each time step and input it to the neural network • Use the output of the neural network (mean 'μ') and standard deviation (sigma 'σ') to sample actions from a Gaussian distribution. This is done at each time step to control the agent. • Record trajectory (state, action and rewards) at each time step • Calculate gradient of parameters w.r.t to the advantage (which is calculated differently depending on the algorithm used) • Update the parameters (weights and biases) of the neural network with respect to these gradients.
REINFORCE algorithm	September 2023	This algorithm updates policy using the total return of a full episode, and the log probabilities of the actions taken. The "advantage" is calculated by normalising the return across the episode. The gradient update follows policy gradient ascent, where the log probability of each action is scaled by the advantage term so that better actions are made more likely.	• Forward propagate through a neural network (as described above) for each time step. • Compute Monte Carlo returns to estimate value of state by taking the discounted sum of all rewards in episode. • Compute "advantage" by taking the difference of the returns and the mean return.

			• Perform backpropagation of neural network (updating weights and biases) and update values using gradient ascent w.r.t advantage.
Monte Carlo Actor Critic (MCAC)	October 2023	A2C implements 2 separate neural networks: the “actor” which outputs actions (just like REINFORCE) and the “critic” which estimates the value function of each state. Both networks are trained simultaneously. The advantage term is now the difference between the returns and the values output by the critic. The returns computed by this method are n-step bootstrapped returns, which compute discounted sum over a fixed number of time steps instead of the full episode.	• Forward propagate through the actor network (same as REINFORCE) to get actions at each time step. • Forward propagate through the critic network at the end of the episode to compute the estimate value of each state • Update actor network using gradient ascent w.r.t advantage • Update critic network using gradient descent w.r.t value loss
Bezier Splines	October 2023	To create tracks with generate from just specifying a few points, I wish to implement a curve tool that is like the curve shape tool in PowerPoint. When the user clicks, a point is added and then smooth curved path is generated between all added points. Bezier splines allow the creation of smooth curves by specifying few points. They also allow for local control, so individual sections of the track can	• Implement a canvas object that can track mouse events and allow the user to create and move points around • Use Catmull-Rom splines to generate smooth curved path between the user defined points. Implement Catmull-rom interpolation for parametric curve generation

		be adapted independently Additionally, to allow greater customisability, I also want to enable to the user to adapt the curvature of the track path at specific points. This is done by creating “handles” at each anchor point that the user can twist to adapt curvature.	• To generate handles at the points, the implicit control points of Catmull-rom splines can be computed by converting back to standard Bezier curves.

1.4 Prototypes

The car class enables dynamic control over the movement of a car sprite, which can be controlled both by an agentic AI, or the user. I intend for this car to be an abstract representation of the mechanics of driving a car.
I intend to have separate AI_Car and Player_Car classes that both inherit from an abstract Car class. This is so that they can both share the same physics, collision detection and rendering mechanism, while ensuring they have separate control mechanisms. I started off by coding a basic prototype of the Car class (with user controls) to test basic physics and updates.

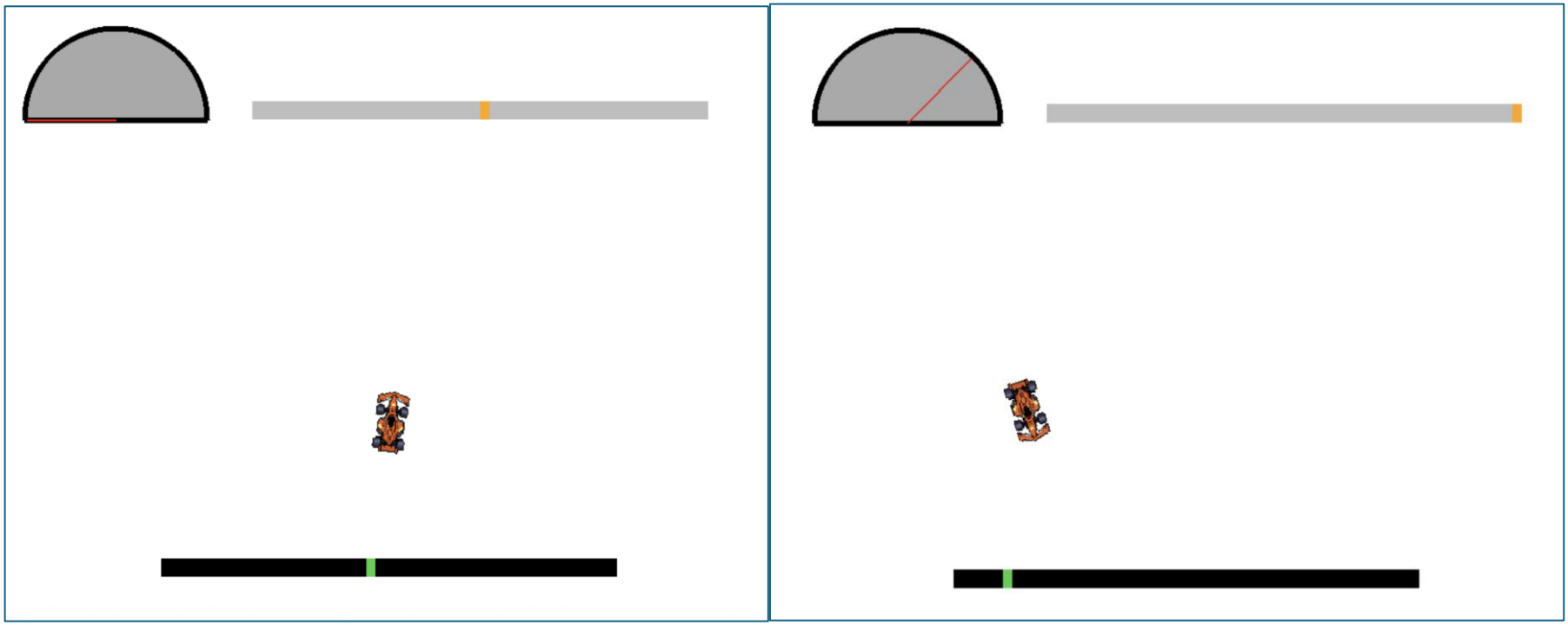
Steering

Initially, I made it so that the user can hold 'D' to increment steering (causing the car to steer right) and can hold 'A' to decrement the steering angle (causing it to steer left). I decided to smoothly increment the variable rather than just rotating the sprite to emulate how a steering wheel turns.

```
15
16      def increment_steering_angle(self, angle):  2 usages
17          self.steer += angle

26      def car_movement():  1 usage
27          if pygame.K_d in PRESSED_KEYS:
28              car.increment_steering_angle(-STEERING_INCREMENT)
29
30          if pygame.K_a in PRESSED_KEYS:
31              car.increment_steering_angle(STEERING_INCREMENT)
32
33          if pygame.K_w in PRESSED_KEYS:
34              car.speed = 5
35
36          if pygame.K_s in PRESSED_KEYS:
37              car.speed = 0
38
39          car.move_car()
40          car.turn_car(car.steer)
```

However, this keyboard control doesn't allow to hold a partial steer angle, since the steering keeps incrementing when the key is held. So, to solve this issue while still maintaining a similar modelling of a real-life steering wheel, I decided to experiment with using a slider to control the steering. This al
For this prototype, I decided to simplify the physics of movement by omitting the modelling of forces affecting the motion of the car. This means not modelling traction, rolling resistance, drag, understeer and oversteer (drifting). Instead, acceleration, velocity and displacement are all modelled in as vectors which increment and decay by arbitrary rates.
I tested this class with a program created a slider for controlling steering, and with other UI elements to be able to see how the speed and throttle values are updated.
I also created a custom Gauge meter GUI element that extends the pygame UIElement class. Extending UIElement allows my cutsom GUI elements to be rendered by the same GUI manager. This gauge meter is used to represent a speedometer, where the rotation of the needle is directly propotional to the speed of the car.

Output:

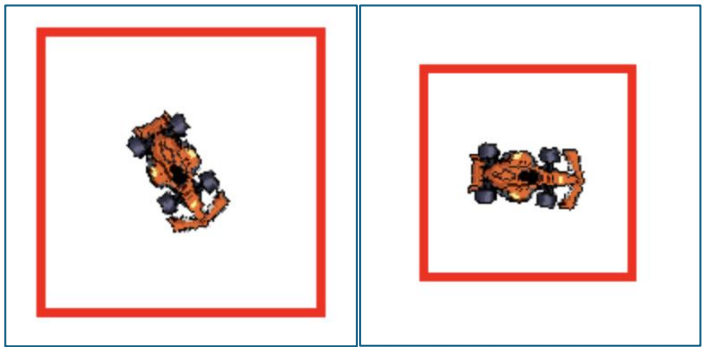


- The car steers and accelerates smoothly
- Frictions works as desires, and the car slows down when no throttle is applied
- The gauge meter functions well as a speedometer, by representing the speed of the car as a fraction of its max speed.

This current setup however allows the car to rotate freely independent of its speed, meaning the car can around while being completely stationary. In real world, the rate of rotation determined by the steering scaled by the velocity. So, to make steering more realistic, I decided to scale the rate of rotation by the velocity.

Hitbox

My initial idea to use the rectangle object of the sprite as the hitbox for collision detection, however when I tested it, the collision detection seemed to be highly inaccurate. This was because in python, rectangle objects are axis aligned, so a rotation in the car surface wouldn't rotate its rectangle, instead it would change size. So, I would have to create a separate polygon object that outlines the car surface to effectively create a hitbox for collision detection.



```
123     # Updates the coordinates of the corners of the hitbox w.r.t the position and orientation of the car
124     def _update_hitbox(self):  2 usages  joanmk +1 *
125         center_x, center_y = self.rect.center
126         width, length = self.car_proportions / 2
127         width -= 5
128         corners = [(-width, length), (width, length), (width, -length), (-width, -length)]
129
130         for i in range(4):
131             x, y = corners[i]
132             angle = math.radians(self.direction)
133             corners[i] = (center_x + x * math.cos(angle) - y * math.sin(angle),
134                           center_y + x * math.sin(angle) + y * math.cos(angle))
135         self.hitbox[:] = corners
```

This new approach has a hitbox that records the 4 corners of the car sprite taking into account its orientation. The polygon hitbox can then be created using these points

Track Creation interface:

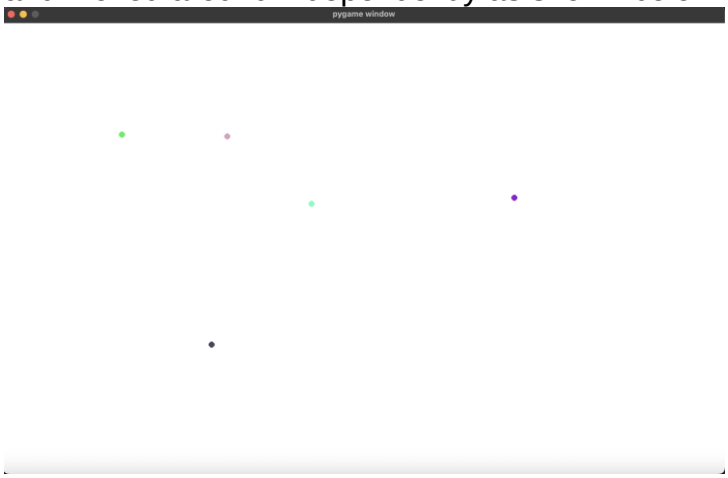
Code:

```
class UITrackCanvas(UIElement):  2 usages  joanmk *
    def __init__(self, relative_rect, manager):  joanmk *
        super().__init__(relative_rect, manager, container=None, starting_height=0, layer_thickness=1)
        self.anchor_points = []
        self.point_colours = []
        self.point_size = 5
        self.image = pygame.Surface((relative_rect.width, relative_rect.height))
        self.is_dragging = False
        self.is_click_buffer = False
        self.drag_point = None
        self.rebuild()

    def process_event(self, event):  joanmk *
        if event.type in (pygame.MOUSEBUTTONDOWN, pygame.MOUSEBUTTONUP, pygame.MOUSEMOTION):
            if self.rect.collidepoint(event.pos):
                relative_mouse_pos = Vector2(event.pos[0] - self.rect.x, event.pos[1] - self.rect.y)
                if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                    self.is_click_buffer = True
                    is_clicked_point = False
                    clicked_point = None
                    for i, point in enumerate(self.anchor_points):
                        if (relative_mouse_pos - point).length() <= self.point_size:
                            is_clicked_point = True
                            clicked_point = (i, point)

                    if is_clicked_point:
                        self.is_dragging = True
                        self.drag_point = clicked_point
                    else:
                        self.anchor_points.append(relative_mouse_pos)
                        r = random.randint( a: 0,  b: 255)
                        g = random.randint( a: 0,  b: 255)
                        b = random.randint( a: 0,  b: 255)
                            self.point_colours.append((r,g,b))

                if event.type == pygame.MOUSEMOTION:
                    if self.is_dragging:
                        self.anchor_points[self.drag_point[0]] = relative_mouse_pos

                if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
                    self.is_click_buffer = False
                    self.is_dragging = False
                    self.drag_point = None

        self.rebuild()

    def rebuild(self):  2 usages  joanmk *
        self.image.fill("White")
        for point in self.anchor_points:
            pygame.draw.circle(self.image, color=self.point_colours[self.anchor_points.index(point)],
                               center = point, radius = self.point_size)

            pygame.draw.lines(self.image,  color: "Black",  closed: False, curve_points,  width: 2)
```

The colour of anchor points have been set to random values for testing purpose to be able uniquely identify points. This will be changed to “Red” in later versions.

Output:

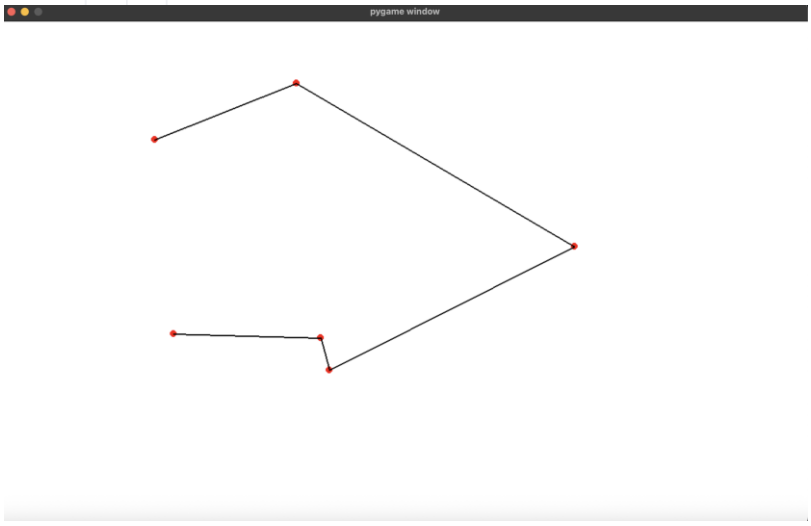
A new point is created for every mouse button click. These points can be dragged and moved around independently as shown below.



Track shape:

Joining all points together using straight lines can be used to create simple polygons.

```
162     def rebuild(self):  2 usages  joanmk *
163         self.image.fill("White")
164         for point in self.anchor_points:
165             pygame.draw.circle(self.image, color= "Red", center = point, radius = self.point_size)
166         if len(self.anchor_points)>1:
167             pygame.draw.lines(self.image, color: "Black", closed: False, self.anchor_points, width: 2)
168
```

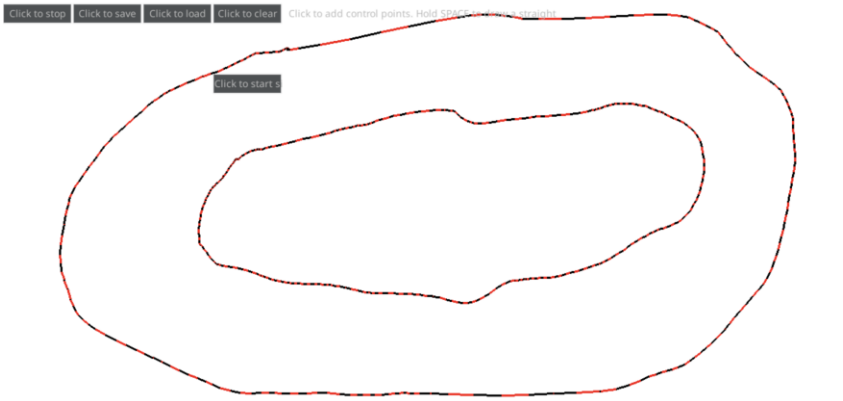


However, this is not desireable since track shapes are often smooth curves instead of straight lines. Using this rendering method would require hundereds of points to create a smoothly curved path. Ideally, the user should be able to specify points where the curve must pass through, and should also have control over curvature and direction of the curve at different points. This would be the idea for the actual track canvas I will make.

To protoype a Track UI for testing, I created a UITrackCanvas class that inherits the pygame_gui UIElement. This means that this is rendered by the pygame_gui manager. The canvas allows basic drawing function, by clicking and draagging. The current mouse position is reecored every cycle and when dragging, a line is drawn connenting the mouse current point to the mouse previous point. Release left click and dragging again starts a new stroke, and allows to draw seperete non-connected segments. The canvas also allows the ability to load a track shape from a file and to save a track trawing to a file (for the protoype only one track is saved to a prenamed file).

```
class UITrackCanvas(UIElement):  2 usages  joanmk +1
    def __init__(self, relative_rect, manager, bg_colour = "White"):  joanmk
        super().__init__(relative_rect, manager, container=None, starting_height=0, layer_thickness=1)
        self.bg_colour = bg_colour
        self.strokes = []
        self.current_stroke = []
        self.is_dragging = False
        self.is_drawing = False
        self.image = pygame.Surface( size: (self.relative_rect.width, self.relative_rect.height), pygame.SRCALPHA)
        self.rebuild()


    def process_event(self, event):  joanmk
        if self.is_drawing:
            if event.type in (pygame.MOUSEBUTTONUP, pygame.MOUSEBUTTONDOWN, pygame.MOUSEMOTION):
                if self.rect.collidepoint(event.pos):
                    if event.type == pygame.MOUSEBUTTONDOWN:
                        self.is_dragging = True
                        self.current_stroke.clear()
                    elif event.type == pygame.MOUSEMOTION and self.is_dragging:
                        relative_pos = pygame.Vector2((event.pos[0] - self.rect.x, event.pos[1] - self.rect.y))
                        self.current_stroke.append(relative_pos)
                    elif event.type == pygame.MOUSEBUTTONUP:
                        self.strokes.append(self.current_stroke.copy())
                        self.current_stroke.clear()
                        self.is_dragging = False
            self.rebuild()

    def rebuild(self):  4 usages  joanmk
        self.image.fill((0,0,0,0))
        r.draw_track_outline(self.image, self.strokes)
        if self.is_dragging:
            for i in range(len(self.current_stroke) - 1):
                pygame.draw.line(self.image,  color: "Red", self.current_stroke[i], self.current_stroke[i+1],  width: 3)

    def save_drawing(self, filename):  1 usage  joanmk +1
        with open("Tracks/"+ filename + ".txt", "w") as file:
            for stroke in self.strokes:
                for point in stroke:
                    file.write(str(point[0]) + "," + str(point[1]) + ",")
                file.write("\n")
            print("saved")

    def load_drawing(self, filename):  1 usage  joanmk
        self.strokes = r.load_track_from_file(filename)
        self.rebuild()

    def clear(self):  8 usages (7 dynamic)  joanmk
        self.strokes.clear()
        self.current_stroke.clear()
        self.rebuild()
```

Testing:



This prototype allows for the creation of temporary tracks, and allows me to test and experiment with collision detection, rendering, ray casting and so on. However, it is not completely ideal, since it is difficult to make smooth continuous tracks. It is also difficult to track car position on the track, since tracks don't have any well defined properties: they are just walls that are connected with no regulation. But it does suffice for the purpose of testing.

1.5 Justification of chosen solution

Considering the research conducted, I decided to code the track maker, car simulation and AI model in python. For the track maker and car simulation. I decided to use the pygame package for developing the game environment. pygame provides a wide range of built-in functions for rendering 2D graphics, event handling, and managing sprite objects. I used this in combination with the pygame_gui package which extends the functionality of pygame by enabling to create and render GUI elements like buttons, label and sliders. These libraries together help save time by simplifying development while still providing me control over the core logic of my program.

To program an AI model using neural networks, I decided to use the NumPy package to optimise matrix operations. Training neural networks involves performing a very large number of linear operations in both forward propagation for determining actions and backward propagation for updating the weights of the network. Without any optimisation, conducting training and performing this large number of operations would cause a severe bottleneck in the number of simulations I can run simultaneously due to expensive computation per simulation. This would mean significantly slower training, and practical difficulties in testing, debugging and fine tuning. NumPy optimises operations through vectorisation and efficient memory management. Vectorisation allows operations on entire arrays or matrices to be executed in a single step rather than through iterative loops. NumPy also uses contiguous blocks of memory for arrays, reducing overhead and enabling CPU-level optimisations such as SIMD (Single Instruction, Multiple Data) instructions. Using NumPy enables me to significantly reduce computation time and therefore implement batch simulations, which allows for faster training. It also allows me to still

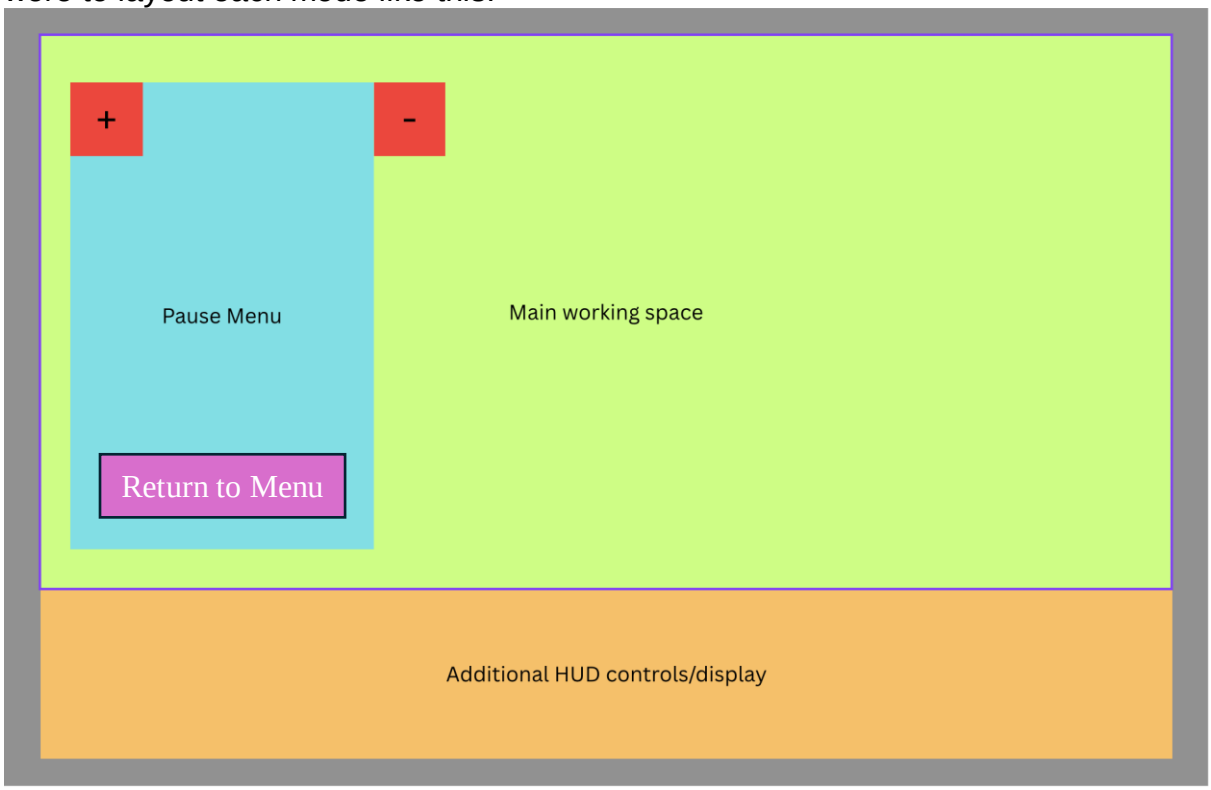
code the core algorithms of forward and backward propagation in the neural network, as well of policy gradient on my own accord.

Along with allowing the user to create their own tracks, and drive around these tracks, I also aimed to create an AI that uses reinforcement learning to dynamically learn the optimal way to drive a car around a track. The idea behind using reinforcement learning is that real life formula 1 cars often have several differences between teams. Additionally, teams change their cars every season when regulations change, and in fact also make several modifications during each season. All these differences can change what the driver must do to attain the fastest lap possible with the configurations and limitations of the car they are driving. This includes things like racing lines, braking points, angle to approach each corner and so on. Such kind of dynamic and versatile modelling is a suitable task for reinforcement learning methods. This project is aimed to be a research-based proof of concept of the ability of agentic AI to model driving a racing car. The simulation I have created is a basic and simplified model of a racing car, but the reinforcement learning AI model can theoretically be used to model even more complex environments just as well.

1.6 Initial Modelling

1.6.1Screen Layout

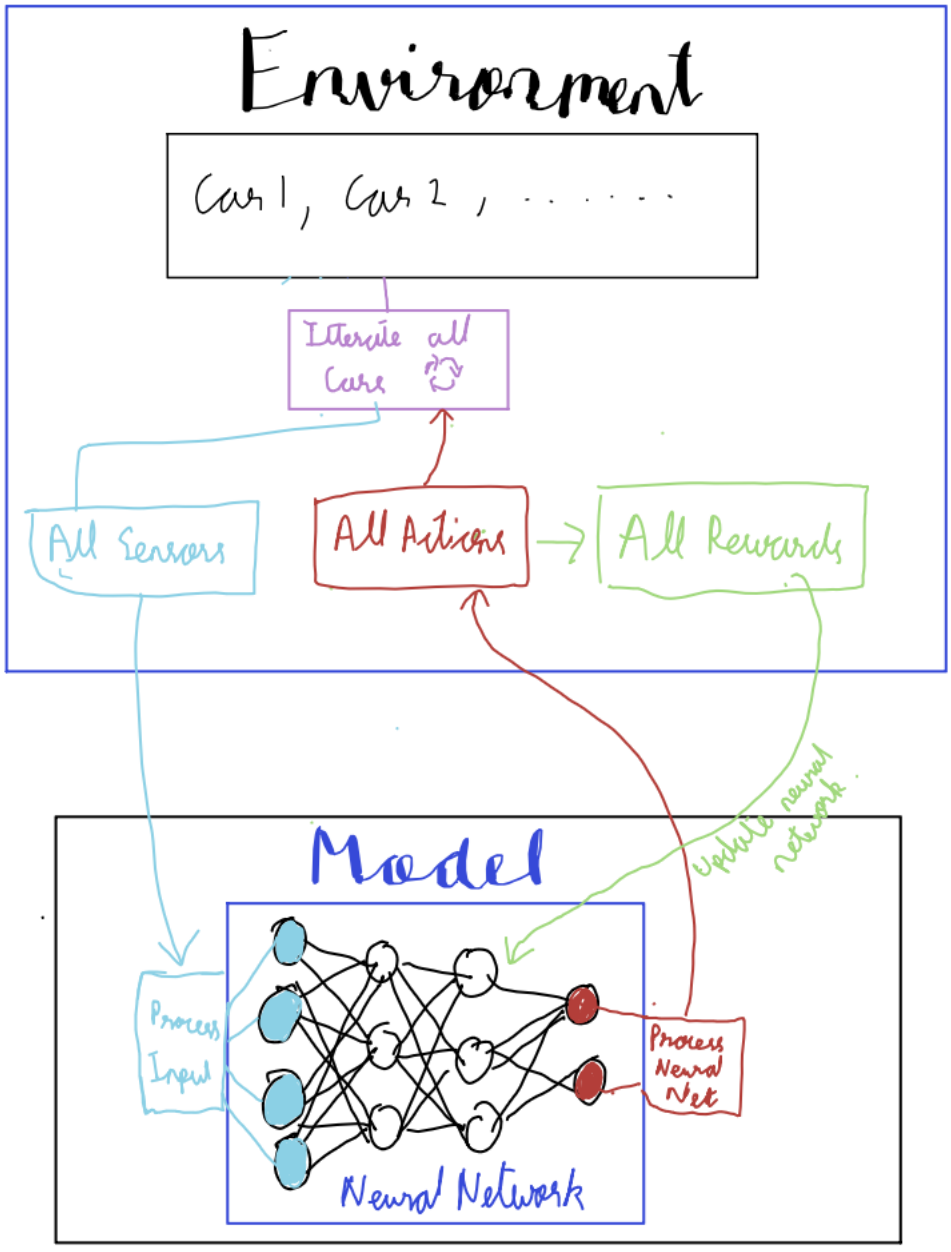
My project has 4 main modes: a track maker, an AI training mode, a solo car simulation mode and a mode to race against a trained AI model. My initial ideas were to layout each mode like this:



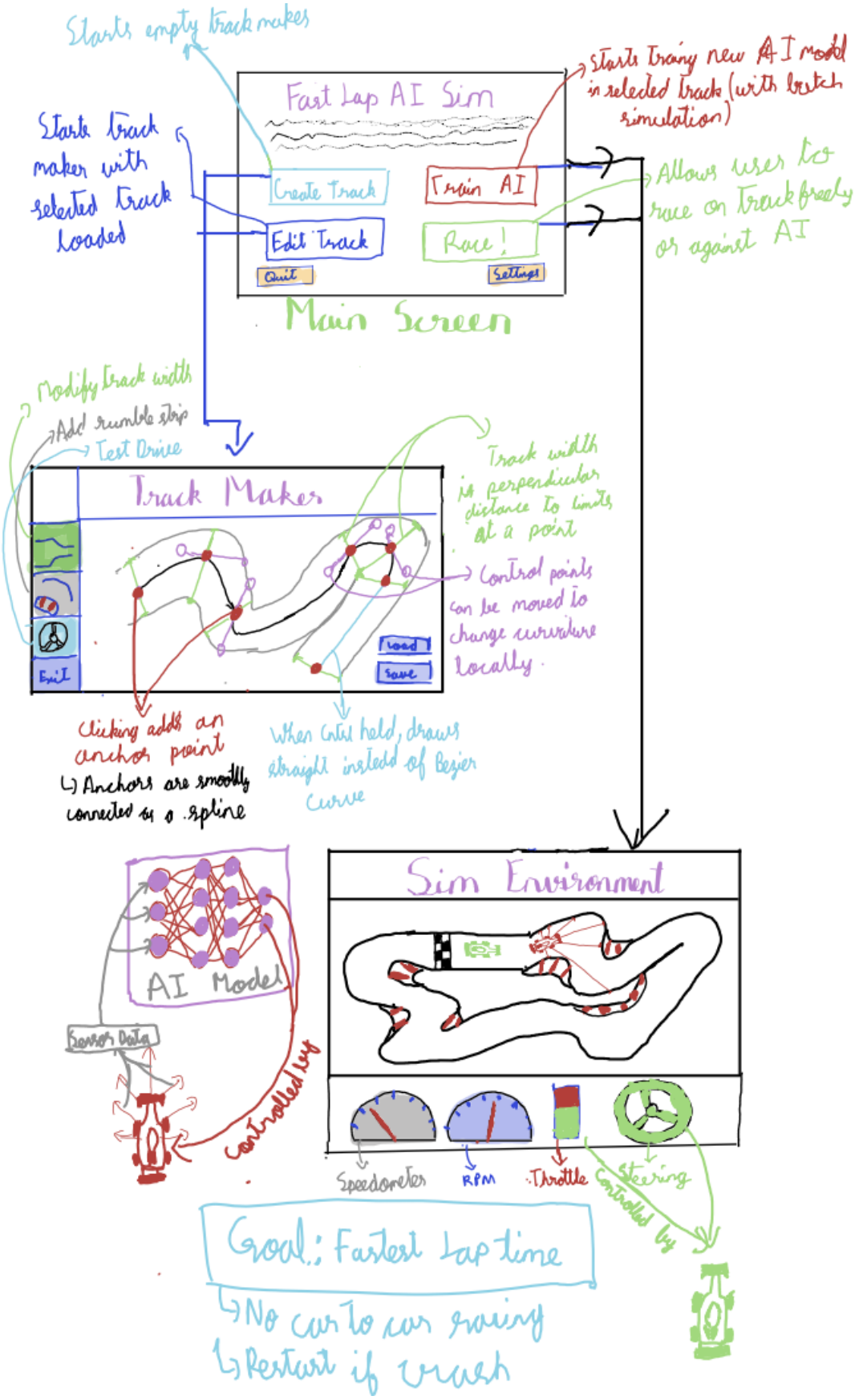
The pause menu should stop the game mode from updating and freeze all HUD control. The return to menu button will display a main menu that will allow the user to navigate to a different game mode.

1.6.2 Training loop

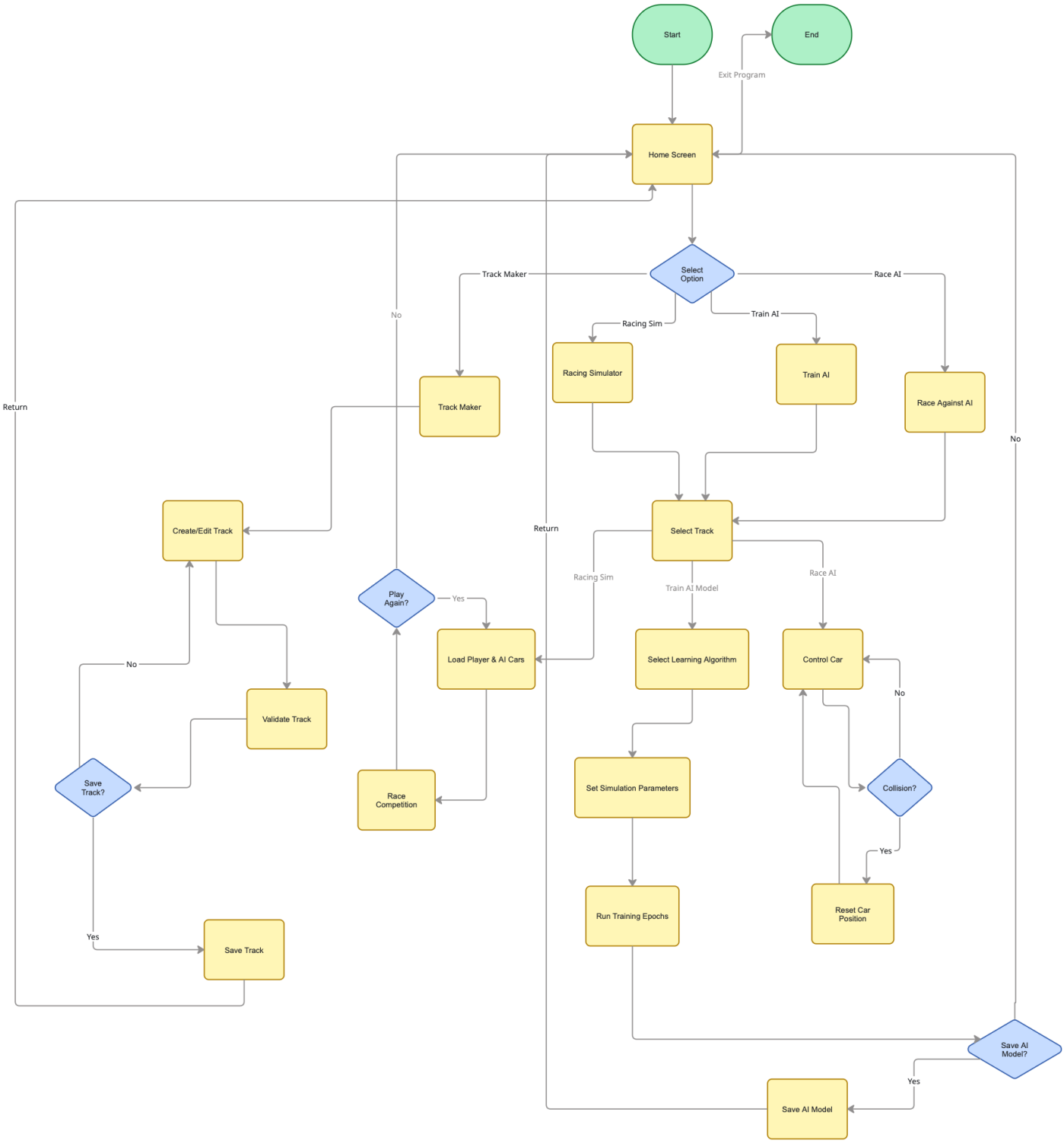
I worked on making an initial model of how to make an AI training environment. To avoid bottleneck in computation, the actions of all the AI cars being simulated should be computed in a single batch. This involves compiling all the states each time step into a matrix and them passing them through the neural network in one go using optimised NumPy matrix calculations.



1.6.3 Story Board



1.6.4 Sketches and process flow of proposed output



1.7 Analysis Data Dictionary

Variable Name	Data Type / Object	Class	Description
pressed_keys	set<int>	GameCore	Set containing the pygame key codes of all currently pressed keyboard keys.
pressed_ - buttons	set<UIElement>	GameCore	Set containing all pygame_gui UI button elements that have been clicked in the current frame.
tick_speedup	int	GameCore	Speedup multiplier for simulation speed set by the user, for faster training AI model training.
unexpected_- error_msg	string	GameCore	Error message string displayed to the user when an unexpected error occurs.
throttle_and_- braking_meter	UIProgressBar	PlayerCar	Progress bar displaying the current throttle (50-100) or braking (0-50) input value to the player.
steering_slider	UIHorizontalSlide r	PlayerCar	Horizontal slider allowing the player to control car steering with the mouse.
is_slider_- enabled	boolean	PlayerCar	A flag indicating whether the steering slider has been enabled. If not, then keyboard controls (‘A’ to turn right and ‘D’ to turn left) is enabled.
speedometer	UIGaugeMeter	PlayerCar	A circular gauge meter displaying the current speed of the player's car with a rotating needle from 0 to 300.
canvas	UITrackCanvas	TrackMakerUI	Custom UI canvas that allows user to create and edits track shapes by placing anchor points and adjusting curves and widths.
anchor_toggle _button	UIButton	TrackMakerUI	Button which switches the canvas to anchor mode for editing track shape.
width_toggle_- button	UIButton	TrackMakerUI	Button which switches the canvas to width mode for

			adjusting track width in sections.
width_slider	UIHorizontalSlider	TrackMakerUI	Slider that appears when a width point is selected, allowing the user to adjust track width (60 to 100 pixels).
mode	string	UITrackCanvas	Current track maker mode set by user. "anchor" mode for shape editing, and "width" for width editing.
is_handles_-enabled	boolean	UITrackCanvas	Flag indicating whether Bezier control handles have been enabled.
validity_issues	string	UITrackCanvas	Validation error message shown to user when attempting to save an invalid track.
reset_button	UIButton	CarSimulation	Button which resets the racing simulation to initial state with cars at start line and timer at zero.
track_loader	UIFileBrowser	CarSimulation	File selector dialog prompting the user to choose which track load in for the racing simulation.
num_of_laps-setter	UIOptionSelector	CarSimulation	Dropdown selector allowing the user to choose the number of laps (1-10) to race for.
end_screen	UIEndScreen	CarSimulation	End screen dialog displaying race results with lap times and options to race again or return to menu.
controls_toggle_button	UIButton	SoloCarSim / RacingAI	Button which toggles the is_steering_enabled flag
model_loader	UIFileBrowser	RacingAI / AITraining - Environment	File selector dialog for the user to choose which trained AI model file to load from .npy file.
episodic_log_-box	UITextBox	AITraining-Environment	Text box displaying episodic logs. Logs episode number, duration, average progress, and average reward.
learning_log_-box	UITextBox	AITraining-Environment	Text box displaying overall learning statistics. Includes

			average and best progress, average and best lap times, minimum progress and lap time thresholds, model readiness, and rollback count.
model_training_box	UITextBox	AITraining-Environment	Text box displaying current training metrics including average return, reward, exploration factor and value estimates, and critic loss (for MCAC model).
model_type_-selector	UIOptionSelector	AITraining-Environment	Dropdown dialog where the user selects type of RL model to train (REINFORCE or MCAC).
hyper_param_-setter	UIModelCreator	AITraining-Environment	Configuration dialog where the user sets neural network architecture and AI training hyperparameters.
model_saver	UIFileBrowser	AITraining-Environment	File selector dialog that allows user enters to save a trained model to a .npy file.
option_list	UIDropDown-Menu	UIOption-Selector	Dropdown menu UI element displaying all available options for the user to choose from.
directory	string	UIFileBrowser	Directory being accessed to save to or load from. Can be "Tracks" or "Models".
mode	string	UIFileBrowser	The file operation mode selected by the user (“save” or “load”).
file_list	UISelectionList	UIFileBrowser	Scrollable list displaying all relevant files in the directory for the user to select from.
text_entry	UITextEntryLine	UIFileBrowser	Text input field for the user to enter a filename for saving or loading.
error_label	UILabel	UIFileBrowser / TrackMakerUI / MainMenu / UIModel - Creator	Text label displaying error messages when the user performs invalid actions.
confirm_-overwrite	boolean	UIFileBrowser	Flag tracking whether the user has been warned about overwriting an existing file and must click OK again.

num_of_cars_-input	UITextEntryLine	UIModel - Creator	Text field to set number of AI cars to train simultaneously (batch size). Input must be a positive integer between 1 and 20 (inclusive).
gamma_input	UITextEntryLine	UIModel - Creator	Text field to set the discount factor (gamma) for future rewards when computing returns. Input must be a decimal value between 0 and 1 (inclusive).
entropy_input	UITextEntryLine	UIModel - Creator	Text field to set the entropy bonus coefficient for exploration during training. Input must be a decimal value between 0 and 1 (inclusive).
l2_lambda_-input	UITextEntryLine	UIModel - Creator	Text field to set the L2 regularization coefficient for preventing overfitting. Input must be a decimal value between 0 and 1 (inclusive)
actor_init_log_-std_input	UITextEntryLine	UIModel - Creator	Text field to set the initial log standard deviation for the actor network's action distribution. Input must be a decimal value between -5 and 5 (inclusive)
actor_learning_rate_input	UITextEntryLine	UIModel - Creator	Text field to set the learning rate (alpha) for the actor network's Adam optimizer. Input must be a decimal value between 0 and 1 (inclusive)
actor_adam_-beta1_input	UITextEntryLine	UIModel - Creator	Text field to set the beta1 parameter for the actor's Adam optimizer momentum. Input must be a decimal value between 0 and 1 (inclusive)
actor_adam_-beta2_input	UITextEntryLine	UIModel - Creator	Text field to set the beta2 parameter for the actor's Adam optimizer variance. Input must be a decimal value between 0 and 1 (inclusive)
actor_layers_-input	UITextEntryLine	UIModel - Creator	Text field to set the actor neural network hidden layer sizes. This is set by

			entering comma-separated layer sizes (e.g., "32,64,64,32"). Input that does not follow this format will return an error.
actor_-activation_-dropdown	UIDropDownMenu	UIModel - Creator	Dropdown menu to select the activation function that will apply to all actor hidden layers (elu, ReLu, tanh, linear). There is also a 'Custom' option that allows the user to set different activations for each layer.
actor_-activation_-custom_input	UITextEntryLine	UIModel - Creator	Text field for the user to set custom activation functions for each actor network layer when "Custom" is selected. This is set by entering comma-separated layer activations (e.g., "ReLu,Tanh,Tanh,elu"). The format must match the format and number of layers set by layer sizes.
critic_init_-log_std_input	UITextEntryLine	UIModel - Creator	Text field to set the initial log standard deviation for the critic network. Input must be a decimal value between -5 and 5 (inclusive) (MCAC only).
critic_learning_rate_input	UITextEntryLine	UIModel - Creator	Text field to set the learning rate for the critic network's Adam optimizer Input must be a decimal value between 0 and 1 (inclusive) (MCAC only).
critic_adam_-beta1_input	UITextEntryLine	UIModel - Creator	Text field to set the beta1 parameter for the critic's Adam optimizer. Input must be a decimal value between 0 and 1 (inclusive) (MCAC only).
critic_adam_-beta2_input	UITextEntryLine	UIModel - Creator	Text field to set the beta2 parameter for the critic's Adam optimizer. Input must be a decimal value between 0 and 1 (inclusive) (MCAC only).
critic_layers_-input	UITextEntryLine	UIModel - Creator	Text field to set the critic neural network hidden layer

			sizes. This is set by entering comma-separated layer sizes (e.g., "32,64,64,32"). Input that does not follow this format will return an error. (MCAC only)
critic_-activation_-dropdown	UIDropDown-Menu	UIModel - Creator	Dropdown menu to select the activation function that will apply to all critic hidden layers (elu, ReLu, tanh, linear). There is also a 'Custom' option that allows the user to set different activations for each layer. (MCAC only)
critic_-activation_-custom_input	UITextEntryLine	UIModel - Creator	Text field for the user to set custom activation functions for each critic network layer when "Custom" is selected. This is set by entering comma-separated layer activations (e.g., "ReLu,Tanh,Tanh,elu"). The format must match the format and number of layers set by layer sizes. (MCAC only)

2 Design

2.1 Design Introduction

This project allows the user to make their own tracks, train AI models on their custom tracks and race against AI models on custom tracks. The MainMenu class displays a starting menu screen that provides the user with 4 options that direct the user to different game modes: “Track Maker”, “Solo Racing Simulation”, “Race against AI”, and “AI Training Mode”. This will be the mode that the user returns to switch mode, or if any unexpected errors occur. I will have a GameCore class that globally handles rendering and event processing for all game modes, which enables modularity. The main game loop calls these methods of the GameCore class.

Track Maker:

The TrackMakerUI class creates a game mode that provides the user an interface to make custom tracks. The TrackCanvas class handles user input and allows the user to create track shapes by creating and moving control points. The game mode has additional options that allow the user to adapt properties of the track such as the curvature and the width. The TrackMakerUI also allows the user to save and load tracks with a unique name. Only tracks that fulfil a set of validity criteria are allowed to be saved.

Car simulation environments:

SoloCarSim, RacingSim and AITrainingEnvironment all start by loading in a user defined track and initialising a set number of laps. These game modes use an instance of a RaceTimeManager class that records lap times, and checks if the total number of laps has been completed. SoloCarSim only creates one instance of the PlayerCar class and the RacingSim creates one instance of the PlayerCar and one of the AICar, which allows the player to race an AI model.

The AITrainingEnvironment class handles training AI models using Reinforcement Learning (RL) methods. First, the user is provided an interface to initialise all the hyperparameters and environment settings such as the number of cars, learning rate, type of RL algorithm, neural network layer settings and so on. A certain number of instances of the AICar are created and run simultaneously by inputting sensor data to the model and taking actions based on the model output. AITrainingEnvironment also handles model returns computation, parameter updates and resetting cars and RaceTimeManager at the end of each episode of training. The game mode also displays training statistics for the user to moderate training progress.

The PlayerCar and AICar classes will inherit from an abstract Car class which handles movement physics, collision detection and update logic. PlayerCar creates a control panel that enables the user to control the movement of the car. AICar has the ray_cast() method, which returns an array of ‘sensor values’ (distances to the nearest wall in different directions), which acts as part of the input to the AI model. The compute_reward() method of the class returns some numerical value that indicates how ‘good’ the most recent action taken was.

Track:

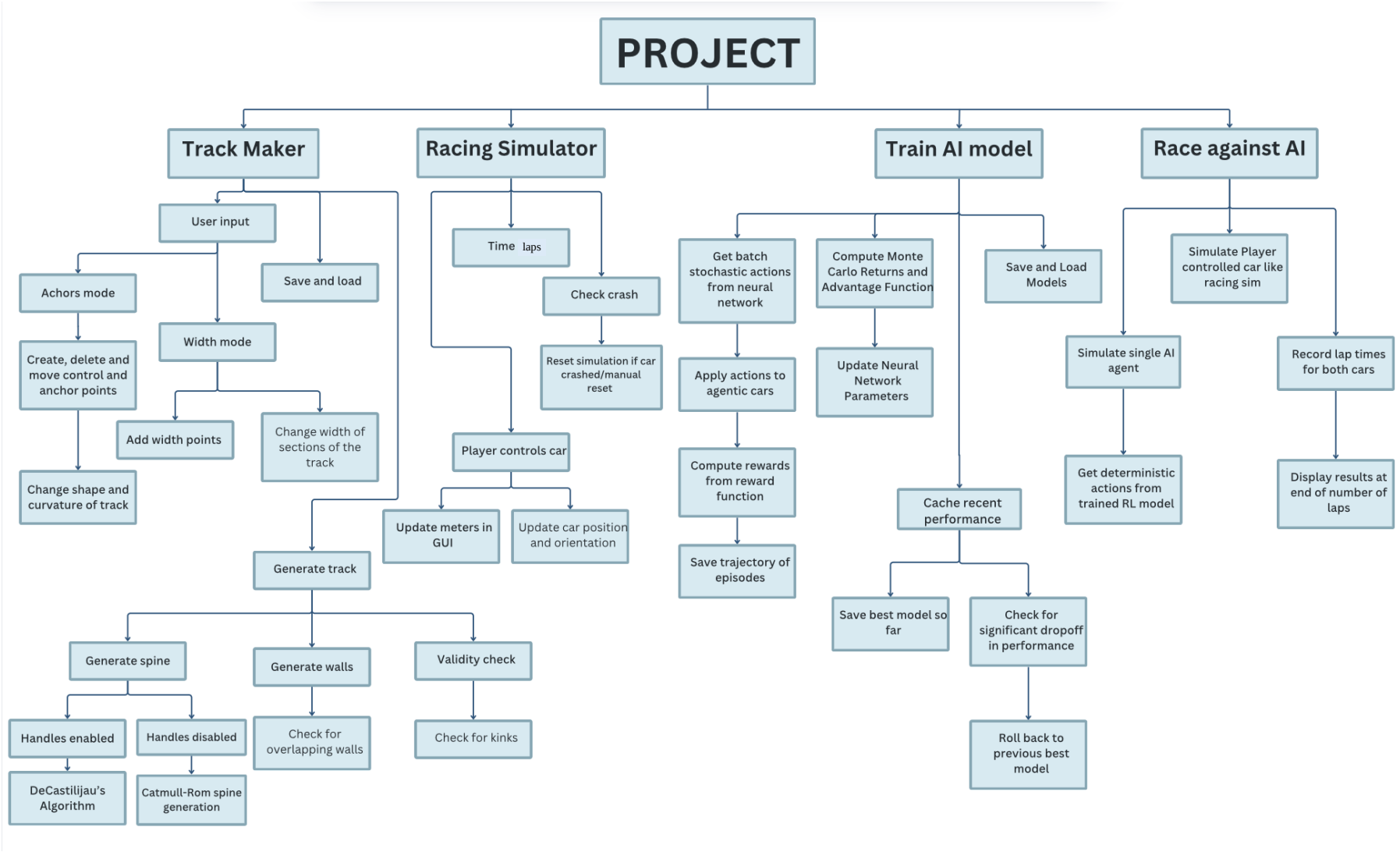
Track class creates a track object with walls being represented as a list of inner_wall_points and outer_wall_points. The track_spine attribute is used to monitor the progress of a car driving along the track. The class also implements a special hash grid data structure that enables for optimised collision detection. This prevents bottleneck when simulating many AICar objects that all need to ray cast and have their hitbox collision detection every frame.

Reinforcement Learning (RL) models:

There will be 2 different RL model classes: REINFORCEModel and MCACModel (Monte Carlo Actor Critic). Both classes implement an ‘actor’ neural network, that takes in an input vector and returns a probability distribution from which the actions are stochastically sampled (since these RL algorithms implement stochastic policy gradient). The compute_monte_carlo_returns() method computes the discounted returns from the recorded trajectory. The 2 model types of handles updating the parameters of their respective neural networks in by implementing their respective algorithms for computing an advantage and their respective update gradients.

To program neural networks from scratch, I will implement a NeuralNetwork class. This class would have instances of a Layer class as attributes. NeuralNetwork class handles iterating through the layers, passing data from one layer to the next in forward propagation, and passes calculated gradients backward through the layers in backpropagation.

2.2 Hierarchy chart:



2.3 Complex Code

2.3.1 Game core and main

To modularise rendering, user event handling and frame processing, I have used an instance of a custom “GameCore” class. This class contains all the global constants, conversion factors and components that are utilised by all the sub-program (game modes).

```
14  class GameCore:  1 usage  joxnrepresent +1 *
15      def __init__(self):  joxnrepresent +1 *
16          # Program constants
17          self.is_debugging = False
18          self.load_model = False
19          self.frame_rate = 60
20          self.tick_speedup = 1
21          self.screen_dimensions = (1200, 750)
22          self.screen_fill = "White"
23          self.current_track = "testing_track"
24          self.meter_pixel_conversion = 10
25          self.eps = 1e-9
26
27          # Core program components
28          self.main_screen = pygame.display.set_mode(self.screen_dimensions)
29          self.clock = pygame.time.Clock()
30          self.pressed_keys = set()
31          self.pressed_buttons = set()
32          self.game_mode = None
33
34          self.gui_manager = None
35          self.game_sprites = pygame.sprite.Group()
36          self.debug_elements = defaultdict(list)
```

- User events are recorded using the self.pressed_keys() and self.pressed_buttons() hash sets.
- self.game_mode: is set based on the options chosen by the user.
- self.gui_manager: handles all pygame_gui components (rendering and event handling)
- self.game_sprites: contains all non-gui game elemtns to be rendered
- self.debug_elements: allows for drawing various graphics (only for development purposes

Changing game modes:

```
38      def set_game_mode(self, new_mode):  2 usages (2 dynamic)  joxnrepresent
39          self.game_sprites.empty()
40          self.debug_elements.clear()
41          self.gui_manager.clear_and_reset()
42
43          self.game_mode = new_mode()
44
```

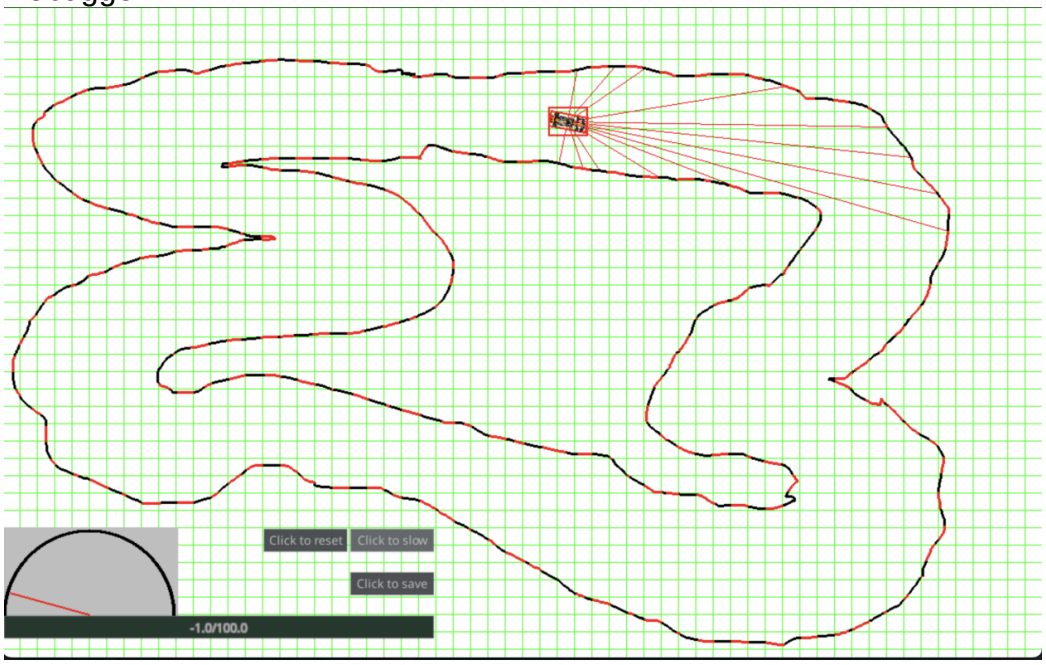
- All rendering groups are cleared, then self.game_mode is updated

Rendering:

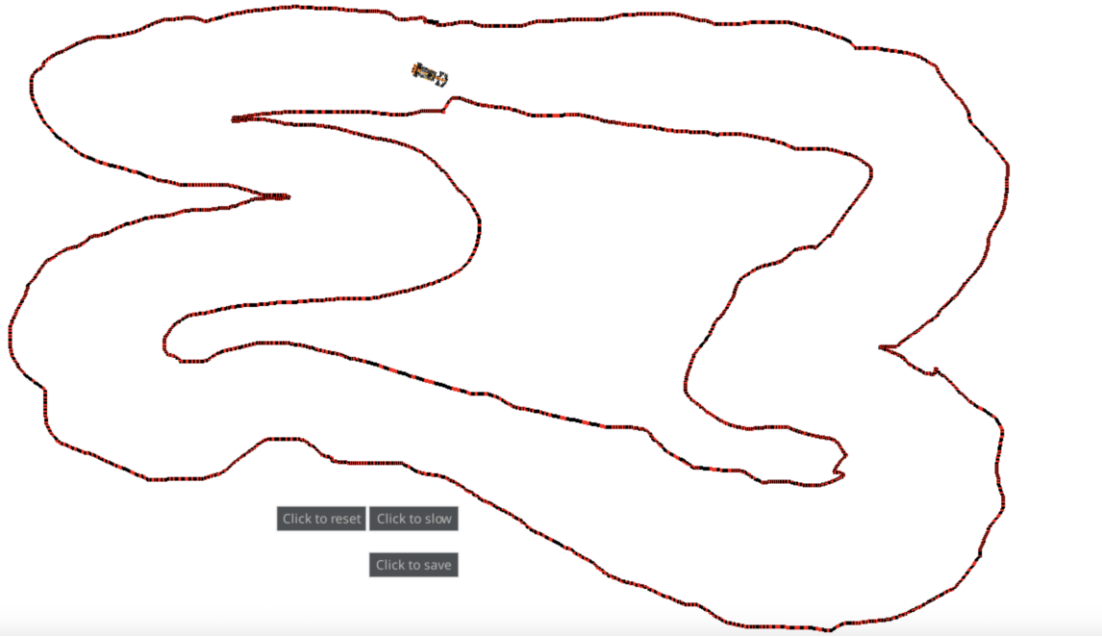
```
45      def render(self):  1 usage (1 dynamic)  ± joxnrepresent *
46          self.main_screen.fill(self.screen_fill)
47
48          # Debug graphics
49          if self.is_debugging:
50              if self.debug_elements:
51                  for element_type, elements in self.debug_elements.items():
52                      for element in elements:
53                          if element_type == "hitboxes":
54                              pygame.draw.polygon(self.main_screen, color: "red", element, width: 2)
55                          elif element_type == "AABB":
56                              pygame.draw.rect(self.main_screen, color: "red", element.rect, width: 2)
57                          elif element_type == "rays":
58                              try:
59                                  for ray in element:
60                                      pygame.draw.line(self.main_screen, color: "red", ray[0], ray[1])
61                              except:
62                                  print(element)
63                          elif element_type == "track spine":
64                              for i in range(len(element) - 1):
65                                  draw_line(self.main_screen, colour: "Blue", element[i], element[i + 1])
66                          if element_type == "grid lines":
67                              if element:
68                                  draw_grid(self.main_screen)
69
70          # Main game rendering
71          self.game_sprites.draw(self.main_screen)
72          self.gui_manager.draw_ui(self.main_screen)
73          pygame.display.flip()
74          self.clock.tick(game_core.frame_rate)
```

- Redering debugger: the debugger is a dictionary containing various elements and their type. Each element is rendering differently depending on what it is.
- Game_spites and gui elements are drawn onto the screen
- Game clock is updated

Debugger:



No debugger:



Event processing:

```
76      def cache_events(self, event):  1 usage (1 dynamic)  joxnrepresent
77          self.gui_manager.process_events(event)
78          if event.type == pygame.QUIT:
79              exit()
80          if event.type == pygame.KEYDOWN:
81              self.pressed_keys.add(event.key)
82          if event.type == pygame.KEYUP:
83              self.pressed_keys.discard(event.key)
84          if event.type == pygame.USEREVENT and  event.user_type == pygame_gui.UI_BUTTON_PRESSED:
85              self.pressed_buttons.add(event.ui_element)
86
87      def process_frame(self):  1 usage (1 dynamic)  joxnrepresent
88          if self.game_mode:
89              self.game_mode.event_handle()
90              self.game_mode.update()
```

- User events are hashed/removed from respectve hash sets

Having the centralised game core allows for game modes to be easily updated without having to reprogram rendering and event handling logic.

Time and frame updates:

The game core includes a clock object that ensures frames are limited to a maximum frame rate. Every time a frame is updated, the clock is ticked and the display is flipped.

Since my project will include an AI training environment, I want to implement the functionality of “speeding up” the simulation to fastrack the training process. The way I decided to do this was to process the physics and logic of multiple frames in a batch and then render a frame once.

```python
# pygame and GUI initialisation
pygame.init()
game_core.gui_manager = UIManager(game_core.screen_dimensions)
game_core.set_game_mode(MainMenu)

def game_loop():  1 usage  joxnrepresent +2 *
    """
    Main game loop that runs the program. Repeats indefinitely until the user closes the program.
    """
    while True:
        # Record user events into respective hash sets.
        for event in pygame.event.get():
            game_core.cache_events(event)

        # Update clock: dt_ms is the time between clock updates
        game_core.dt_ms = game_core.clock.tick(game_core.frame_rate)

        # dt_per_step is time per frame processing call -> handles speedups
        dt_per_step = game_core.dt_ms / game_core.tick_speedup

        for _ in range(game_core.tick_speedup):
            """
            When tick speedup is used, many frames of the program are processed and are only rendered once. This helps
            reduce bottleneck caused by rendering.
            """
            game_core.process_frame()
            if not game_core.is_paused:
                game_core.sim_time_ms += dt_per_step

        game_core.render()

game_loop()
```

- dt_ms is the time between frame rendering.
- dt_per_time_step is time in between frame processing
- sim_time_ms is the total time that has been simulated (used for lap timing)

2.3.1 Game Modes

Since my project involves several different functionalities, I decided to modularise each different subprogram independently. I did this by create an abstract GameMode class, and every subprogram is programmed as a child class of this. The GameMode class includes the methods to handle user events and frame updates. These methods are overridden in the respective child classes so that each game mode can have its unique function. Having the GameMode class be the parent class of all the game modes allows for the GameCore to have a universal set of event processing, rendering and update methods by calling the methods of the GameMode class.

```
22  class GameMode(ABC):  2 usages  joxnrepresent
23      def __init__(self):  joxnrepresent
24          self.game_sprites = game_core.game_sprites
25          self.debug_elements = game_core.debug_elements
26          self.gui_manager = game_core.gui_manager
27
28      @abstractmethod  7 usages (6 dynamic)  joxnrepresent
29      def update(self, *args):
30          self.game_sprites.update(*args)
31          self.gui_manager.update(1/game_core.frame_rate)
32          pass
33
34      @abstractmethod  1 usage (1 dynamic)  joxnrepresent
35      def event_handle(self):
36          pass
```

These methods are overridden by the different game mode subplacces (MainMenu, TrackMaker, SoloCarSim, RacingAI and AITrainingEnvironment).

Main Menu methods:

```
def event_handle(self):  1 usage (1 dynamic)  joxnrepresent
    """Handle mode selection."""

    if self.track_maker_button in game_core.pressed_buttons:
        game_core.set_game_mode(TrackMakerUI)

    elif self.solo_sim_button in game_core.pressed_buttons:
        game_core.set_game_mode(SoloCarSim)

    elif self.racing_sim_button in game_core.pressed_buttons:
        game_core.set_game_mode(RacingAI)

    elif self.ai_sim_button in game_core.pressed_buttons:
        game_core.set_game_mode(AITrainingEnvironment)

    elif self.ack_error_button in game_core.pressed_buttons:
        self.hide_error()

    # This closes the program permanently
    elif self.exit_button in game_core.pressed_buttons:
        pygame.quit()
        exit()

    super().event_handle()

def update(self, *args):  joxnrepresent
    """Update GUI manager."""
    self.gui_manager.update(1 / game_core.FRAME_RATE)
```

Track Maker Methods:

```
def update(self):  joxnrepresent
    """Update GUI manager. Create width slider if in width mode,"""
    self.gui_manager.update(1 / game_core.FRAME_RATE)
    if self.canvas.mode == "width":
        self.changing_widths()
```

```
def event_handle(self):  1 usage (1 dynamic)  joxnrepresent *
    if self.anchor_toggle_button in game_core.pressed_buttons:
        self.toggle_anchor_mode()

    if self.width_toggle_button in game_core.pressed_buttons:
        self.toggle_width_mode()

    if self.save_button in game_core.pressed_buttons:
        if self.canvas.validity_issues is not None:
            self.display_validity_issues()
        else:
            self.save_track()

    if self.load_button in game_core.pressed_buttons:
        self.load_track()

    if self.clear_button in game_core.pressed_buttons:
        self.reset_canvas()

    if self.menu_button in game_core.pressed_buttons:
        game_core.set_game_mode(MainMenu)

    if self.toggle_handles_button in game_core.pressed_buttons:
        self.toggle_handles()

    if self.ack_error_button in game_core.pressed_buttons:
        self.ack_error_button.hide()
        self.error_label.hide()
    super().event_handle()
```

SoloCarSim methods:

```
def update(self, *args):  new *
    """Update player car. Reset car to start of recent lap on crash."""
    super().update()
    if game_core.is_paused or self.end_screen is not None:
        return
    if self.cars:
        self.cars[0].update()
    if self.is_crashed[0]:
        self.reset_cars()
        self.cars[0].progress = math.floor(self.cars[0].progress)

def event_handle(self):  1 usage (1 dynamic)  new *
    """Handle steering control toggle."""
    if self.controls_toggle_button in game_core.pressed_buttons:
        self.cars[0].toggle_slider()
        if self.cars[0].is_slider_enabled:
            self.controls_toggle_button.set_text("Disable steer slider")
        else:
            self.controls_toggle_button.set_text("Enable steer slider")

    super().event_handle()
```

RacingAI methods:

```
def update(self, *args):  new *
    super().update()
    if game_core.is_paused or self.end_screen is not None:
        return
    for i, car in enumerate(self.cars):
        if self.time_manager.is_all_laps_finished[i]:
            if game_core.sim_time_ms - self.time_manager.lap_times[i][-1] > 60 * 1000:
                results = self.get_results()
                self.display_end_screen(results)
            continue
        if self.is_crashed[i]:
            car.reset()
            self.is_crashed[i] = False
        else:
            if isinstance(car, AICar):
                sensors = car.get_sensors(self.track)
                state = sensors + [car.velocity.magnitude() / car.MAX_SPEED, car.steer / car.MAX_STEER_RAD]
                actions = self.rl_model.get_stochastic_actions(state)[0]
                car.update(actions[0])
            else:
                car.update()
```

```
def event_handle(self):  1 usage (1 dynamic)  new *
    if self.controls_toggle_button in game_core.pressed_buttons:
        self.cars[0].toggle_slider()
        if self.cars[0].is_slider_enabled:
            self.controls_toggle_button.set_text("Disable steer slider")
        else:
            self.controls_toggle_button.set_text("Enable steer slider")
    super().event_handle()
```

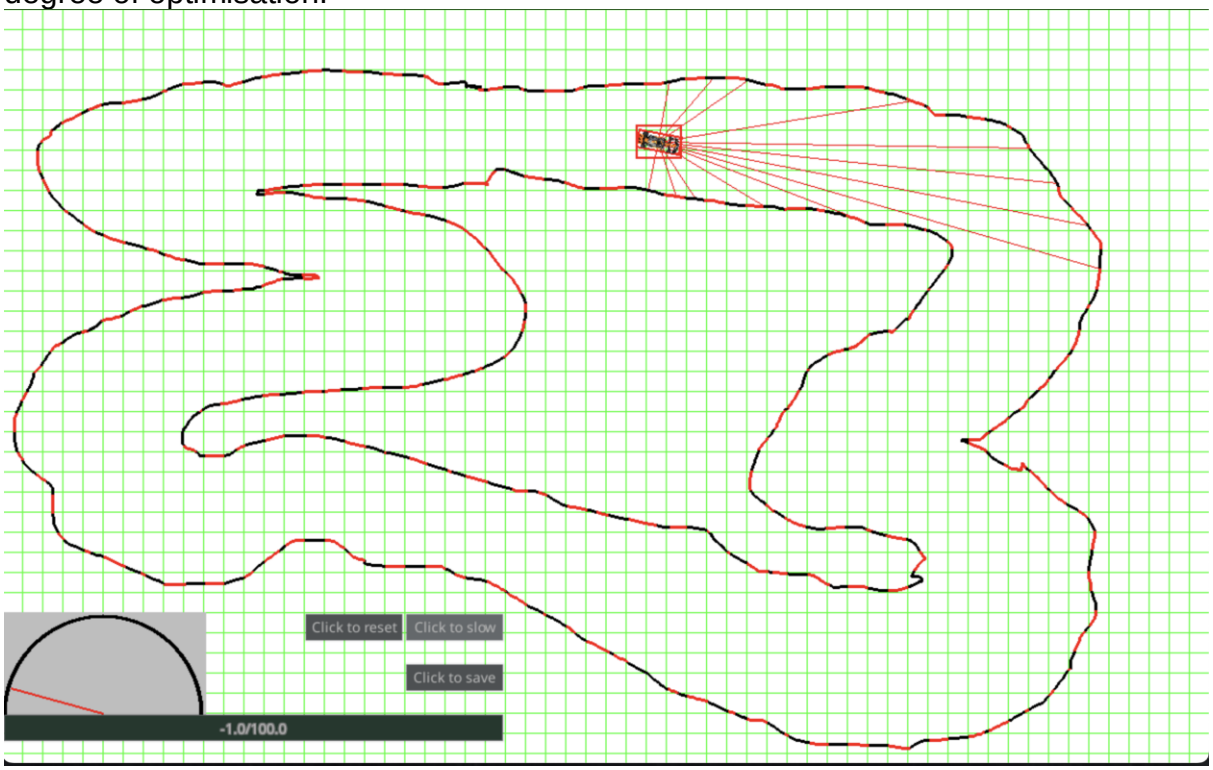
AI Training Environment methods:

```
def update(self, *args):  new *
    """Update AI simulation and handle episode completion."""
    super().update()
    if not game_core.is_paused and self.rl_model is not None:
        self.update_ai_step()

    if all(self.is_crashed):
        super().update()
        self.reinitialise_simulation()
        return
```

```
def event_handle(self):  1 usage (1 dynamic)  new *
    """Handle user input events."""
    # Reset simulation
    super().event_handle()
    if self.reset_button in game_core.pressed_buttons:
        self.reinitialise_simulation()

    # Speed control
    if self.tick_speedup_button in game_core.pressed_buttons:
        self.toggle_speedup()

    # Save model
    if self.save_ai_button in game_core.pressed_buttons:
        self.initialise_model_saver()
```

2.3.1 Spatial Hash Grid, Ray casting and Collision Detection:

To enable efficient ray casting and collision detection, I decided to store the track as a Spatial Hash Grid data structure. This works by diving the 2D world into equally sized square cells and storing the references of each wall segment in the cell(s) they pass through. With my setup, most walls are present in one cell and cells can contain several wall segments. This allows for local collision detection rather than interating through every wall segment every cycle, since only wall segments in the cells that the object is in are checked. This makes the algorithm highly scalable and allows for many car simulations to be run simultaneously with minimal load.
I have implemented the grid using a dictionary (hash map) with the keys being integer coordinates of the cell and the paired value being the list of wall segment indices in that cell. Hashing is more memory efficient that just a large array since only cells containing segments are stored in memory. Using the wall segment indices also makes it so that only one copy of the wall segments array is stored in memory. Using this data structure is suitable for my use case since the the ray casting algorithm I am using would be too computationally heavy for simultaneous simulation otherwise. Moreover, the track segments are static and therefore only need to be hashed once during creation, so reccursive methods like quadtree would actually lead to more load due to larger overheads. Spacial hash grid provides a suitable degree of optimisation.



Traversal Algorithm

To traverse the grid for hashing and querying, I have implemented the digital differential analyser (DDA) algorithm. The algorithm works by calculating the differences in x and y coordinates between endpoints and determines the number of steps needed to move from the start cell to the end cell. It then incrementally advances along the line by incrementing along both the x and y axes to traverse the line. Increments are made along the axis that has a closer perpendicular cell boundary in the direction of the traversal from the current position. This method steps through every intersected grid cell in order exactly once, therefore allowing to accurately hash all relavent cells.

Pseudocode:

```
dda_grid_travese(start, end):
    start_cell ← cell indices of start
    end_cell ← cell indices of end

    Δt_x ← change in x each stepped cell
    Δt_y ← change in y each stepped cell
    t_max_x ← distance to next x-boundary
    t_max_y ← distance to next y-boundary

    while start_cell is not end_cell:
        if t_max_x < t_max_y:
            step along x axis by 1 unit
            t_max_x ← t_max_x + Δt_x
        else:
            step along y axis by 1 unit
            t_max_y ← t_max_y + Δt_y
        end if
    end while
```

on_visit lamda function:

Python lamda functions are small anonymous functions that can take any number of arguments, but can only have one expression. This is useful when an anonymous function is to be called inside another function. In my case, I use the on_visit lamda function to determine what the dda_grid_traverse() function returns. This allows me to use the traversal algorithm to perform the tasks of hasing wall segments of the track to cells in the spacial hash grid, as well as for collision detection with hitboxes and ray casting.

```
    # Set maximum number of steps
    max_steps = (abs(end_cell_x - start_cell_x) + abs(end_cell_y - start_cell_y) + 10)
    for i in range(max_steps):

        # Call on visit function at each step
        result = on_visit(start_cell_x, start_cell_y)

        if result is not None:
            # Return if the required value has been found (Like collision point).
            return result

        if start_cell_x == end_cell_x and start_cell_y == end_cell_y:
            # Exit if end of segment is reached
            break

        # Step in the x or y direction depending upon distance to boundary
        if t_max_x < t_max_y:
            start_cell_x += cell_step_x
            t_max_x += t_delta_x
        else:
            start_cell_y += cell_step_y
            t_max_y += t_delta_y
```

When the dda_gird_traverse() method is implemented a specific task, it is also parsed the on visit lambda function which defines the action taken at each step.

```
def hash_segment(self, wall_segment, segment_index):  2 usages  joanmk
    self.dda_grid_traverse(wall_segment, on_visit=lambda ix, iy: self._add_segment_to_cell(ix, iy, segment_index))

def return_collision_point(self, ray, wall_segments):  5 usages  new *
    return self.dda_grid_traverse(ray, on_visit=lambda ix, iy:
                                              self._check_cell_for_collision(ix, iy, ray, wall_segments))
```

Wall sement hashing:

To hash a wall segment to the grid, dda_grid_traverse is used to identify the cells that the posses through. The add_segment_to_cell() method then hashes the wall segment index to the list of values stored by the cell.

```
def _add_segment_to_cell(self, ix, iy, segment_index):  1 usage  joanmk
    self.cells[(ix, iy)].append(segment_index)
```

This function returns None and so dosent break the dda_grid_traverse loop. So it hashes the segment to every cell that it passes through.

Testing Hashing

Code:

```
grid = SpatialHashGrid()

walls = [
    ((0, 0), (200, 0)),
    ((0, 0), (0, 200)),
    ((0, 0), (200, 200)),
    ((230, 20), (120, 175)),
]

print("Testing segment hashing")
print()
for i, seg in enumerate(walls):
    print(f"Segment {i + 1}:")
    grid.hash_segment(seg, i)
    print()
```

Output:

```
pygame-ce 2.5.5 (SDL 2.32.6, Python 3.12.4)
Testing segment hashing

Segment 1:
cell x: 0, cell y: 0
cell x: 1, cell y: 0
cell x: 2, cell y: 0
cell x: 3, cell y: 0
cell x: 4, cell y: 0
cell x: 5, cell y: 0
cell x: 6, cell y: 0
cell x: 7, cell y: 0
cell x: 8, cell y: 0
cell x: 9, cell y: 0

Segment 2:
cell x: 0, cell y: 0
cell x: 0, cell y: 1
cell x: 0, cell y: 2
cell x: 0, cell y: 3
cell x: 0, cell y: 4
cell x: 0, cell y: 5
cell x: 0, cell y: 6
cell x: 0, cell y: 7
cell x: 0, cell y: 8
cell x: 0, cell y: 9

Segment 3:
cell x: 0, cell y: 0
cell x: 0, cell y: 1
cell x: 1, cell y: 1
```

```
cell x: 1, cell y: 2
cell x: 2, cell y: 2
cell x: 2, cell y: 3
cell x: 3, cell y: 3
cell x: 3, cell y: 4
cell x: 4, cell y: 4
cell x: 4, cell y: 5
cell x: 5, cell y: 5
cell x: 5, cell y: 6
cell x: 6, cell y: 6
cell x: 6, cell y: 7
cell x: 7, cell y: 7
cell x: 7, cell y: 8
cell x: 8, cell y: 8
cell x: 8, cell y: 9
cell x: 9, cell y: 9
cell x: 9, cell y: 10

Segment 4:
cell x: 11, cell y: 1
cell x: 10, cell y: 1
cell x: 10, cell y: 2
cell x: 10, cell y: 3
cell x: 9, cell y: 3
cell x: 9, cell y: 4
cell x: 8, cell y: 4
cell x: 8, cell y: 5
cell x: 7, cell y: 5
cell x: 7, cell y: 6
cell x: 7, cell y: 7
cell x: 6, cell y: 7
```

Segment 1 (horizobntal segment): Gradient = 0, therefore only moves along the x axis, i.e the cell x increases each step while the cell y remains constant.
Segment 2 (vertical segment): Gradient = ∞, therefore only moves along the y axis, i.e the cell y increases each step while the cell x remains constant.
Segment 3 (Diagonal): Gradient = 1, therefore for every 1 step in x direction, moves 1 step in y direction, i.e alternates between incrementing cell x and cell y.
Segment 4 (Diagonal): Gradient ≈ -1.41, therefore for every step in the negativge x direction, moves 1 or 2 steps in y direction, i.e irregular patttern of incrementing cell x and cell y.

Ray intersection:

Ray intersection is used for ray casting. This returns the first collision point owith a wall along the ray. The lambda function check_cell_for_collision() itterates through all wall segments hashed to that cell and checks for a hit point using a line segment intersection check. If a hitpoint is found, it is returned and breaks the dda traversal loop. Otherwise the loop continues.

```
def _check_cell_for_collision(self, ix, iy, ray, wall_segments):  1 usage  joanmk
    if (ix, iy) in self.cells:
        for seg_index in self.cells[(ix, iy)]:
            seg = wall_segments[seg_index]
            hit_point = r.get_line_segments_intersection(ray, seg)
            if hit_point:
                return hit_point
    return None
```

Testing Ray Intersection:

Code:

```
132  grid = SpatialHashGrid()
133
134  walls = [
135      ((0, 0), (200, 0)),
136      ((0, 0), (0, 200)),
137      ((0, 0), (200, 200)),
138      ((230, 20), (120, 175)),
139  ]
140
141  for i, seg in enumerate(walls):
142      grid.hash_segment(seg, i)
143
144  print()
145  print("Testing ray intersection")
146  print()
147
148  print("Test 1: dx > dy")
149  ray1 = ((100, 50), (250, 100))
150  hit1 = grid.return_collision_point(ray1, walls)
151  print("Collision point:", hit1)
152  print()
153
154  print("Test 2: dy > dx")
155  ray2 = ((100, 20), (130, 250))
156  hit2 = grid.return_collision_point(ray2, walls)
157  print("Collision point:", hit2)
158  print()
159
160  print("Test 3: no collision")
161  ray3 = ((250, 250), (400, 400))
162  hit3 = grid.return_collision_point(ray3, walls)
163  print(" Collision point:", hit3)
```

Output:

```
Testing ray intersection

Test 1: dx > dy
cell x: 5, cell y: 2
cell x: 6, cell y: 2
cell x: 6, cell y: 3
cell x: 7, cell y: 3
cell x: 8, cell y: 3
Collision point: (187.91304347826087, 79.30434782608695)

Test 2: dy > dx
cell x: 5, cell y: 1
cell x: 5, cell y: 2
cell x: 5, cell y: 3
cell x: 5, cell y: 4
Collision point: (112.0, 112.0)

Test 3: no collision
cell x: 12, cell y: 12
cell x: 12, cell y: 13
cell x: 13, cell y: 13
cell x: 13, cell y: 14
cell x: 14, cell y: 14
cell x: 14, cell y: 15
cell x: 15, cell y: 15
cell x: 15, cell y: 16
cell x: 16, cell y: 16
cell x: 16, cell y: 17
cell x: 17, cell y: 17
cell x: 17, cell y: 18
cell x: 18, cell y: 18
cell x: 18, cell y: 19
cell x: 19, cell y: 19
cell x: 19, cell y: 20
 Collision point: None

Process finished with exit code 0
```

The first ray has a shallow gradient and therefore moves furhter along the x direction than along the y direction.
The second ray has a steep gradient and therefore moves futher along the y direction than x direction
The thrid ray misses all walls and so moves all the way across the ray without hitting any segments and finally returns None.

All collision points retuned are geometrically accurate as demonstrated earlier while testing the parametric line segment intersection algorithm.

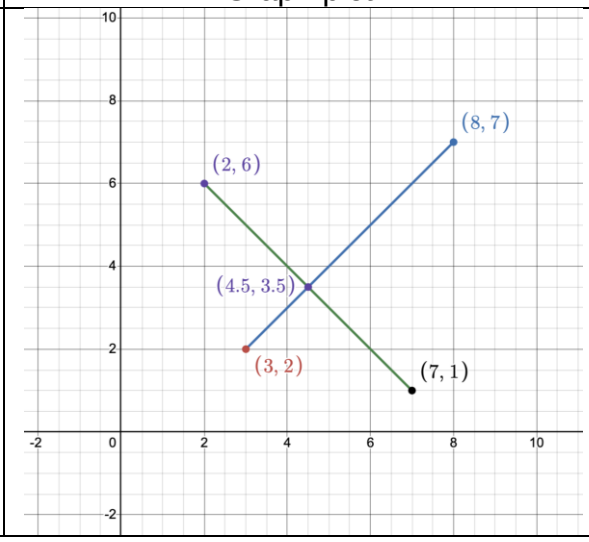
Line segment intersection check

To check whether 2 line segements are intersecting and determine their point of intersection, I have implemented a parametric line segmetic intersection algorithm.

```python
EPS = 1e-12 # Very small number to avoid division by zero
def get_line_segments_intersection(seg1, seg2):  1 usage  new *
    (x1, y1), (x2, y2) = seg1
    (x3, y3), (x4, y4) = seg2

    # Early reject intersection if AABB's do not intersect
    if (max(x1, x2) < min(x3, x4) or max(x3, x4) < min(x1, x2) or
            max(y1, y2) < min(y3, y4) or max(y3, y4) < min(y1, y2)):
        return None

    # Intersection parameters
    determinant = (x2 - x1) * (y4 - y3) - (y2 - y1) * (x4 - x3)
    alpha_numerator = (x3 - x1) * (y4 - y3) - (y3 - y1) * (x4 - x3)
    beta_numerator = (x3 - x1) * (y2 - y1) - (y3 - y1) * (x2 - x1)

    if abs(determinant) < EPS: # Parallel lines

        if abs(alpha_numerator) < EPS and abs(beta_numerator) < EPS: # Checking collinear

            # dot_prod checks for direction
            dot_prod = (x2 - x1) * (x4 - x3) + (y2 - y1) * (y4 - y3)

            # Overlapping regions
            overlap_start_x = max(min(x1, x2), min(x3, x4))
            overlap_end_x = min(max(x1, x2), max(x3, x4))
            overlap_start_y = max(min(y1, y2), min(y3, y4))
            overlap_end_y = min(max(y1, y2), max(y3, y4))

            # Checking if they overlap
            if overlap_start_x <= overlap_end_x and overlap_start_y <= overlap_end_y:
                # Case 1: seg1 first endpoint lies on seg2
                if min(x3, x4) <= x1 <= max(x3, x4) and min(y3, y4) <= y1 <= max(y3, y4):
                    return x1, y1

                p1 = (overlap_start_x, overlap_start_y)
                p2 = (overlap_end_x, overlap_end_y)
                if dot_prod > 0:
                    # Case 2: Return Overlap start point
                    return p1
                else:
                    # Case 3: Return Overlap end point
                    return p2
            # Case 4: Parallel and not collinear lines
            return None

        else: # Non-Parallel lines
            alpha = alpha_numerator / determinant
            beta = beta_numerator / determinant

            if alpha >= 0 and 0 <= beta <= 1:

                x = x1 + alpha * (x2 - x1)
                y = y1 + alpha * (y2 - y1)
                return x, y
            else:
                return None
```

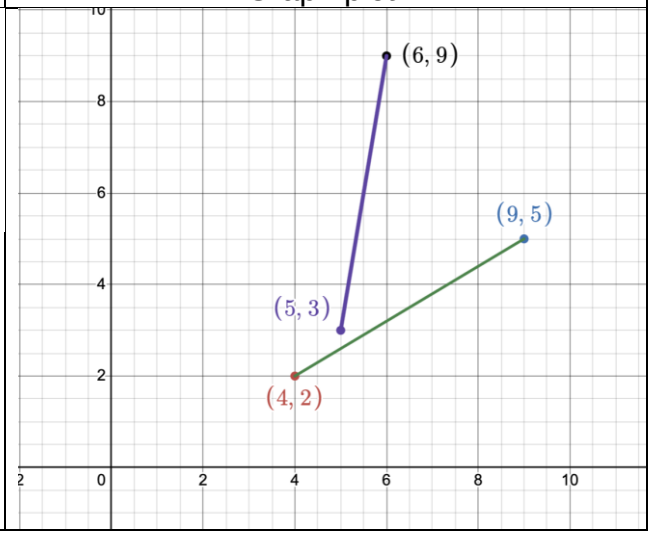
Test 1: Intersecting segments

```
seg1 = ((2,6), (7,1))
seg2 = ((3,2), (8,7))
intersection_point = get_line_segments_intersection(seg1, seg2)
print("Line segment 1" + str(seg1))
print("Line segment 2" + str(seg2))
print("Point of intersection" + str(intersection_point))
```

Code Output	Graph plot
Line segment 1((2, 6), (7, 1)) Line segment 2((3, 2), (8, 7)) Point of intersection(4.5, 3.5) Process finished with exit code 0	

Test 2: Non-Intersecting (Non parallel) segments

```
seg1 = ((5,3), (6,9))
seg2 = ((4,2), (9,5))
intersection_point = get_line_segments_intersection(seg1, seg2)
print("Line segment 1: " + str(seg1))
print("Line segment 2: " + str(seg2))
print("Point of intersection: " + str(intersection_point))
```

Code Output	Graph plot
Line segment 1: ((5, 3), (6, 9)) Line segment 2: ((4, 2), (9, 5)) Point of intersection: None Process finished with exit code 0	

2.3.2 Track Maker:

To allow the user to create their own custom track, I designed a track maker interface. The interface allows the user to customise the shape, width and other

features of the track. It also the user to save their track to a file and this saved track can be loaded to train an AI model to drive on it, or to race on it.

Track Canvas:

The track canvas allows the user to create a unique track shape using a curve tool. The user clicks on the canvas to create an anchor point and the tool connects these anchor points using a smooth curve. To achieve this, the track canvas records any mouse events on the screen and creates new points when the mouse button is pressed. Created points are stored in a list. The TrackCanvas class inherits the UIElement Class, which means its event handling and rendering is called my the pygame_gui manager.

Catmull-Rom spline:

A Catmull-Rom spline generates a smooth curve that passes through all the control points set by the user, without needing the user to specify any other information. The "control points" of the individual cubic bezier curve segments are inferred from the neighbouring control points. Since each bezier segment is only influenced by the 2 anchor points directly adjacent, moving a single point only affects nearby segments (i.e there is local control over the curve).

The splines are generated by Catmull-rom interpolation, which evaluates each bezier segment parametrically. Each bezier segment is computing using 4 consecutive anchor points: p0, p1, p2 and p3, where p1 and p2 are the end points of the segment and p0 and p3 are the adjacent segment anchor points. For the first segment generated, p0 = p1, and for the last segment generated, p3 = p2 (since there is no adjacent anchor point to use for calculation.

Each position along the curve is determined by the continuous parameter t, where 0 ≤ t ≤ 1. Each point is defined by:

$$\boldsymbol{p}(t) = \frac{1}{2}\begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix}\begin{bmatrix} -1 & 3 & -3 & 1 \\ 2 & -5 & 4 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix}\begin{bmatrix} \boldsymbol{p}_{k-1} \\ \boldsymbol{p}_k \\ \boldsymbol{p}_{k+1} \\ \boldsymbol{p}_{k+2} \end{bmatrix}$$

(In the given example, k = 1)

This same equation can be represented as a polynomial equation in terms of t:

$$C(t) = at^3 + bt^2 + ct + d$$

Where:

$$a = -p_0 + 3p_1 - 3p_2 + p_3$$

$$b = 2p_0 - 5p_1 + 4p_2 - p_3$$

$$c = -p_0 + p_2$$

$$d = 2p_1$$

So each segment of the Catmull-Rom spline can be rendered by stepping through the continuous parameter t at descrete intervals (which defines the resolution of the curve), and then calculating points on the segmnent using this parametric polynomial.

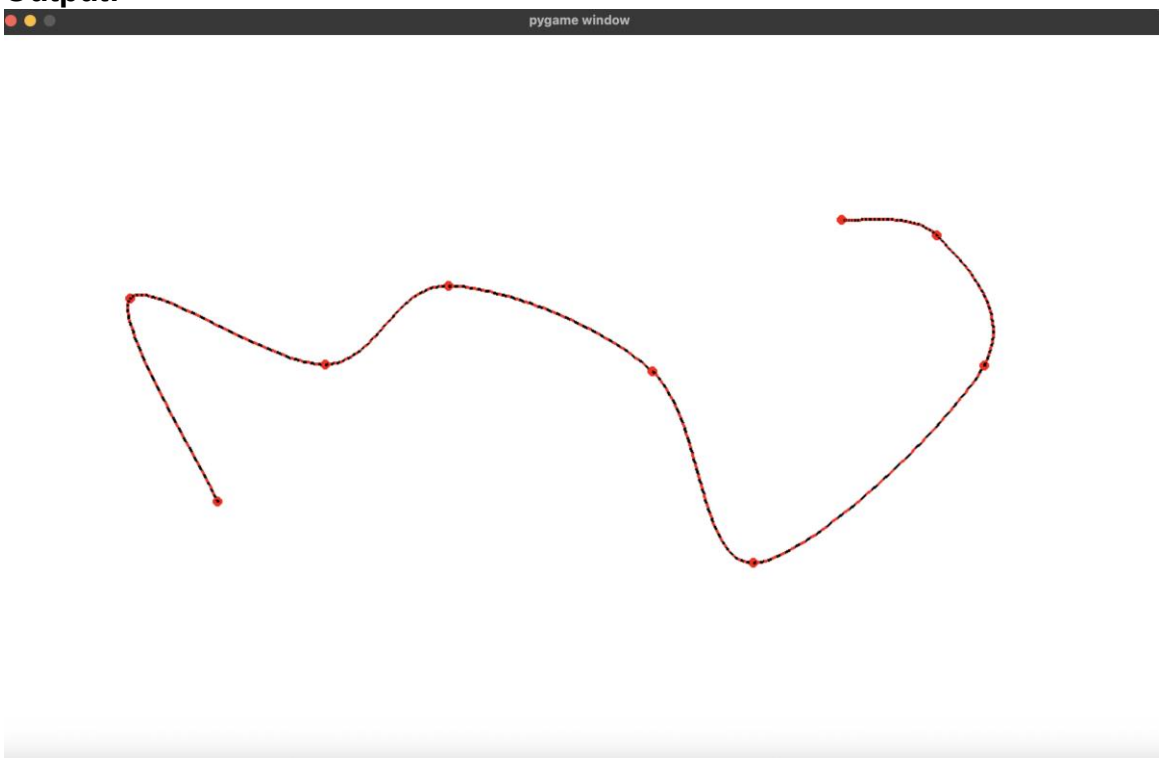
Catmull-Rom interpolation pseudocode:

```
FUNCTION CatmullRom(p0, p1, p2, p3, resolution):
    a ← -p0 + 3*p1 – 3*p2 + p3
    b ←  2*p0 – 5*p1 + 4*p2 - p3
    c ← -p0 + p2
    d ←  2*p1

    curvePoints ← empty list

    FOR t FROM 0 to 1:
        point ← 0.5 × (a·t³ + b·t² + c·t + d)
        ADD point TO curvePoints

    RETURN curvePoints
```

Code:

```
    def rebuild(self):  2 usages  joanmk *
        self.image.fill("White")
        for point in self.anchor_points:
            pygame.draw.circle(self.image, color= "Red", center = point, radius = self.point_size)
        if len(self.anchor_points) >= 2:
            for i in range(len(self.anchor_points)-1):
                p0 = self.anchor_points[i-1] if i>0 else self.anchor_points[0]
                p1 = self.anchor_points[i]
                p2 = self.anchor_points[i+1]
                p3 = self.anchor_points[i+2] if i+2 < len(self.anchor_points) else self.anchor_points[i+1]

                curve_segments = self.catmull_rom(p0, p1, p2, p3)
                r.draw_alternating_line_segments(self.image, curve_segments)

    def catmull_rom(self, p0, p1, p2, p3, resolution = 50):  1 usage  new *
        # Caching coefficients of cubic in terms of t (at^3 + bt^2 + ct + d)
        a = -p0 + 3*p1 - 3*p2 + p3
        b = 2*p0 - 5*p1 + 4*p2 - p3
        c = -p0 + p2
        d = 2*p1

        curve_segments = []
        for i in range(resolution + 1):
            t = i/resolution
            t2 = t*t
            t3 = t2 * t
            point = 0.5 * (a*t3 + b*t2 + c*t + d)
            if t != 0:
                curve_segments.append((cache_point, point))
            cache_point = point

        return curve_segments
```

Output:

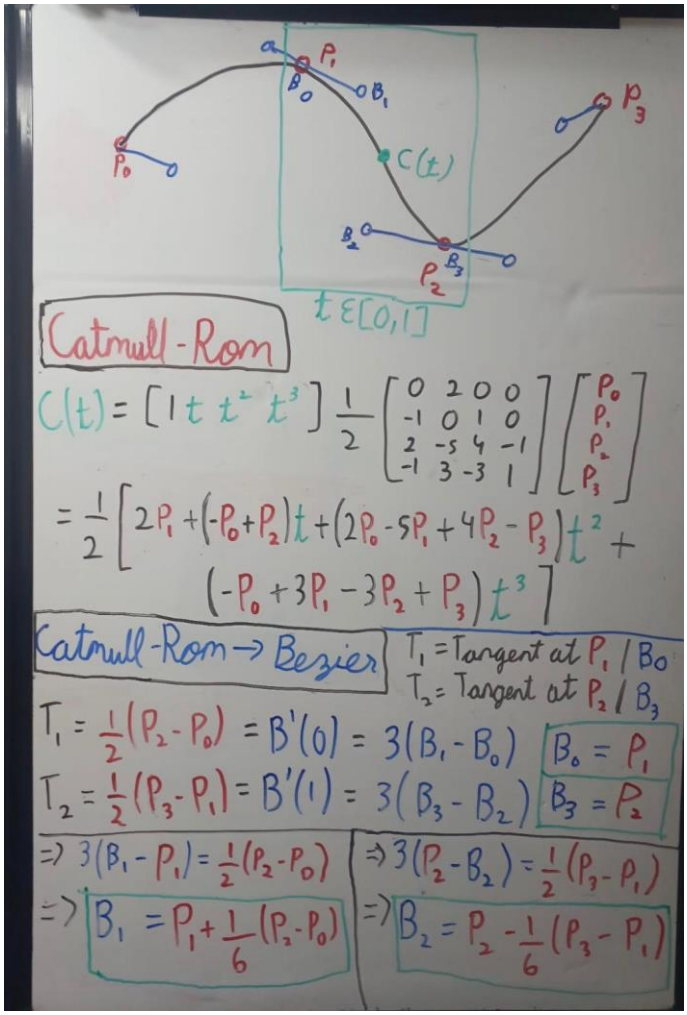


The advantage of using a Catmull-Rom spline is that moving any of the anchor points only significantly changes the segment is lies on and minorly changes the directly adjacent segments to allow for smoother curve. This allows for significant local control, which is suitable for a track creation interface (fifth point has been moved to extreme position, but only that local segment has been altered).
The anchor points are smoothly connected by a Catmull-Rom spline with no kinks. The red and black sections identify individual track segments (here it is 50 between any 2 points. This value can be altered).

Adding Control points:

The Catmull-Rom spline is a variation of the hermite spline that automatically computes the internal control points based on the positions of the neighbouring anchor points. However, since control points are implicit, they are not directly rendered and the user cannot control the exact curvature of each segment. Therefore, to enable this flexibility, I decided to explivitly generate the cubic bezier control points for each segment, which would allow the user to manualy move them to alter curvature. This is done by the Catmull-Rom to Bezier conversion, which uses the points p0 p1 p2 and p3 to compute the Bezier control points b0 b1 b2 and b3 for a segment. Since a bezier curve passes through the first and last point, and here p1 and p2 are the end points of the segment, b0 = p1 and b3 = p2. The other points can be calculated as:

$$
\begin{aligned}
B_0 &= P_1 \\
B_1 &= P_1 + \frac{1}{6}(P_2 - P_0) \\
B_2 &= P_2 - \frac{1}{6}(P_3 - P_1) \\
B_3 &= P_2
\end{aligned}
$$



Code:

```
class UITrackCanvas(UIElement):  2 usages  joanmk *
    def __init__(self, relative_rect, manager):  joanmk *
        super().__init__(relative_rect, manager, container=None, starting_height=0, layer_thickness=1)
        self.anchor_points = []
        self.control_points = []
        self.point_size = 5
        self.image = pygame.Surface((relative_rect.width, relative_rect.height))
        self.is_dragging = False
        self.is_click_buffer = False
        self.is_anchor_point_selected = False
        self.drag_point = None
        self.rebuild()

    def process_event(self, event):  joanmk *
        if event.type in (pygame.MOUSEBUTTONDOWN, pygame.MOUSEBUTTONUP, pygame.MOUSEMOTION):
            if self.rect.collidepoint(event.pos):
                relative_mouse_pos = Vector2(event.pos[0] - self.rect.x, event.pos[1] - self.rect.y)
                if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                    self.is_click_buffer = True
                    is_point_clicked = False
                    clicked_point = None
                    for i, point in enumerate(self.anchor_points):
                        if (relative_mouse_pos - point).length() <= self.point_size:
                            is_point_clicked = True
                            self.is_anchor_point_selected = True
                            clicked_point = (i, point)

                    for i, point in enumerate(self.control_points):
                        if (relative_mouse_pos - point).length() <= self.point_size:
                            is_point_clicked = True
                            self.is_anchor_point_selected = False
                            clicked_point = (i, point)

                    if is_point_clicked:
                        self.is_dragging = True
                        self.drag_point = clicked_point
                    else:
```

- Second dragging check to see is any control points are selected
- Is_anchor_point_selected flag to identify whether to index the control_points or anchor_points list.

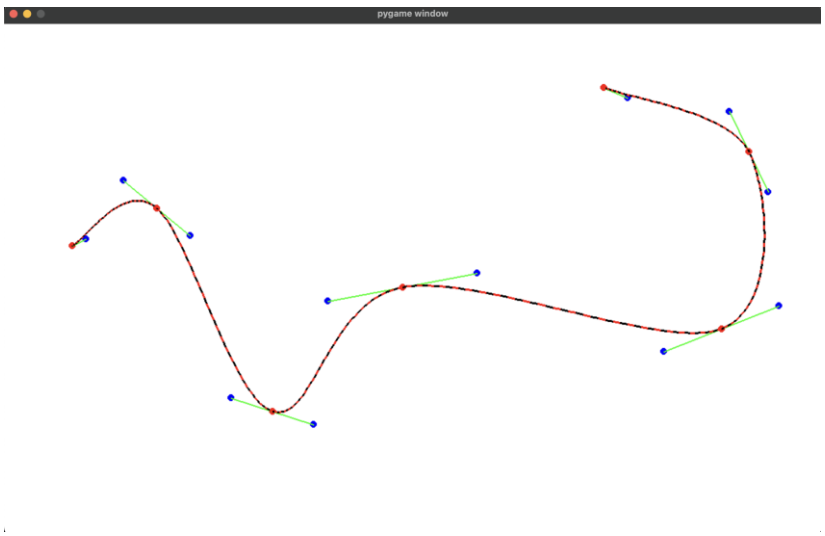
```
                        self.anchor_points.append(relative_mouse_pos)

                if event.type == pygame.MOUSEMOTION:
                    if self.is_dragging:
                        if self.is_anchor_point_selected:
                            self.anchor_points[self.drag_point[0]] = relative_mouse_pos
                        else:
                            self.control_points[self.drag_point[0]] = relative_mouse_pos

                if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
                    self.is_click_buffer = False
                    self.is_dragging = False
                    self.drag_point = None

            self.update_control_points()
            self.rebuild()

    def rebuild(self):  2 usages  joanmk *
        self.image.fill("White")
        for point in self.anchor_points:
            pygame.draw.circle(self.image, color= "Red", center = point, radius = self.point_size)
        for i, point in enumerate(self.control_points):
            pygame.draw.circle(self.image, color="Blue", center=point, radius=self.point_size)
            if i%2 == 0 and i != 0:
                pygame.draw.line(self.image,  color: "green", point, self.control_points[i-1],  width: 2)
            if i == 0:
                pygame.draw.line(self.image,  color: "green", point, self.anchor_points[0],  width: 2)
            if i == len(self.control_points) - 1:
                pygame.draw.line(self.image,  color: "green", point, self.anchor_points[-1],  width: 2)

        if len(self.anchor_points) > 1:
            for i in range(len(self.anchor_points)-1):
                p0, p1, p2, p3 = self.get_anchor_points(i)
                curve_segments = self.catmull_rom(p0, p1, p2, p3)
                r.draw_alternating_line_segments(self.image, curve_segments)
```

- Control points are rendered as Blue circles
- A green line is drawn to connect the 2 control points that are tangent to the spline at the same point. This creates a “handle”

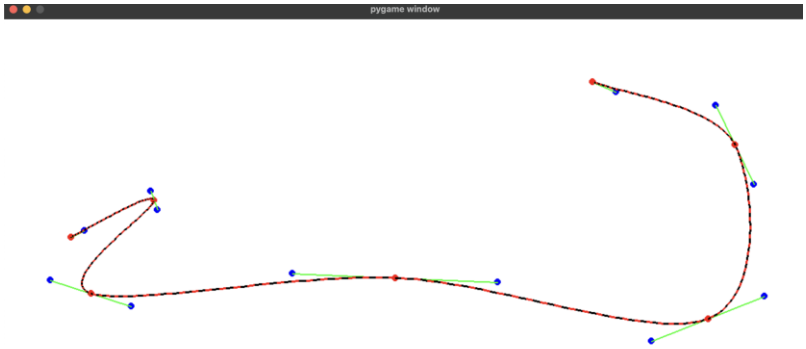
```
194     def get_anchor_points(self, i):  2 usages  new *
195         p0 = self.anchor_points[i - 1] if i > 0 else self.anchor_points[0]
196         p1 = self.anchor_points[i]
197         p2 = self.anchor_points[i + 1]
198         p3 = self.anchor_points[i + 2] if i + 2 < len(self.anchor_points) else self.anchor_points[i + 1]
199         return p0, p1, p2 ,p3
200
201     def catmull_rom(self, p0, p1, p2, p3, resolution = 50):  1 usage  new *
202         # Caching coefficients of cubic in terms of t (at^3 + bt^2 + ct + d)
203         a = -p0 + 3*p1 - 3*p2 + p3
204         b = 2*p0 - 5*p1 + 4*p2 - p3
205         c = -p0 + p2
206         d = 2*p1
207
208         curve_segments = []
209         for i in range(resolution + 1):
210             t = i/resolution
211             t2 = t*t
212             t3 = t2 * t
213             point = 0.5 * (a*t3 + b*t2 + c*t + d)
214             if t != 0:
215                 curve_segments.append((cache_point, point))
216             cache_point = point
217
218         return curve_segments
219
220     def update_control_points(self):  1 usage  new *
221         self.control_points.clear()
222         for i in range(0, len(self.anchor_points) - 1):
223             p0, p1, p2, p3 = self.get_anchor_points(i)
224             self.control_points.extend(self.get_bezier_points(p0, p1, p2, p3))
225
226     def get_bezier_points(self, p0, p1, p2, p3):  1 usage  new *
227         b1 = p1 + (p2 - p0) / 6
228         b2 = p2 - (p3 - p1) / 6
229         return (b1, b2)
```

- get_bezier_points() method performs Catmull-Rom to Bezier conversion.
- update_control_points() method recomputes all control points each cycle.

Output:



The control points generate handles that are tangent to the spline at the anchor points. The distance of the 2 control points on the same tangent from their anchor point is the same. This is the characteristic shape of a Catmull-Rom spline. Local control is also maintained as moving a single point does not change the shape of the full spline as observed below:



In a Catmull-Rom spline, the tangent control points are calculated based on the distance to the neighbouring points. The Moving the third anchor point closer to the first anchor point decreases the magnitude (dstance between the control points on at the same tangent) of the second control handle decreses. Since it is also further away from the fifth anchor point, the magnitude of the fourth control handle increases.

Conversely, moving the third anchor point closer to the fifth anchor point and furhter from the first anchor point decreases the magnitude of the third control handle and increases the magnirude of the second control handle.

Enabline control Handles:

Rendering: To allow these control points to actually modify the shape of the curve, The curve can no longer be rendered using Catmull-Rom interpolation since this simply redefines the control points based on the distances to adjacent anchor points. To overcome this, I decided to implement De Castolijau's algorithm for rendering the curve. De Castilijau's algorithm is a reccursive algorithm that uses successive linear interpolation to define a bezier curve with a set of control points. In this case, each segment between 2 anchor points will be a defined by a cubic bezier curve.

Pseudocode:

```
Function DeCastelijau(points, t)
    If number of points = 1
        Return points[0]
    End if
    new_points ← empty list
    For point in points
        x ← ((1 - t) * point x) + (t * next point x)
```

y ← ((1 - t) * point y) + (t * next point y)

ADD (x, y) TO new_points

End for

Return DeCastelijau(new_points, t)

End function

Code:

```
199     def get_bezier_points(self, p0, p1, p2, p3):  2 usages  joxnrepresent
200         b1 = p1 + (p2 - p0) / 6
201         b2 = p2 - (p3 - p1) / 6
202         return (b1, b2)
203
204     def evaluate_curve(self, p1, b1,b2,p2, resolution = 50):  1 usage  joxnrepresent
205         curve_segments = []
206         for i in range(resolution + 1):
207             t = i/resolution
208             points = [p1, b1, b2, p2]
209             curve_point = self.de_casteljau(points, t)
210             if t != 0:
211                 curve_segments.append((cache_point, curve_point))
212             cache_point = curve_point
213         return curve_segments
214
215     def de_casteljau(self, points, t):  2 usages  joxnrepresent
216         if len(points) == 1:
217             # Base case
218             return points[0]
219
220         # Recursive case
221         new_points = []
222         for i in range(len(points) - 1):
223             new_points.append((1 - t) * points[i] + t * points[i + 1])
224         return self.de_casteljau(new_points, t)
```

Curve Continuity:

One way of ensuring track shapes follow a certain degree of smoothness without sharp kinks is to ensure G1 continuity i.e tangent continuity. This ensures that all curves are connected, and their positions and directions match at the connection point. To implement this, whenever a control point is moved, the other controllment point tangent to the same anchor point must also be translated by the same amount in the opposite direction. This ensures that both points still lie on the same tangent line.

Code:

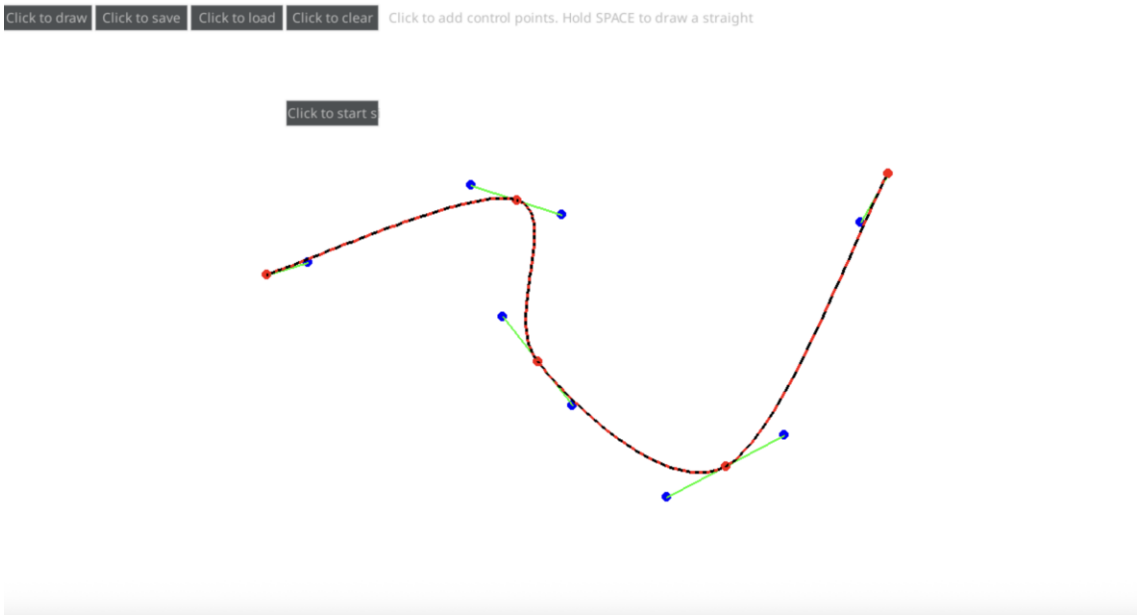
```
86  def process_event(self, event):  ± joxnrepresent +1 *
87      if event.type in (pygame.MOUSEBUTTONDOWN, pygame.MOUSEBUTTONUP, pygame.MOUSEMOTION):
88          if self.rect.collidepoint(event.pos):
89              relative_mouse_pos = Vector2(event.pos[0] - self.rect.x, event.pos[1] - self.rect.y)
90              if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
91                  self.is_click_buffer = True
92                  is_point_clicked = False
93                  clicked_point = None
94                  for i, point in enumerate(self.anchor_points):
95                      if (relative_mouse_pos - point).length() <= self.point_size:
96                          is_point_clicked = True
97                          self.is_anchor_point_selected = True
98                          clicked_point = (i, point)
99
100                 for i, point in enumerate(self.control_points):
101                     if (relative_mouse_pos - point).length() <= self.point_size:
102                         is_point_clicked = True
103                         self.is_anchor_point_selected = False
104                         clicked_point = (i, point)
105
106                 if is_point_clicked:
107                     self.is_dragging = True
108                     self.drag_point = clicked_point
109                 else:
110                     self.anchor_points.append(relative_mouse_pos)
111                     if len(self.anchor_points) > 1:
112                         p0 = self.anchor_points[-3] if len(self.anchor_points) >2 else self.anchor_points[-2]
113                         p1 = self.anchor_points[-2]
114                         p2 = self.anchor_points[-1]
115                         p3 = relative_mouse_pos
116                         self.control_points.extend(self.get_bezier_points(p0, p1, p2, p3))
117                         if len(self.anchor_points) > 2:
118                             anchor = self.anchor_points[-2]
119                             handle_length = (self.control_points[-2] - anchor).length()
120                             direction = (self.control_points[-2] - anchor).normalize()
121
122                             updated_handle_point = anchor - handle_length * direction
123                             self.control_points[-3] = updated_handle_point
```

- When new anchor point is created, the handle in previous segment is re - computed to ensure it remains tangent to the curve

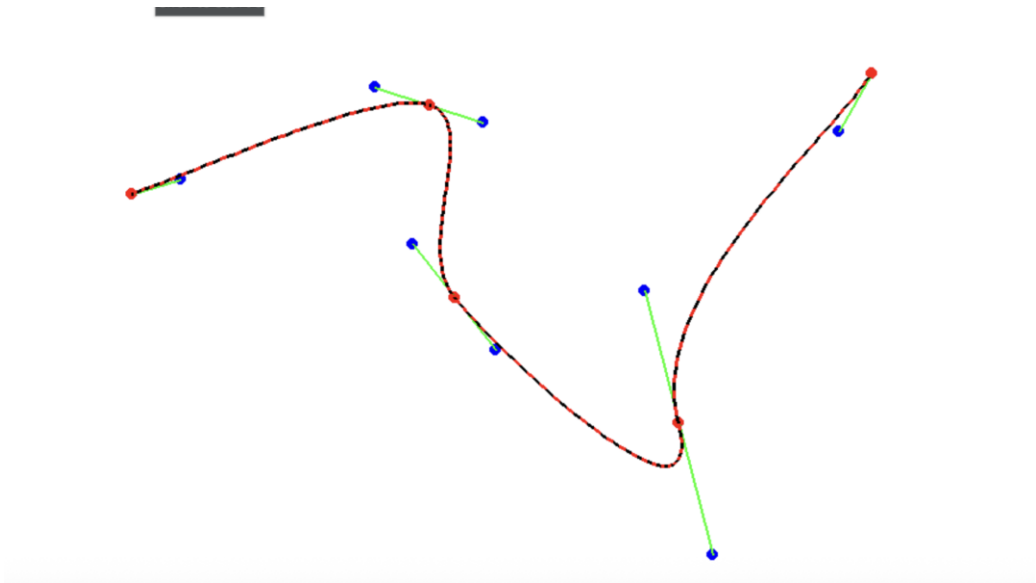
```
124
125             if event.type == pygame.MOUSEMOTION:
126                 if self.is_dragging:
127                     if self.is_anchor_point_selected:
128                         index = self.drag_point[0]
129                         translate_vector = relative_mouse_pos - self.anchor_points[index]
130                         self.anchor_points[index] = relative_mouse_pos
131                         if index*2 < len(self.control_points):
132                             self.control_points[index*2] += translate_vector
133                         if index != 0:
134                             self.control_points[index*2 - 1] += translate_vector
135
136                     else:
137                         translate_vector = relative_mouse_pos - self.control_points[self.drag_point[0]]
138                         self.control_points[self.drag_point[0]] = relative_mouse_pos
139                         mirror_point_index = None
140                         if (self.drag_point[0] % 2 == 0
141                                 and self.drag_point[0] != 0
142                                 and self.drag_point[0] != len(self.control_points) - 1):
143                             mirror_point_index = self.drag_point[0] - 1
144                         elif (self.drag_point[0] != 0
145                                 and self.drag_point[0] != len(self.control_points) - 1):
146                             mirror_point_index = self.drag_point[0] + 1
147                         if mirror_point_index:
148                             self.control_points[mirror_point_index] -= translate_vector
149                     self.mouse_pos = None
150                 else:
151                     self.mouse_pos = relative_mouse_pos
```

- Translation vector of the moved control point is calculated
- The correspoding handle point (mirror point) is moved by the inverse of the translation vector

Output:

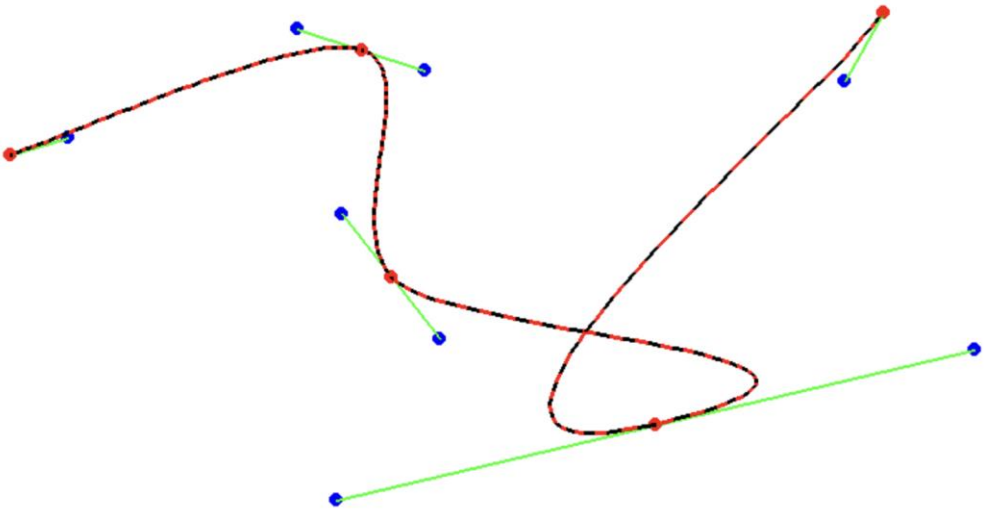


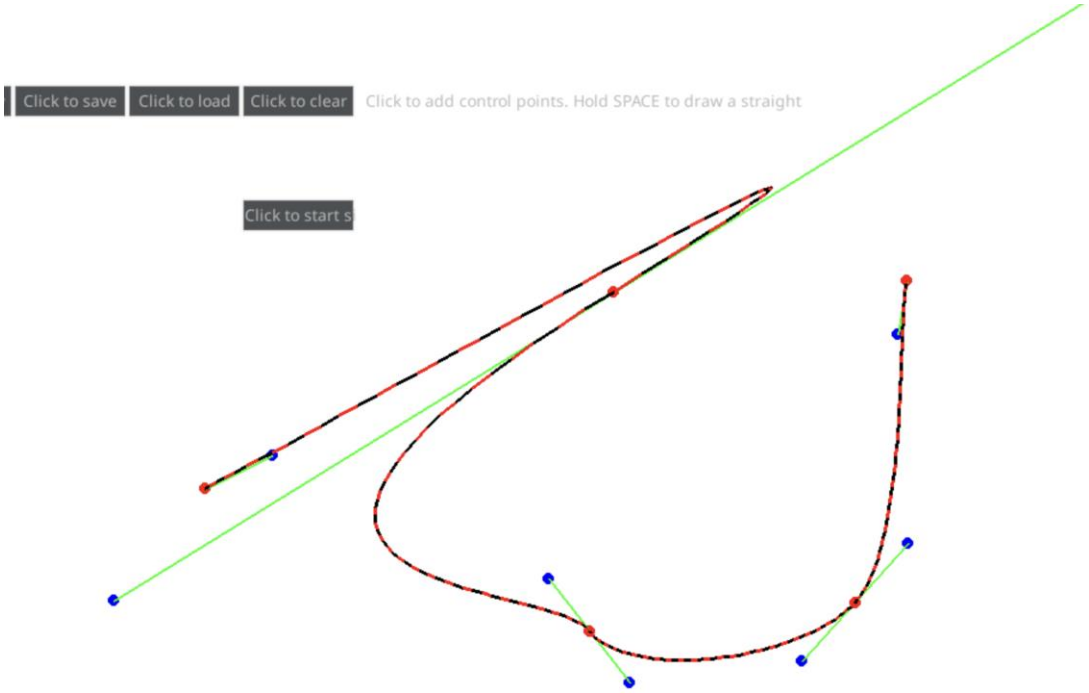
- New anchor points still joined using catmull-rom spline
- Catmull-rom control points ensure handles are generated tangent and symmetric.



- When control point (on the 4th handle) is moved, the corresponding control point also moves, ensuing handle remains tangent, and symmetric
- Moving one handle only changes the shape of the local segment
- De castelijau''s algorithm correctly computes new bezier spline using manually set control points.

Although G1 continuity ensures a considerable degree of regularity, there are still other parameters that must be moderated. These are mainly overlaps and kinks, which are both still possible even under the enforced G1 continuity





A contraint that can be added to limit kinks is to set a limit on how long the handles can be, i.e, how far away a control point can be from the respective anchor point.

```
153                clamped_translation = (relative_mouse_pos - anchor_point).clamp_magnitude(self.max_handle_length)
154                moved_point = anchor_point + clamped_translation
```

Another contraint I experimented with is to set a limit on how close any 2 anhchor points can be.

```
if self.is_anchor_point_selected:
    valid_pos = True
    for anchor_point in self.anchor_points:
        if (((relative_mouse_pos - anchor_point).length() < 2 * self.max_handle_length) and
        anchor_point != self.anchor_points[point_index]):
            valid_pos = False
            break
    if valid_pos:
        translate_vector = relative_mouse_pos - self.anchor_points[point_index]
        self.anchor_points[point_index] = relative_mouse_pos
        if point_index*2 < len(self.control_points):
            self.control_points[point_index * 2] += translate_vector
        if point_index != 0:
            self.control_points[point_index * 2 - 1] += translate_vector
    else:
        self.is_dragging = False
        self.drag_point = None
```

However when implementing this contraint, I began to realise that this method of ensures every single movement adheres to a set of rules actually leads to a clunky user experience, since there are so many invalidities to manage. Hence I decided to only implement the max handle length, and to enfore all other contraints like overlaps, overly sharp coners etc. I would simply highlisght the invalid portions of the track. This would still allow the user considerable freedom and a smooth experience while also ensure track validitiy.

Adding walls:

Since the center line of the track (the track spine) is represented as a series of points, I can consider the spine as a series of line segments betwween each pair of points. My idea to generate walls is to generate 2 line segments parallel and equidistant to each spine segment. The series of line segments on one side of the spine can be considered the ‘inner wall’ and the other can be considered the ‘outer wall’. Here I have rendered inner wall to be orange and the outer wall to be purple.

```
256    def generate_track_walls(anchor_points, control_points, width = 50):  1 usage   joxnrepresent *
257        track_spine = generate_track_spine(anchor_points, control_points)
258        outer_wall_points = []
259        inner_wall_points = []
260        for i in range(len(track_spine)-1):
261            spine_point, next_point = track_spine[i], track_spine[i+1]
262            forward = (next_point - spine_point).normalize()
263            if i > 0:
264                backward = (spine_point - track_spine[i - 1]).normalize()
265            else:                                          # Getting vectors to next and prev points
266                backward = forward
267
268            tangent = (forward + backward)
269            if tangent.length_squared() == 0:              # Getting the tangent vector
270                tangent = forward
271
272            # Getting normal to be able to generate 2 pairs of equidistant parallel points
273            normal_vector = tangent.rotate(90).normalize()
274            outer_wall_point = spine_point - normal_vector * width
275            inner_wall_point = spine_point + normal_vector * width
276            is_outer_wall_valid = True
277            is_inner_wall_valid = True
278            valid_distance = width ** 2  # Getting squared valid distance to avoid expensive sqrt operation
279
280            # Creating padding to not generate points in corners where they are too close and overlapping
281            for j in range(i-40, i+40):
282                if (outer_wall_point - track_spine[j % len(track_spine)]).length_squared() < valid_distance:
283                    is_outer_wall_valid = False
284                    break
285            for j in range(i - 40, i + 40):
286                if (inner_wall_point - track_spine[j % len(track_spine)]).length_squared() < valid_distance:
287                    is_inner_wall_valid = False
288                    break
289
290            if is_outer_wall_valid:
291                outer_wall_points.append(outer_wall_point)
292            if is_inner_wall_valid:
293                inner_wall_points.append(inner_wall_point)
294        return outer_wall_points, inner_wall_points
```

Output:

Toggleable control handles:

Having now experiemented with rendering curves both comepletely with Catmull-Rom interpolation and no control handles, as well as with De Castiljau's algorithm with editable control handles, I decided to implement both of these into my final track maker. The default setting is the Catmull-rom spline without the control handles, but the user can click on a button to enable the control handles, which then switches to rendering with De Castiljau's algorithm, and the user can edit the ocntrol handles.

In the TrackMakerUI gamemode:

```
if self.toggle_handles_button in game_core.pressed_buttons:

    # Sets the curve rendering mode based on if control handles are enabled
    self.canvas.is_handles_enabled = not self.canvas.is_handles_enabled
    self.canvas.selected_point = None
    if self.canvas.is_handles_enabled:
        self.canvas.generate_all_controls()
        self.toggle_handles_button.set_text("Disable Handles")
    else:
        self.canvas.control_points.clear()
        self.toggle_handles_button.set_text("Enable Handles")
```

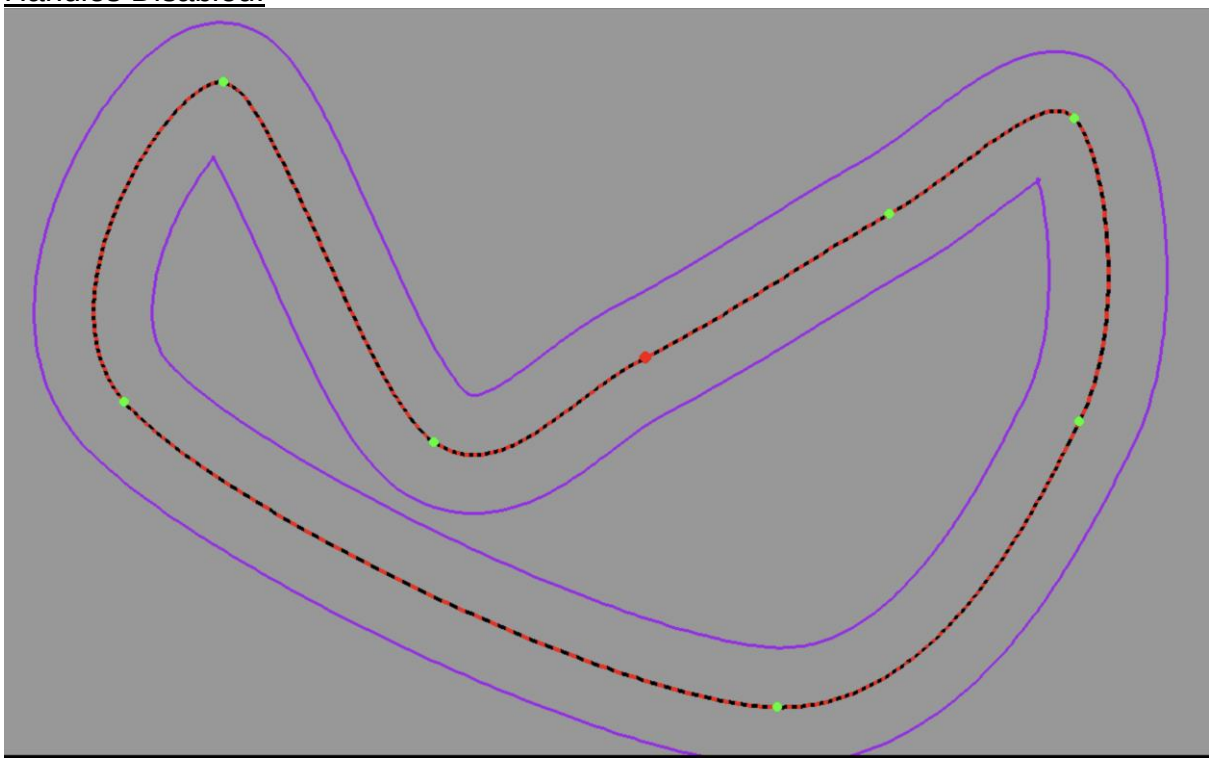
- If handles are enabled, the list of control points is generating using the Catmull-rom to bezier conversion.
- If handles are disabled, list of control points is cleared and curve is rendered using Catmull-rom interpolation.

```
def generate_all_controls(self):  2 usages   joxnrepresent *
    self.control_points.clear()
    if len(self.anchor_points) < 2:
        return

    for i in range(n - 1):
        p1 = self.anchor_points[i]
        p2 = self.anchor_points[i + 1]

        if self.is_track_complete:
            # wrap neighbors for closed track:
            p0 = self.anchor_points[i - 1] if i - 1 >= 0 else self.anchor_points[-2]
            p3 = self.anchor_points[i + 2] if i + 2 < len(self.anchor_points)  else self.anchor_points[1]
        else:
            # keep current behaviour for open track
            p0 = self.anchor_points[i - 1] if i - 1 >= 0 else p1
            p3 = self.anchor_points[i + 2] if i + 2 < len(self.anchor_points)  else p2

        b1, b2 = self.get_bezier_points(p0, p1, p2, p3)
        self.control_points.append(b1)
        self.control_points.append(b2)

    # ensure the handle is mirrored at connection point for closed tracks
    if self.is_track_complete and len(self.control_points) >= 2:
        handle_point = self.anchor_points[0]  # anchor at seam
        mirror_handle_translation = self.control_points[0] - handle_point
        self.control_points[-1] = handle_point - mirror_handle_translation
```

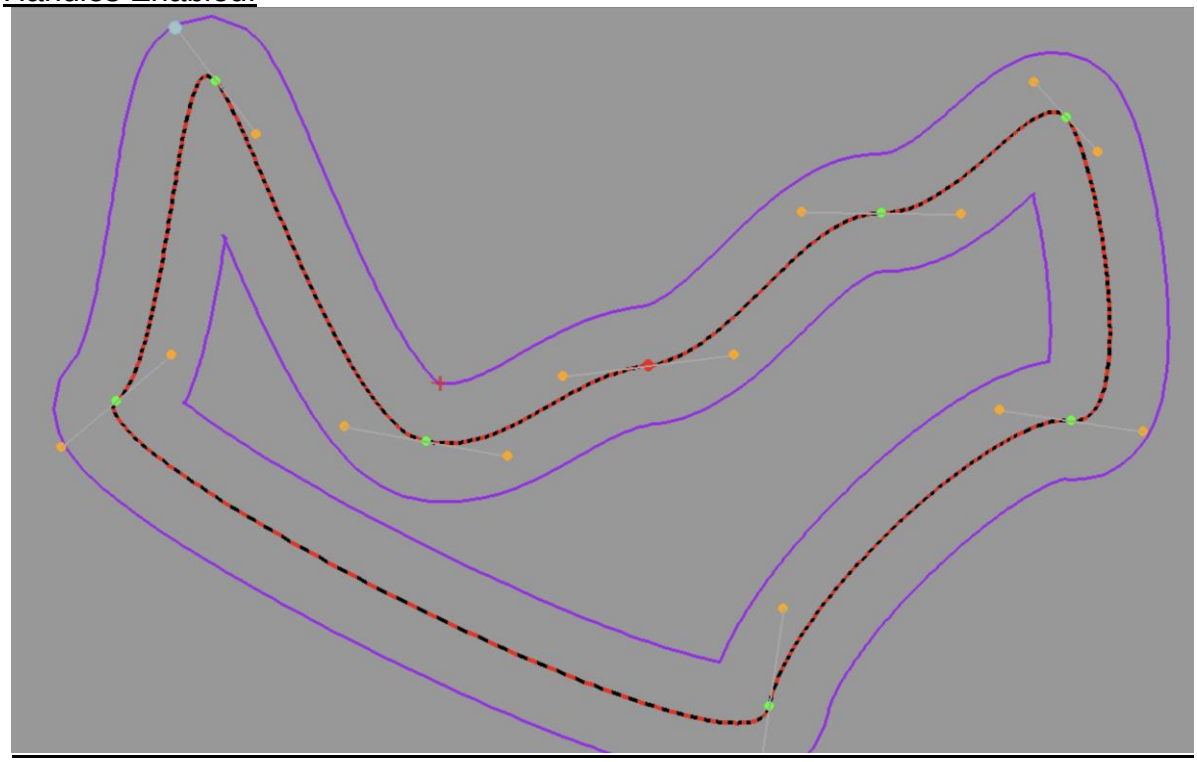
- Control points are generated taking into account both closed and open tracks

Additionally, for all rendering and event handling, the functions bypass anything to do with the control points when the handles are disabled.

Handles Disabled:



Handles Enabled:



Anchor insertion: -

I wanted to implement anchor insertion to allow the user to create anchor points mid way the track instead of only the end. This means that turns can be added to the track without having to move all anchor points along the track spine. To do this, I implemented a point-to-segment distance method to measure the shortest distance between a line segment and a point.

This closest point of the line segment to the point is calculated by projecting point ‘p’ onto the the vector ‘ab’ and then clamping the projection scalar parameter ‘t’ within range [0,1] to ensure that the closent point lies on the segment. The scalar parameter ‘t’ is how far along the vector ‘ab’ the closest point is.

$$t = \frac{\vec{AP} \cdot \vec{AB}}{\|\vec{AB}\|^2}$$

The distance from the point to the segment is then the euclidian distance to the point on ab where p = a + t * ab.

```
def point_segment_distance(point, segment):  2 usages   joxnrepresent *
    """
    Gets pythagorean distance from point to a line segment

    Args:
        point (tuple): (x,y) coordinates of a point
        segment (tuple): End points of a line segment ((x1,x2),(y1,y2))

    Returns:
        float: Closest distance
    """
    a, b = segment
    ap = point - a
    ab =  b - a

    t = max(0, min(1, ap.dot(ab) / (ab.length_squared()+ 1e-10)))
    closest_point = a + ab * t
    return (point - closest_point).length()
```

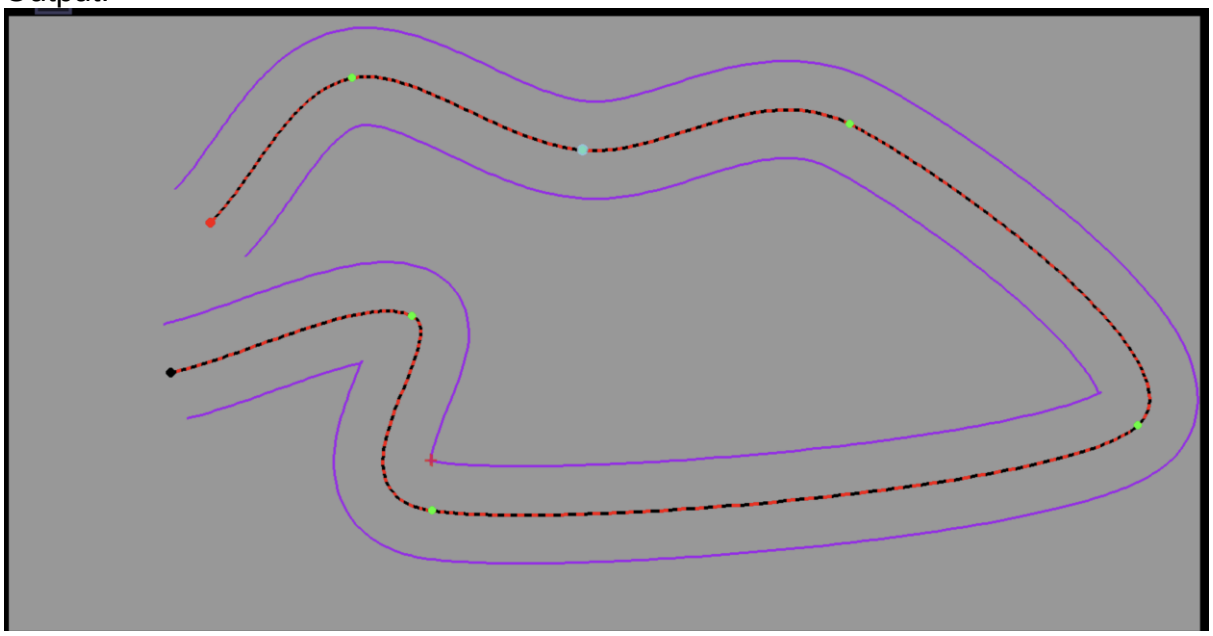
To insert anchors, if the distance to a spine point is less than the threshold of 40, then a anchor point is inserted at that index ‘i’.

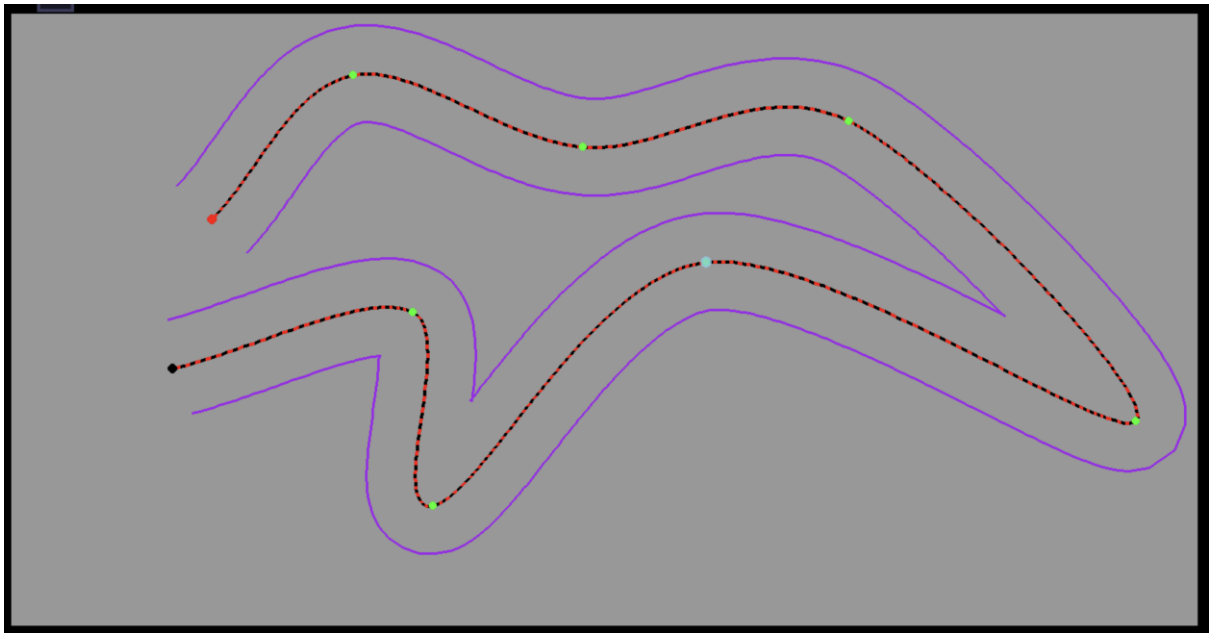
```
def handle_anchor_insertion(self, mouse_pos):  1 usage   joxnrepresent *
    for i in range(len(self.anchor_points) - 1):
        a1, a2 = self.anchor_points[i], self.anchor_points[i + 1]
        if self.is_handles_enabled:
            c1, c2 = self.control_points[2 * i], self.control_points[2 * i + 1]
            spine_points = r.generate_bezier_track_spine( anchor_points: [a1, a2],  control_points: [c1, c2])
        else:
            spine_points = r.generate_catmull_rom_track_spine( anchor_points: [a1, a2], self.is_track_complete)

        for j in range(len(spine_points) - 1):
            segment = (spine_points[j], spine_points[j + 1])

            dist = r.point_segment_distance(mouse_pos, segment)
            if dist <= 40:
                return i
    return None
```

Inserting anchor code:

```
def insert_anchor(self, mouse_pos, insert_index):  1 usage   joxnrepresent
    self.anchor_points.insert(insert_index, mouse_pos)
    self.control_points = []
    if not self.is_handles_enabled:
        return
    for i in range(len(self.anchor_points) - 1):
        p1 = self.anchor_points[i]
        p2 = self.anchor_points[i + 1]
        p0 = self.anchor_points[i - 1] if i - 1 >= 0 else p1
        p3 = self.anchor_points[i + 2] if i + 2 < len(self.anchor_points) else p2
        b1, b2 = self.get_bezier_points(p0, p1, p2, p3)
        self.control_points.append(b1)
        self.control_points.append(b2)

    if self.is_track_complete and len(self.control_points) >= 2:
        handle_point = self.anchor_points[0]
        mirror_handle_translation = self.control_points[0] - handle_point
        self.control_points[-1] = handle_point - mirror_handle_translation
```

Output:





- Inserted anchor updates the track shape without needing to move any other points

Track Widths:

To allow the user to adapt the width of the track in different segments, I implemented separate width mode and anchor mode.

```
def event_handle(self):  1 usage (1 dynamic)   joxnrepresent *
    """Handles button presses."""
    if self.anchor_toggle_button in game_core.pressed_buttons:
        self.canvas.mode = "anchor"
        self.canvas.selected_point = None
        self.anchor_toggle_button.disable()
        self.width_toggle_button.enable()

    if self.width_toggle_button in game_core.pressed_buttons:
        self.canvas.mode = "width"
        self.canvas.selected_point = None
        self.width_toggle_button.disable()
        self.anchor_toggle_button.enable()
```

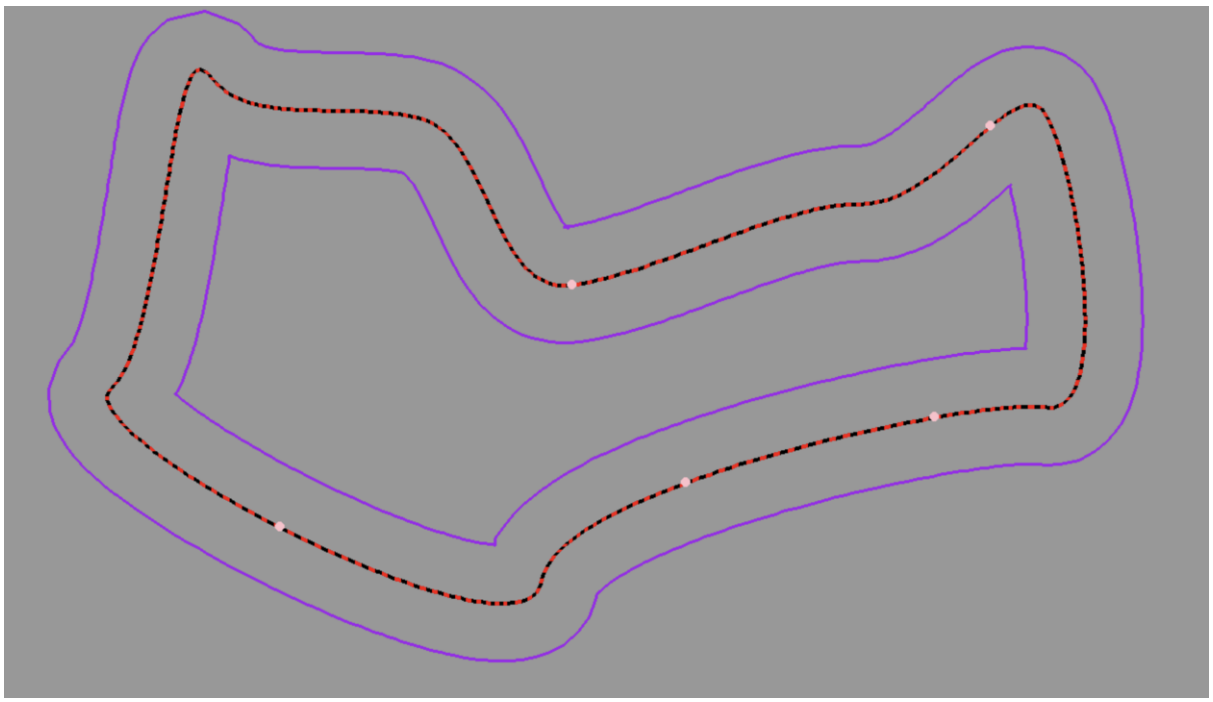
The anchor mode allows the user to create, delete and move anchor and control points to change the shape of the track, while width mode allows user to locally change the width of the track in individual sections. The user modifies track widths by creating “width points” alont the track spine, and then setting the sidth of the track at these width points

I store the data of tracks in a dictionary data structure (widths_dict), with the keys being the location of the width points along the spine, and the values being the associated width value. The width point positions are stored as a portion along the track spine length (value between 0 to 1, 0 being the start and 1 being the end of the track). The points are then determined by finding the closest track spine point that corresponds to this value. The reason why the widths dictionary stores the width points as perventages of the track spine is because the size, shape, and number of spine points change dynamiccally when the user moves around anchor points.

Creating width points

```
def create_width_point(self, mouse_pos):  1 usage  joxnrepresent *
    if len(self.track_spine) > 1:
        # Minimum 2 anchors
        for i in range(len(self.track_spine) - 1):
            # Iterate through all spine segments
            segment = (self.track_spine[i], self.track_spine[i+1])
            dist = r.point_segment_distance(mouse_pos, segment)
            if dist <= 2:
                # Create points if the track spine point is close enough
                track_proportion = i/(len(self.track_spine)-1)
                self.widths_dict[track_proportion] = 60
                self.selected_point = ('w', track_proportion)
                break
```

- Width point is created with default with value of 60
- The point_segment_distance() method is used to identify where on the track spine to create the width point.



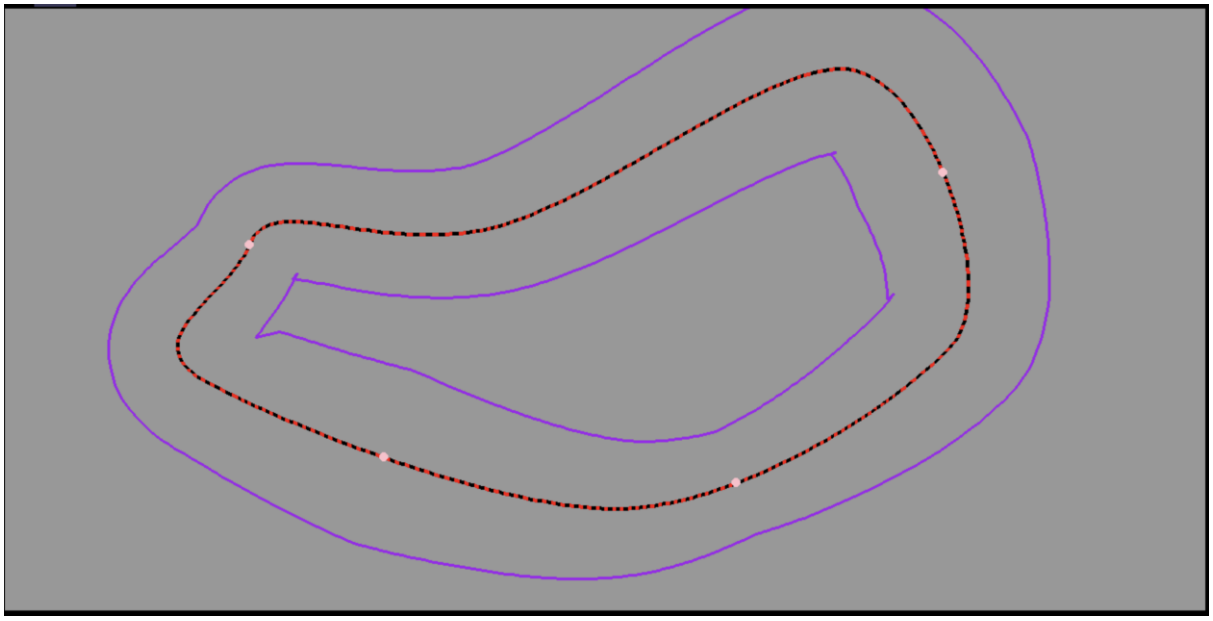
- Pink points are the width points

Generating track walls:

Having implemented width points, I re-implemented the method to generate track walls. First, a list of the track spine indices corresponding to the stored width values. For each pair of points, the change in width is calculated to determine how much the width should increase at each segment of the track wall. The segments in between pairs of width points are generated as before, by calculating the normal vector and then created wall points equidistant on either side of the spine point. Wall points generated are screened near corners to minimise overlapping.

```
def generate_track_walls(track_spine, widths_dict, is_track_complete = False):  2 usages  joxnrepresent
    """
    Generate track walls from spine.

    Args:
        track_spine (list[Vector2]): Track spine points
        widths_dict (dict): Width values (fractional positions)
        is_track_complete (bool): Whether to connect walls as continuous loop.

    Returns:
        tuple(list[Vector2], list[Vector2]): lists of outer and inner wall point coordinates.
    """
    outer_wall_points = []
    inner_wall_points = []

    # Convert width position to track spine indices.
    width_point_locations = list(widths_dict.keys())
    width_point_locations.sort()
    width_indices = []
    for location in width_point_locations:
        width_indices.append(math.floor(location * (len(track_spine)-1)))

    # Interpolate between width points
    for i in range(1, len(width_indices)):
        start_width_index, end_width_index = width_point_locations[i - 1], width_point_locations[i]
        start_track_index, end_track_index = width_indices[i - 1], width_indices[i]
        start_width = widths_dict[start_width_index]
        d_width = widths_dict[end_width_index] - widths_dict[start_width_index]

        for t in range(start_track_index, end_track_index):
            segment_width = start_width + (t - start_track_index) / (end_track_index - start_track_index) * d_width
```

```
        spine_point, next_point = track_spine[t], track_spine[t+1]

        forward = (next_point - spine_point).normalize()
        if t > 0:
            backward = (spine_point - track_spine[t - 1]).normalize()
        else:                                          # Getting vectors to next and prev points
            backward = forward

        tangent = (forward + backward)
        if tangent.length_squared() == 0:              # Getting the tangent vector
            tangent = forward

        # Getting normal to be able to generate 2 pairs of equidistant parallel points
        normal_vector = tangent.rotate(90).normalize()
        outer_wall_point = spine_point - normal_vector * segment_width
        inner_wall_point = spine_point + normal_vector * segment_width
        # Getting squared valid distance to avoid expensive sqrt operation


        def check_wall_point_validity(wall_point):  joxnrepresent
            """Check if wall point is too close to track spine. Ensures smooth corners."""

            valid_distance = (segment_width - 3) ** 2  # Included padding
            tollerence_limit = 80
            for j in range(t - tollerence_limit, t + tollerence_limit):

                if is_track_complete:
                    spine_index = j % len(track_spine)
                else:
                    spine_index = max(0, min(j, len(track_spine) - 1))

                if(wall_point - track_spine[spine_index]).length_squared() < valid_distance:
                        if is_track_complete:
                            index_distance = abs(spine_index - t)
                            wrapped_index_distance = min(index_distance, len(track_spine) - index_distance)
                        else:
                            wrapped_index_distance = abs(spine_index - t)
                        if wrapped_index_distance <= 1:
                            continue
                        return False
            return True

        is_outer_wall_valid= check_wall_point_validity(outer_wall_point)
        is_inner_wall_valid = check_wall_point_validity(inner_wall_point)

        if is_outer_wall_valid:
            outer_wall_points.append(outer_wall_point)
        if is_inner_wall_valid:
            inner_wall_points.append(inner_wall_point)

    return outer_wall_points, inner_wall_points
```



- Track wall width smoothly interpolates from one poitn to the next

Selecting width points

```
elif self.mode == "width":
    if not self.track_spine:
        return False
    width_point_positions = list(self.widths_dict.keys())
    for position in width_point_positions:
        width_point_index = max(min(math.floor(position * len(self.track_spine)), len(self.track_spine)-1),0)
        point = self.track_spine[width_point_index]
        if (mouse_pos - point).length() <= self.point_size:
            self.selected_point = ('w', position)
            self.is_point_selected = True
            return True
```

- Select a point by finding itterating through the closest spine points to every width point position

Track Validity Checks: -

There are 2 main criteria to check for validity of track walls:

- Walls must not overlap
- Walls must not kink/ cross over the width of the track blocking the car.

These checks require all the wall segments to be checked against each other. With long tracks, this causes lots of delay in user input. To counteract this, I implemented an instance of the Spacial hash Grid (shg_grid), and hash all wall segments into it. Then all wall intersection checks are handled by checking collisiongs using this data structure.

```
def hash_grid(self, outer_wall_points, inner_wall_points):  1 usage  joxnrepresent
    self.shg_grid.clear_grid()
    hash_points = outer_wall_points + inner_wall_points
    for i in range(len(hash_points) - 1):
        if i != len(outer_wall_points) - 1:
            self.shg_grid.hash_segment( wall_segment: (hash_points[i], hash_points[i + 1]), i)
```

- Grid is cleared to remove all older segments
- All outer and inner wall segments hashed to the spacial hash grid
- Outer walls are added first, then inner walls are added. This means when itterating through segments, index correction has to be applied.

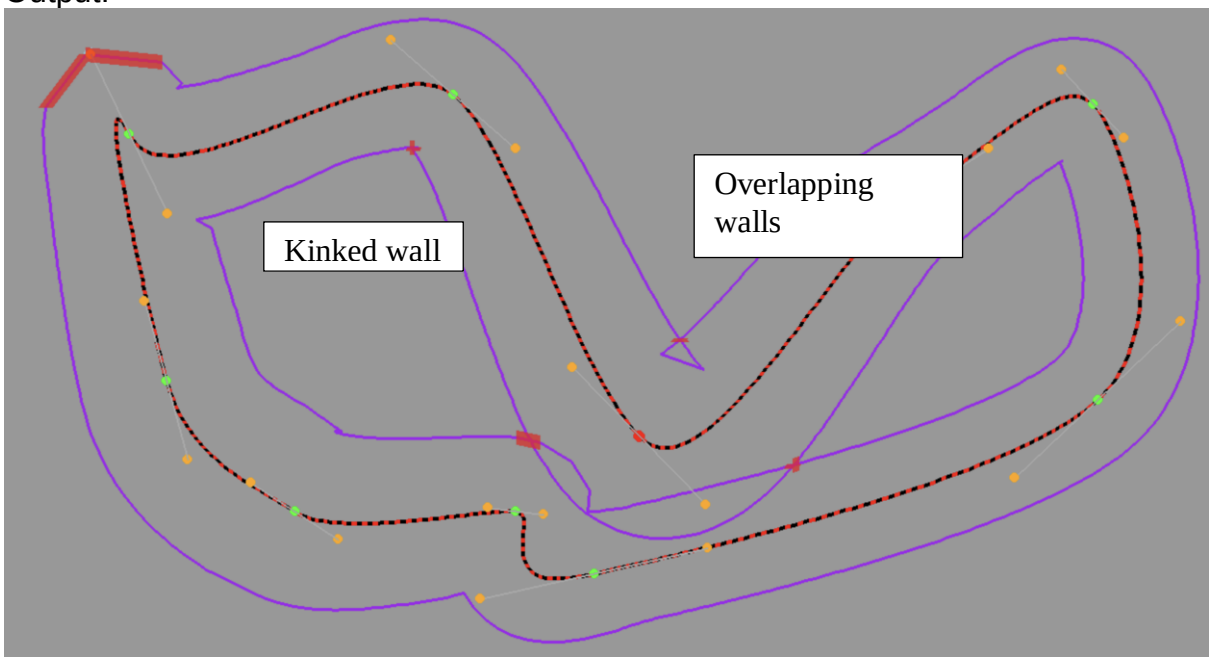
To flad validity issues, I made a check_wall_validity() function that returns a list of all invalid segments. These segments can be highlighted in red to notify the user where the issues of validity are. If the size of this list is 0, then it means there are no issues with wall validity.

```
def check_wall_validity(wall_points, hashed_grid_start_index):  joxnrepresent *
    hashed_points = outer_wall_points + inner_wall_points
    invalid_segments = []
    for i in range(len(wall_points) - 1):
        segment = (wall_points[i], wall_points[i + 1])
        collisions = self.shg_grid.return_all_collisions(segment, hashed_points)
        hashed_segment_index = hashed_grid_start_index + i

        for collision_index in collisions:
            distance = min(abs(collision_index - hashed_segment_index),
                           len(wall_points) - 1 - abs(collision_index - hashed_segment_index))

            # Checking for collision between walls that are not adjacent/very close (preventing false positives)
            if ((0< collision_index < len(hashed_points)-1 and 0< i < len(hashed_points)-1)and
                    (distance > 4)):
                invalid_segments.append((hashed_points[collision_index], hashed_points[collision_index + 1]))

        # Checking for kinks in the track walls
        if Vector2(segment[0] - segment[1]).length_squared() > 4000:
            invalid_segments.append(segment)

    return invalid_segments
```

Invalid segments are rendered with a red overlay to flag invalidity to the user visually In rebuild:

```
    invalid_walls = self.get_validity_issues(outer_wall_points, inner_wall_points)
    for wall in invalid_walls:
        r.draw_line(overlay,  colour: (255, 0, 0, 150), wall[0], wall[1],  width: 15)
    self.image.blit(overlay, (0, 0))
```

Output:



Along with wall validity issues, there are 2 other validity criteria that I intend to enforce:

- Track must be complete
- Track must have a minimum length (To ensure that there is a full loop to drive along)

So the total track validation enforces these. I made a function that returns the specific validity issue so that the user can know clearly why they cannot save the track.

```
        if len(invalid_inner_walls + invalid_outer_walls) > 0:
            self.validity_issues = "invalid walls"
        elif not self.is_track_complete:
            self.validity_issues = "incomplete"
        elif len(outer_wall_points) < 300 or len(inner_wall_points) < 300:
            self.validity_issues = "too short"
        else:
            self.validity_issues = None
        return invalid_outer_walls + invalid_inner_walls
```

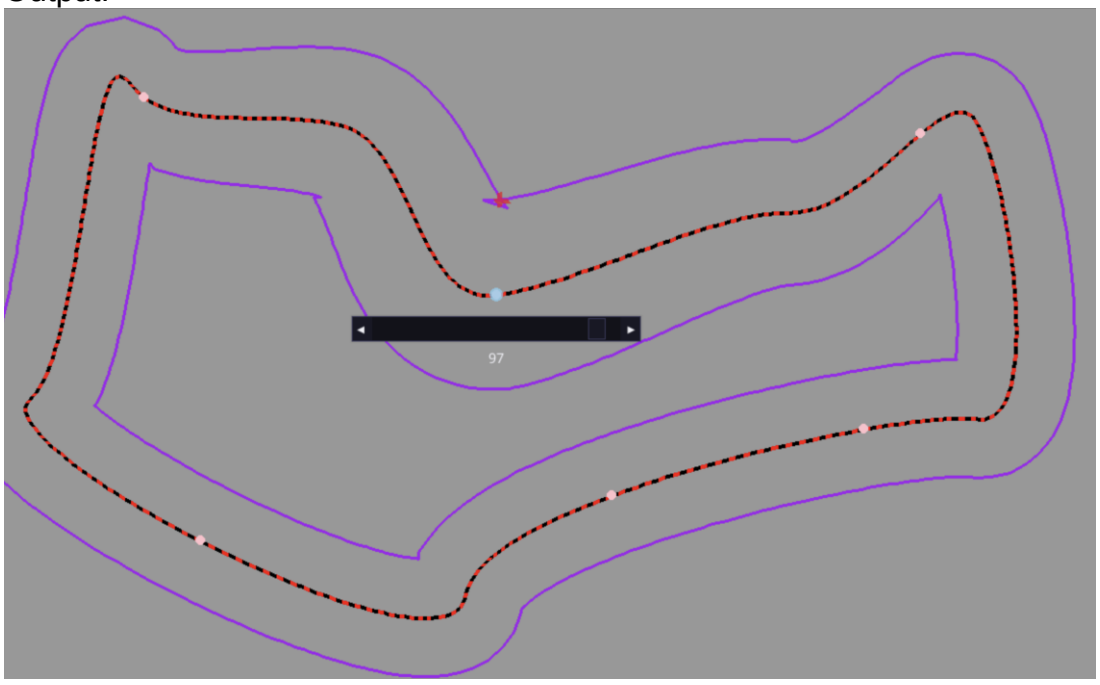
<u>Changing widths</u>

When a width point is selected, a slider is created directly below it. This slider allows the user to change the width of the track at the selected width point

In TrackMakerUI gamemode class:

```
def changing_widths(self):  1 usage   joxnrepresent *
    """
    Handles user changing track widths.

    Updates width slider position based on where width point is. Updates track width values
    based on slider input.
    """
    if self.canvas.selected_point and self.canvas.selected_point[0] == 'w':
        width_val_index = self.canvas.selected_point[1]
        width_point_index = math.floor(max(min(width_val_index * len(self.canvas.track_spine),
                                               len(self.canvas.track_spine) - 1), 0))
        point_position = self.canvas.track_spine[int(width_point_index)]
        if self.width_slider is None:
            # Create slider
            start_val = self.canvas.widths_dict[width_val_index]

            self.width_slider = pygame_gui.elements.UIHorizontalSlider(
                relative_rect=pygame.Rect((point_position[0] -150, point_position[1] + 100),
                                          (300, 30)),
                start_value=start_val,
                value_range=(60, 100),
                manager=self.gui_manager
            )

            self.width_label = pygame_gui.elements.UILabel(
                relative_rect=pygame.Rect((point_position[0] -30, point_position[1] + 130), (60, 30)),
                text=str(int(start_val)),
                manager=self.gui_manager
            )

        else:
            # Set width value from slider value
            val = int(round(self.width_slider.get_current_value()))
            self.canvas.widths_dict[width_val_index] = val
            self.width_label.set_text(str(val))
else:
    # Remove slider if changing away from width mode.
    if self.width_slider and self.width_label:
        self.width_slider.kill()
        self.width_label.kill()
    self.width_slider = None
    self.width_label = None
```

Output:



- Track width is smoothly transitioned between the values at different width points.
- Width slider is visible below the selected width point
- Width changes as the value of the slider is modified.

Saving and Loading

If track meets all validity criteria, the use can save the track into a json file.

```
def write_track_data(self, filename=game_core.current_track):  1 usage   joxnrepresent *
    """
    Create data frame with correct formatting and save it to a json file. Ensures handles are generated
    before saving.

    Args:
        filename (str): Track name
    """
    if filename is not None:
        if not self.canvas.is_handles_enabled:
            self.canvas.generate_all_controls()
        data = {
            "anchors": [(point.x, point.y) for point in self.canvas.anchor_points],
            "controls": [(point.x, point.y) for point in self.canvas.control_points],
            "widths": {str(k): v for k, v in self.canvas.widths_dict.items()}
        }
        with open(f"Tracks/{filename}.json", "w") as file:
            json.dump(data, file, indent=2)
```

- Anchor points, control points and widths are saved as a dictionary
- File is created/ written to using json.dump
- Saved track can be fully reporduced using this track data

Load Track:

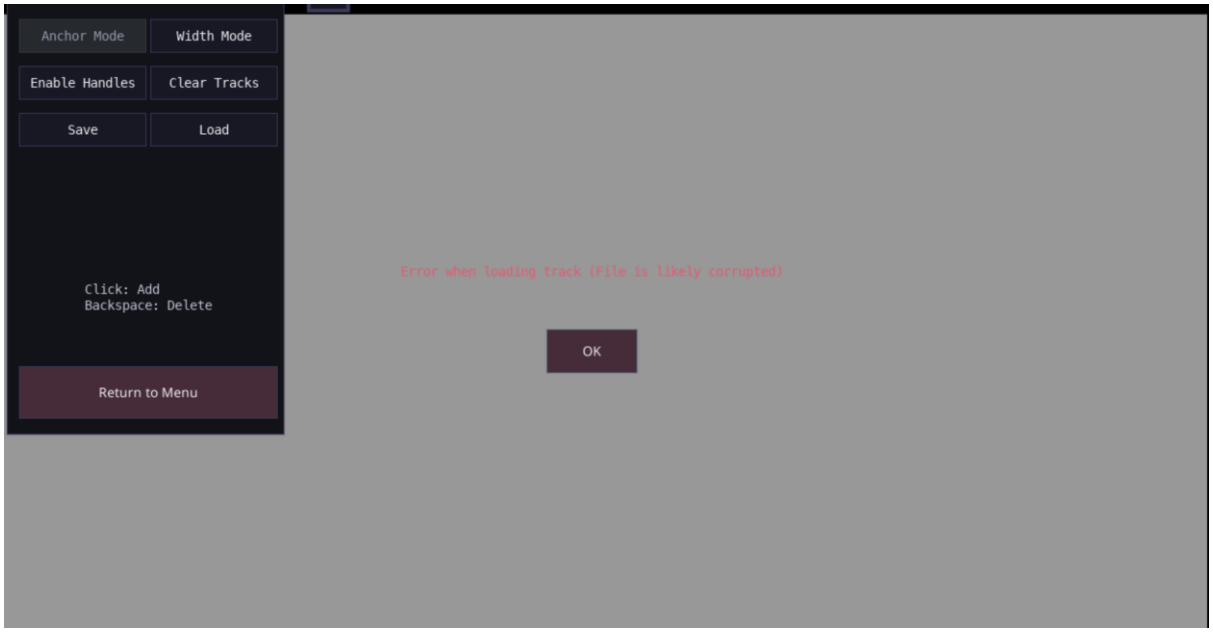
To load tracks from saved json files, I implemented a load_bezier_track() helper function that extracts the track data. Additionally, I added a “centering” functionality. This is usegul when loading tracks to car simulations, since it ensures drawn tracks are rendered in the middle of the screen.

```
def load_bezier_track(filename, enforce_centering = False):  2 usages  joxnrepresent *
    with open(f"Tracks/{filename}.json", "r") as f:
        data = json.load(f)

    anchor_points = [Vector2(point[0], point[1]) for point in data["anchors"]]
    control_points = [Vector2(point[0], point[1]) for point in data["controls"]]
    widths_dict = {float(k): v for k, v in data["widths"].items()}

    if enforce_centering:
        all_points = anchor_points + control_points

        min_x = min(point.x for point in all_points)
        max_x = max(point.x for point in all_points)
        min_y = min(point.y for point in all_points)
        max_y = max(point.y for point in all_points)

        track_center = Vector2((min_x + max_x) / 2, (min_y + max_y) / 2)
        screen_center = (game_core.screen_dimensions[0] / 2, (game_core.screen_dimensions[1] - 200)/2)
        translation = screen_center - track_center

        anchor_points = [p + translation for p in anchor_points]
        control_points = [p + translation for p in control_points]

    return anchor_points, control_points, widths_dict
```

- Extract anchor points, contorl points, and widths from save data.
- Itterate thorugh all points and translate them if cenetering is enforced.

Then the TrackMakerUI game mode handles file curruption or file not found errors.

```
def load_track_data(self, filename=game_core.current_track):  1 usage  joxnrepresent *
    """
    Load track from json file.

    Sets the canvas anchor points, control points and widths attributes to the data stored in the file.
    Displays error if file is corrupted.

    Args:
        filename (str): Track file name
    """
    try:
        if filename is not None:
            (self.canvas.anchor_points, self.canvas.control_points,
             self.canvas.widths_dict) = r.load_bezier_track(filename)
        else:
            self.error_label.set_text("File not found")
            self.error_label.show()
            self.ack_error_button.show()
    except Exception as e:
        self.error_label.set_text("Error when loading track (File is likely corrupted)")
        self.error_label.show()
        self.ack_error_button.show()
        print(e)
```



2.3.3 Track

The Track class creates a sprite object that allows cars to drive along tracks in car simulation game modes. The Track is initialised with bezier track data (anchor points, control points and widths dictionary) that is loaded from a JSON file. The other track elements (track spine, inner walls, outer walls and the sprite image) are calculated and rendered from this data. The Track object also allows for hitbox collision detection to handle cars crashing into walls, and ray casting to allow for AI car sensors to detect distances.

Load bezier track helper method:
To enable loading bezier track data from a file, I creater a load_bezier_track() method in the resources module. This method extracts the anchor points, control points and widths data and converts it to a suitable format. Additionally, it also handles optional “centering”. If centering Is enforces, the track is rendered so that the bounding box is at the middle of the screen.

```python
def load_bezier_track(filename, enforce_centering = False):  2 usages  joxnrepresent
    """
    Load bezier track from JSON file.

    Args:
        filename (str): Track filename
        enforce_centering (bool): Whether track is centered on screen

    Returns:
        tuple: (anchor_points, control_points, widths_dict)
    """
    with open(f"Tracks/{filename}.json", "r") as f:
        data = json.load(f)

    anchor_points = [Vector2(point[0], point[1]) for point in data["anchors"]]
    control_points = [Vector2(point[0], point[1]) for point in data["controls"]]
    widths_dict = {float(k): v for k, v in data["widths"].items()}

    # Calculate bounding box
    if enforce_centering:
        all_points = anchor_points + control_points

        min_x = min(point.x for point in all_points)
        max_x = max(point.x for point in all_points)
        min_y = min(point.y for point in all_points)
        max_y = max(point.y for point in all_points)

        track_center = Vector2((min_x + max_x) / 2, (min_y + max_y) / 2)
        screen_center = (game_core.screen_dimensions[0] / 2, (game_core.screen_dimensions[1] - 200)/2)
        translation = screen_center - track_center

       #Translate all points
       anchor_points = [p + translation for p in anchor_points]
       control_points = [p + translation for p in control_points]

    return anchor_points, control_points, widths_dict
```

Track Rendering

The track sprite renders to display the track walls, and the start line. The Track walls are rendered as a sequeze of red and white strips (to mimic F1 track colour scheme), and a black and white checkered line is rendered at the start line. The road in between the walls is rendered in grey.

```python
88      def draw_track(self):  2 usages  joxnrepresent +1 *
89          """Draw track graphics with walls, road and finish line."""
90          self.image.fill((0, 0, 0, 0))
91
92          # Draw road in grey
93          track_polygon = self.outer_wall_points + self.inner_wall_points
94          pygame.draw.polygon(self.image,  color: (80, 80, 80), track_polygon)
95          self.draw_checkered_line()
96          self.draw_alternating_wall()
97
98          game_core.game_mode.debug_elements["track spine"] = [self.track_spine]
99
```

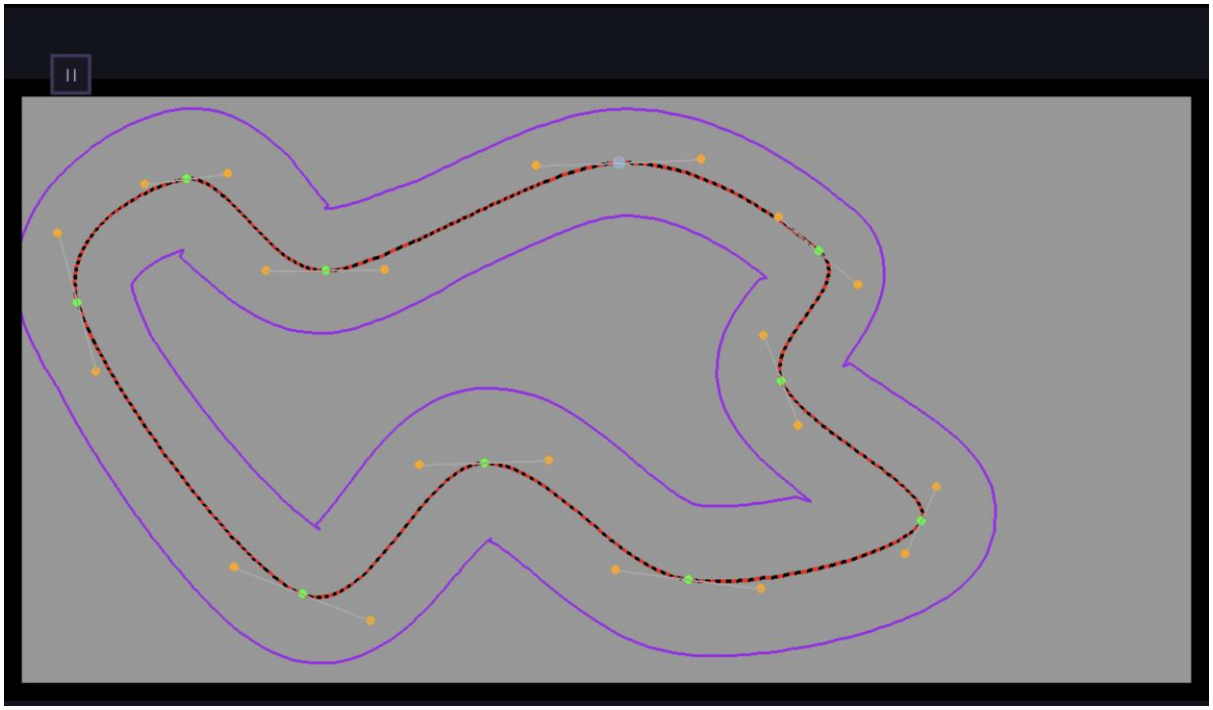
```
def draw_alternating_wall(self,width=10):  1 usage  joxnrepresent *
    """
    Draw walls with alternating colours.

    Args:
        width (int): Wall line width
    """

    is_black = True
    current_segment_length = 0
    for segment in self.wall_segments:
        color = (200, 200, 200) if is_black else (200, 0, 0)
        pygame.draw.line(self.image, color, segment[0], segment[1], width)
        current_segment_length += Vector2(segment[0] - segment[1]).length()
        if current_segment_length >= 50:
            current_segment_length = 0
            is_black = not is_black
```

- Red and white segments are set to a defualt length of 50
- Width is the width of walls being rendered

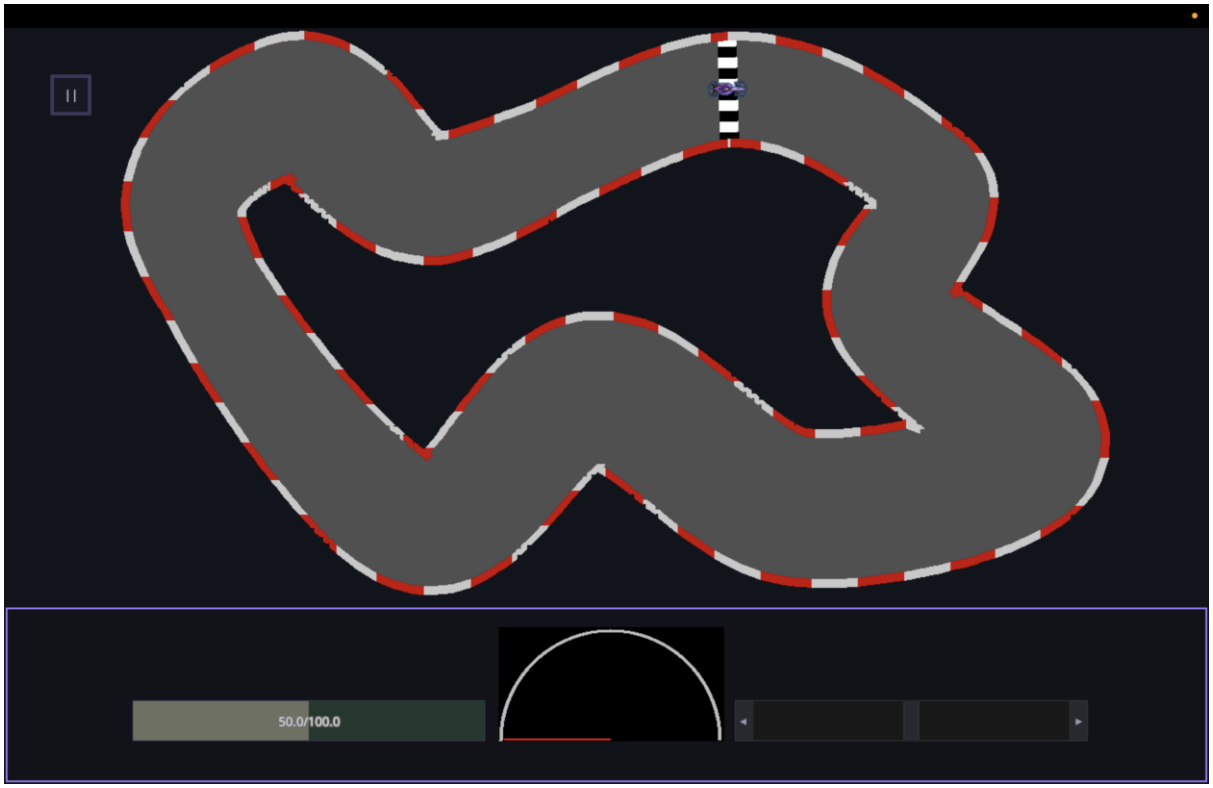
To draw a checkered line, I first find the finish line by connecting the start point of the inner wall point list and the start point of the outer wall point list. By finding the normal vector to this line, I then sequetially generate checkered squares along the finish line. The number of squares along the line is set by checkered_rows. The thickness of the line determines how thick the line generates

```
def draw_checkered_line(self, thickness = 20, checkered_rows = 10):  1 usage  joxnrepresent *
    """
    Draw checkered finish line across track start.

    Args:
        thickness (int): Line thickness perpendicular to track
        checkered_rows (int): Number of checker squares
    """
    finish_line = self.inner_wall_points[0] - self.outer_wall_points[0]
    length = finish_line.length()

    unit_vector = finish_line.normalize()
    normal_vector = Vector2(-unit_vector.y, unit_vector.x)

    check_length = length / checkered_rows

    for i in range(checkered_rows):
        start = self.outer_wall_points[0] + unit_vector * (i * check_length)
        end = self.outer_wall_points[0] + unit_vector * ((i + 1) * check_length)

        color = (255, 255, 255) if i % 2 == 0 else (0, 0, 0)

        # Get checkered square corners
        p1 = start + normal_vector * (thickness / 2)
        p2 = end + normal_vector * (thickness / 2)
        p3 = end - normal_vector * (thickness / 2)
        p4 = start - normal_vector * (thickness / 2)

        pygame.draw.polygon(self.image, color,  points: [p1, p2, p3, p4])
```

Output:
Drawn Track:



Rendered track:



- Track has identical features to the drawn track (same shape, curvature and start)
- Track walls generated as regular red and white pattern
- Thick checkered line at the start.
- Centering has been enforced when loading the track into a car simulation

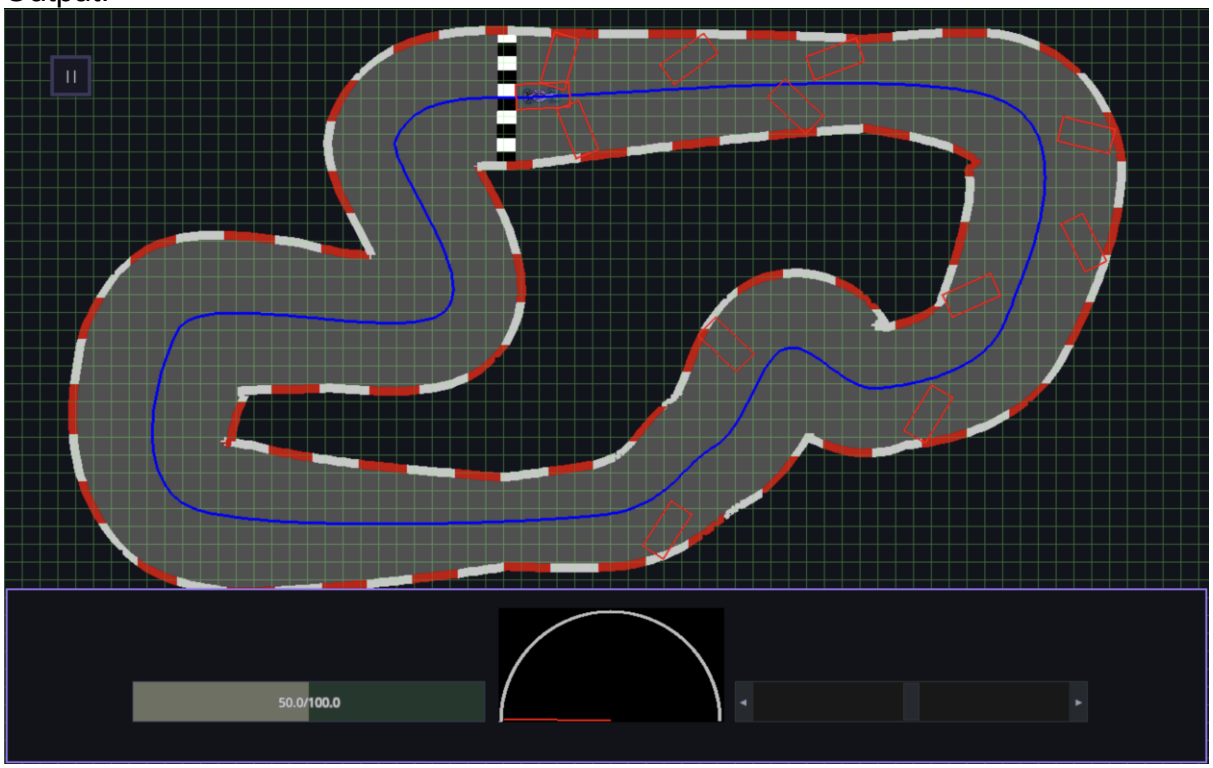
Hitbox collission detection

To detect collisions between the car and the walls of the track, I have used the return_collision_point() method of the SpacialHashGrid Class to run the DDA traversal algorithm with the correct lamda function. I have represented the car hitbox as 4 line segments that outline the car as a rectangle. To check for collisions, I check each individual line with track segments in the cells that it intersects with.

```
37      # Checks each border of the hitbox for collision with track segments in the cells it passes through
38      def hitbox_collision_detection(self, hitbox):  1 usage (1 dynamic)  joanmk
39          for i in range(len(hitbox)):
40              hitbox_border_line = (hitbox[i], hitbox[(i + 1) % len(hitbox)] )
41              collision = self.grid.return_collision_point(hitbox_border_line, self.wall_segments)
42              if collision is not None:
43                  return True
44          return False
```

To visualise crash points, I added “ghost hitboxes” to the debugger to show the places where cars are flagged as “crashed” and ensure they do not clip through walls. I testing crashing at different angles, speeds, orentations and locations on the track.

Output:



- Cars are reset without crossing over the boundaries of the track
- Hitbox overlap with track wall is consistent across all orientations.

Ray Casting

To get sensor data of distances to the nearest wall in different direction, I have implemented a simplified version of ray casting. Instead of manually casting the ray cell by cell, I have the rays represented as very long line segments (to ensure the ray always collides with some wall of the track). The ray cast method then runs the DDA traversal and returns the coordinates of the nearest collision point with a wall. Along

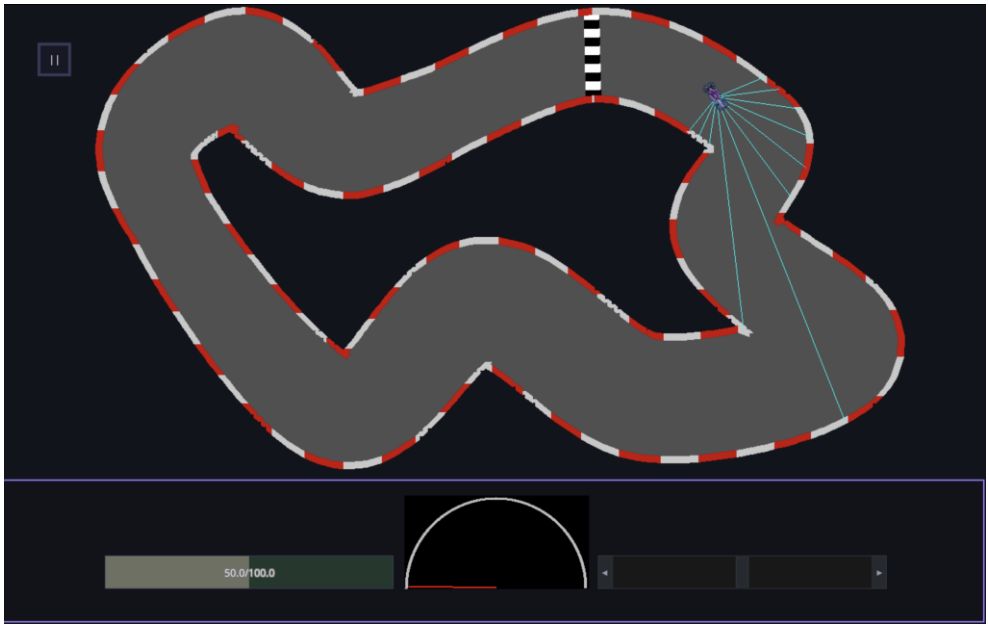
with the absolute coordinate of the collision point, the ray cast method also returns a normalised value of the distance to the nearest wall.

```
32
33      # Cast ray and return shortened ray up till nearest point of intersection
34      # Returns normalised distance which is the percentage of max length
35      def ray_cast(self, ray):  1 usage (1 dynamic)  joanmk +1
36          hit_point = Vector2(self.grid.return_collision_point(ray, self.wall_segments))
37          normalised_distance = (hit_point - ray[0]).length() / r.METER_PIXEL_CONVERSION
38          return hit_point, normalised_distance
39
```

In the actual program, only AICar will implement this ray casting, but for testing, I have temporarily implemented ray casting for the PlayerCar.

```
def get_sensors(self, track):  2 usages (2 dynamic)  new *
    """
    Computes and returns a list of sensor values.
    Sensors values are the distance to the nearest wall in different directions. Distances are determined using
    ray-casting.

    Args:
        track (Track): The track which the car is currently driving on.
    """
    center = pygame.Vector2(self.rect.center)
    rays = []

    forward_ray = pygame.Vector2(0, 1).rotate(self.direction) * self.MAX_RAY_CAST
    rays.append((center, center + forward_ray))
    for angle in self.RAY_CAST_ANGLES:
        rays.append((center, center + forward_ray.rotate(angle)))
        rays.append((center, center + forward_ray.rotate(-angle)))

    collided_rays = []
    sensors = []
    for ray in rays:
        hit_point, normalised_collision_distance = track.ray_cast(ray)
        collided_rays.append((center, hit_point))
        sensors.append(normalised_collision_distance)

    return sensors
```

Output:



To ensure the sensor data being returned is also within the required range of [0,1], I testing printing max and min sensor values while driving.

Sample of print log

```
Min sensor value: 0.039756495769282134, Max sensor value: 0.5339547331549841
Min sensor value: 0.04040787369851925, Max sensor value: 0.5326732698203055
Min sensor value: 0.04012093795208429, Max sensor value: 0.532156276824693
Min sensor value: 0.039844364757935555, Max sensor value: 0.5314399483635892
Min sensor value: 0.04053589338387169, Max sensor value: 0.530201974713031
Min sensor value: 0.04028404554711278, Max sensor value: 0.52944077835776
Min sensor value: 0.040995082814378615, Max sensor value: 0.5281727116846504
Min sensor value: 0.040774065633824165, Max sensor value: 0.5273898110359043
Min sensor value: 0.04054614036774047, Max sensor value: 0.5267525722494903
Min sensor value: 0.04130000328824462, Max sensor value: 0.5253004924311704
Min sensor value: 0.04109987795411598, Max sensor value: 0.5246747990792152
Min sensor value: 0.040908927418395, Max sensor value: 0.523877464363461
Min sensor value: 0.041699113011437125, Max sensor value: 0.5225931628304111
Min sensor value: 0.04133692937885768, Max sensor value: 0.5209141455708725
Min sensor value: 0.042157042892161475, Max sensor value: 0.5196305485705062
Min sensor value: 0.04198466474558857, Max sensor value: 0.5188303681844298
Min sensor value: 0.04180218573518444, Max sensor value: 0.5182083199857612
Min sensor value: 0.04263701242314371, Max sensor value: 0.5167468928729112
Min sensor value: 0.04231167092489739, Max sensor value: 0.5152439843932118
Min sensor value: 0.04316193409317378, Max sensor value: 0.5137813838934596
Min sensor value: 0.042991350427064014, Max sensor value: 0.5131590669732624
Min sensor value: 0.042831731540872116, Max sensor value: 0.5123568802976972
Min sensor value: 0.043685571945087716, Max sensor value: 0.5110696597908531
Min sensor value: 0.04353645650870845, Max sensor value: 0.5102653930642004
Min sensor value: 0.04424702583663648, Max sensor value: 0.5081041765984196
Min sensor value: 0.04409801408843201, Max sensor value: 0.5072977506866378
```

2.3.4 Car

Controlling Car:

I designed a Car class that simulates a simplified model of the real-world physics of driving. The car has 2 parameters that are controlled (by user or AI model): throttle and steering, both values are floats within the range [-1, 1]. The throttle controls both throttle and braking, where -1 is maximum braking, 0 is no throttle or braking (coasting) and 1 is maximum throttle. The steer controls the angle by which the car turns, with -1 being full turn left and 1 being full turn right.

In real world driving, it is not possible to instantly rotate a steering wheel from its maximum left position to its maximum right position due to actuator lag. Similarly, it is not possible to instantly go from full throttle to full braking. Therefore, to model actuator lag in controlling throttle, braking and steering with better accuracy, I decided to implement “target actions”. The target actions are what is determined by input (from user or AI model), and the actual throttle and steer values are smoothly increments until they reach the target value. The rate at which the controls are updated is determined by response factors.

```
STEER_RESPONSE = 0.3
THROTTLE_RESPONSE = 0.1
BRAKE_RESPONSE = 0.04
```

```
def _update_controls(self, target_steer, target_throttle):  1 usage   joxnrepresent
    """
    Smooth control inputs toward target values. Limits rate of increase with response factors.

    Args:
        target_steer (float): Desired steering [-1, 1]
        target_throttle (float): Desired throttle [-1, 1]
    """

    # Increment steer input
    d_steer_input = target_steer - self.current_steer_input
    if abs(d_steer_input) > 0.05:
        self.current_steer_input += 0.15 * r.sign(d_steer_input)
    else:
        self.current_steer_input = target_steer
    self.current_steer_input = max(-1.0, min(1.0, self.current_steer_input))

    # Increment throttle input
    d_throttle_input = target_throttle - self.current_throttle_input
    if abs(d_throttle_input) > 0.05:
        throttle_update_factor = self.THROTTLE_RESPONSE if target_throttle >= 0 else self.BRAKE_RESPONSE
        self.current_throttle_input += throttle_update_factor * r.sign(d_throttle_input)
    else:
        self.current_throttle_input = target_throttle
    self.current_throttle_input = max(-1.0, min(1.0, self.current_throttle_input))
```

- Steering uses the same steer response factor for turning left and right
- Throttle has a smaller response factor than the braking. This models how it takes more effort to fully press down a brake pedal than a throttle pedal

Vector parameters:

To simplify the modelling of the physics, I decided to model the motion of the car in terms of vectors instead of forces since modelling complex forces like engine forces, resistive forces of drag and friction and slip forces exceeds the scope of this project. I directly calculate the velocity and displacement each frame by taking derivatives of the acceleration. The acceleration is obtained by multiplying the throttle input by some constant throttle force factor. Friction is modelled by decaying the velocity of the car each frame where no throttle is applied.

```
def _update_car_vector_values(self):  1 usage   joxnrepresent +1 *
    """
    Update velocity, position, and heading based on physics.

    Implements simplified car physics with:
    - Velocity decay when no throttle
    - Grip (smoothly interpolates velocity)
    - Bicycle model for turning
    """

    dt = game_core.dt_ms/1000

    # Update acceleration in heading direction
    direction_unit_vector = pygame.Vector2(0, 1).rotate(self.direction)
    self.acceleration = self.throttle * direction_unit_vector

    # Update velocity
    if self.throttle == 0:
        self.velocity *= 0.992
    else:
        self.velocity += self.acceleration * dt
    if self.velocity.length_squared() != 0:
        self.velocity.clamp_magnitude_ip(self.MAX_SPEED)

    # Apply grip
    speed = self.velocity.magnitude()
    self.velocity = self.velocity.lerp(direction_unit_vector * speed, self.GRIP)

    # Update position
    self.position += self.velocity * dt
```

- Grip factor allows car to drive smoothly without sliding by aligning velocity vector with current heading direction

<u>Steering:</u>

To create a simplified model of steering, I decided to implement the kinematic bicycle model of steering. This assumes the steering is only applied from a single point at the front of the car, and that there is no tire slip. It omits the need to model tyre and vehicle forces. The main limitation of this model is that it is inaccurate for high speeds, however this will not have a huge impact on the range of speeds that I will implement.

I first apply a conversion factor to convert pixels to meters, which allows me to directly implement the bicycle model formula. It ignores low speed to avoid division by 0 errors. I then apply the bicycle model formula to calculate the angular velocity.

$$\omega = \frac{v}{L}\tan(\delta)$$

where:

- ω is the angular velocity (yaw rate)
- v is the forward speed
- L is the wheelbase
- δ is the steering angle

This angular velocity is added on to the heading direction. The angular velocity is also clamped to a maximum value to make up for unrealistic steering at high speeds.

```
# Update position
self.position += self.velocity * dt

# Update direction based on bicycle model
speed_m = speed / game_core.meter_pixel_conversion      # Speed in m/s
if speed_m > 0.1:
    # Calculate clamped angular velocity
    max_angular_velocity = self.MAX_LATERAL_ACCELERATION / speed_m
    angular_velocity = max(-max_angular_velocity,
                           min(max_angular_velocity,
                                   (math.tan(self.steer) * speed_m) / self.WHEELBASE))

    # Update direction
    self.direction += math.degrees(angular_velocity) * dt
```

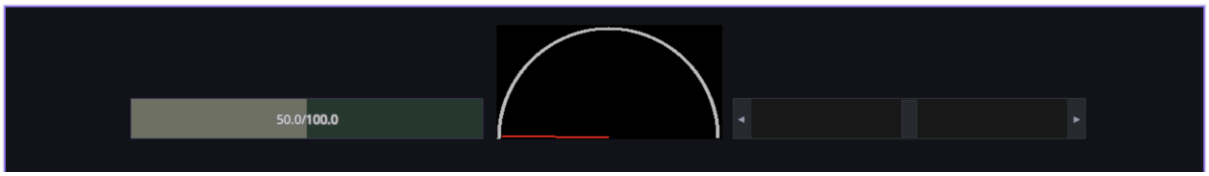
To test how well this works, I temporarily made it so that the path of the car can be traced by holding q.

```
    if pygame.K_q in game_core.pressed_keys:
        pygame.draw.circle(track.image, color="red", center=self.position, radius=1)
```

Output

Turning when accelerating -> Turning radius increases as speed (and angular velocity) increases

Turning at low speeds -> small turning radius



Tracking Progress:

To track the cars progress through the track, I find the closest point of the track spine to the car. By finding the index of this track spine point in the list of spine points, and finding the ratio to the number of points, I can approximate how far along the track the car is. This is based on the idea that the progress of the car along the track is the same as this ratio.

```
def update_and_get_progress(self, track_spine):  2 usages (2 dynamic)  joxnrepresent *
    """
    Calculate car's progress along track and detect lap completion.

    Finds nearest point on track spine to calculates fractional progress (0-1). Handles wraparound at track
    boundaries, and detects lap completion or wrong-way driving.

    Args:
        track_spine (list[Vector2]): Ordered list of track centerline points

    Returns:
        tuple(float, bool): (cumulative_progress, is_lap_finished)
            - cumulative_progress: Total laps + fractional progress (e.g., 2.5)
            - is_lap_finished: True if lap boundary was just crossed
    """

    # Calculate fractional progress by finding the nearest track spine point
    center = Vector2(self.rect.center)
    closest_point = min(track_spine, key=lambda point: (center - point).length_squared())
    current_progress = track_spine.index(closest_point) / (len(track_spine) - 1)

    # Handle wrap-around by checking change in progress
    change_in_progress = current_progress - (self.progress % 1)
    if change_in_progress > 0.5:
        change_in_progress -= 1.0
    elif change_in_progress <= -0.5:
        change_in_progress += 1.0

    # Update progress
    new_progress = self.progress + change_in_progress
```

Since the progress update code implicitly handles the wraparound when cross the start/finish line boundary, I implemented a separate flag to indicate if a wraparound has occurred and if a car has finished a lap. This is useful for lap timing and for rewarding AI car behaviour. Additionally, if the car drives in the opposite direction along the track, I also handle resetting the car back to the start line, to prevent it from finishing laps in the opposite direction.

```
is_lap_finished = False
if new_progress > self.progress:
    # Check if lap was completed
    if math.floor(new_progress) - math.floor(self.progress) > 0:
        print("lap completed!")
        self.progress = new_progress
        is_lap_finished = True
    else:
        self.progress = new_progress
else:
    # Check if car is travelling wrong way
    if new_progress <= -0.5:
        print("Wrong way! Please reset")
        self.is_crashed = True
        self.progress = 0
    else:
        self.progress = new_progress

return self.progress, is_lap_finished
```

To test this, I added temporary progress printing and drove 2 laps of the track.

```
        print(self.progress)
        return self.progress, is_lap_finished
```

Start of lap 1:

```
0.0
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.0029069767441860465
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.005813953488372093
0.00872093023255814
0.00872093023255814
0.00872093023255814
0.00872093023255814
```

End of Lap 1 and Start of Lap 2

```
0.9563953488372093
0.9593023255813954
0.9622093023255814
0.9651162790697675
0.9680232558139535
0.9680232558139535
0.9709302325581395
0.9738372093023255
0.9767441860465116
0.9796511627906976
0.9796511627906976
0.9825581395348837
0.9854651162790697
0.9883720930232558
0.9883720930232558
0.9912790697674418
0.997093023255814
lap completed!
1.0
1.002906976744186
1.005813953488372
1.005813953488372
1.0087209302325582
1.0116279069767442
1.0145348837209303
1.0145348837209303
1.0174418604651163
1.0174418604651163
1.0203488372093024
```

End of Lap 2

```
1.9389534883720931
1.941860465116279
1.9447674418604652
1.947674418604651
1.947674418604651
1.9505813953488373
1.9534883720930232
1.9563953488372094
1.9593023255813953
1.9622093023255816
1.9651162790697674
1.9680232558139537
1.9709302325581395
1.9709302325581395
1.9767441860465116
1.9767441860465116
1.9796511627906976
1.9796511627906976
1.9825581395348837
1.9854651162790697
1.9854651162790697
1.9883720930232558
1.9912790697674418
1.994186046511628
1.997093023255814
lap completed!
2.0
```

Car Hitbox:

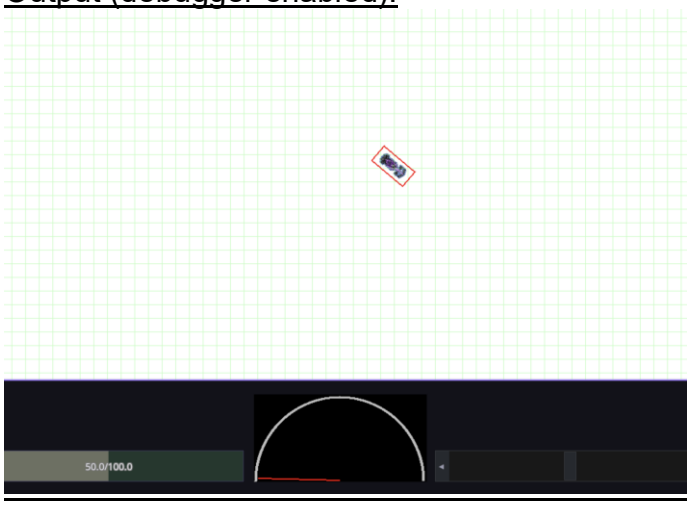
As mentioned during prototyping, the Car sprite “rect” attribute cannot directly be used as a hitbox for collision detection since it is an axis aligned bounding box. To create a hitbox that properly encapsulates the car and rotates as the heading direction of the car changes, I implemented an algorithm that identifies the 4 corners of the car. The hitbox is then the rectangle that connects these 4 points. Since I have already implemented optimised dda traversal methods using the Spatial hash grid for line segment intersection checks, I decided to apply collision detection of the hitbox

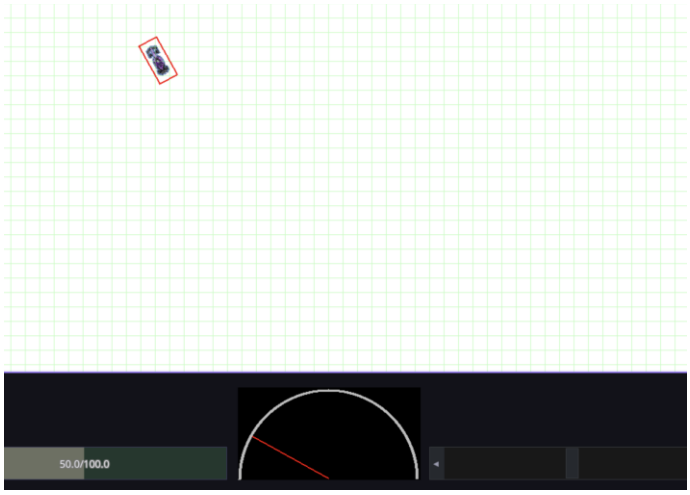
as 4 individual lines instead of a single rectangle. Although this means that 4 separate collision checks need to be done, the speedup provided by the dda traversal still makes this method optimal.

```
def _update_hitbox(self):  3 usages  joxnrepresent +1 *
    """
    Recalculate hitbox corners based on current position and rotation.
    """
    center_x, center_y = self.rect.center
    width, length = self.car_proportions / 2
    width -= 5
    corners = np.array([(-width, length), (width, length), (width, -length), (-width, -length)])
    angle = math.radians(self.direction)
    rotation_matrix = np.array([(math.cos(angle), -math.sin(angle)), (math.sin(angle), math.cos(angle))])
    rotated_corners = corners @ rotation_matrix.T
    rotated_corners[:, 0] += center_x
    rotated_corners[:, 1] += center_y

    self.hitbox[:] = rotated_corners.tolist()
```

- The corners of the hitbox are represented in a vector
- Matrix multiplication with rotation matrix performs linear transformation of rotation on all points
- Current position of each point is updated

Output (debugger enabled):





Car Polymorphism:

There are 2 different type of cars that can be simulated in the project: a player car and an AI car. To enable different game modes to handle updating cars each frame in a modularised manner, I implemented polymorphism. The PlayerCar and AICar classes inherit from the Car base class and override the update method to handle their specific logic.

The PlayerCar is updated by getting the throttle and braking input from user input. It also updates the on-screen GUI elements relating to the car (speedometer, throttle meter and steering slider (if enabled). The AICar update method takes the target actions as arguments (target actions are determined from AI model output). It also has methods for ray casting (sensor data) and reward computation.

The different car types also override the reset() method, since they have different on-screen components that need to be reset when the car is reset. For the PlayerCar, resetting car must also reset all GUI elements (speedometer, throttle meter, steering slider).

PlayerCar update:

```
def update(self):  joxnrepresent
    """
    Handles car movement and control panel updates
    """
    self.drive_car(self._handle_steering(), self._handle_throttle())
    self.throttle_and_braking_meter.set_current_progress((self.current_throttle_input+1)*50)
    self.speedometer.update_value(self.velocity.magnitude())
```

PlayerCar reset:

```
def reset(self):  joxnrepresent
    """
    Resets car to starting state.
    Re-initialises the HUD control panel
    """
    super().reset()
    self.steering_slider.kill()
    self.throttle_and_braking_meter.kill()
    self.speedometer.kill()
    self.throttle_and_braking_meter =UIProgressBar(
        relative_rect=pygame.Rect((game_core.screen_dimensions[0] / 4 - 200, 100), (400, 50)),
        manager=game_core.gui_manager,
        container= self.hud_panel,
        object_id = '#UIProgressBar'
    )

    self.steering_slider = UIHorizontalSlider(
        relative_rect=pygame.Rect((game_core.screen_dimensions[0] *3/4 - 200, 100), (400, 50)),
        start_value=0,
        value_range=(-100, 100),
        manager=game_core.gui_manager,
        container= self.hud_panel,
        object_id = '#UIHorizontalSlider'
    )
    self.is_slider_enabled = False
    self.steering_slider.disable()

    self.speedometer = UIGaugeMeter(
        relative_rect=pygame.Rect((game_core.screen_dimensions[0]/2  - 125 , 20), (250, 125)),
        manager=game_core.gui_manager,
        container= self.hud_panel
    )
```

AICar update:

```
def update(self, target_actions):  joxnrepresent
    """
    Handles car movement. Car is controlled by actions output by the RL model

    Args:
        target_actions (tuple(float, float)): (target_throttle, target_steer) with both values [-1, 1]
    """
    self.drive_car(*target_actions)
```

2.3.5 Lap Timer

At the end of a Car simulation run, I want to display the user their individual lap times and the total time taken to finish the laps they set. To do this, I created a RaceTimeManager class that handles recording lap times.

```
class RaceTimeManager:  3 usages  joxnrepresent
    def __init__(self):  joxnrepresent
        self.start_time = None
        self.is_all_laps_finished = [False]
        self.lap_times = [[None]]
        self.num_of_laps = 0

    def initialise_manager(self, num_of_cars):  2 usages  joxnrepresent
        self.start_time = game_core.sim_time_ms
        self.is_all_laps_finished  = [False] * num_of_cars
        self.lap_times = [
            [None for _ in range(self.num_of_laps)]
            for _ in range(num_of_cars)
        ]
```

- Start time is set when the manager is initialised. This is subtracted from all lap times to find the durations of each lap
- Lap times are organised in a list ordered [car_index][lap_index]

If lap is completed, then the lap time for the given car and lap number is updated

```
    def update(self, i, laps_completed, is_lap_completed):  joxnrepresent
        if is_lap_completed:
            self.lap_times[i][laps_completed - 1] = game_core.sim_time_ms
            if laps_completed >= self.num_of_laps:
                self.is_all_laps_finished[i] = True
```

I already have a print statement in progress computation to record whenever a car finishes a lap. To test lap timer, I added additional print statement at the lap timer update to print whenever the time manager values is updated. There are 12 cars being simulated, and I decided to simulate 3 laps.

Output:

```
lap completed!
Lap time updated: Car6, lap1
Lap time (ms): 2867.9999999999736

lap completed!
Lap time updated: Car7, lap1
Lap time (ms): 2874.799999999973

lap completed!
Lap time updated: Car8, lap1
Lap time (ms): 2908.7999999999693

lap completed!
Lap time updated: Car9, lap1
Lap time (ms): 3001.4999999999727

lap completed!
Lap time updated: Car1, lap1
Lap time (ms): 3174.399999999975

lap completed!
Lap time updated: Car6, lap2
Lap time (ms): 4887.799999999951

lap completed!
Lap time updated: Car7, lap2
Lap time (ms): 4956.199999999954

lap completed!
Lap time updated: Car1, lap2
Lap time (ms): 5241.399999999948

lap completed!
Lap time updated: Car6, lap3
Lap time (ms): 6948.200000000066

lap completed!
Lap time updated: Car7, lap3
Lap time (ms): 7080.300000000047

lap completed!
Lap time updated: Car1, lap3
Lap time (ms): 7204.400000000031
```

- Lap times are updated every time a lap is completed
- Lap times are updated for the correct car and correct lap number
- Print statements show increasing lap times, which means timer is updated in the correct order.

2.3.6 Car Simulation

The abstract base CarSimulation class is an abstract class that handles the basic functionality of a car simulation. The individual simulation modes (SoloCarSim, RacingAI and AITrainingEnvironment) all extend this and apply their own unique set of functionalities. The functions carried out by the base CarSimulation class are:

- Load and initialise track
- Initialise number of laps and the race time manager
- Handle environment updates (update cars, race timer)

- Handle environment reinitialization

Track and number of laps initialisation:
To initialise a track, a new track loader object is created to allow the user to select a file. The set_track() method then loads the track data from the file with the help of the load_bezier_track() helper method in the resources folder. A new instance of the Track class is initialised with this data.

```
def initialise_track(self):  1 usage   joxnrepresent
    """Open file selector for track loading."""
    self.track_loader = UIFileBrowser(pygame.Rect((0, 0), game_core.screen_dimensions),
                                      self.gui_manager,
                                       directory: "Tracks",
                                       mode: "load",
                                      callback=lambda filename: self.set_track(filename),
                                      return_mode= MainMenu)


def set_track(self, filename):  4 usages   joxnrepresent
    """
    Load and initialise track from file. Loads bezier curve and width data and creates Track object.
    Returns to main menu if error.

    Args:
        filename (str): Track file to load (without extension)
    """
    try:
        anchors, controls, widths = r.load_bezier_track(filename, enforce_centering=True)
        self.track = Track(anchors, controls, widths)
        self.game_sprites.add(self.track)
    except Exception as e:
        r.game_core.unexpected_error_msg = f"Error occurred when loading {filename}: (File is likely corrupted)"
        r.game_core.set_game_mode(MainMenu)
        print("Track load failed:", e)
```

The number of laps is initialised in a similar way, by initialising the num_of_laps_setter UIOptionSelector instance, which sets the num_of_laps attribute.

```
def initialise_num_of_laps(self):  3 usages   joxnrepresent
    """Open dialog for selecting number of laps."""
    self.num_of_laps_setter = UIOptionSelector(pygame.Rect((0, 0), game_core.screen_dimensions),
                      self.gui_manager,
                      title= "Select number of laps",
                      options =["1", "2", "3", "4", "5", "6", "7", "8", "9", "10"],
                      callback=lambda num_of_laps: self.set_num_of_laps(num_of_laps),
                      return_mode=MainMenu)

def set_num_of_laps(self, num_of_laps):  1 usage   joxnrepresent
    """
    Set race length and initialise cars.

    Args:
        num_of_laps (str): Number of laps
    """
    self.time_manager.num_of_laps = int(num_of_laps)
    self.time_manager.initialise_manager(self.num_of_cars)
    self.init_cars()
```

Update method:

The base CarSimulation update method iterates through all cars, checks for collisions and updates the crashed flags of each car. It also updates the progress of each car, and the race time manager lap times.

```
def update(self):  joxnrepresent
    """
    Updates state of all active cars:

    Calls collision detection
    Updates progress
    check for race completion and updates time manager on lap completions
    Skips updates if paused or race finished.
    """
    self.gui_manager.update(1 / game_core.FRAME_RATE)

    if game_core.is_paused or self.end_screen is not None:
        return

    if self.track:
        self.track.update()
    for i, car in enumerate(self.cars):

        # Skip crashed cars
        if self.is_crashed[i] or self.time_manager.is_all_laps_finished[i]:
            continue

        car.collision_detection(self.track)
        if car.is_crashed:
            self.is_crashed[i] = True

        progress, is_lap_completed = car.update_and_get_progress(self.track.track_spine)
        laps_completed = math.floor(progress)
        self.time_manager.update(i, laps_completed, is_lap_completed)

    # If all laps finished, show end screen
    if all(self.time_manager.is_all_laps_finished):
        results = self.get_results()
        self.display_end_screen(results)
        return
```

Simulation Types

To enable different types of car simulation environements, I created SoloCarSim, RacingAI and AITrainingEnvironment classes that inherit the CarSimulation base class. This implements multi-level inheritence to have greater modularity. These different modes override the init_cars(), display_end_screen(), reset_gui(), reset_environement() and reset_cars() methods to implement their specific logic

Abstract methods:

```python
@abstractmethod  2 usages  joxnrepresent
def init_cars(self):
    """Initialise cars. Child classes must implement."""
    pass

@abstractmethod  1 usage  joxnrepresent
def display_end_screen(self, results):
    """
    Handle end screen display of race statistics. Child classes must implement this

    Args:
        results (dict): Race statistics from time manager
    """
    pass

@abstractmethod  1 usage  new *
def reset_gui(self):
    """Reset GUI elements. Child classes must implement this."""
    pass

@abstractmethod  2 usages  new *
def reset_environment(self):
    """Reset time manager for new race."""
    self.time_manager.initialise_manager(self.num_of_cars)

@abstractmethod  1 usage  new *
def reset_cars(self):
    """Reset all cars to starting positions and progress and clear crash states."""
    for car in self.cars:
        car.reset()
    self.is_crashed = [False] * self.num_of_cars
```

2.3.7 Neural Network

To create an AI agent that can learn to drive the car around the track, I will use reinforcement learning algorithms that train neural network. The sensor data of the AI car is what the model gets as input that allows it to observe the environment (which is the track).

I have designed my own neural network class using only numpy for optimised matrix operations. The neural network is composed of ‘layers’, which consists of ‘nodes’ which perform linear operations. Each layer receives an input vector ‘x’ and has an associated weights ‘W’ and a bias ‘b’. The weights and biases of the network are the trainable parameters of the network and are what is optimised through training. There are also other non-trainable parameters (known as hyperparameters) which are fine-tuned outside of training.
The process by which a Neural network takes in an input and performs some computation to determine the output is known as the “forward pass” or “forward propagation”. The input is in the form of an “input vector” which is passed into the input layer. Each layer receives an input and computes an output which is passed on to the next layer. At the output layer, the output is the final output of the network. At each node, linear operations compute the output ‘z’. An activation function (e.g. Tanh, ReLu) is applied to ‘z’ to produce the activation output ‘a’ which is the input to the next node

2.3.8 Layers

To optimise computation, I have decided to compute the linear operations at each node as a group vectorised operation at each layer. This allows for the use of numpy arrays, which allow for faster calculations.
To enable this, the input vector and biases are stored as vectors and the weights are stored in a matrix.

Forward pass

Forward pass is performed by taking the dot product of the input and weights and then adding the bias vector (z = x.W + b), then passing it through activation function to get the layer output (a = ϕ(z)). The size of the output vector is the same as the number of nodes in the next layer, since the output of one layer is the input to the next layer. In this way, data is passed through the network, and the output of the final layer is returned.

```
42      def layer_forward_pass(self, x):  1 usage   joanmk
43          z = x.dot(self.weights) + self.biases
44          a = self.activation(z)
45          self.last_x = x
46          self.last_z = z
47          self.last_a = a
48          return a
```

Backward pass:

The purpose of the backward pass is to compute the gradient of the advantage function “A” w.r.t the weights and biases at each layer (dA/dW and dA/db). These gradients determine the direction and magnitude of the update applied to the parameters.
To calculate the gradients, chain rule is used to differentiate the advantage function w.r.t the weights and biases respectively

Gradient Calculation	Formula
Gradient w.r.t. pre-activation	$\frac{\partial A}{\partial \mathbf{z}} = \frac{\partial A}{\partial \mathbf{a}} \odot \frac{\partial \mathbf{a}}{\partial \mathbf{z}}$
Gradient w.r.t. Weights	$\frac{\partial A}{\partial W} = \frac{1}{M}(X^T \cdot \frac{\partial A}{\partial \mathbf{z}})$
Gradient w.r.t. Biases	$\frac{\partial A}{\partial b} = \text{mean}(\frac{\partial A}{\partial \mathbf{z}}, \text{axis} = 0)$
Gradient w.r.t. Input	$\frac{\partial A}{\partial X} = \frac{\partial A}{\partial \mathbf{z}} \cdot W^T$

In addition to the numerical gradients, I have also decided to incorporate an architecture that allows for batch processing of data. Hence, vectors are computed as 2D arrays, which allows me to simulate multiple agents simultaneously to allow better learning.

```
50      def layer_backward_pass(self, d_mu):  1 usage   joanmk +1
51          batch_size = d_mu.shape[0]                              #d_mu is dL/d_mu
52          d_z = d_mu * self.activation_derivative(self.last_z) #dL/d_z = dL/d_mu * d_mu/d_z
53          d_W = self.last_x.T.dot(d_z) / batch_size                    #dL/d_W = x.T * dL/d_z
54          """x.t has shape (input_size, batch_size)
55          d_z has shape (batch_size, output_size)
56          So x.T.dot(d_z) will have shape (input_size, output_size), same as weights"""
57          d_b = d_z.mean(axis=0)                                  #dL/d_b = mean of dL/d_z
58          d_x = d_z.dot(self.weights.T)                           #dL/d_x = dL/d_z * W.t
59
60          return d_W, d_b, d_x
```

- d_x is also computed to be passed to the previous layer, since this is the same as the gradient of the output of the previous layer.

Activation Functions:

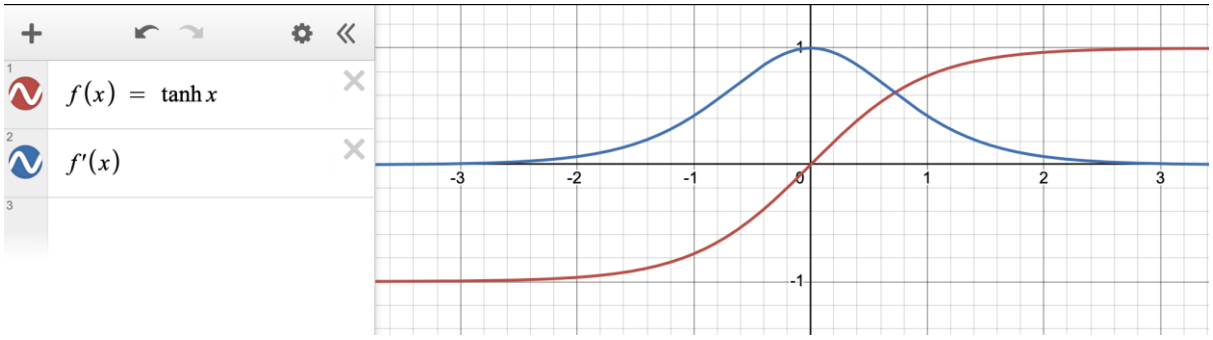
Since neural networks are universal function approximators, it is essential that the network can model non-linear patterns to allow it to model a driving car. However, the operations at nodes are all linear operations. Therefore, to introduce non-linearity to the computation, the output of each layer is wrapped in an activation function, which simply maps the output onto a new non-linear function (unless the activation function itself is linear). I have decided to implement 2 non-linear activation functions I can choose from.

```
def activation(self, z):  4 usages   joanmk +1
    if self.activation_name == 'relu':
        return np.maximum(0, z)
    if self.activation_name == 'elu':
        return np.where(z > 0, z, np.exp(z) - 1)
    elif self.activation_name == 'linear':
        return z
    elif self.activation_name == 'tanh':
        return np.tanh(z)
```

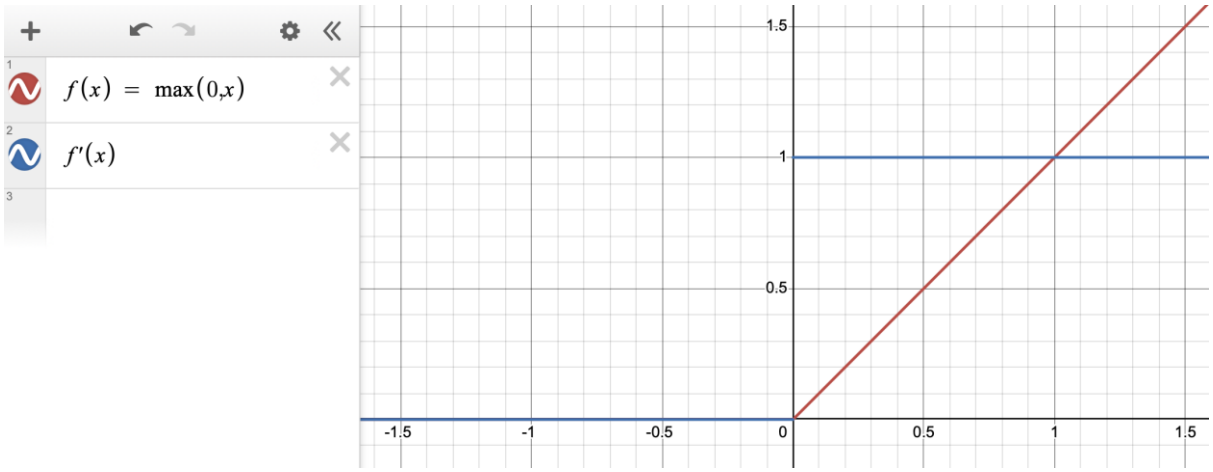
In addition to being non-linear, it is also important for the functions to be continually differentiable. This is because to calculate derivatives in backward pass, it is also necessary to differentiate with respect to the activation function to apply chain rule. Hence the respect derivatives for the activation functions are as follows:

```
def activation_derivative(self, z):  1 usage   joanmk +1
    if self.activation_name == 'relu':
        return (z>0).astype(float)
    elif self.activation_name == 'elu':
        return np.where(z > 0, 1, np.exp(z))
    elif self.activation_name == 'linear':
        return np.ones_like(z)
    elif self.activation_name == "tanh":
        return 1 - np.tanh(z) ** 2
```

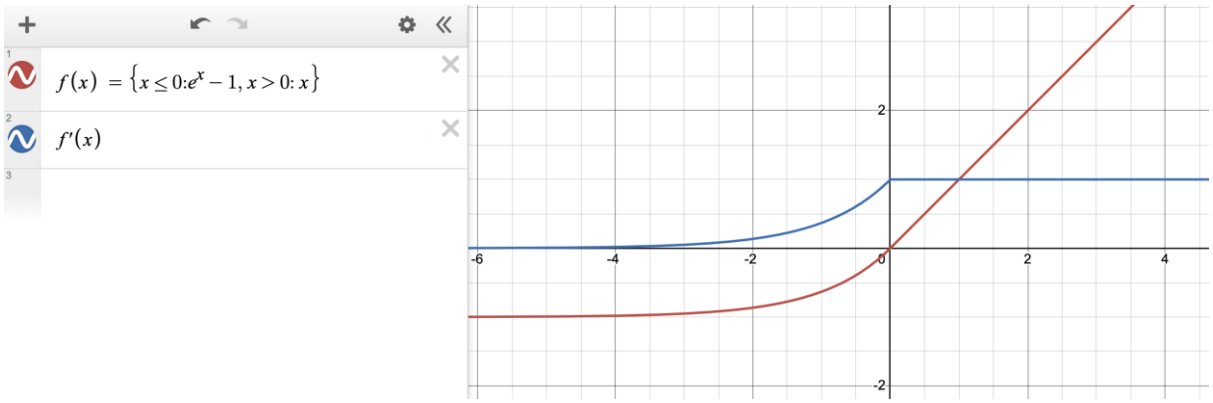
Tanh:



ReLu:



Elu:



2.3.9 Neural Network Class:

Initialising weights:
At the start of training, the weights and biases are initialised to some starting values. The biases are initialised to an array of zeros, while the weights are assigned random values. However, it is important that while weights are initialised randomly, a good initialisation can improve learning stability and convergence speed by avoiding vanishing/exploding activations and gradients.

```
17
18      def initialise_weights(self):  1 usage  joanmk *
19          if self.activation_name == 'relu':
20              # He initialisation
21              sigma = np.sqrt(2/self.input_size)
22              return np.random.randn(self.input_size, self.output_size) * sigma
23          else:
24              # Xavier initialisation
25              limit = np.sqrt(6/(self.input_size +self.output_size))
26              return np.random.uniform(-limit, limit,  size: (self.input_size, self.output_size))
27
```

He initialisation: This is a method of weights initialisation that is specifically tailored to the ReLu activation function. It addresses the problem of “dying” nodes, caused by nodes being permanently inactive if the input tends to the negative direction.

Xavier initialisation: This weight initialisation technique helps to prevents gradients of updates from exploding or vanishing. It works by drawing weights from a normal distribution with mean as 0 and a variance that depends on the number of input and output nodes. This helps to stabilise the neural network.

Network forward propagation:
The process of computing actions at each time step beings with plugging the input vector into the neural network of the network. The neural network performs forward propagation and returns the output.

```
    def forward_propagation(self, a):  2 usages  joanmk +1
        # a -> input to next layer
        activation_outputs = [a]
        for layer in self.layers:
            a = layer.layer_forward_pass(a)
            activation_outputs.append(a)
        return a, activation_outputs
```

- Iterates through all layers, passing the output of one layer as the input to the next
- Finally returns the output of the final layer

Updating Parameters:
To update the parameters of the neural network (weights and biases), we must compute their gradients w.r.t the advantage function. The gradient of the output (μ) w.r.t the advantage is passed to the network the RL model. This gradient is then passed through the layers in backpropagation to compute all gradients.

```
101
102     def compute_gradients(self, d_mu):  1 usage   joanmk +1
103         weight_grads = []
104         bias_grads = []
105         for layer in reversed(self.layers):
106             d_W, d_b, d_mu = layer.layer_backward_pass(d_mu)
107             weight_grads.insert( __index: 0, d_W)
108             bias_grads.insert( __index: 0, d_b)
109         return weight_grads, bias_grads
```

- Iterates through all the neural network layers in reverse and gets respective gradients for weights and biases.

2.3.10 Adaptive Moment Estimation (Adam)

The Adam optimiser is a technique that helps the model converge quicker by dynamically adjusting learning rates during training. This technique combines the benefits of Momentum and RMSProp techniques.
With Vanilla stochastic gradient ascent, the weights are updated directly by the gradient with respect to the advantage function. The model essentially "climbs uphill" along the advantage function to reach the globally maximum value of the advantage. The maximum value of the advantage corresponds to the best possible policy that acts optimally in the environment.
Adam optimiser implements 2 optimisation "moments" that speed up the hill climbing process (in mathematics, 'moment' refers to an expected value). The moments are represented by m_t and v_t.

<u>First Moment - Momentum:</u>
The momentum moment works similarly to how momentum works in physics. It accumulates velocity when consecutive gradient updates occur in the same direction. This smoothens updates by reducing oscillations and speeds up movement in consistent direction.

The equation for the momentum moment is given by:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1).\text{grad}$$

<u>Second Moment – Adaptive learning rate (or velocity):</u>
This moment tracks the average squared gradient.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) grad^2$$

This means if the gradients are large, this term becomes large. Since the final update term is divided by this moment, this has the effect of having a smaller learning rate when recent gradients have been big. This means that each weight ends up having its own adaptive learning rate based of how large the gradient updates of that weight have been.

Bias Correction:

Using these 2 moments, the gradient in the update term is now replaced by $\frac{m_t}{\sqrt{v_t}}$ (second moment is square rooted to counteract the squaring the gradient.)

Since both moments are initialised to 0, this means that during early learning, these moments are severely downscaling gradients, since these calculations assume there is already a history of momentum and gradient square calculations. For example, at t = 0, if the gradient is 10, and both β values are 0.9,
$m_t = 0.9 * 0 + (1 - 0.9) * 10 = 0.1$

$v_t = 0.9 * 0 + (1 - 0.9)10^2 = 10$

And the update term becomes $\frac{0.1}{10} = 0.01$. So, the gradient in the first-time step has gone 1000 times smaller. Such high down scaling would completely collapse early-stage learning. To counteract this, we introduce 2 new bias correction terms, $\hat{m}$ and $\hat{v}$, which are defined as:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

This correction has a big impact when for early time steps, and negligible impact for later time steps. So, it ensures accurate estimations during early training without having impact on later training

The final update equation now becomes:

$$\theta = \theta + \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Where ϵ is just a very small constant to avoid division by 0 errors.

This overall replacement of the grad term with this Adam estimation term significantly reduces instability caused by high variance in gradients and allows for faster convergence.

Formula in code:

```
@staticmethod  3 usages  joanmk +1 *
def adam_estimation(m, v, grad, t, alpha, beta1, beta2):
    m = beta1 * m + (1 - beta1) * grad
    v = beta2 * v + (1 - beta2) * grad ** 2
    m_hat = m / (1 - beta1 ** t)
    v_hat = v / (1 - beta2 ** t)
    update_value = -alpha * m_hat / (np.sqrt(v_hat) + 1e-8)
    return (m, v), update_value
```

To apply the Adam moment estimation, the previous moment values need to be cached so that they can be applied to calculate update terms when updating parameters at the end of each episode. I store the Adam values in a dictionary, with keys mapping to the moments for the weights ‘w’, the biases ‘b’, the log_std and the time step ‘t’.

```
#Adaptive Moment (Adam) Optimiser
"""
1st moment - mean
2nd moment - variance
"""
self.adam_moments = {'w':[], 'b':[],
                     'log_std':(np.zeros_like(self.log_std), np.zeros_like(self.log_std)), 't':0}
for layer in self.layers:
    self.adam_moments['w'].append((np.zeros_like(layer.weights), np.zeros_like(layer.weights)))
    self.adam_moments['b'].append((np.zeros_like(layer.biases), np.zeros_like(layer.biases)))
```

When updating parameters, the weight update and bias update terms are computing through the Adam estimation, and the Adam moments dictionary is updated. After all layer parameters are updated, the log_std is also updated in a similar way

```
def update_params(self, d_mu, d_log_std = None):  4 usages (1 dynamic)  joxnrepresent +1
    # Adam optimisation
    # w_moments, b_moments, log_stds_moments, t = self.adam_moments.values()
    weight_grads, bias_grads = self.compute_gradients(d_mu)

    self.adam_moments['t'] += 1
    for i, layer in enumerate(self.layers):
        self.adam_moments['w'][i], weight_update = self.adam_estimation(self.adam_moments['w'][i][0],
                                                                        self.adam_moments['w'][i][1],
                                                                        weight_grads[i],
                                                                        self.adam_moments['t'],
                                                                        self.alpha,
                                                                        self.beta1,
                                                                        self.beta2)

        self.adam_moments['b'][i], bias_update = self.adam_estimation(self.adam_moments['b'][i][0],
                                                                      self.adam_moments['b'][i][1],
                                                                      bias_grads[i],
                                                                      self.adam_moments['t'],
                                                                        self.alpha,
                                                                        self.beta1,
                                                                        self.beta2)
        layer.weights += weight_update
        layer.biases += bias_update

    if d_log_std is not None:
        self.adam_moments['log_std'], log_std_update = self.adam_estimation(self.adam_moments['log_std'][0],
                                                                            self.adam_moments['log_std'][1],
                                                                            d_log_std,
                                                                            self.adam_moments['t'],
                                                                            self.alpha,
                                                                            self.beta1,
                                                                            self.beta2)

    if d_log_std is not None:
        self.log_std += log_std_update
        self.log_std = np.clip(self.log_std, -3,  a_max: 1)
```

2.3.11 Stochastic Policy Gradient Reinforcement Learning Model

I decided to implement stochastic policy gradient as a heuristic technique to approximate the most optimal policy for driving around a track. The algorithm I implement uses gradient ascent to hill-climb and improve the value of an "advantage" function. The advantage function is the product of the log probability of the action taken and the Monte Carlo returns for that action.
The basic workflow of a stochastic policy gradient algorithm implemented here is as follows.
Forward propagation:

- State vector is passed into the actor network for forward propagation
- The output of the neural network is the 'μ' (mean or peak) value of a gaussian distribution
- The 'σ' (standard deviation or randomness) parameter is obtained from the log_std parameter of the network
- Pre-squash actions are computed and cached by adding the generated noise to the output value
- Actions are obtained by clipping pre-squash actions into the domain of [-1, 1] (Since the target throttle and steer input values are limited within this range)
- Trajectory (state, pre squash actions and reward at each time step) of episode is stored and actions are returned for the agent to act on

Backpropagation:

- Stored trajectory is retrieved from memory
- Stored states are stacked into single matrix and passed through actor again to recompute predicted 'μ' values
- Stored pre squash actions, 'μ' values and the 'σ' value are used to compute the probabilities of the different actions taken under the current policy
- The Monte Carlo returns (G_t) are computed from the stored rewards in trajectory
- The model uses the Monte Carlo returns and the computed probabilities to estimate the value of the advantage function (the way this is done depends on what algorithm is used).
- This advantage function is passed through the layers in the backward pass. The gradients of this advantage w.r.t the individual parameters in computed as shown above.
- The adam optimiser is applied along with these gradients to update all parameters.

The exploration of having a random element to the way actions are computed for a given state is a critical component of stochastic policy gradient methods of reinforcement learning. Policy gradient essentially works by predicting a certain set of actions, and then through exploration, measures how much better or worse nearby actions would have been. This exploration allows for the algorithm to compute gradients of updates in the direction of the actions that were "better" at each stage.

At its core, the objective function of stochastic policy gradient is to maximise the expected return over an episode when picking actions according to the current policy.

$$J(\theta) = E_{\tau \sim \pi}[R(\tau)]$$

Where J(θ) is the objective function.
Since the policy parameters (neural network weights) are updated by their gradient with respect to the objective function, we can directly calculate this gradient rather than the function itself using the policy gradient theorem (derivation and proof on page 22).

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot R(\tau)]$$

The term $\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot R(\tau)$ is called the policy gradient estimator. The term describes which direction the parameters need to be updated.

To solve this optimisation problem, RL usses heuristics to run episodes in batches to estimate the value of the objective some number of trajectories. This allows us to compute a numerical value of the this objective function

Computing actions from neural netowork
The output of the neural network isnt the action taken by the agent. It is the parameters of a Gaussian probability distribution from which the actions are sampled. The reason for this is if the network directy outputs actions, the agent always takes the same action for the same state. Random Gaussian sampling of actions is what enables stochastic policy gradient to compute a numerical value for the term $\nabla\theta \log \pi\theta(a_t \mid s_t)$, and therefore carry out gradient updates.

```
19      def get_stochastic_actions(self, state_vector):  1 usage
20          # x: input vector
21          x = np.array(state_vector, np.float32)
22          if x.ndim == 1:
23              x = np.expand_dims(x, axis=0)
24
25          mu = self.actor.forward_propagation(x)
26          sigma = np.exp(self.actor.log_std)
27          noise = np.random.randn(*mu.shape) * sigma
28          pre_squash = mu + noise
29          actions = np.clip(pre_squash, -1,  a_max: 1)
30          return actions, pre_squash
```

Computing gradients:
In order to carry out gradient updates, we must compute a numeric value for the term $\nabla_\theta J(\pi_\theta)$. The first part of this is to calculate the gradients of the parameters θ w.r.t the log probabilities. This is the $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ term. The second part is the action value function estimate, which is linked to the estimated return along the

trajectory R(τ). In practice, this is done by calculating the advantage function. At each step, the advantage value is A_t . The way this is calculated depends on the algorithm implemented. The following is the derivation of this value for a stochastic gradient policy implementing Gaussian sampling like I have done:

Let one Trajectory (τ) be :-

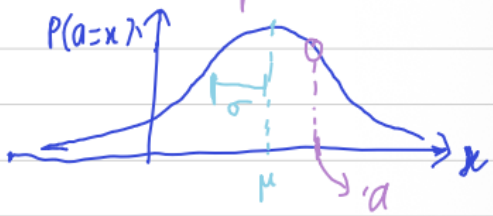
$$(s_0, s_1, s_2 \ldots\ldots s_T)$$
$$(a_0, a_1, a_2 \ldots\ldots a_T)$$
$$(r_0, r_1, r_2 \ldots\ldots r_T)$$

Where s_k ← State at time step k
a_k ← Action taken in s_k at time step k.
r_k ← Reward recieved from taking a_k in s_k.

Output from following policy π_θ (Output from neural network)
↳ mean action (μ), gaussian noise (σ)

Actions are obtained by randomly sampling from a Gaussian distribution with these parameters.

$a \sim N(\mu, \sigma^2)$
↳ Gaussian dist



Probability of 'a' taking a particular value :

$$P(a \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \times e^{\left(\frac{-(a-\mu)^2}{2\sigma^2}\right)}$$

↳ Definition of Gaussian dist.

$$\Rightarrow \log P(a \mid \mu, \sigma) = \log\left((2\pi\sigma^2)^{-1/2} e^{\left(\frac{-(a-\mu)^2}{2\sigma^2}\right)} \right)$$

↳ $\log P = \log_e P$

$$= \log\left((2\pi\sigma^2)^{-1/2}\right) + \log\left(e^{\left(\frac{-(a-\mu)^2}{2\sigma^2}\right)}\right)$$

$$= -\frac{1}{2}\log(2\pi\sigma^2) + \frac{-1}{2}\left(\frac{(a-\mu)^2}{2\sigma^2}\right)$$

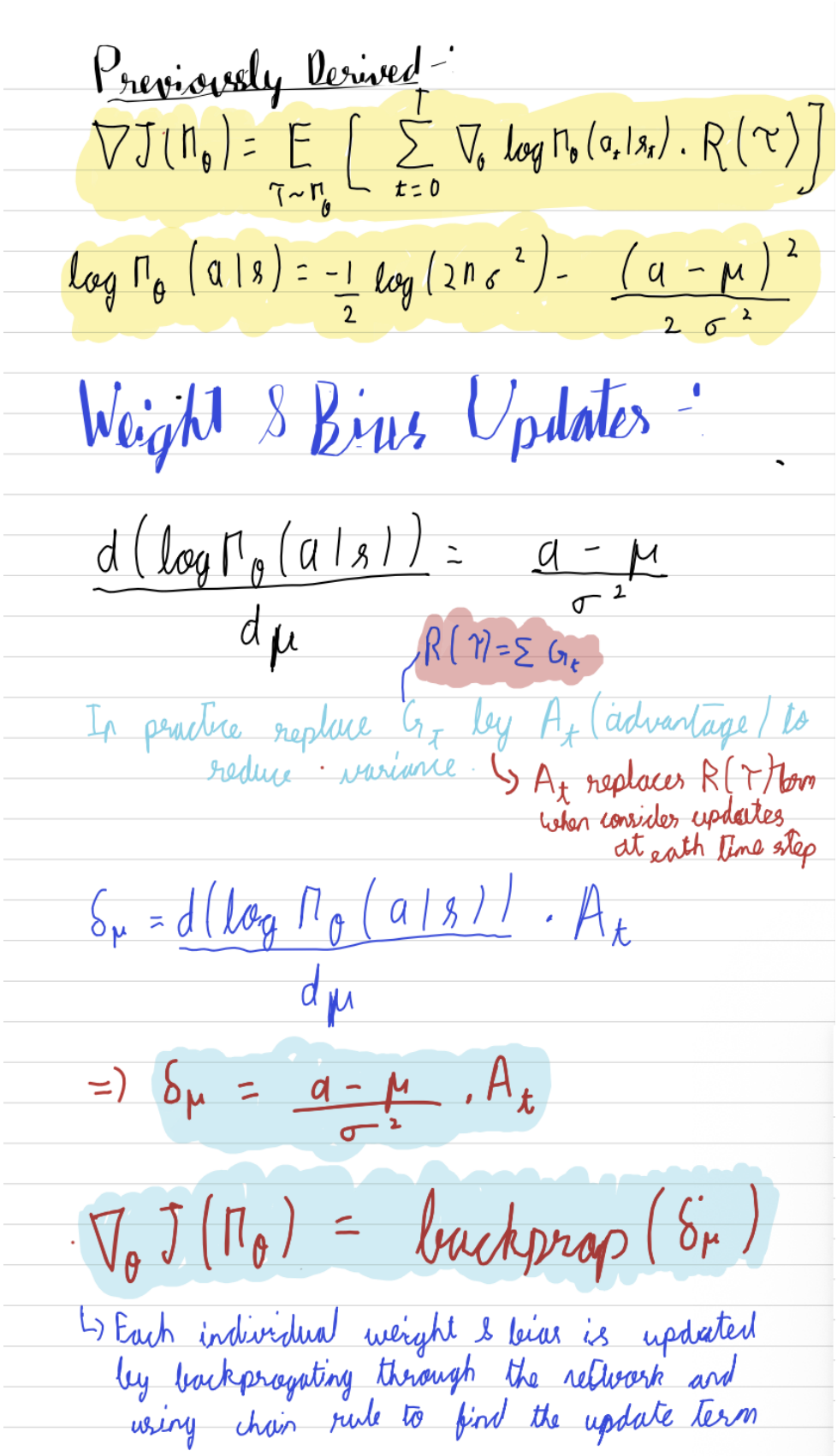
$$= -\frac{1}{2}\left(\log(2\pi\sigma^2 + \frac{(a-\mu)^2}{2\sigma^2}\right)$$

Since $\log P(a \mid \mu, \sigma) = \log \Pi_\theta(a_t \mid s_t)$

Probability of taking a certain action in a given state

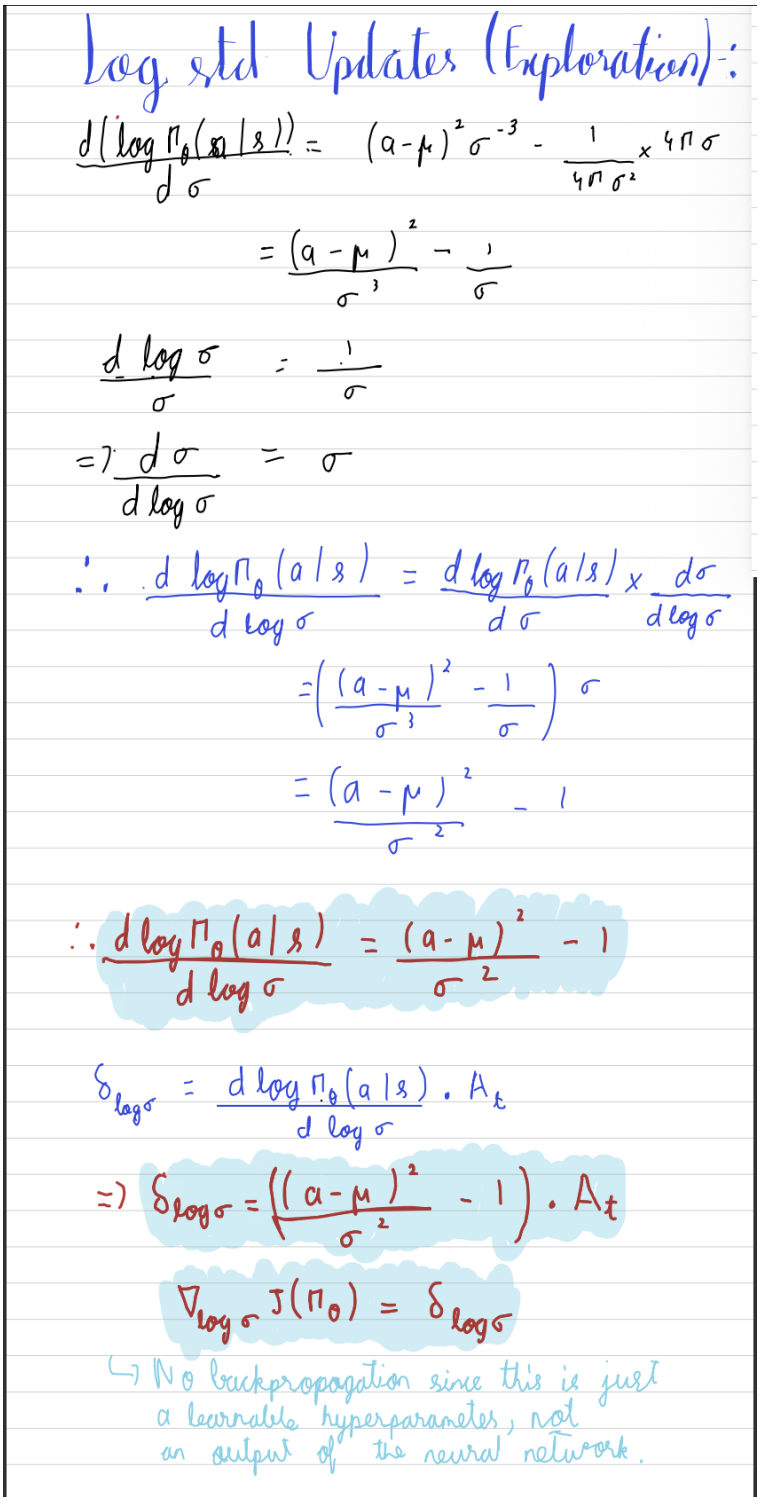
$$\log \Pi_\theta(a_t \mid s_t) = \frac{-1}{2}\left(\log(2\pi\sigma^2 + \frac{(a-\mu)^2}{2\sigma^2}\right)$$

So the derivative of the log probabilities can be found by differentiating this term. To get the numeric value of the term $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$, The log probability is differentiated w.r.t the parameter θ that is being updated. For the weights and biases, this is the individual weight and bias values at each layer. The following calculation shows how weights and bias gradients are passed into the backpropagation algorithm. The steps for backpropagation have already been discussed on page 28, 29 and 115.



Updaing weights and biases using this gradient has he effect of increaing the probabiltiy of actions by a factor that is directly propotional to how much it influenced positive returns, and inversely propotional to how probable it already was.

To update the log σ parameter, the log probality is differentiated w.r.t the log σ. This parameter is a trainable hyperparameter of the neural network, not an output, so there is no backpropagation that needs to happen since this value is updated seperately.



Updating the log σ parameter by this gradient has the effect of increasing randomness when actions do not lead to good returns (increase exploration) and

decreasing randomness when returns are getting better (decrease exploration and increase exploitation).

```
def update_actor(self, advantage, states, pre_squash_actions):  2 usages  joxnrepresent *
    mu = self.actor.forward_propagation(states)
    sigma = np.exp(self.actor.log_std)

    d_mu = -((pre_squash_actions - mu) / (sigma ** 2 + 1e-12)) * advantage[:, None]
    d_mu += self.l2_lambda * mu
    d_mu = np.clip(d_mu, -1,  a_max: 1)

    d_log_std = (np.mean(-(((pre_squash_actions - mu) ** 2) / (sigma ** 2) - 1) * advantage[:, None], axis = 0) +
                 self.entropy)
    self.actor.update_params(d_mu, d_log_std)
```

- d_mu is the δ_μ as derived above
- d_log_std is the $\delta_{\log \sigma}$ as derived above
- the actor.update_params() method is called to update weights and biases, and log_std using Adam optimiser.

2.3.12 Monte Carlo Returns

For each time step and action, the model receives a reward. But these rewards only related to the present action. To ensure that gradients are not short sighted, and consider long term consequences, an additional value called the 'return' G_t is calculated. This return term is a discounted sum of all future rewards (Only future rewards are considered to follow the Markov Property discussed earlier. Any action only influences the present and future, and its value is not determined by the past).

The value of the returns can be written recursively as:

$$G_t = r_t + \gamma G_{t+1}$$

Where γ is the discount factor

The recursive form is useful when trajectory sizes are not defined. However, since in this project every trajectory has a fixed number of time steps, the returns can instead be defined iteratively:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \cdots + \gamma^{T-t-1} r_{T-1}$$

```
def compute_monte_carlo_return(self):  2 usages  joxnrepresent
    rewards = np.array(self.rewards, np.float32)

    time_steps, batch = rewards.shape
    returns = np.zeros(rewards.shape, np.float32)
    G = np.zeros(batch, np.float32)

    for i in reversed(range(time_steps)):
        G = rewards[i] + self.gamma * G
        returns[i] = G

    return returns.flatten()
```

Since the discount factor γ is in range [0,1), later rewards get discounted more. The higher the discount factor, the more the model considers long term impact. The typical value is 0.9 - 0.99.

2.3.13 REINFORCE Model:

REINFORCE (also known as Vanilla Policy Gradient) implements the most basic model of Stochastic Policy Gradient. The advantage function is estimated using only the Monte Carlo Returns G_t

$$A_t = G_t$$

To reduce variance caused by plainly using Monte Carlo returns, I implement a technique called advantage normalisation. The form of normalisation I have used is the z-score normalisation.

$$\tilde{A}_t = \frac{A_t - \mu_A}{\sigma_A + \varepsilon}$$

Where μ_A is the mean advantage, σ_A is the variance of the advantage and ε is a small constant to avoid division by 0 errors.

```
def update_params(self):  1 usage (1 dynamic)  joxnrepresent
    states = np.vstack(self.states)
    pre_squash_actions = np.vstack(self.pre_squash_actions)
    returns = self.compute_monte_carlo_return()
    advantage = (returns - returns.mean()) / (returns.std() + 1e-8)
    self.update_actor(advantage, states, pre_squash_actions)
    avg_rewards = np.array(self.rewards).mean()
    avg_returns = np.array(returns).mean()
    avg_std = np.exp(self.actor.log_std).mean()
    self.clear_trajectory()
    return (f"Reward: {avg_rewards:.3f} \n"
            f"Returns: {avg_returns:.3f} \n"
            f"Exploration (Std): {avg_std:.3f} \n")
```

- Actor is updated w.r.t normalised advantage.
- Average rewards, returns, and standard deviation are returned for logging.

2.3.14 Monte Carlo Actor Critic (MCAC):

The MCAC model trains 2 neural networks. These are the ‘Actor’ (which is the same as the REINFORCE algorithm) is a policy network that outputs actions, and the ‘Critic’ is a state-value network that aims to predict the state value function at each time step of a trajectory.

When updating parameters, Forward propagation is run on the critic network for all the recorded states at each time step of the trajectory and the network outputs predicted state values $\mathrm{V}^{\pi}(s)$. This is done by passing all the sates over all time steps as a batch input to the neural network to get an output vector of state values to enable optimised numpy matrix operations and speedup the computation. The Advantage value for each time step can be estimated by:

$$A_t = G_t - \mathrm{V}^{\pi}(s_t)$$

Where $V^{\pi}(s_t)$ is the state value function for the given time step that the critic network has predicted. This advantage is normalised using z-score normalisation just like REINFORCE and the actor network is then updated.

The critic network is also trained simultaneously along with the actor to be able to provide better value estimates for given states. This network is not updated using stochastic policy gradient, since there is no stochasticity in how values are predicted. Instead, the loss function is the mean squared error between the predicted value and the actual returns.

$$L = \frac{1}{2}(V^{\pi}(s_t) - G_t)^2$$

And differentiating the loss w.r.t the predicted value, we get:

$$\frac{\partial L}{\partial V^{\pi}(s_t)} = V^{\pi}(s_t) - G_t$$

This gradient is used for critic updates to allow the critic to make better predictions of the state value. The critic predictions act as a baseline to stabilise learning.

```
def update_params(self):  1 usage (1 dynamic)  joxnrepresent

    states = np.vstack(self.states)
    actions = np.vstack(self.pre_squash_actions)
    values = self.critic.forward_propagation(states).reshape(-1)
    returns = self.compute_monte_carlo_return()
    advantage = returns - values
    advantage = (advantage - advantage.mean()) / (advantage.std() + 1e-8)

    self.update_actor(advantage, states, actions)
    d_value = (values - returns).reshape(-1, 1)
    self.critic.update_params(d_value)

    avg_rewards = np.array(self.rewards).mean()
    avg_returns = np.array(returns).mean()
    avg_value = np.array(values).mean()
    avg_critic_loss = np.sqrt(np.mean((values - returns) ** 2))
    avg_std = np.exp(self.actor.log_std).mean()
    self.clear_trajectory()

    return (f"Reward: {avg_rewards:.3f} \n"
            f"Returns: {avg_returns:.3f} \n"
            f"Value: {avg_value:.3f} \n"
            f"Critic Loss: {avg_critic_loss:.3f} \n"
            f"Exploration (Std): {avg_std:.3f} \n")
```

- Critic predicted values used to calculate advantage
- Advantage is still normalised to reduce variance
- Critic network is updated by separate gradients to the actor.

2.3.15 Reward Engineering:

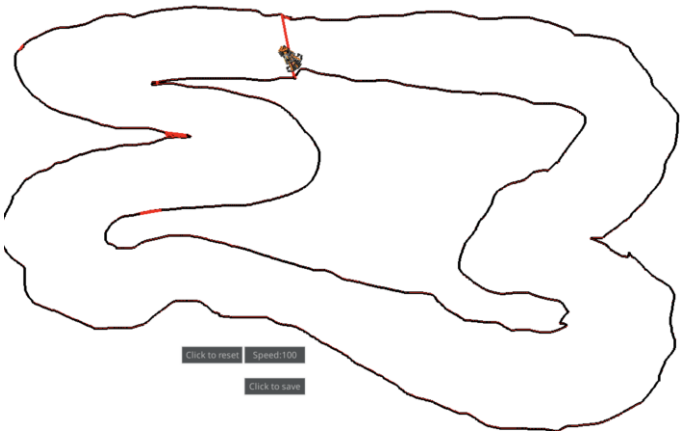
For every action taken by the agent, the agent is given a reward for that action which indicates how well the action contributes to achieving the final goal. In other words, the rewards numerical represent how “good” each action taken is. The rewards received for an action taken by the agent in each state is determined by a ‘reward function’. The reward function maps the action to a numerical value that is calculated by the characteristics of the state it is in the environment (and possibly the next state). Reward engineering is the process of designing this reward function.
The reward function rewards specific behaviours of the agent by different amounts, and these individuals rewards for behaviours are summed to get the reward value. This function must therefore be able to effectively decompose the objective of the agent into measurable signals obtained from its interaction with the given environment.
The function must decide which signals/behaviours are negative and positive but also must normalise and scale the magnitudes of these rewards based on how ‘good’ or ‘bad’ they are. This is usually done by having sparse large terminal rewards (crashing, finishing lap) and dense shaping rewards (staying away from walls, going fast). The terminal rewards create a large signal which help the agent understand these states have significantly determined whether it has achieved its goal. The shaping rewards create gradients of change as the agent explores the environment, which helps the agent to pick actions that are better each episode. It is very important that all rewards are scaled well to represent the environment and objective accurately.
I have my reward function as a method of the AiCar class, and it is called after the AI car carries out a pair of actions (steer and throttle). The function I started with computed rewards/ penalties for 9 events:

- Crash penalty
- Progress (distance moved in the last time step) reward
- Imbalance (If car is not at centre of the track) penalty
- Speed reward
- Max progress (How far the car has moved compared to the maximum progress it has attained) reward/ penalty
- Wall hugging (driving too closely to a wall) penalty
- Jittery acceleration (If car keeps setting the target throttle to polarised values every cycle) penalty
- Jittery steering (If car keeps setting the target steer to polarised values every cycle) penalty
- Time penalty (small negative reward each time step to encourage faster laps.

```python
def compute_reward(self, prev_progress, max_progress, sensors, actions):  1 usage (1 dynamic)  new *
    reward = 0
    rewards_breakdown = []

    # Crash penalty
    if self.is_crashed:
        rewards_breakdown = [-60, 0, 0 ,0 ,0 ,0,0, 0]
        return -60, rewards_breakdown
    else:
        rewards_breakdown.append(0)
    # ---------------------------------------------------- #

    # Progress reward
    distance_moved = self.progress - prev_progress
    if distance_moved < -0.5:
        distance_moved += 1.0
    if distance_moved > 1e-5:
        rewards_breakdown.append(distance_moved * 400)
        reward += distance_moved * 400
    else:
        rewards_breakdown.append(-0.02)
        reward-= 0.02
    # ---------------------------------------------------- #

    # Imbalance reward
    left_sensors = [sensors[2], sensors[4], sensors[6], sensors[8], sensors[10]]
    avg_left = sum(left_sensors) / 5
    right_sensors = [sensors[1], sensors[3], sensors[5], sensors[7], sensors[9]]
    avg_right = sum(right_sensors) / 5
    imbalance = abs(avg_left - avg_right)
    reward -= imbalance * 0.9
    rewards_breakdown.append(-imbalance * 0.9)
    # ---------------------------------------------------- #

    # Speed reward
    speed = self.velocity.magnitude()
    normalized_speed = speed / self.max_speed
    if sensors[0] > 0.04:
        rewards_breakdown.append(normalized_speed * 0.2)
        reward += normalized_speed * 0.2
    else:
        reward -= normalized_speed * 1
        rewards_breakdown.append(-normalized_speed * 1)
    # ---------------------------------------------------- #

    # Max progress reward/penalty
    progress_change = self.progress - max_progress
    reward += 0 * progress_change
    rewards_breakdown.append(0 * progress_change)
    # ---------------------------------------------------- #

    # Wall hugging penalty
    min_sensor = min(sensors) if sensors else 0
    if min_sensor < 0.03:
        reward -= (0.03 - min_sensor) * 70.0
        rewards_breakdown.append(-(0.03 - min_sensor) * 70.0)
    else:
        rewards_breakdown.append(0)
    # ---------------------------------------------------- #
    steer, throttle = actions

    # Jittery steering penalty
    steer_change = abs(self.current_steer_input - steer)
    reward -= steer_change * 0.015
    rewards_breakdown.append(-steer_change * 0.015)

    # ---------------------------------------------------- #
    # Jittery acceleration penalty
    throttle_change = abs(self.current_throttle_input - throttle)
    reward -= throttle_change * 0.015
    rewards_breakdown.append(-throttle_change * 0.015)
    # ---------------------------------------------------- #
    # Time penalty
    reward += 0.005
    rewards_breakdown.append(0.05)
    # ---------------------------------------------------- #

    return reward, rewards_breakdown
```

- I have tried to keep the scale small to account for the scaling issue with the MCAC value network mentioned earlier
- The rewards_breakdown is used for debugging

In testing however, I observed incorrect learning behaviour from the agent. The agent would sometimes learn to drive straight till the first turn but then would always learn to then crash into the nearest wall as shown in the image.



The reason for this behaviour can be diagnosed by observing the training logs.

```
Episode:1
Epoch rewards: [-40.0, 68.232, -4.005, 0.647, 0.667, -10.036, -3.331, -3.772, -1.615]
Time step rewards: [-0.123, 0.211, -0.012, 0.002, 0.002, -0.031, -0.01, -0.012, -0.005]

Episode:2
Epoch rewards: [-40.0, 52.32, -5.224, 0.483, -175.636, -12.481, -3.641, -4.134, -1.753]
Time step rewards: [-0.114, 0.149, -0.015, 0.001, -0.5, -0.035, -0.01, -0.012, -0.005]

Episode:3
Epoch rewards: [-40.0, 32.971, -3.079, 0.285, -197.252, -7.974, -2.383, -2.709, -1.148]
Time step rewards: [-0.173, 0.143, -0.013, 0.001, -0.855, -0.035, -0.01, -0.012, -0.005]

Episode:4
Epoch rewards: [-40.0, 46.2, -3.233, 0.398, -193.505, -8.291, -2.491, -2.837, -1.21]
Time step rewards: [-0.165, 0.19, -0.013, 0.002, -0.796, -0.034, -0.01, -0.012, -0.005]

Episode:5
Epoch rewards: [-40.0, 63.404, -3.255, 0.594, -232.239, -8.299, -2.972, -3.413, -1.456]
Time step rewards: [-0.137, 0.217, -0.011, 0.002, -0.795, -0.028, -0.01, -0.012, -0.005]

Episode:6
Epoch rewards: [-40.0, 65.162, -3.459, 0.59, -269.78, -8.89, -3.196, -3.566, -1.547]
Time step rewards: [-0.129, 0.21, -0.011, 0.002, -0.869, -0.029, -0.01, -0.011, -0.005]

Episode:7
Epoch rewards: [-40.0, 70.804, -3.383, 0.688, -270.086, -8.964, -3.444, -3.831, -1.649]
Time step rewards: [-0.121, 0.214, -0.01, 0.002, -0.816, -0.027, -0.01, -0.012, -0.005]

Episode:8
Epoch rewards: [-40.0, 38.844, -3.196, 0.345, -302.076, -8.686, -2.531, -2.88, -1.223]
Time step rewards: [-0.163, 0.158, -0.013, 0.001, -1.23, -0.035, -0.01, -0.012, -0.005]

Episode:9
Epoch rewards: [-40.0, 39.791, -3.042, 0.337, -306.631, -8.056, -2.458, -2.784, -1.174]
Time step rewards: [-0.17, 0.169, -0.013, 0.001, -1.3, -0.034, -0.01, -0.012, -0.005]

Episode:10
Epoch rewards: [-40.0, 63.609, -2.595, 0.603, -242.98, -5.974, -2.367, -2.606, -1.119]
Time step rewards: [-0.178, 0.283, -0.012, 0.003, -1.081, -0.027, -0.011, -0.012, -0.005]
```

The reason for this behaviour is incorrect scaling of the rewards. As seen in the logs, the first 2 rewards, (which is the crash penalty and the progress reward) are higher magnitude that the rest. Both contribute around 1E-1 each time step while most of the other rewards contribute 1E-2. This may not be necessarily a huge problem, since these are the most important rewards that should shape the learning the most.

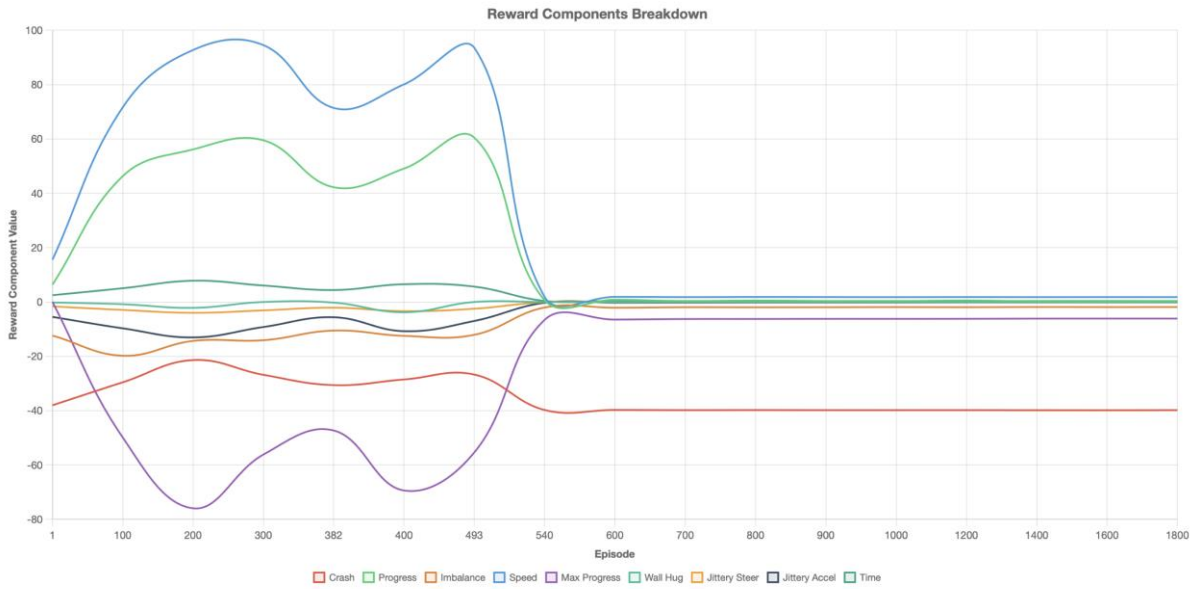
The most significant contributor to poor learning is the fifth reward in the log, which is the max progress reward. We see that it very quickly grows to large negative values. This is because the car initially explores some progress, but then if it crashes early once, it immediately receives a huge penalty.
Large rewards per time step leads to large gradients in updating the network. So, these large max progress penalties dominate learning and overpower all other rewards. The agent learns it receives a huge negative reward each time step regardless of what it does, and there simply tries to end each episode as fast as possible, since the crash penalty is still small compared to the max progress penalty.

Following these observations, I made some changes to the reward function to try to ensure that gradients of change reflect the environment well and allow the model to learn effectively:

```
275     def compute_reward(self, prev_progress, max_progress, sensors, actions):  1 usage (1 dynamic)
276         reward = 0
277         rewards_breakdown = []
278 
279         # Crash penalty
280         if self.is_crashed:
281             rewards_breakdown = [-20*(2-self.progress), 0, 0 ,0 ,0 ,0,0, 0]
282             return -20*(2-self.progress), rewards_breakdown
283         else:
284             rewards_breakdown.append(0)
285         # ------------------------------------------------- #
286 
287         # Progress reward
288         distance_moved = self.progress - prev_progress
289         if distance_moved < -0.5:
290             distance_moved += 1.0
291         if distance_moved > 1e-5:
292             rewards_breakdown.append(distance_moved * 100)
293             reward += distance_moved * 100
294         else:
295             rewards_breakdown.append(-0.01)
296             reward-= 0.01
297         # ------------------------------------------------- #
298 
299         # Imbalance reward
300         left_sensors = [sensors[2], sensors[4], sensors[6], sensors[8], sensors[10]]
301         avg_left = sum(left_sensors) / 5
302         right_sensors = [sensors[1], sensors[3], sensors[5], sensors[7], sensors[9]]
303         avg_right = sum(right_sensors) / 5
304 
305         imbalance = abs(avg_left - avg_right)
306         if imbalance > 0.1:
307             reward -= imbalance * 0.1
308             rewards_breakdown.append(-imbalance * 0.1)
309         else:
310             rewards_breakdown.append(0)
311         # ------------------------------------------------- #
312 
313         # Jittery steering penalty
314         steer_change = abs(self.current_steer_input - steer)
315         reward -= steer_change * 0.005
316         rewards_breakdown.append(-steer_change * 0.005)
317 
318         # ------------------------------------------------- #
319         # Jittery acceleration penalty
320         throttle_change = abs(self.current_throttle_input - throttle)
321         reward -= throttle_change * 0.015
322         rewards_breakdown.append(-throttle_change * 0.015)
323         # ------------------------------------------------- #
324 
325         # Time penalty
326         reward += 0.005
327         rewards_breakdown.append(0.005)
328         # ------------------------------------------------- #
329 
330         return reward, rewards_breakdown
```

- Created new crash penalty which scales down as progress increases. This ensures crashing early is punished more than crashing later which encourages more exploration
- Scaled down the progress reward

However, this reward function still didn't allow for good learning. Analysing the reward breakdown graph enabled me to diagnose this issue



- Max progress reward caused the collapse of learning
- Progress and speed reward have identical patterns, meaning they reward the same behaviour.
- Rewards components are still not scaled correctly

To resolve these issues, I first worked to rescale all rewards. I then implemented a single 'efficiency' reward to replace the progress and speed rewards. This efficiency reward is scaled by both the progress per time step and the current speed, which avoids the double rewarding

```
286         # Efficiency (progress and speed) reward
287         distance_moved = self.progress - prev_progress
288         if distance_moved < -0.5:
289             distance_moved += 1.0
290
291         if distance_moved > 1e-5:
292             speed = self.velocity.magnitude()
293             normalized_speed = speed / self.max_speed
294             efficiency_reward = distance_moved * 100 * (0.5 + 0.5 * normalized_speed)
295             rewards_breakdown.append(efficiency_reward)
296             reward += efficiency_reward
297         else:
298             rewards_breakdown.append(-0.01)
299             reward -= 0.01
300         # ------------------------------------------------------------------------------
```

I also experimented with a different way of implementing max progress rewards. Instead of rewarding at each time steps, I tried to make a terminal bonus reward that discounts some of the crash penalty

```
current_progress = self.progress
progress_change = current_progress - mean_progress
exploration_coef = 140 if progress_change >  0  else 40
exploration_reward = (r.sign(progress_change) + 2.5*
                      math.log(abs(exploration_coef * progress_change) + 1))
reward += exploration_reward
rewards_breakdown.append(exploration_reward)
```

However, this ‘exploration reward’ was still incredibly noisy and caused bad learning so I decided to omit it entirely.

I used Claude Sonnet 4.5 to generate me a reward analyser artifact that allowed me to graph my episode rewards and reward component breakdown conveniently.

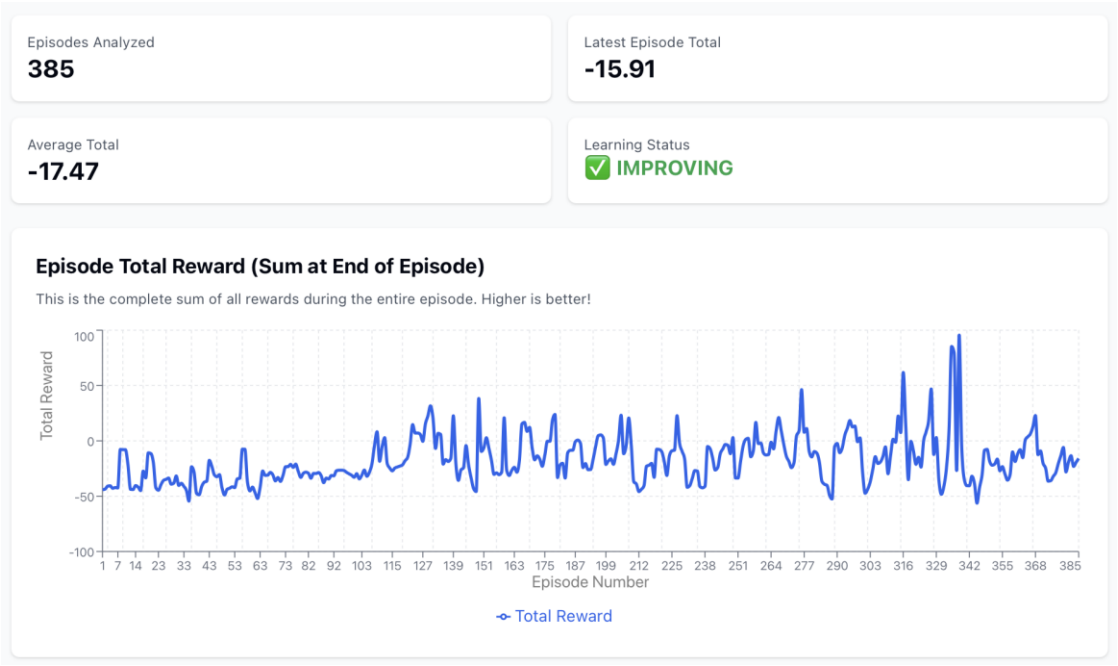
With the help of this tool and with a lot more work on fine tuning the rewards, the final reward function I have obtained is this:

```
def compute_reward(self, is_stuck, is_lap_finished, prev_progress, sensors):  1 usage (1 dynamic)  joxnrepresent *
    """
    Calculate rewards for most recent actions taken by RL model.
    Combines multiple reward components. Returns both total reward, and breakdown of individual components.

    Args:
        is_stuck (bool): If car is stuck being still.
        is_lap_finished (bool): If car finished lap
        prev_progress (float): Previous progress for computing change in progress
        sensors (list[float]): Sensor data of distances to walls in different directions

    Returns:
        tuple(float, list[float]): (total_reward, rewards_breakdown)
            - total_reward: Sum of all rewards
            - rewards_breakdown: Individual reward components for analysis
    """

    rewards_breakdown = []

    # ------------------------------------------------- #
    """
    1) Stopping penalty
    """
    if self.is_crashed:
        if is_stuck:
            stopped_penalty = self.STUCK_PENALTY
        else:
            stopped_penalty = self.CRASH_PENALTY
```

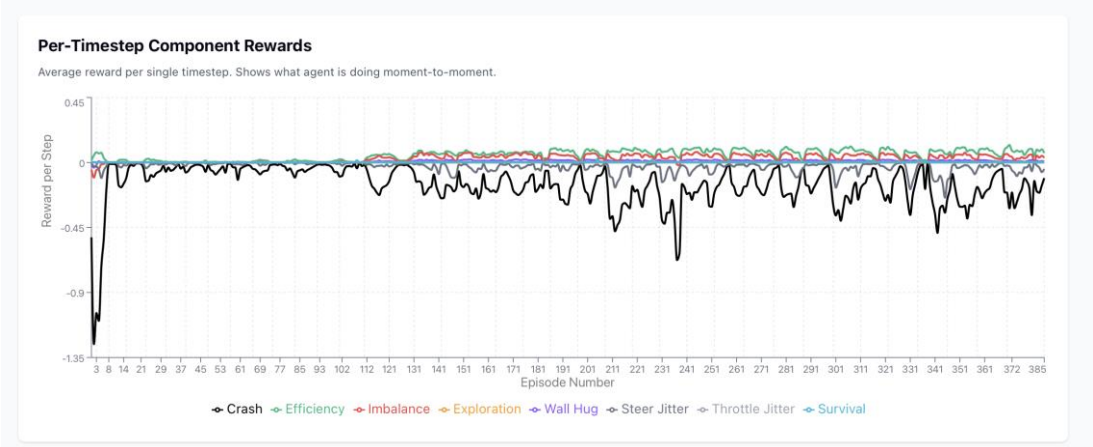
```
        """
        OMITTED: Exploration bonus
        """
        # current_progress = self.progress
        # progress_change = current_progress - mean_progress
        # exploration_coef = 140 if progress_change >  0  else 40
        # exploration_reward = (r.sign(progress_change) + 2.5*
        #                       math.log(abs(exploration_coef * progress_change) + 1))
        # reward += exploration_reward
        # rewards_breakdown.append(exploration_reward)

        rewards_breakdown = [stopped_penalty, 0, 0 , 0 , 0 , 0 , 0, 0]
        return sum(rewards_breakdown), rewards_breakdown
    else:
        rewards_breakdown.append(0)
    # ------------------------------------------------- #

    # ------------------------------------------------- #
    """
    2) Efficiency reward
    """
    distance_moved = self.progress - prev_progress

    if distance_moved < -0.5:
        distance_moved += 1.0

    if distance_moved > 1e-4:
        speed = self.velocity.magnitude()
        normalized_speed = speed / self.MAX_SPEED
        speed_reward = self.SPEED_REWARD_COEF * normalized_speed**2
        efficiency_reward = self.PROGRESS_REWARD_COEF * distance_moved + speed_reward

        rewards_breakdown.append(efficiency_reward)
    else:
        rewards_breakdown.append(-0.001)
    # ------------------------------------------------- #

    # ------------------------------------------------- #
    """
    3) Imbalance reward
    """
    left_sensors = [sensors[2], sensors[4], sensors[6], sensors[8]]
    avg_left = sum(left_sensors) / 4
    right_sensors = [sensors[1], sensors[3], sensors[5], sensors[7]]
    avg_right = sum(right_sensors) / 4
    imbalance = avg_right - avg_left

    left_clearance = sum(left_sensors)
    right_clearance = sum(right_sensors)

    aligned_steering_reward = (((right_clearance - left_clearance) * self.current_steer_input )
                               * self.ALIGNED_STEERING_REWARD_COEF)
    if distance_moved > 1e-4:
        rewards_breakdown.append(aligned_steering_reward)
    else:
        rewards_breakdown.append(0)
    # ------------------------------------------------- #
```

```
# ------------------------------------------------- #
"""
4) Wall hugging penalty
"""
min_sensor = min(sensors) if sensors else 0
if min_sensor < self.WALL_PROXIMITY_THRESHOLD:
    rewards_breakdown.append((0.02 - min_sensor) * self.WALL_PROXIMITY_PENALTY_COEF)
else:
    rewards_breakdown.append(0)
# ------------------------------------------------- #

# ------------------------------------------------- #
"""
5) Steer anticipation reward
"""
front_sensor = sensors[0]
corner_strength = max(0.0, 0.5 - front_sensor) * abs(imbalance)
steer_anticipation_reward = corner_strength * self.current_steer_input  * self.ANTICIPATED_STEERING_REWARD_COEF
if distance_moved > 1e-4:
    rewards_breakdown.append(steer_anticipation_reward)
else:
    rewards_breakdown.append(0)
# ------------------------------------------------- #

# ------------------------------------------------- #
"""
6) No steer penalty
"""
if min(sensors) <self.NO_STEER_PENALTY_THRESHOLD and distance_moved > 1e-4:
    no_steering_penalty = self.NO_STEER_PENALTY_COEF * (1- abs(self.current_steer_input))
    rewards_breakdown.append(no_steering_penalty)
else:
    rewards_breakdown.append(0)
# ------------------------------------------------- #

# ------------------------------------------------- #
"""
7) Survival bonus
"""
rewards_breakdown.append(self.SURVIVAL_BONUS)
# --------------------------------------------------

# ------------------------------------------------- #
"""
8) Finish lap bonus
"""
if is_lap_finished:
    rewards_breakdown.append(self.LAP_COMPLETION_BONUS)
else:
    rewards_breakdown.append(0)
# ------------------------------------------------- #

return sum(rewards_breakdown), rewards_breakdown
```

By running this reward function for many episodes, I used the reward analyser artifact to graph the training data and analyse the reward function.



- Total Reward is overall increasing
- Model is improving overall



- Overwhelming crash penalty in early training, but training still stabilises

- Components are scaled appropriately

2.3.16 AI Training Environment

The AITrainingEnvironment class extends the CarSImulation class, and handles running training epochs. A certain number of cars are initialised, and every update, the class iterates through all these cars to get their sensor data and update their physics. Sensor data is batched together each time step to allow it all to be processed by the neural network to produce batch actions. This batch processing significantly speeds up computation since it allows for optimised numpy matrix calculations. The batch of actions is then applied to the cars.

```
def update(self, *args):  joxnrepresent *
    super().update()
    if not game_core.is_paused and self.rl_model is not None:
        self.update_ai_step()

    if all(self.is_crashed):
        super().update()
        self.reinitialise_simulation()
        return

def update_ai_step(self):  1 usage  joxnrepresent *
    """Perform single AI training step for active cars."""
    states = np.zeros( shape: (self.num_of_cars, self.STATE_SIZE), np.float32)
    for i, car in enumerate(self.cars):
        if not self.is_crashed[i]:
            # State vector: sensors + current speed + current steering.
            sensors = car.get_sensors(self.track)
            states[i] = sensors + [car.velocity.magnitude() / car.MAX_SPEED, car.steer / car.MAX_STEER_RAD]

    # Get batch actions
    self.actions, pre_squash = self.rl_model.get_stochastic_actions(states)
    rewards = np.zeros(self.num_of_cars, np.float32)

    for i, car in enumerate(self.cars):
        # Iterate through active cars.
        if not self.is_crashed[i]:
            car.update(self.actions[i])
            # Get sensors at new position for reward computation
            sensors = car.get_sensors(self.track)
            rewards[i] = self.process_car_and_get_reward(i, car, sensors)

    self.rl_model.update_trajectory(states, pre_squash, rewards)
```

- First states are computed
- Batch actions are output from the model
- Cars are updated based on taken actions, and the corresponding rewards are recorded
- Trajectory is cached

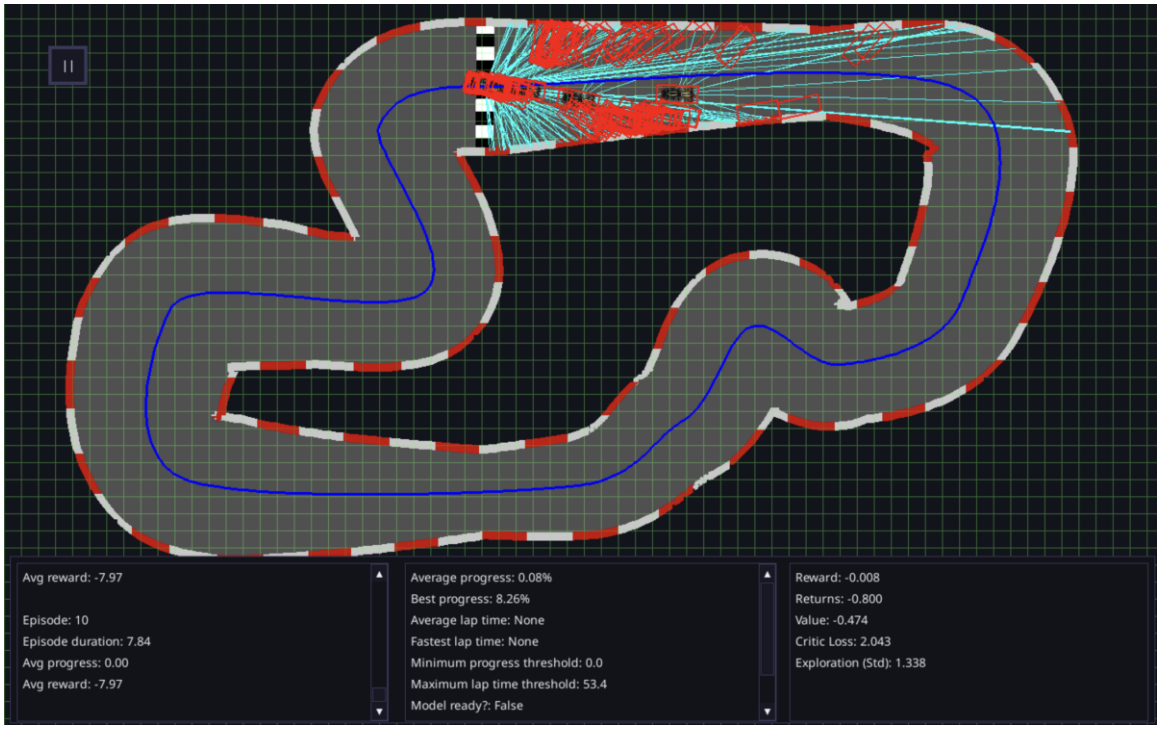
To process car rewards, the reward function needs to know if car is stuck, if a lap was finished, the progress in last frame and the sensors. So before computing the reward, all this data is first processed, and then the reward is computed

```python
def process_car_and_get_reward(self, i, car, sensors):  1 usage  joxnrepresent *
    """Process a single car's reward after action is taken."""
    car.collision_detection(self.track)
    progress, is_lap_finished = car.update_and_get_progress(self.track.track_spine)

    if is_lap_finished:
        laps_completed = math.floor(progress)
        # Ensure we don't double count if super().update() also runs
        if self.time_manager.lap_times[i][laps_completed - 1] is None:
            self.time_manager.update(i, laps_completed, is_lap_finished)

    is_stuck = self.check_stuck(i, progress)

    # Compute reward
    reward, rewards_array = car.compute_reward(is_stuck, is_lap_finished, self.prev_progress[i], sensors)

    # Check if completed max laps
    if car.progress >= self.time_manager.num_of_laps:
        self.is_crashed[i] = True  #
        car.is_crashed = True
        reward += 1.5 * car.LAP_COMPLETION_BONUS
        rewards_array[-1] += 1.5 * car.LAP_COMPLETION_BONUS

    clipped_reward = np.clip(reward, -50, 100 * self.time_manager.num_of_laps)

    # Check if completed max laps
    if car.progress >= self.time_manager.num_of_laps:
        self.is_crashed[i] = True  #
        car.is_crashed = True
        reward += 1.5 * car.LAP_COMPLETION_BONUS
        rewards_array[-1] += 1.5 * car.LAP_COMPLETION_BONUS

    clipped_reward = np.clip(reward, -50, 100 * self.time_manager.num_of_laps)

    # Log rewards for debugging
    reward_breakdown = np.array(rewards_array)
    self.epoch_rewards_breakdown[i] += reward_breakdown
    self.time_step_rewards_breakdown.append(reward_breakdown)

    self.max_epoch_progress[i] = max(progress, self.max_epoch_progress[i])
    self.prev_progress[i] = progress

    return clipped_reward
```

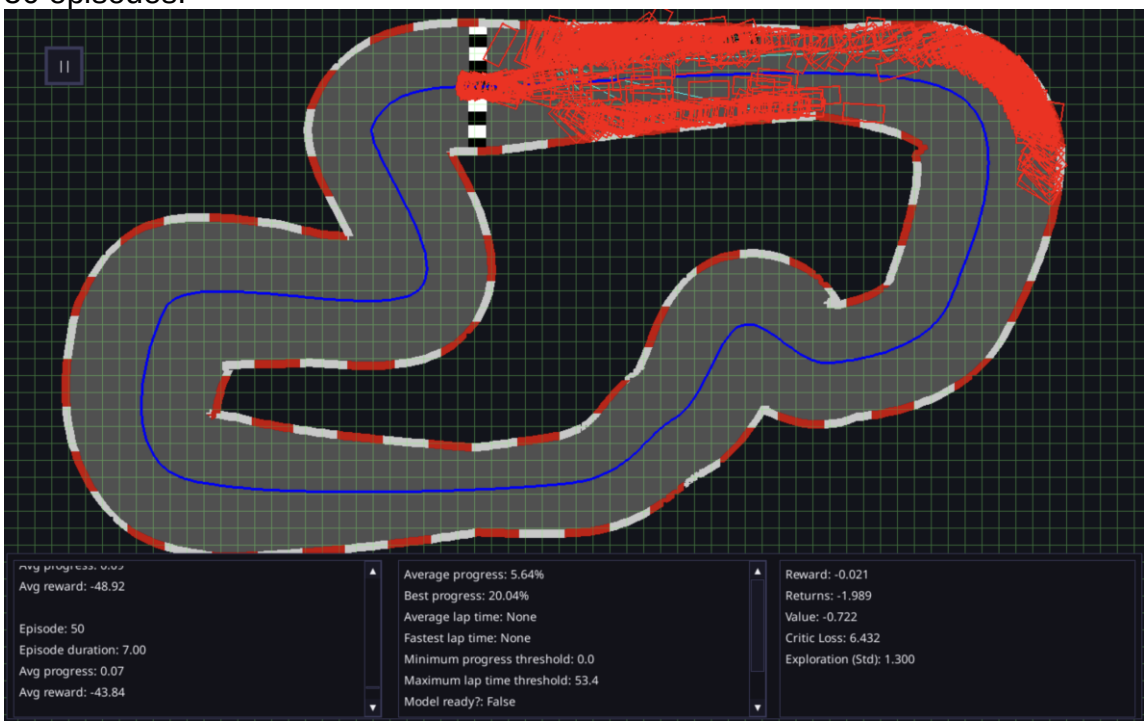
- Additional bonus reward is provided if the car finishes the required number of laps.
- Reward logs are updating for tracking.

Output training:

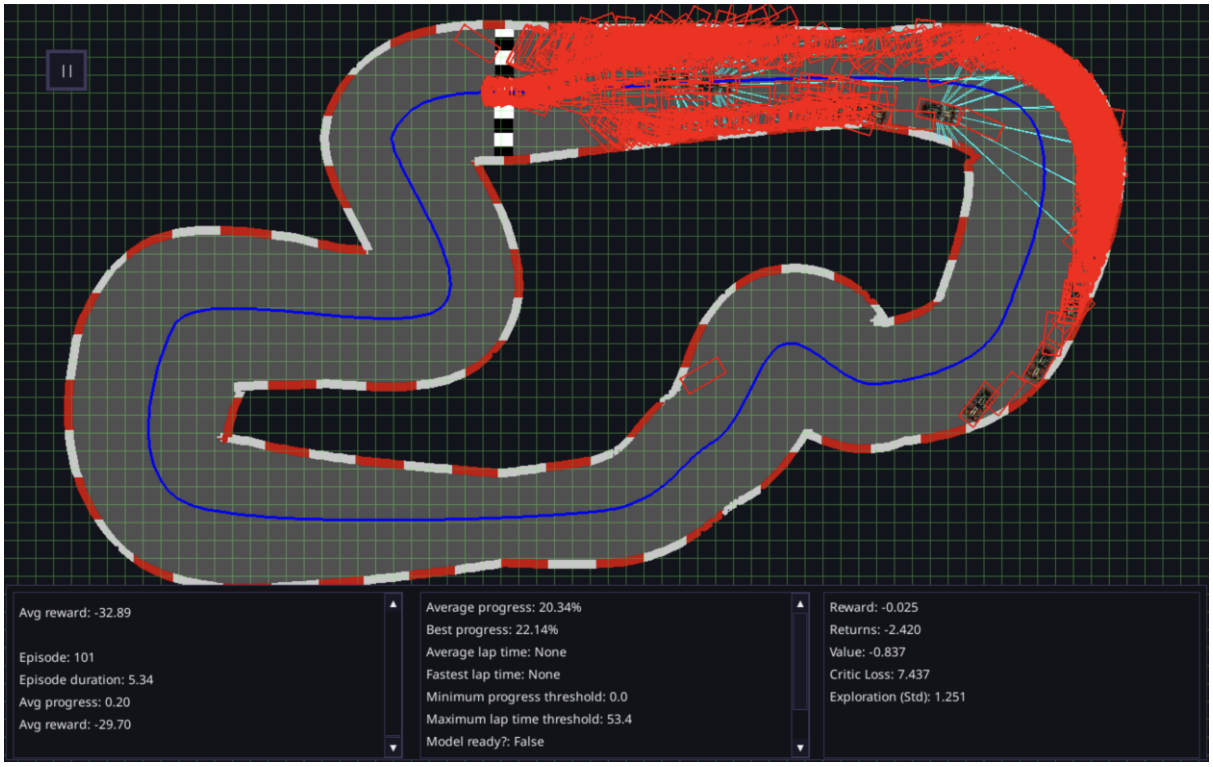
10 episodes:



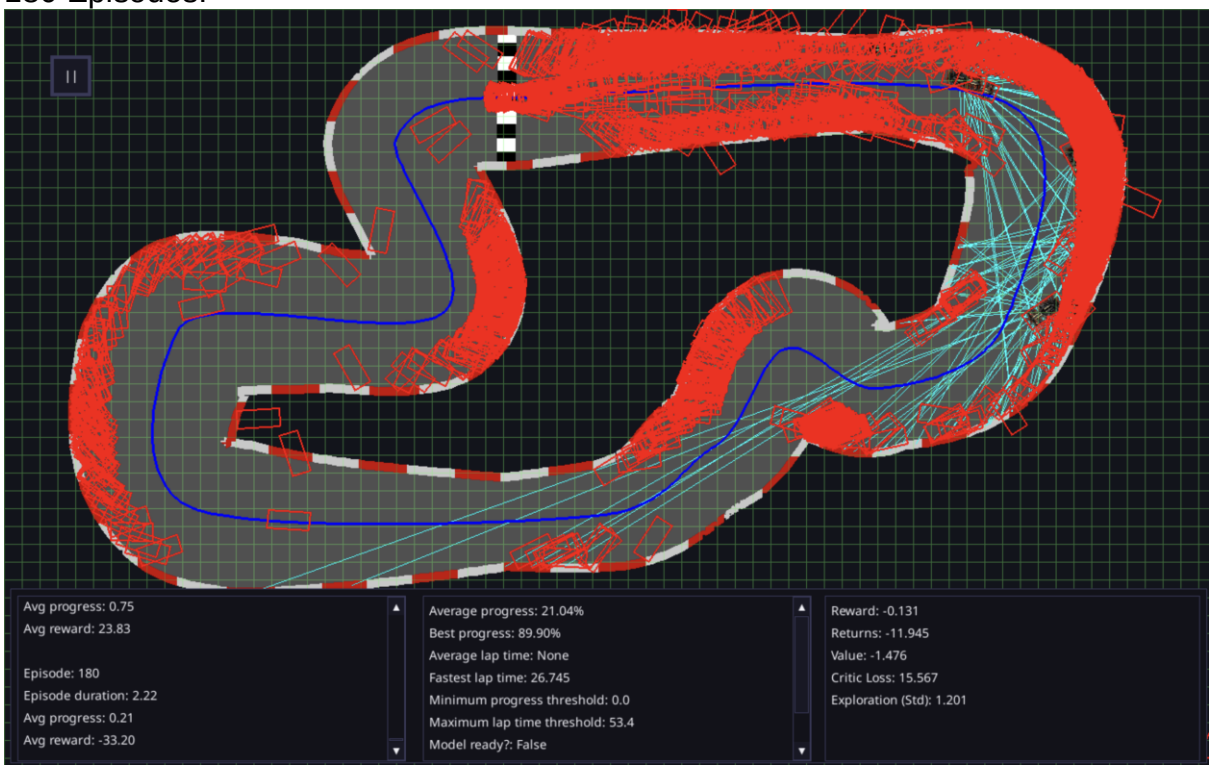
50 episodes:



100 Episodes:



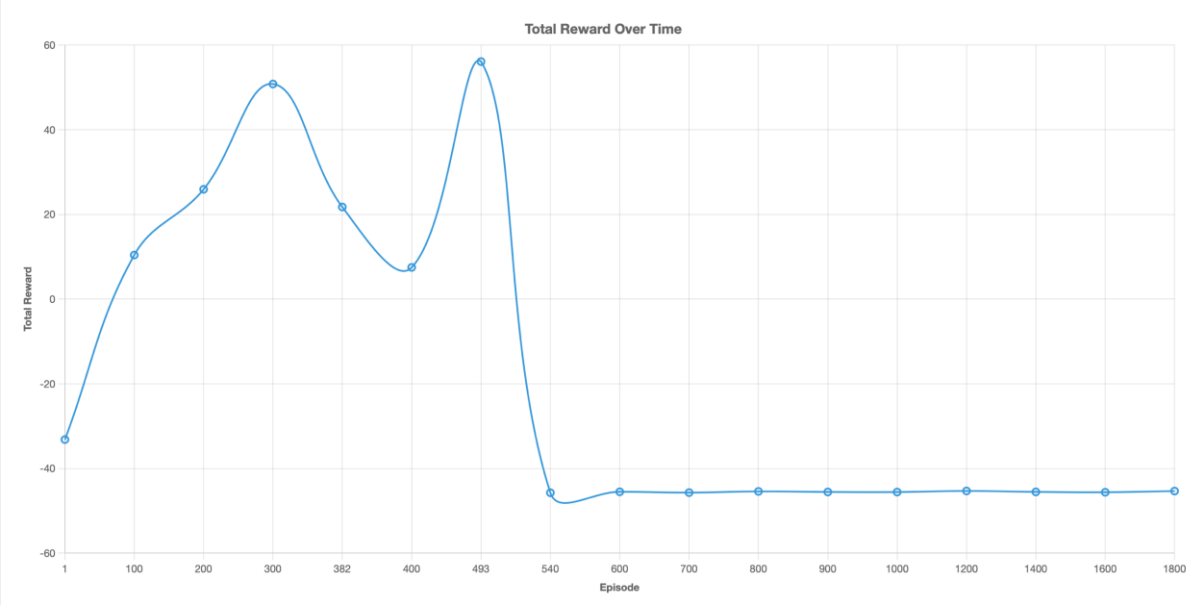
180 Episodes:



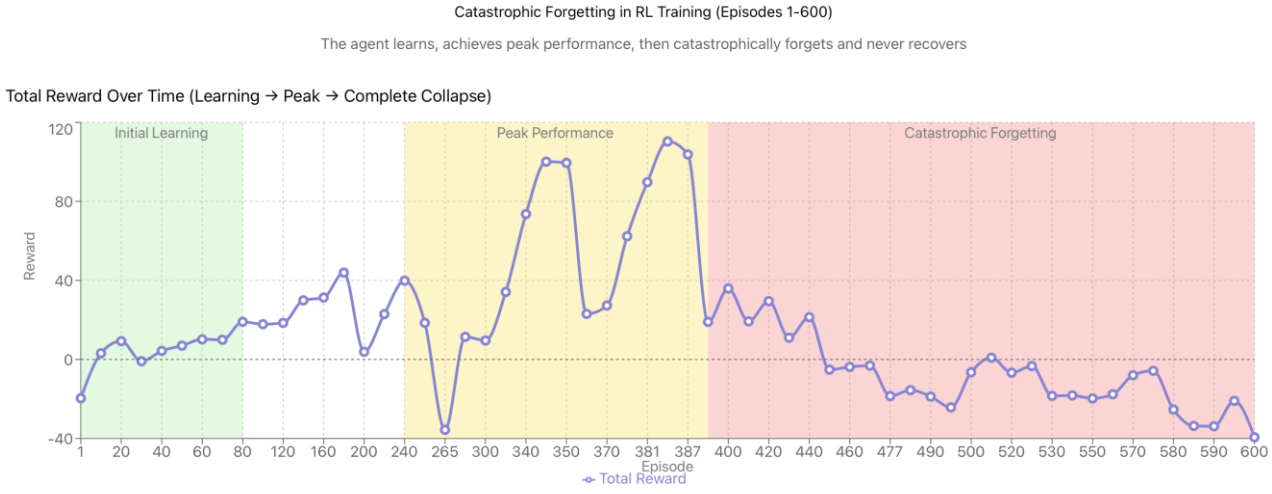
Evidently, training is taking place, and the model is improving with more episodes.

Catastrophic Forgetting and Rollbacks:

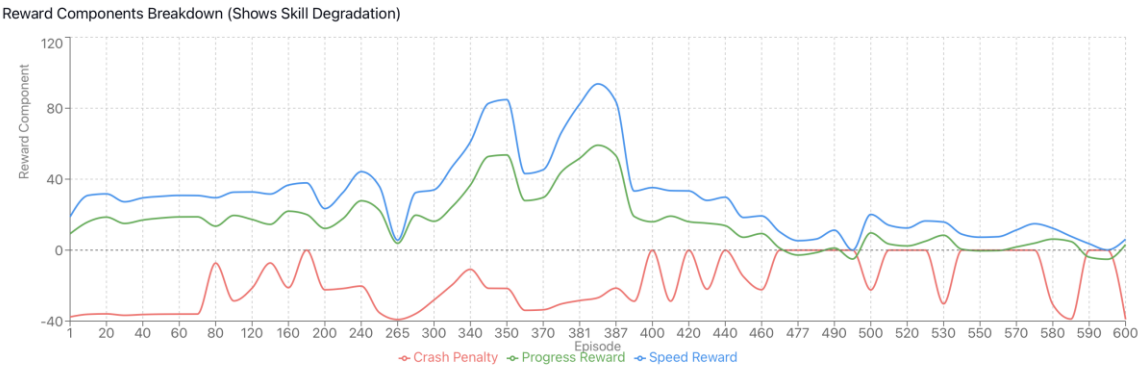
Methods like REINFORCE and MCAC do not have “memory” of their historic rewards. They simply learn based on gradients experienced by their present and future actions. As a result, such models can experience a phenomenon known as catastrophic interference, or catastrophic forgetting. This occurs when a model abruptly loses its gained knowledge of the environment. This is caused by the continuous sequential nature of stochastic policy gradient algorithms that can cause the erasure of previously learnt skilled. In practice, this is caused by a sequence of unlucky runs, that cause gradients to bias in the wrong direction and then the model gets stuck in a local minimum.
This phenomenon is seen more evidently in the REINFORCE algorithm. This is because the Value baseline provided by the critic network in MCAC helps prevent several unlucky runs from overpowering gradient updates.
By logging episodic returns over many episodes, I was able to graph the returns with respect to time to be able to illustrate the catastrophic forgetting problem:



We can observe a gradual increase in progress until episode 540, after which, rewards completely stagnant at a baseline with no improvement even after 1000 episodes
With the help of the Sonnet 4.5 Claude AI model, I was able to create a more interactive visualisation artifact of the catastrophic forgetting observed in a separate training run. This is the evidence of terminal policy collapse.



I also used it to graph individual reward components to observe degradation of behaviours:



We can observe the degradation after 440 episodes.

To counter this problem, I implemented a system of model rollback. This involves recording the best episodic return and maintaining a moving history window. If the most recent episodes in this history window are significantly worse than the best recorded episode, then the model is ‘rolled back’ to its older parameters and retries training from that point again.

```
def check_rollback(self):  1 usage   joxnrepresent *
    """Check if model should be rolled back due to poor performance."""
    if len(self.performance_history) < self.HISTORY_WINDOW:
        return ""

    recent_performance = np.mean(self.performance_history[-self.HISTORY_WINDOW:])

    if recent_performance < self.ROLLBACK_THRESHOLD * self.best_hist_avg:
        # Rollback to best model
        self.rl_model.actor.set_params(self.best_model_params['actor'])
        if hasattr(self.rl_model, 'critic'):
            self.rl_model.critic.set_params(self.best_model_params['critic'])

        print()
        print("----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----")
        print("Model Rolled Back")
        print("----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----")
        print()

        self.rollback_counter += 1
        self.rl_model.clear_trajectory()
        self.performance_history.clear()
        return "----xx" * 5 + "\n" + "Model Rolled Back" + "\n" + "----xx" * 5 + "\n \n"
    return  ""
```

This essentially gives the environment “memory” and prevents catastrophic forgetting due to unlucky trajectories. When the model is reset back to older parameters, it will likely take a different course of learning since stochastic policy gradient is powered by randomness, and so it would be able to resume learning.

Testing rollbacks

```
Episode: 35
Epoch rewards: [-40.0, 4.723, 0.024, 0.0, -0.001, -22.952, 3.793, 0.0]
Time step rewards: [-0.032, 0.004, 0.0, 0.0, -0.0, -0.018, 0.003, 0.0]

Episode: 36
Epoch rewards: [-22.5, 2.812, 0.79, 0.0, 0.237, -11.121, 6.33, 0.0]
Time step rewards: [-0.011, 0.001, 0.0, 0.0, 0.0, -0.005, 0.003, 0.0]

Episode: 37
Epoch rewards: [-32.5, 2.634, 0.346, 0.0, 0.03, -1.062, 2.317, 0.0]
Time step rewards: [-0.042, 0.003, 0.0, 0.0, 0.0, -0.001, 0.003, 0.0]


----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----
Model Rolled Back
----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----

Episode: 38
Epoch rewards: [-37.5, 6.019, 0.615, 0.0, 0.182, -20.734, 3.849, 0.0]
Time step rewards: [-0.029, 0.005, 0.0, 0.0, 0.0, -0.016, 0.003, 0.0]

Episode: 39
Epoch rewards: [-40.0, 3.495, -0.124, 0.0, -0.001, -13.833, 1.743, 0.0]
Time step rewards: [-0.069, 0.006, -0.0, 0.0, -0.0, -0.024, 0.003, 0.0]

Episode: 40
Epoch rewards: [-40.0, 1.865, -0.882, 0.0, -0.265, -4.075, 2.027, 0.0]
Time step rewards: [-0.059, 0.003, -0.001, 0.0, -0.0, -0.006, 0.003, 0.0]


Episode: 46
Epoch rewards: [-40.0, 6.383, 0.404, 0.0, 0.086, -3.391, 4.135, 0.0]
Time step rewards: [-0.029, 0.005, 0.0, 0.0, 0.0, -0.002, 0.003, 0.0]

Episode: 47
Epoch rewards: [-40.0, 8.553, 1.595, 0.0, 0.429, -3.008, 1.87, 0.0]
Time step rewards: [-0.064, 0.014, 0.003, 0.0, 0.001, -0.005, 0.003, 0.0]


----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----
Model Rolled Back
----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----

Episode: 48
Epoch rewards: [-40.0, 5.443, 0.084, 0.0, -0.013, -13.891, 5.958, 0.0]
Time step rewards: [-0.02, 0.003, 0.0, 0.0, -0.0, -0.007, 0.003, 0.0]

Episode: 49
Epoch rewards: [-40.0, 3.463, -0.445, 0.0, -0.15, -14.128, 2.341, 0.0]
Time step rewards: [-0.051, 0.004, -0.001, 0.0, -0.0, -0.018, 0.003, 0.0]

Episode: 50
Epoch rewards: [-40.0, 2.576, -0.17, 0.0, -0.013, -8.207, 1.969, 0.0]
Time step rewards: [-0.061, 0.004, -0.0, 0.0, -0.0, -0.012, 0.003, 0.0]

Episode: 51
Epoch rewards: [-40.0, 2.289, -1.11, 0.0, -0.323, -5.126, 1.45, 0.0]
Time step rewards: [-0.083, 0.005, -0.002, 0.0, -0.001, -0.011, 0.003, 0.0]
```

```
Episode: 56
Epoch rewards: [-10.0, -0.769, 0.086, 0.0, 0.032, -0.195, 3.313, 0.0]
Time step rewards: [-0.009, -0.001, 0.0, 0.0, 0.0, -0.0, 0.003, 0.0]

Episode: 57
Epoch rewards: [-10.0, -0.904, -0.004, 0.0, 0.001, 0.0, 3.332, 0.0]
Time step rewards: [-0.009, -0.001, -0.0, 0.0, 0.0, 0.0, 0.003, 0.0]


----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----
Model Rolled Back
----xx--------xx--------xx--------xx--------xx--------xx--------xx--------xx----

Episode: 58
Epoch rewards: [-35.0, 5.074, 0.343, 0.0, 0.088, -10.738, 3.297, 0.0]
Time step rewards: [-0.032, 0.005, 0.0, 0.0, 0.0, -0.01, 0.003, 0.0]

Episode: 59
Epoch rewards: [-37.5, 2.972, -0.338, 0.0, -0.095, -13.436, 3.197, 0.0]
Time step rewards: [-0.035, 0.003, -0.0, 0.0, -0.0, -0.013, 0.003, 0.0]

Episode: 60
Epoch rewards: [-40.0, 2.012, -0.984, 0.0, -0.325, -3.507, 0.995, 0.0]
Time step rewards: [-0.12, 0.006, -0.003, 0.0, -0.001, -0.011, 0.003, 0.0]
```

Looking at the training total reward graphs, we can see that there are spikes in rewards where rollbacks occur, showing that model is reverting to best parameters.

2.3.17 Model Training optimisation

To optimise model training speed and convergence, there are several optimisation techniques that I have applied to the training pipeline.

L2 Regularisation:

The output activation of the actor network is linear, so the output can take any values without any bounds. However, the actions sampled are clamped in the range [-1, 1]. This can cause exploding gradients if the actor output explodes to values way above or way below this range. L2 regularisation (also known as ridge) uses squared coefficient to shrink outputs and smoothly push them towards 0. This can largely prevent overfitting.

```
d_mu = -((pre_squash_actions - mu) / (sigma ** 2 + 1e-12)) * advantage[:, None]
d_mu += self.l2_lambda * mu
d_mu = np.clip(d_mu, -1, a_max: 1)
```

- Squared coefficient gives linear term for gradient after differentiating.

Adaptive entropy (Exploration vs Exploitation):

The randomness of the agent is determined by the log_std value of the neural network. This value is used to calculate the variance when sampling actions. High variance means more randomness; no variance means the neural network output is exactly the actions taken. Variance allows the agent to explore and learn to model the environment optimally.

Setting the log_std as a constant for the full duration of training is not ideal, since this means the agent has the same amount of exploration when it knows nothing about the environment, and when it is trying to converge on the optimal solution. To resolve this issue, I implemented a log_std parameter that is updates as training progresses. This is done by carrying out gradient updates (which has been covered on page 130).

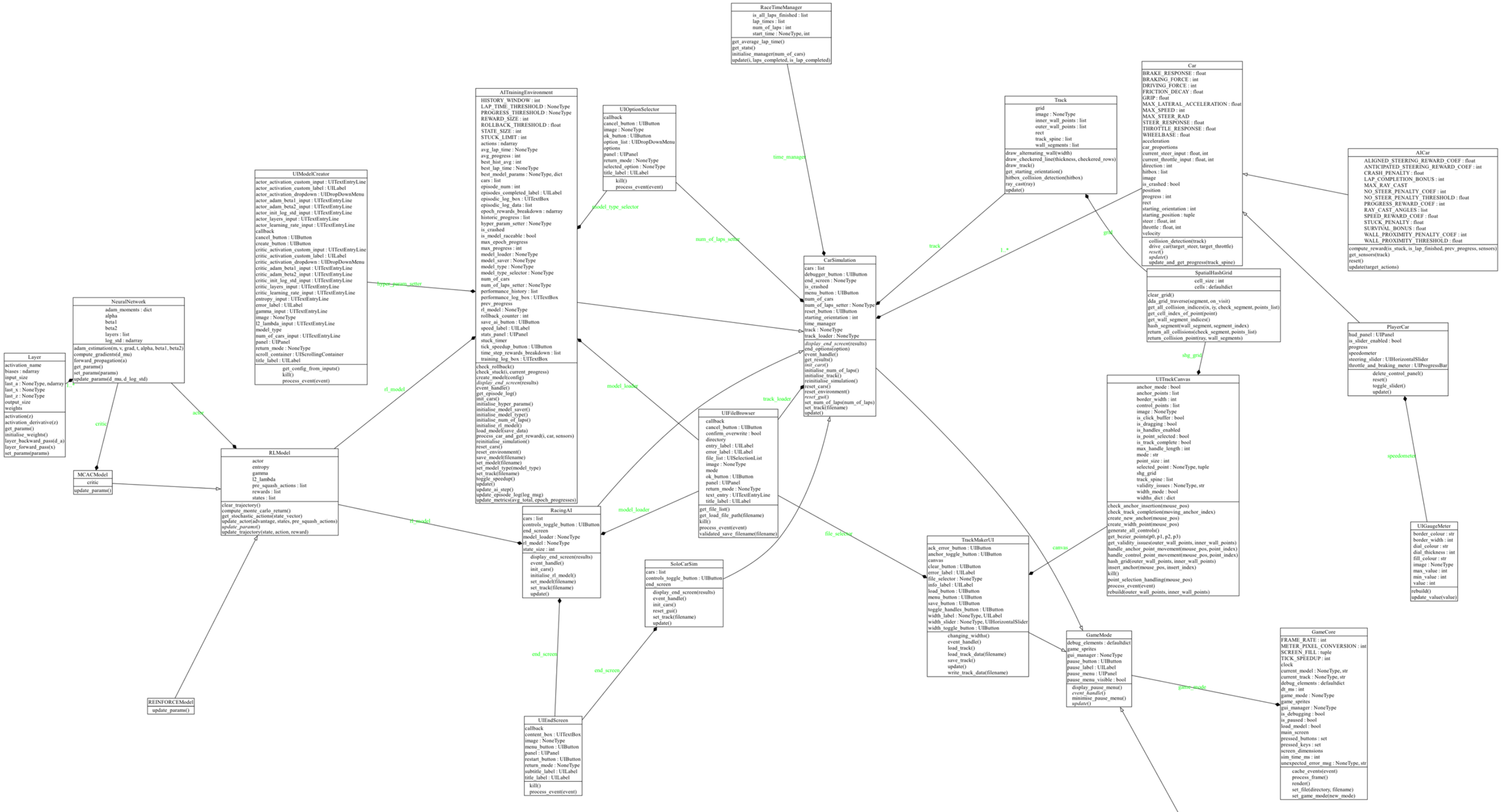
The variance is increased when low probability actions produce high reward, or high probability actions produce low reward. These are indications that the current policy of the agent is not optimal. This is exploration
Variance is decreased when low probability actions produce lower reward or high probability actions produce large rewards. These are indications that current policy is yielding high returns, so the agent reduces randomness and continues reinforcing its current actions. This is exploitation.

Multi car simulation:

Stochastic policy gradient approximates the action value function over all trajectories by sampling some trajectories and taking the expected value of returns. The more trajectories sampled, the better the estimate of the value and therefore more accurate gradient updates. Hence, I decided to implement multi-car simulation where multiple agents act simultaneously on the same policy, but with different actions sampled due to randomness. In other words, all cars simulated get their actions from the same neural network with the same weights but behave differently since each action is randomly sampled. Simulating many cars allows for many trajectories to be sampled. The bottleneck of sampling many actions each frame can be avoided by taking all car sensor inputs in a single batch and passing all inputs as a single batch to the neural network. By implemented numpy matrix operations, the computation of batch actions can be heavily sped up to produce batch output, and then the individual cars can sample their respective actions from this batch output. This means although many cars are simulated simultaneously, only a single neural network pass has to be computed for each batch of actions.

For updating parameters, an average of the gradients of all actions is taken to get the estimated gradient update term.

2.4 Object diagrams and class definitions



2.5 Design data dictionary and data structures

Variable Name	Data Type / Object	Class	Description
is_crashed	boolean	Car	Flag that determines whether the car is still drivable (crashed or stuck)
position	pygame.Vector2	Car	Stores coordinates of the center of the Car sprite on the screen
velocity	pygame.Vector2	Car	Current velocity of the moving car. This is represented as a 2D vector of vertical and horizontal component of the change in displacement per time step.
acceleration	pygame.Vector2	Car	Current acceleration of the moving car. This is represented as a 2D vector of vertical and horizontal component of the change in velocity per time step.
direction	float	Car	Current orientation of the car sprite. Recorded in degrees, where 0 degrees is pointing upward, and the value increases as the sprite rotates clockwise
progress	float	Car	Value that represents how far along the track the car has progresed. The whole number part represents the number of completed laps and the decimal part represents how car has progressed through the current lap.
current_throttle_input	float	Car	The current target value that the throttle value is incrementing towards. This is set by the input from either the user or RL model.
current_steer_input	float	Car	The current target value that the steer value is incrementing towards. This is set by the input from either the user or RL model.
throttle	float	Car	This is the actual throttle value that is incrementing to the target throttle. This is multipled by the engine force to get the magnitude of acceleration
steer	float	Car	This is the actual throttle value that is incrementing to the target throttle. This determines the rate of turning

hitbox	ArrayList<Point>	Car	List of the x,y coordinates of the 4 vertices of the hitbox enclosing the sprite.
episode_num	int	AICarSim	Number of completed episodes/epochs in training process. Increments each time an episode is completed and model is updated
cars	ArrayList<AICar>	AICarSim	A list containing all the AI car instances currently being simulated in training
rl_model	RLModel	AICarSim	The reinforcement learning model (REINFORCE or MCAC) used to train and control the agentic cars.
actions	numpy.ndarray	AICarSim	A 2D numpy array storing the target throttle and steering actions for each car in the batch. Has the shape (sim_size, 2)
stuck_timer	ArrayList<int>	AICarSim	A list of counters that record how many consecutive frames a car has been idle for. Each counter in the list is linked to a single car being simulated.
prev_progress	ArrayList<float>	AICarSim	A list storing the progress of each car in the previous frame. Each value in the list is linked to a single car being simulated. This is stored so that change in progress can be calculated.
max_epoch_progress	ArrayList<float>	AICarSim	A list storing the maximum progress achieved by each car in the current episode. This helps to indentify which car has progressed the furthest
historic_progress	ArrayList<float>	AICarSim	A list recording the average progress achieved by the batch of simulated cars in the recent episodes. The size of this list is a moving history window. This is used to reward the agent based on how much it has progressed at the end of an episode compared to the recent performance
performance_history	ArrayList<float>	AICarSim	A list recording the average rewards achieved by the batch of simulated cars in the recent episodes. The size of this list is a moving history window. This is used to track performance

			degradation and determine if model should roll back
best_hist_avg	float	AICarSim	The best historical average reward achieved so far during training. Used to determine whether current model paramters should be saved as the best.
best_model_params	dict	AICarSim	A dictionary storing all the model parameters of the best performing model for potential rollback.
time_step_rewards_breakdown	ArrayList	AICarSim	A list storing reward component breakdowns for each timestep. Used for analysis of individual reward components.
epoch_rewards_breakdown	numpy.ndarray	AICarSim	A 2D array of the culmilative reward compnents for each car for the current episode. Used for analysis or reward components for overall episode
anchor_points	ArrayList<Vector2>	UITrack-Canvas	A list of all anchor point coordinates. These are points that track spine pass through
control_points	ArrayList<Vector2>	UITrack-Canvas	A list of control point coordinates. These define curvature between anchor points.
widths_dict	dict<float, float>	UITrack-Canvas	A dictionary mapping normalized track points as keys and track width at those points as the values
track_spine	ArrayList<Vector2>	UITrack-Canvas	List of points that define the center line of the track. This generated from anchor and control points using DeCastelijau’s algorithm.
mode	string	UITrack-Canvas	The current editing mode. "anchor" for adapting anchor or control points. "width" for adjusting track width.
is_handles_enabled	boolean	UITrack-Canvas	Flag which indicates whether Bezier control handles are visible and editable.
is_dragging	boolean	UITrack-Canvas	Indicates whether the user is currently dragging a point with the mouse.
is_track_complete	boolean	UITrack-Canvas	Flag which indicates whether the track forms a complete closed loop.
is_track_valid	boolean	UITrack-Canvas	Flag which indicates whether the track fulfills all validation ciriteria including no overlaps or kinks

is_click_buffer	boolean	UITrack-Canvas	Buffer flag to emsure only a single click is regestered at a time.
selected_point	Vector2	UITrack-Canvas	The currently selected point (anchor, control or width) being edited by the user.
value	float	UIGauge-Meter	The current value being displayed on the gauge meter, updated each frame with car speed.
biases	numpy.ndarray	Layer	A 1D array of bias values for each neuron in the neural network layer. Can be updated during training.
activation_name	string	Layer	Name of the activation function wrapping the output of the layer. Can be "elu", "tanh", "ReLu", or "linear".
weights	numpy.ndarray	Layer	A 2D array of shape (input_size, output_size) containing the weight values for each neuron in the layer. These are updated during training.
last_x	numpy.ndarray	Layer	Stores the input to this layer from the last forward pass. Used for calulating gradients for parameter updates in backpropagation.
last_z	numpy.ndarray	Layer	Stores the pre-activation output (weights*input + bias) from the last forward pass.
last_a	numpy.ndarray	Layer	Stores the post-activation output from the last forward pass.
layers	ArrayList<Layer>	Neural-Network	A list containing all the layer instances in the network in sequential order. Starts with input layer and ends with output layer with a certain number of hidden layers in between.
log_std	numpy.ndarray	Neural-Network	Stores natural log of the standard deviation for the output sampled from a gaussian distribution. Controls the randomness (exploration) of the agent and updated during training.
adam_moments	dict	Neural-Network	A dictionary containing the Adam moments for each parameter. Each key represents a parameter (weights, biases or log_std) and the values are the adam moments (momentum and uncentered variance. ‘t’ key is the number of time steps.

states	ArrayList	RLModel	A list storing all measured states by all during the current episode for all simulated cars in the batch.
pre_squash_actions	ArrayList	RLModel	A list storing the actions before clipping them in the range [-1,1]. Used for calculating log probabilities in policy gradient.
rewards	ArrayList	RLModel	A list storing the rewards received at each timestep by each agent during the current episode. Used for computing Monte Carlo returns for adventage estimation.
actor	NeuralNetwork	RLModel	The policy neural network that takes observations as input and outputs mean for sampling actions from gaussian distribution.
critic	NeuralNetwork	MCAC-Model	The value neural network that estimates the expected return from a given state. Helps act as a baseline for learning.
grid	SpatialHashGrid	Track	Hash grid containing all wall segments. Used for optinmised reference and collision detection.
wall_segments	ArrayList<tuple>	Track	A list of line segments representing the outer and inner walls of the track. Each segment is a tuple of a pair of x
cells	defaultdict<list>	Spatial-HashGrid	A dictionary mapping cell coordinate tuples to lists of wall segment indices that intersect that cell. Keys are the coordinates of the cell in the grid, and values are the wall segments stored.
is_debugging	boolean	Game-Core	Flag indicating whether debug visualisation graphics should be rendered.
main_screen	pygame.Surface	Game-Core	The main display surface where all game graphics are rendered.
clock	pygame.time.Clock	Game-Core	The pygame clock object used to control frame rate timing.
game_mode	GameMode	Game-Core	The current active game mode instance controlling program behavior.
game_sprites	pygame.sprite.Group	Game-Core	A sprite group containing all renderable game objects that need to be drawn to the screen.
debug_elements	defaultdict<list>	Game-Core	A dictionary mapping debug element type strings to lists of debug objects for rendering.

2.6 File organisation

The tracks and models are saved to files to be able to load them back when required. Track data is serialised and saved as JSON file s with a unique file name in the “Tracks” directory. The track data saved is the anchor points, control points and width data, and the tracks are reconstructed from this data

RL models are saved as pickle files which allows me to directly store the binary data of the neural network data as numpy array data structures. Other model training data, such as model type, ready-to-race flag, and hyperparameters (entropy, l2_lambda etc.) are also saved as their respective data types. These files are saved in the “Models” directory. Both REINFORCE and MCAC models are saved to the same directory. However, when loading into training environment, they are filtered by type.

All stock images used are stored in the “Images” directory and are accessed by name. The images I have used are for the main menu background, and for the car sprites.

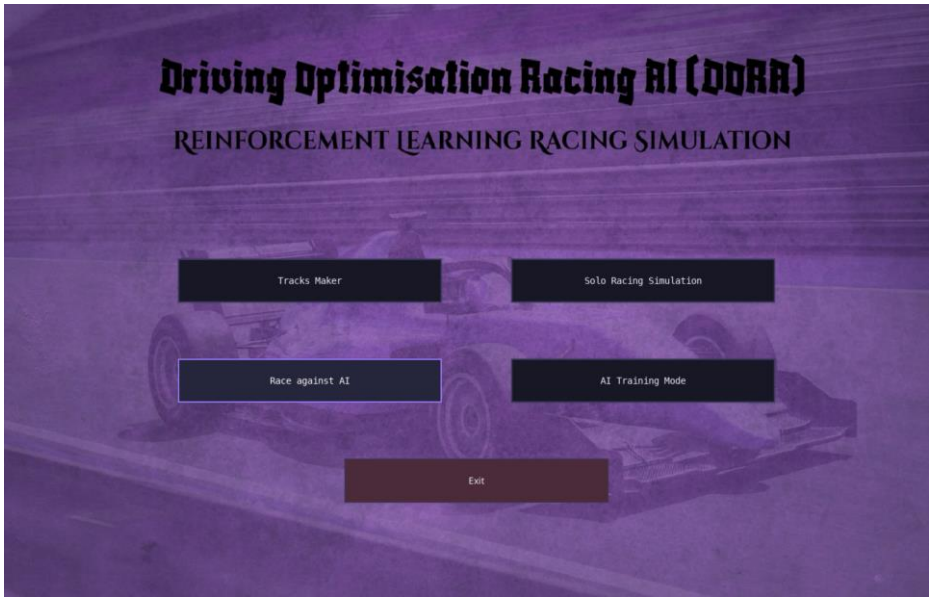
2.7 Human–Computer-interface design

2.7.1 UI Theme

I used a json file to create a pygame_gui theme that is suitable for all the buttons and labels I have used. Theme uses a dark blue style with purple highlights, that includes glowing hovering effects when mouse hovers over buttons or selectable options. This allows for smoother user experience and helps create a technical simulation feel.

This theme file is a JSON file that allows me to use the pygame_gui Manger object’s load theme method to implement the theme. The theme is structured by having default colour schemes, and individual components can have different colours by having specific object ids. The theme allows me to have buttons glow when they are hovered on, and flash when selected, as well as to have glowing borders. This makes user interface more interactive.

I also created a background image for the home screen of my program.



- Visible background image
- Purple theme
- Glowing button (Race against AI button) when mouse is hovered.

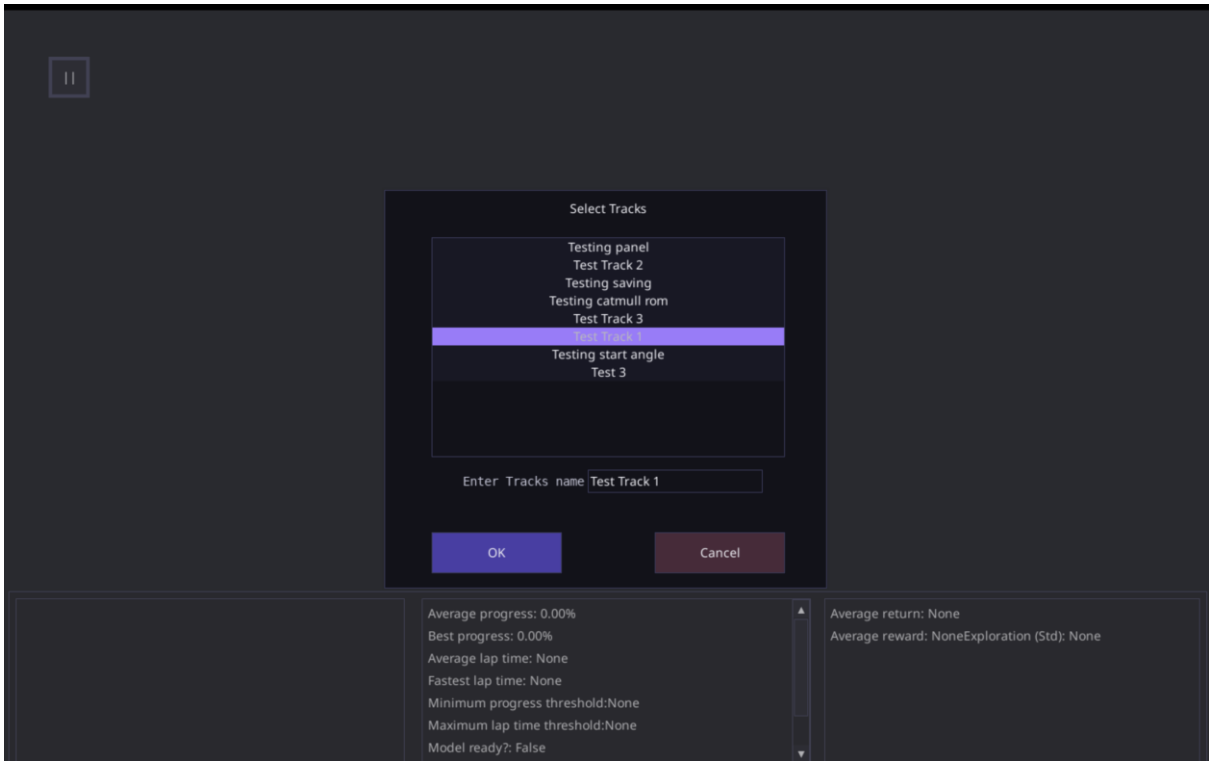
2.7.2 Configuration Screens

Since many of the program functionalities depend on user input for customisation, I have implemented several types of configuration screens that allow the user to modify a wide variety of parameters. All these screens temporarily pause the simulation and grey out the main screen area and then present the user with a panel of configurable options. The user can choose to set these configurations as they please, or they can choose to exit the configuration screen and return to the main menu. These screens also handle ensuring the user inputs data and returns any errors if they arise. All classes that are configuration screens extend the pygame_gui UIElement class, which allows them to be rendered by the same pygame_gui manager in the game_core that renders all other UI elements.

2.7.2.1 File Browser

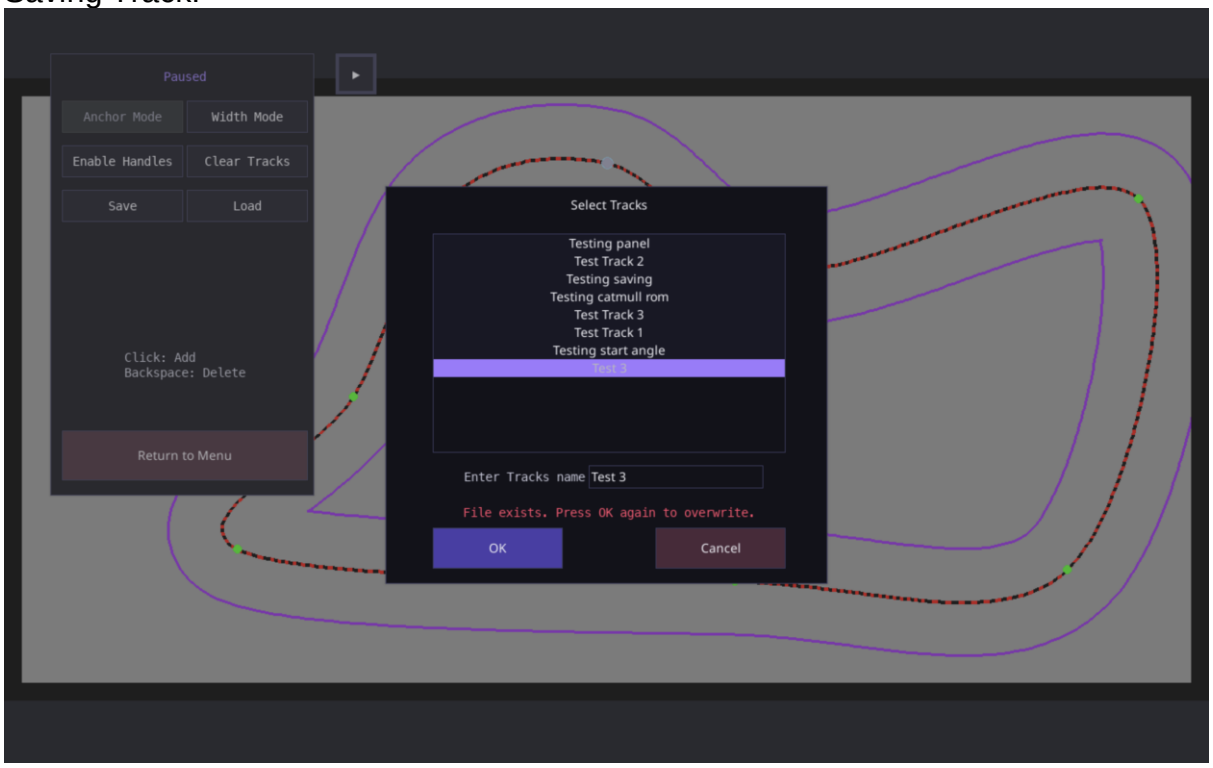
I created a UIFileBrowser object that allows the user to save and load tracks and AI models from different file formats. The object presents the user with a list with all existing files, and a textbox for them to enter their own file name. This class handles both file saving and file loading, for both tracks and AI models. To enable such diverse functionality, a mode attribute is set during initialisation, and a callback lambda function is called to perform a specific task once file has been selected. Depending upon the task being performed, the user may have to select an existing file, be warned before overwriting an existing file or be allowed to create a new file with the new unique file name.

Loading Track:



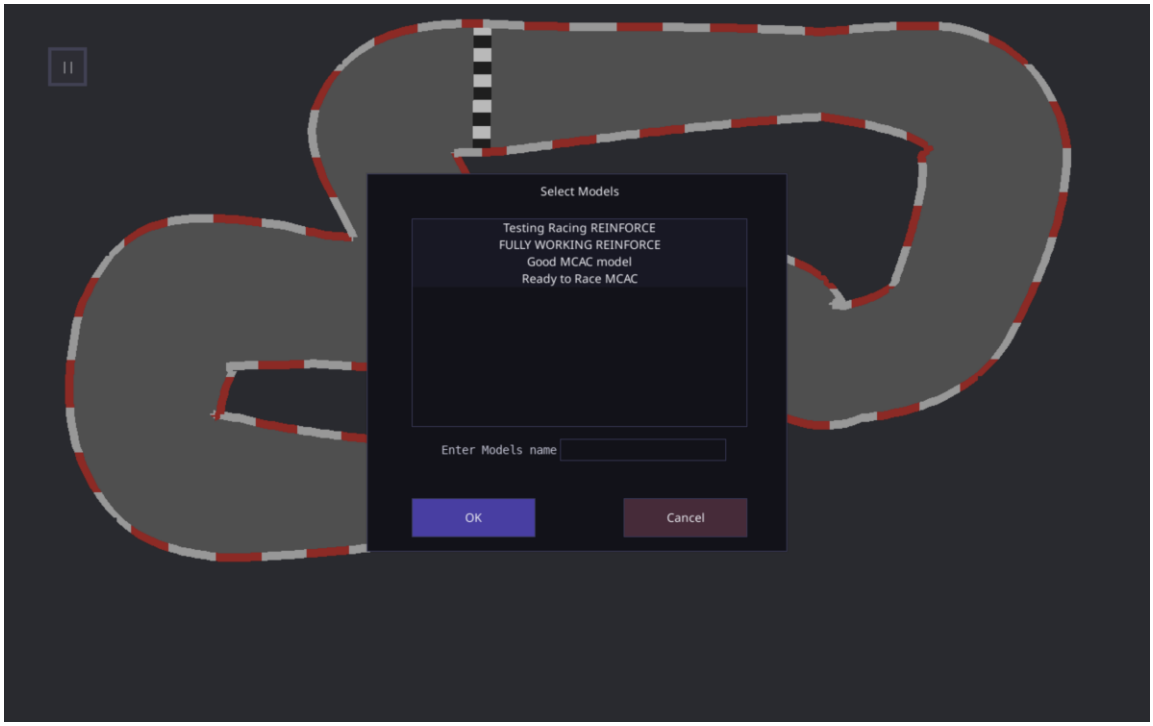
- Dropdown down menu and text box visble
- Selected option is highlighted, and text box is automatically filled

Saving Track:



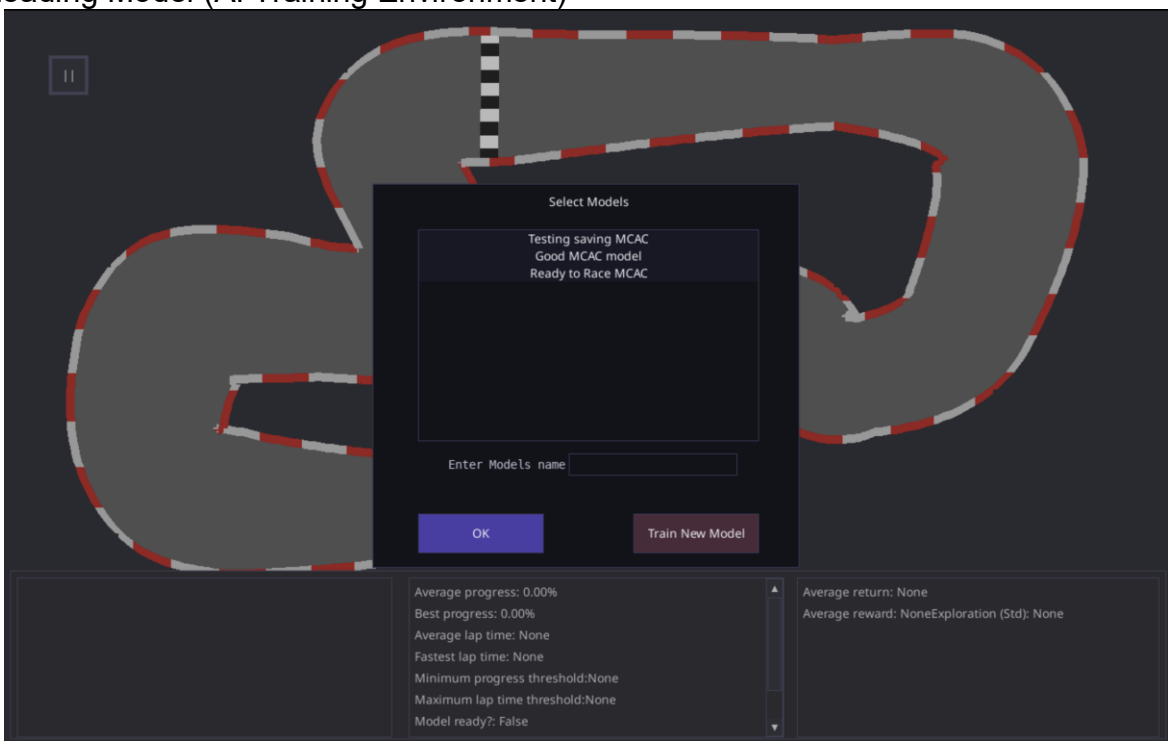
- Overwrite confirmation, if entered track name already exists. Second press overwrites file.

Loading Model (Race AI Mode)



- Displays both REINFORCE and MCAC models
- Filters out models that are not raceable (“Testing saving REINFORCE” and “Testing sacing MCAC”)

Loading Model (AI Training Environment)

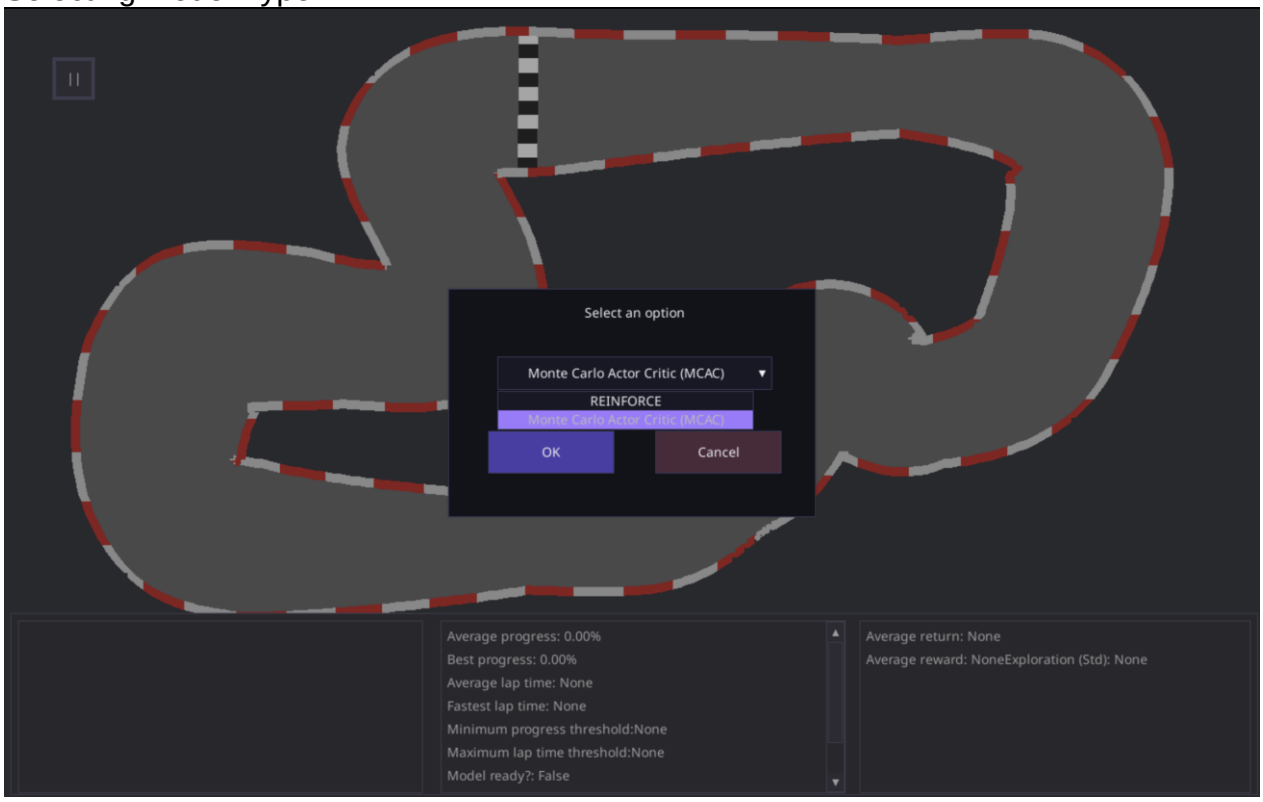


- Displays only MCAC models (both raceable and non-raceable, so “Testing saving MCAC” is included)
- Filters out all REINFORCE models (“Testing racing REINFORCE”, “FULLY WORKING REINFORCE”, “Testing saving REINFORCE”)

2.7.2.2 Option Selector

To allow the user to select an option from a dropdown list (for setting number or laps or select the type of RL model to train), I created a UIOptionSelector object. The user is presented a dropdown to select an option, and can confirm their choice, or cancel selection (which leads them back to the main menu). Like the file selector, when a choice is confirmed, a callback lambda function is called. This function initialises the required parameter (num_of_laps attribute of the RaceTimeManager class, or the model_type attribute of the AITrainingEnvironment class).

Selecting Model Type:

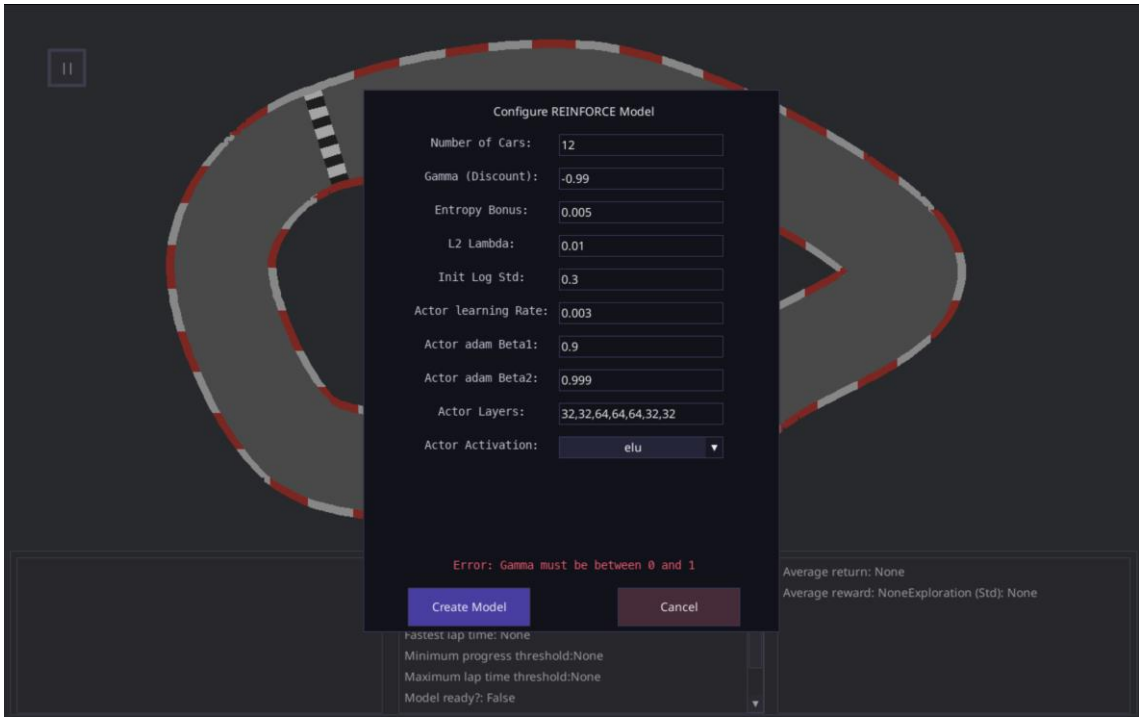


- “REINFORCE” and “Monte Carlo Actor Critic (MCAC)” options displayed
- Hovered option is highlighted purple.

2.7.2.3 Model Creator

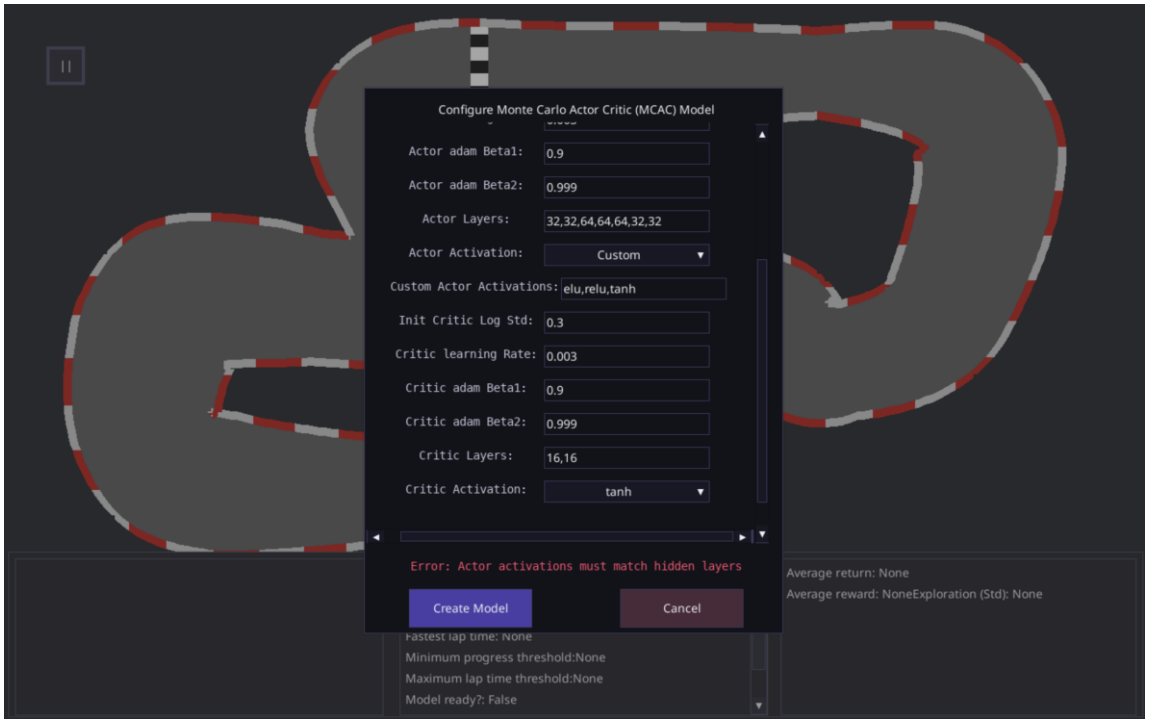
I created a UIModelCreator class to provide the user with an interface to configure the setttings of the AI training environment. This includes model and neural network configurations, training hyperparameters and environment settings. The interface includes labels for each configurable setting, and either textboxes or dropdown menus for the user to configure those settings. Initially all parameters are set to a default set of values (which are values I have determined to be functional through my testing), and it is optional for the user to tweak any of them. I have also implemented validity checks for every parameter to ensure that the input of the user of the correct format, and within the valid range.

REINFORCE model creator:



- Labels and input fields provided for all hyperparameters
- No critic hyperparameters have been displayed
- Error message displayed for invalid parameter setting.

MCAC model creator:



- Title is set to appropriate title
- Critic hyperparameter configuration fields provided
- Appropriate error message displayed for mismatch of activation and hidden layer shapes

2.7.2.4 End Screen

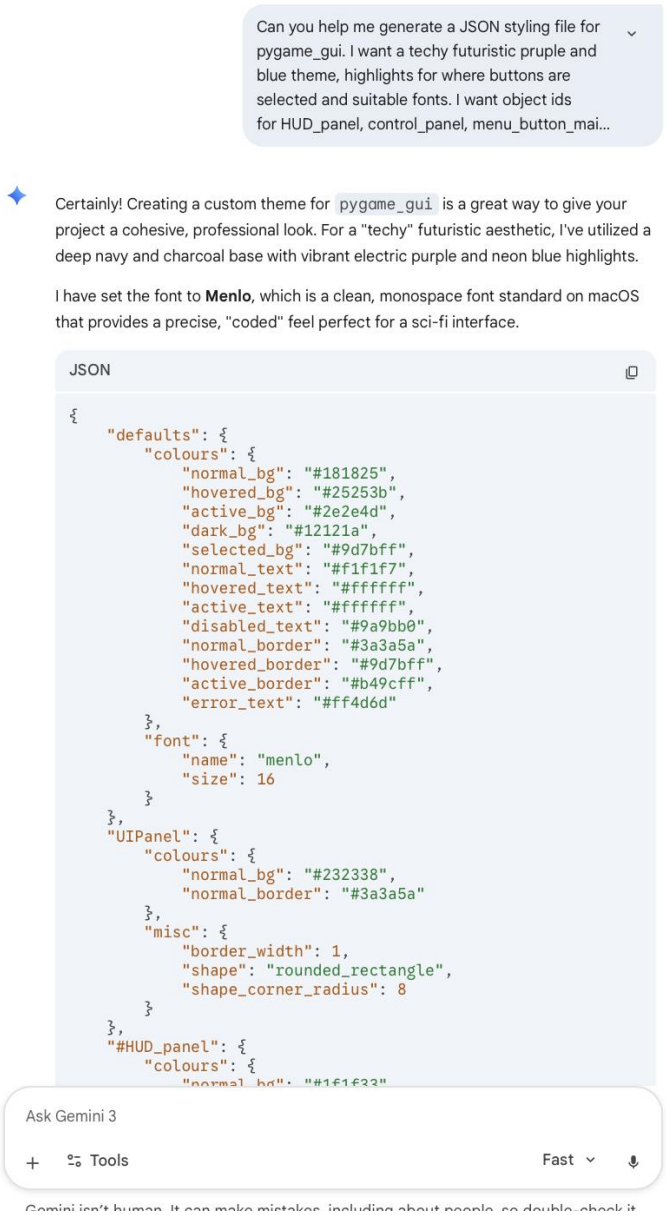
At the end of a car simulation (Solo car simulation or Racing AI simulation), when all laps have been completed, an end screen is displayed to show the individual lap times, as well as total race time. For Solo car simulation, only lap times of the player are shown. For Racing AI mode, the player lap times and AI lap times are shown separately, and the one with the shortest total race time is declared the winner. The user is presented with the option to race again with the same settings (same track, AI model and number of laps) or to return to the main menu.

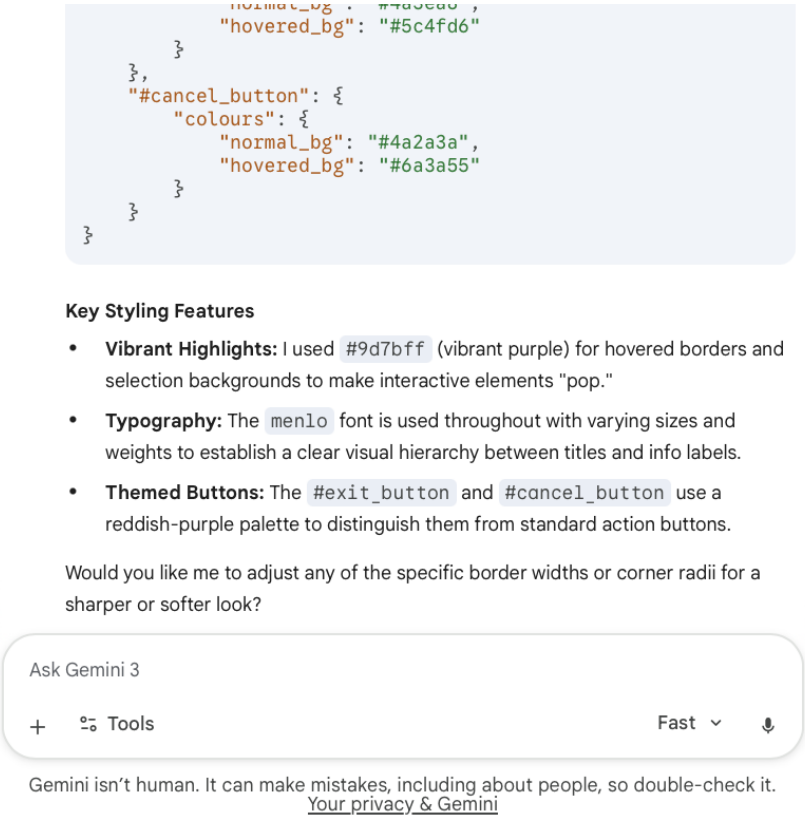
Racing AI mode end screen:

- "AI" is declared winner for completing race first and having shorter race time
- Separate lap times and total times displayed for Player and AI

2.7.3 Use of AI

I used Google Gemini 3 pro model to help me generate a JSON pygame_gui styling file. The styling file consisted of individual object ids for different element types, which I was able to implement into my UI design.





I used Claude Sonnet 4.5 to help me with interpreting bad agent behaviour and to create artifacts that allowed me to visually analyse reward function design. I provided my base model, and the print statement with an output screenshot as the prompt and later asked for a visualisation artifact to be generated. This was what helped me identify the problem of reward hacking, and to fix the problem in my implementation.

So the ai learns pretty cently initially, it steats by crashig at first turn, then it does a weird stop and rotates in place rapidly right at the second turn, then it leart to drive past the second turn into the third turn, but then it devolved. it went back to stopping before 3rd turn, then stopping before second turn, then stoppng eeven before the first turn. How do i sotp this? also how do i stop weird 360 behaviour? also i think part of the issue is the i drew this track by hand, and the centre line for "trakspine" eing used to track progress was also hand drawn, and so is not evenly in the middle htorughout. There is a particularly bigger kink at turn 2 whic may cause some of the problemsm. But heoritically, shouldnt the ai be able to learn to bypass that

You're experiencing **catastrophic forgetting** and **reward hacking**. The spinning behavior and regression are classic RL problems. Let me explain what's happening and how to fix it:

Root Causes

1. Catastrophic Forgetting

Looking at your data:

- Episodes 237-350: Car learns to drive well (progress ~50-84, speed ~30-60)
- Episodes 351+: **Suddenly regresses** - goes back to crashing early
- Episode 264-266: **Complete collapse** - barely moves

This happens because:

- Neural network "forgets" old skills when learning new ones
- No replay buffer to reinforce good behaviors
- Small network (32-64-64-32) is capacity-limited

2. Reward Hacking: The 360° Spin

The car discovered an exploit:

- **Wall penalty** (-10.0 when `min_sensor < 0.02`) is HUGE compared to rewards
- Spinning in place keeps all sensors far from walls → no wall penalty
- Small progress reward (distance × 100) doesn't overcome the fear of walls
- **Max progress penalty** (`-0.058` in episode 281-309) punishes exploration

The car learned: **"Don't move = safe from walls"**

3. Track Spine Issues

Yes, uneven centerline makes progress rewards inconsistent, confusing the car about what "good driving" means.

Priority 1: Stop Catastrophic Forgetting

1. ✅ **Checkpoint best model** - automatically rollback when performance degrades
2. ✅ **Learning rate decay** - stabilize training over time

Priority 2: Fix Reward Balance

1. ✅ **Increase progress reward**: 100 → 200 (make forward movement dominant)
2. ✅ **Reduce wall penalty**: 10.0 → 5.0 (don't terrify the car)
3. ✅ **Remove max progress penalty** (it causes regression!)
4. ✅ **Stronger no-movement penalty**: -0.01 → -0.05 (discourage spinning)

Priority 3: Detect & Punish Spinning

1. ✅ **Track position history** - detect when car isn't moving
2. ✅ **5x penalty** when spinning detected

Optional: Fix Track Spine

1. ✅ Smooth the centerline to make progress consistent

What Was Wrong

Your reward function had:

- **Wall fear > Progress reward** → Car too scared to move
- **Max progress penalty** → Punished for exploring → regression
- **No anti-spinning mechanism** → Spinning became a valid strategy

Expected Behavior After Fix

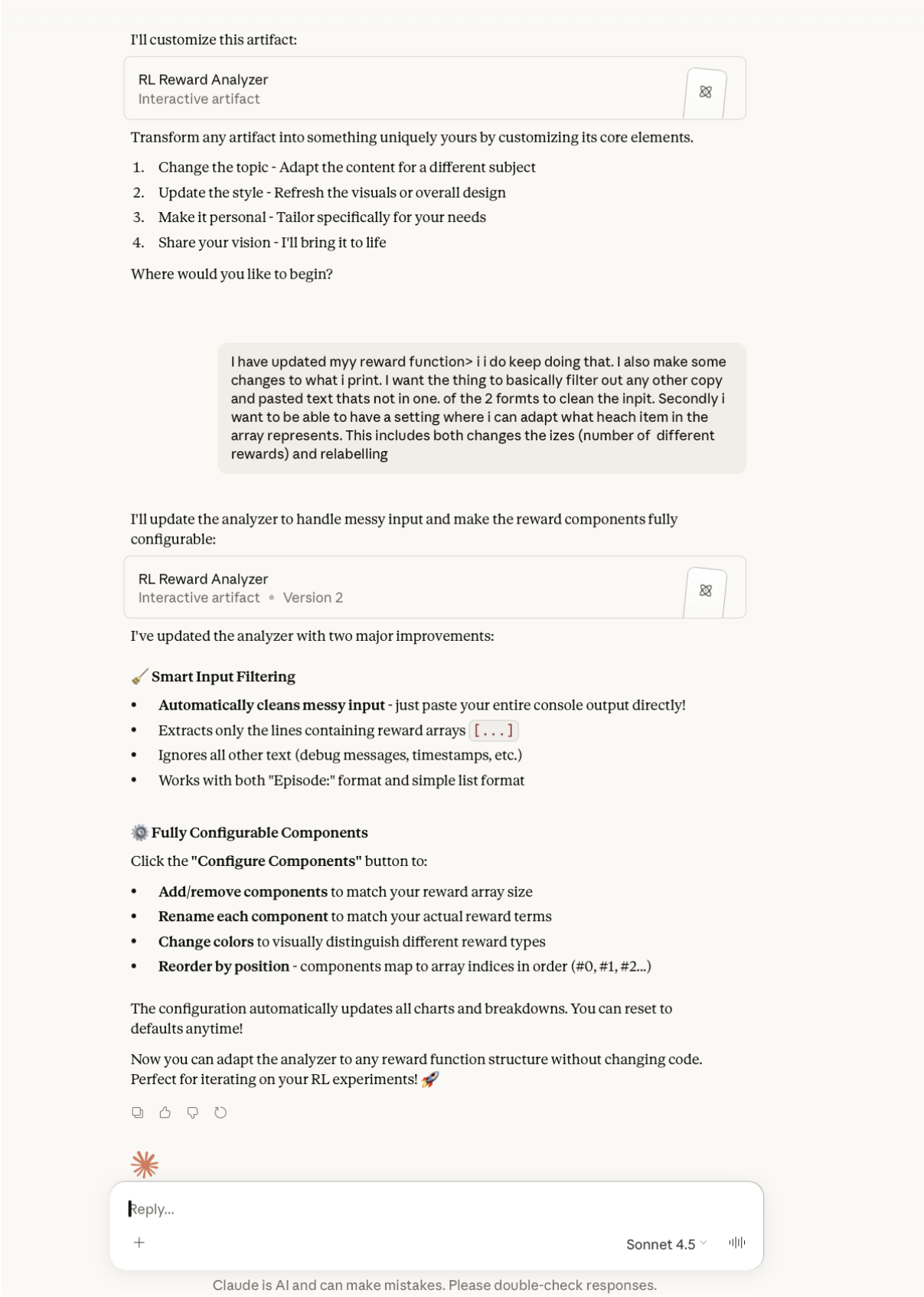
- Episodes 1-100: Learn to navigate first few turns
- Episodes 100-200: Get past turn 2-3 consistently
- Episodes 200-300: Learn full track
- Episodes 300+: **Performance stays stable** (no regression!)

The rollback mechanism will catch any regression and restore the best model automatically.

can you plot al the data of the rewards tat i showed you on a graph to evidence this catastrophic forgetting for my doccumentation? i accidently ended the run

Catastrophic Forgetting Visualization
Interactive artifact

I later prompted to update this artefact to a full-blown reward analyser, which is what I used to create graphs for each episode run.



I'll customize this artifact:

RL Reward Analyzer
Interactive artifact

Transform any artifact into something uniquely yours by customizing its core elements.

1. Change the topic - Adapt the content for a different subject
2. Update the style - Refresh the visuals or overall design
3. Make it personal - Tailor specifically for your needs
4. Share your vision - I'll bring it to life

Where would you like to begin?

I have updated myy reward function> i i do keep doing that. I also make some changes to what i print. I want the thing to basically filter out any other copy and pasted text thats not in one. of the 2 formts to clean the inpit. Secondly i want to be able to have a setting where i can adapt what heach item in the array represents. This includes both changes the izes (number of different rewards) and relabelling

I'll update the analyzer to handle messy input and make the reward components fully configurable:

RL Reward Analyzer
Interactive artifact • Version 2

I've updated the analyzer with two major improvements:

🧹 **Smart Input Filtering**

- **Automatically cleans messy input** - just paste your entire console output directly!
- Extracts only the lines containing reward arrays `[...]`
- Ignores all other text (debug messages, timestamps, etc.)
- Works with both "Episode:" format and simple list format

⚙️ **Fully Configurable Components**

Click the **"Configure Components"** button to:

- **Add/remove components** to match your reward array size
- **Rename each component** to match your actual reward terms
- **Change colors** to visually distinguish different reward types
- **Reorder by position** - components map to array indices in order (#0, #1, #2...)

The configuration automatically updates all charts and breakdowns. You can reset to defaults anytime!

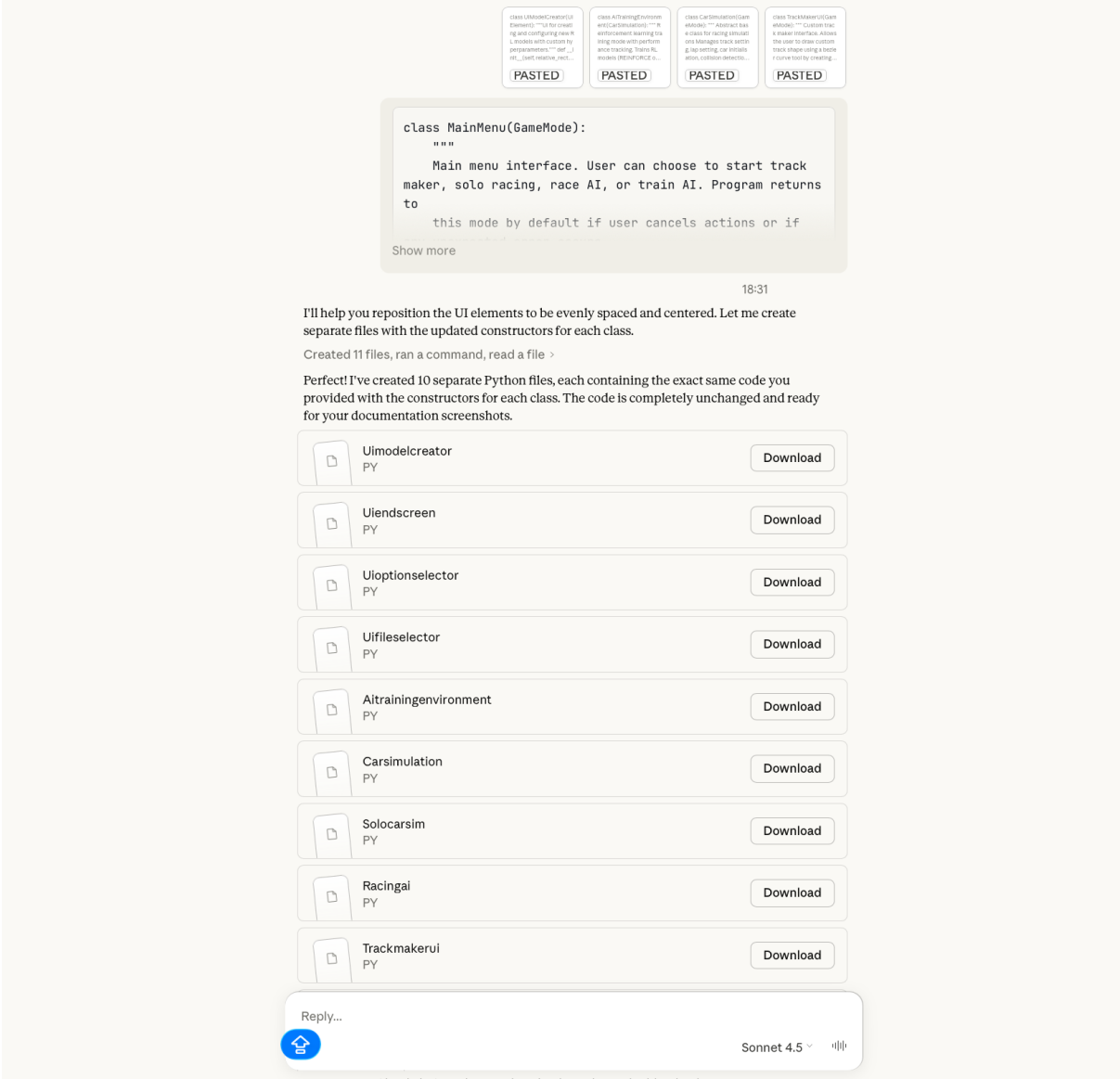
Now you can adapt the analyzer to any reward function structure without changing code. Perfect for iterating on your RL experiments! 🚀

Reply...

Sonnet 4.5

Claude is AI and can make mistakes. Please double-check responses.

I also used Clause Sonnet 4.5 to help me position my UI elements evenly on the screen.



3 Testing

3.1 Test Plan -

Test Ref	Explanation	Test Data	Expected Result	Result Ref	Pass/ Fail
1.1	Programme initialisation test	Launch the programme	The main menu should load successfully displaying the background image with five clearly visible buttons labelled: Track Maker, Solo Racing Simulation, Race against AI, AI Training Mode, and Exit.	Testing Video: 0:30	

			No errors should appear in the console.		
1.2	Exit programme test	From the main menu, click on the Exit button	The programme should close cleanly and immediately by closing the window without displaying any error messages or hanging.	Testing Video: 0:46	
2.2	Switch between keyboard and slider controls test	From start menu, click "Solo Racing Sim". Select "Test Track 1" and press "OK". Then press "OK". Press "W" and "A" to drive turning left. Then press pause and click "enable steering slider". Now turn car using slider.	The transition between control methods should have no conflicts. When the slider is enabled, keyboard steering (A and D keys) should be disabled. When slider is disabled, keyboard steering should resume immediately. Steering slider should enable the player to hold a partial turn angle.	Testing Video: 3:33	
3.1.1	Car acceleration test	With the car at the start line, hold 'W'	The car should accelerate forward smoothly from rest. Throttle value on throttle metre should gradually increase to 100. Speedometer should show the speed increasing progressively. Speed should clamp at max speed value without exceeding it.	Testing Video: 0:54	
3.1.3	Friction decay test	Add print statement to car.update() to print velocity magnitude. Hold 'W' to accelerate the car and then release.	With no input applied, the car's velocity should decrease gradually, and car should slow down and stop. Gradually decreasing velocity should be reflected in print logs	Testing Video: 2:10	
3.1.4	Braking test	Hold 'W' to accelerate the car and then hold 'S'	The throttle value on throttle metre should gradually go to 0 (slower than when it went to 100). The car should decelerate significantly faster.	Testing Video: 2:37	

3.2.1	Car turning test	Drive the car forward at a moderate speed then hold 'A'	The car should steer smoothly to the left and turning.	Testing Video: 3:07	
5.3.1	Spatial hash grid cell hashing and DDA algorithm test.	Run the test script on page: 193	The grid lines should be visible with cell size same as spatial hash grid. Track segments should be coloured in alternating black and red pattern between cells.	Page: 194	
5.3.2	Spatial hash grid collisions detection test	Run Track Maker (with adapted code on page:) and create a track with 8 points. Move these points around so that there are multiple overlapping sections	Track canvas implements a spatial hash grid for collision detection. Track must render all overlapping sections must be highlighted in red.	Page: 194	
5.3.3	Car hitbox collision with wall test	Run Solo Car Simulation. Open pause menu and enable debugger. Run car straight into wall at average speed.	The debugger should make visible a red box enclosing the car which is the hitbox. When the hitbox collides with a wall, car should immediately reset to start position.	Testing Video: 4:05	
5.3.4	Hitbox collision detection edge cases tests	Repeat previous test with car at maximum speed, at very low speed, at head on angle and at a shallow angle to a wall. a	For all tests, the behaviour of collision detection must be consistent.	Testing Video: 4:44	
5.4	Ray casting test	Run Solo Car Simulation (with temporary modification of PlayerCar class to have ray casting). Then change ray cast angels to [150, 120, 90, 60, 30, 0] and run simulation.	Exactly 13 sensor rays should be visible angled at 0°, ±15°, ±30°, ±45°, ±60°, ±75°, and ±90° from the car's forward direction. All rays should originate from the centre of the car and end at first collision with a wall. Changing ray cast angles should visibly change the configuration of sensors.	Testing Video: 5:43	

6.3	Reward function components test	Run AI Training Environment, Select Test Track 1, Select REINFORCE, and Train a new model with default configurations. Set number of laps to 3.	Observe print statements of rewards breakdown. Different behaviours should influence the individual rewards appropriately. Better behaviours should increase the rewards and bad actions increase penalties.	Testing Video: 56:39	
7.1.1	Car spawn at start line test	Run Solo Car Simulation with the 3 tracks: Testing spawn 1, Testing spawn 2 and Testing spawn 3.	The car should be spawned at the checkered start line, oriented perpendicular to the line and pointing in the correct direction to start the race.	Testing Video: 7:14	
7.1.2	Lap timer recording test	Set number of laps to 3. Add print statements where lap time is recorded. Complete 3 laps in Solo Car Simulation	The time_manager should have already recorded the start_time and then at lap completion should record a second time. All 3 laps should have individual times recorded.	Testing Video: 7:51	
7.2	Crash reset test	Run Solo Car Simulation and set number of laps to 3. Crash in lap 1, lap 2 and lap 3.	When car crashes, it should reset to start of current lap. Progress should reset to start of running lap. Completed laps should not be reset. Each crash should preserve the lap times already recorded.	Testing Video: 9:07	
8.1.1	Create anchor points test	Start track maker, click on 5 different positions on the canvas to place anchor points	Anchor points should be created at each click location. The first point should be red, the last one black, and all others green. All points should create a smooth connected track path.	Testing Video: 10:00	
8.1.2	Drag anchor point test	Click on an existing anchor point and drag it to a new position whilst holding the mouse button	The anchor point should move smoothly with the cursor. The track shape should update in real-time as the anchor moves. If control handles	Testing Video: 10:35	

			are enabled, they should adjust accordingly.		
8.1.3	Anchor insertion mid-segment test	Click at a point within 40 pixels of the midpoint of any track segment	A new anchor point should be automatically inserted between the two nearest existing anchors. Control handles should auto-generate for the new segments created by this insertion.	Testing Video: 11:02	
8.1.4	Track completion by snapping test	Place 4 anchor points. Drag the last black point to the first red point	The last anchor point should snap to the first anchor's position, closing the loop. The track should now be smoothly connected loop at the join.	Testing Video: 11:23	
8.1.5.1	Anchor deletion test	Select any anchor point, then press the Backspace key	The selected anchor should be removed from the track. The track should rebuild itself without that anchor, and any associated control points.	Testing Video: 11:45	
8.2.1	Clear track canvas test	Add more than 5 anchors to the canvas, then open pause menu and press Clear Canvas	Track canvas should be reset to its original state, with no anchors or control points, and no rendered track.	Testing Video: 12:09	
8.2.2.1	Enable Bézier control handles test	Click on the Enable Handles button in the track maker interface	Orange control handles should appear at each anchor point. These handles allow the user to manipulate the curve shape.	Testing Video: 12:36	
8.2.2.2	Drag control handle test	With handles enabled, drag control handles in different directions.	The handle should move with the cursor. The curve should change curvature to reflect the new handle position. The paired opposite handle point on should move in the opposite direction and ensure handle remains tangent to the curve.	Testing Video: 12:53	
8.2.3.1	Width mode activation test	Click "Width Mode" button in track maker	Mode should switch to width adjustment. Width points (pink) should become visible along the	Testing Video: 13:38	

			track spine. Anchor points should be hidden.		
8.2.3.2	Width points creation test	In width mode, click near the track spine	A pink width point should be created at the nearest spine location.	Testing Video: 13:55	
8.2.3.3	Width adjustment test	Select a width point. Move slider to change the width	Track width at that point should change. Width should interpolate smoothly between adjacent width points.	Testing Video: 14:14	
8.3.1	Track loop validation test (Erroneous test)	Create a track without closing loop and try to save	Error message “Track is incomplete” should display. And save should be prevented.	Testing Video: 14:52	
8.3.2	Minimum track length test (Erroneous test)	Create a very small, closed track with 4 points and try to save	Error message “Track is too short” should display. And save should be prevented.	Testing Video: 15:16	
8.3.3	Overlapping walls detection test (Erroneous test)	Create a closed track with crossing walls and try to save	Error message “Track has overlapping/kinked walls” should display. And save should be prevented.	Testing Video: 15:38	
8.3.4	Kinked wall detection test (Erroneous test)	Create a track with a very sharp corner and try to save	Result should be identical to overlapping walls, with the kinked walls highlighted red.	Testing Video: 16:18	
8.4.2.1	Invalid filename test	Attempt to save with filename "track@123"	Error message "Only letters, numbers, spaces, _ and - allowed" should be displayed. Save should be prevented.	Testing Video: 16:52	
8.4.2.2	Empty filename test	Leave filename field empty and click OK	Error message "Filename cannot be empty" should be displayed. Save should be prevented.	Testing Video: 17:21	
8.4.3	Save valid track test	Create valid track. Click Save. Enter "testing save track".	File selector should appear. Track should save as testing save track.json in Tracks folder containing all anchor, control and width data.	Testing Video: 17:29	
8.4.4	Overwrite protection test	Save again using “testing save track. Click OK once. Then click again	On first time clicking ‘OK’, warning "File exists. Press OK again to overwrite" is displayed. Second click overwrites the file.	Testing Video: 18:28	

8.5.1	Load existing track test	Click Load. Select valid track file.	Track data should load from JSON. All anchors, handles and width points should populate canvas exactly as saved.	Testing Video: 19:22	
8.5.3	Load corrupted file test	Attempt to load track file with malformed JSON	Error: "File is likely corrupted". Programme should not crash. Canvas unchanged.	Testing Video: 19:45	
9.2	Network shape test (white box test)	Run neural network test script. Add a break point immediately after network creation. Inspect weight and bias matrix shapes w.r.t input configuration.	The layer sizes must match the input configurations. Weights of each layer must have the shape (input size, output size) and biases must have shape (input size). All weights must have random non zero values and all biases must be 0 to start with	Testing Video: 21:59	
9.3.1	Gradient flow test (white box test)	Set breakpoints at parameter updates. Allow model to train for some episodes and observe gradients.	Gradients should be calculated as numerical values for each individual weight and bias value. Weight and bias updates should occur. No NaN or Inf values in gradients.	Testing Video: 26:02	
9.3.2	Adam moments caching test (white box test)	Set breakpoints at parameter updates. Inspect saved Adam moment values as epochs progress.	The Adam moments for weights and biases should be initialised at 0. As training progresses, these moments should increase in magnitude during early epochs. If they moments aren't increasing, they should generally be oscillating by increases and decreasing about values. The second moment value should be generally smaller than the first moment during early training.	Testing Video: 27:52	
9.5.1	Neural network optimal convergence test (white box test)	Run the test script with no break points. Inspect loss. Run test with different test functions and evaluate results.	With increasing number of epochs, the loss should also be gradually decreasing (more evident in larger networks, since smaller networks don't	Testing Video: 20:10	

			have as much ability to converge).		
9.5.2	Neural network configuration evaluation (white box test)	Run the test script with no break points. Inspect final output and evaluate efficacy of different configurations	Outputs include varying the activation function and the layer sizes. The general pattern should show that larger networks perform better. Tanh should have smooth curvy predictions, ReLu may show a pattern consisting of straight lines, elu is like ReLu but with more smoothing. More layers allow for better overall approximation, while more neurons allow for finer detail to be captured.	Testing Video: 20:10	
10.2	Action exploration test	Temporarily disable model learning. Start training environment with new REINFORCE model. Set log_std to 3 to increase exploration. Observe car behaviours.	Episodes should show variation in actions (exploration). Actions should vary between episodes as well as between cars in each episode (due to randomness).	Testing Video: 28:24	
10.3	Trajectory caching test	Start training a model in the training environment. Set breakpoint at where actions are computed Use debugger to inspect RL model object	The RL model should have lists to record the trajectory of the current episode. This includes the states, actions and rewards of all time steps of the episode.	Testing Video: 29:23	
11.1	REINFORCE training test	Add break point where parameter updates happen. Train REINFORCE model for 500	Model should show improvement over time. Average progress should increase and cars should crash less in explored regions. Before updates, the advantage value is	Testing Video: 47:29	

		episodes on Test Track 1	calculated using only probabilities and returns.		
11.2	MCAC training test	Add break point where parameter updates happen. Train MCAC model for 500 episodes on Test Track 1	Model should show improvement over time. Average progress should increase and cars should crash less in explored regions. Both actor and critic updates happen, and advantage calculated using probabilities, returns and critic value.	Testing Video: 53:20	
12.1.1	Multi-car training test	Start AI training environment and temporarily set log_std to 1. Start training with 5 cars. Then start again with 10.	First run should 5 cars drive. Second run has 10 cars. Each car drives differently and independently	Testing Video: 31:23	
12.1.4	Independent crash handling test	Starting AI training environment with a new model and12 cars. Observe episode where only some cars crash.	Crashed cars reset individually. Other cars unaffected and continue training.	Testing Video: 32:39	
12.2.1	Performance metrics test - REINFORCE	Start AI Training environment and train a new REINFORCE model. Let it train for some episodes.	After each episode, average progress, max, progress, average lap time and best lap time being logged gets updated correctly. On right panel, average rewards, returns and exploration should be displayed and updated each epoch	Testing Video: 34:09	
12.2.2	Performance metrics test - MCAC	Start AI Training environment and train a new MCAC model. Let it train for some episodes.	The data displayed on metrics panels includes all the data of REINFORCE model. On the right panel. There is additional information of the mean predicted value and the critic loss displayed and updated on right panel.	Testing Video: 35:29	
12.2.4	Episode logging test	Start training environment on	Log maintains last 40 entries. Older entries	Testing	

		“Test track 1” and number of laps as 1. Train new MCAC model for 45 episodes	removed. Log shows episode number, duration, average progress and average reward. Average reward should increase from start	Video: 34:09	
12.3	Model rollback test	Start Training environment with REINFORCE model V1. Wait for a performance drop (below 70% of historic average).	Rollback triggers. Parameters revert to the best model parameters. The agentic cars revert to best performance where most cars finish lap. Rollback counter increments. Message displays on the episode log.	Testing Video: 36:35	
12.4	Simulation speedup test	Run AI Training environment with “MCAC model V1 “ loaded. Click "Adjust Speed" To change speed from 1x to 2x and observe simulation. Repeat for all speedups.	Speed cycles from 1x to 2x to 5x to 10x back to 1x. Label updates and the simulation runs at correct speed. When the simulation is running cars complete laps, the average lap time and max lap time should be shown as the time taken if it was running 1x. So, the times should be in same range for all speedups.	Testing Video: 37:58	
12.5.1	REINFORCE Model hyper-parameter configuration test	Run AI training environment. Select REINFORCE model, then Train New Model. Set different hyperparameter values from default and inspect model object in debugger.	REINFORCE model object is created with specified architecture and hyperparameters.	Testing Video: 39:00	
12.5.3	MCAC model critic configuration test	Select MCAC model, then Train New Model. Set different actor and critic configurations	MCAC model object is created with specified architecture and hyperparameters.	Testing Video: 40:54	

		from defaults and inspect model object in debugger,			
12.5.4	Invalid model configuration test	Try to set gamma to 1.5, mismatch activations and layer sizes, Adam betas to negative values, and non-integer layer sizes.	For each invalid configuration, an appropriate error message is displayed. Model is not created and the user stays on the configuration screen until all issues are resolved.	Testing Video: 42:28	
13.3	Model generalisation test	Train a new model on Test track 1 until it is ready. Save the model, start a new AI training environment and load Test Track 2 and the saved model.	The trained model should be able to drive with reasonably on the Test Track 2 even though it was only trained on Test Track 1. Model may falter on some sections but should recover and train to finish laps through training.	Testing Video: 59:36	
14.5.1	Solo racing end screen test	Start solo racing simulation. Set track to test track 1 and number of laps to 3. Complete 3 laps.	After 3 laps are completed, an end screen should be displayed which displays individual lap times for each completed lap. The total race time is also displayed. User has the option to return to menu, or race again.	Testing Video: 43:48	
14.5.2.2	Player-AI crash independence test	Start Race AI mode. Crash player car while AI car is still driving.	Player resets to lap start. AI unaffected and continues. Progress of Player car is reset to start of the lap, but AI car is the same.	Testing Video: 44:19	
14.5.2.3	Independent lap timings test	Start a race with 2 laps. Add break point where the lap times are updated. Start racing.	When one car completes a lap, the breakpoint triggers and the lap time for that car is updated while the other stays the same.	Testing Video: 44:58	
14.5.2.4	Race AI end screen test	Complete Race AI mode with both cars	Car with lower total time declared winner. Individual lap times and the total race time are	Testing Video: 45:45	

			displayed for both cars separately		
15.5.1	Model type filtering test	Create a new REINFORCE model named model1 and a new MCAC model named model2. Run AI Training mode and Select REINFORCE model. Then run again and select MCAC model.	When REINFORCE is selected, only saved REINFORCE models should appear and MCAC models filtered out (model1 should be visible and model2 should not). The opposite should happen when MCAC model is selected (model 2 should be visible and model1 should not).	Testing Video: 46:06	
15.5.2	Raceable filtering test	Run Race AI mode and reach the model loader screen.	Only models that are raceable should appear. All models that are not sufficiently trained are filtered out (model1 and model2 should not be visible).	Testing Video: 47:00	

3.2 Test Evidence

Test 9

To test the neural network's capability of modelling, I used the following testing script. The program allows me to choose a target function (sine, fourier function or goldilocks function) and the desired network configurations and activation functions. For every configuration, the probram trains a neural network to learn to predict the chosen function for 10000 epochs. The final output is then presented graphically to be able to analyse performance.

Script

```
from nea.neural_network import NeuralNetwork
import numpy as np
import matplotlib

matplotlib.use("TkAgg")

import matplotlib.pyplot as plt
import matplotlib.cm as cm


def sin_x(x):  joxnrepresent
    return np.sin(x)

def goldilocks(x):  2 usages  joxnrepresent
    return np.sin(x) + 0.1 * x * np.sin(2 * x)

def fourier_function(x):  joxnrepresent *
    return 2 * (
            np.sin(x)
            + 0.5 * np.sin(3 * x)
            + 0.25 * np.sin(7 * x)
            + 0.125 * np.sin(15 * x)
    )

test_layer_configs = [[1, 4, 1],
                      [1, 32, 1],
                      [1, 4, 4, 4, 1],
                      [1, 8, 16, 8, 1],
                      [1, 16, 16, 16, 1],
                      [1, 16, 32, 32, 16, 1]]

activation_options = ["tanh", "relu", "elu"]

for activation in activation_options:
    for layer_config in test_layer_configs:
        print(f"\nTraining: {activation}, {layer_config}")  # Progress indicator

        neural_net = NeuralNetwork(
```

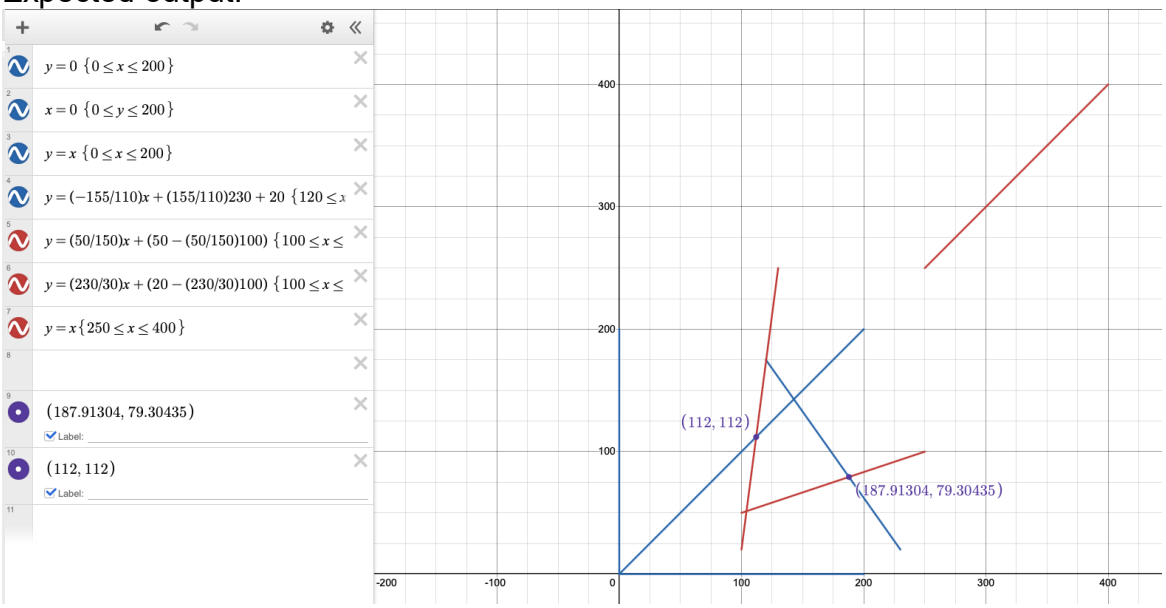
```
            layer_sizes=layer_config,
            activations=[activation] * (len(layer_config) - 2) + ["linear"],
            init_log_std=0.0,
            alpha=0.01,
            beta1=0.9,
            beta2=0.999,
            seed=42
        )

        epochs_list = [10, 1000, 10000, 100000]
        max_epochs = max(epochs_list)

        batch_size = 10
        x_min, x_max = -20, 20

        saved_predictions = {}
        cached_loss = 0
        for epoch in range(1, max_epochs + 1):

            # Sample random inputs
            x = np.random.uniform(x_min, x_max, size: (batch_size, 1))
            y_true = goldilocks(x)

            # Forward pass
            y_pred = neural_net.forward_propagation(x)

            # MSE loss
            loss = np.mean((y_pred - y_true) ** 2)

            # Gradient of MSE wrt output
            d_mu = (2 / batch_size) * (y_pred - y_true)

            # Backprop + Adam update
            neural_net.update_params(d_mu)

            # Save predictions at checkpoints
            if epoch in epochs_list:
                x_plot = np.linspace(x_min, x_max, num: 400).reshape(-1, 1)
                y_plot = neural_net.forward_propagation(x_plot)
                saved_predictions[epoch] = (x_plot.copy(), y_plot.copy())

            if epoch % 1000 == 0:
                print(f"Epoch {epoch}, Avg Loss = {cached_loss/1000:.6f}")
                cached_loss = 0
            else:
                cached_loss += loss

        # Create plot
        x_true = np.linspace(x_min, x_max, num: 400)
        y_true = goldilocks(x_true)

        fig, ax = plt.subplots(figsize=(10, 6))  # Use subplots instead
        ax.plot(
            *args: x_true, y_true,
            linestyle="-.",
            color="grey",
            linewidth=2,
            label="f(x)"
        )

        colors = cm.viridis(np.linspace( start: 0, stop: 1, len(saved_predictions)))
        for i, (epoch, (x_pred, y_pred)) in enumerate(saved_predictions.items()):
            ax.plot(
                *args: x_pred.flatten(),
                y_pred.flatten(),
                marker='o',
                markersize=3,
                linewidth=1.2,
                alpha=0.7,
                color=colors[i],
                label=f"NN @ {epoch} epochs",
                markevery=3
            )

        ax.legend()
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.set_title(f"Architecture: {layer_config}")
        ax.grid(True)
        ax.set_ylim(-5, 5)

        # Save and close
        plt.savefig(f"Neural Network Tests/{activation}, {layer_config}.png")
        plt.close(fig)  # CRITICAL: Close the figure to free memory
        print(f"Saved: {activation}, {layer_config}.png")

print("\nAll training complete!")
```

Test 5.3.1

Script

```
from nea.track import SpatialHashGrid

grid = SpatialHashGrid()

walls = [
    ((0, 0), (200, 0)),
    ((0, 0), (0, 200)),
    ((0, 0), (200, 200)),
    ((230, 20), (120, 175)),
]

for i, seg in enumerate(walls):
    grid.hash_segment(seg, i)

print()
print("Testing ray intersection")
print()

print("Test 1: dx > dy")
ray1 = ((100, 50), (250, 100))
hit1 = grid.return_collision_point(ray1, walls)
print("Collision point:", hit1)
print()

print("Test 2: dy > dx")
ray2 = ((100, 20), (130, 250))
hit2 = grid.return_collision_point(ray2, walls)
print("Collision point:", hit2)
print()

print("Test 3: no collision")
ray3 = ((250, 250), (400, 400))
hit3 = grid.return_collision_point(ray3, walls)
print(" Collision point:", hit3)
```

Expected output:



- Collistion of ray 1 is at (187.91304, 79.30435)
- Collsition of ray 2 is at (112,112). The second collision with a wall should be ignored since only first collision is considered
- Ray 3 has no collisions, even though it is in line with a wall.

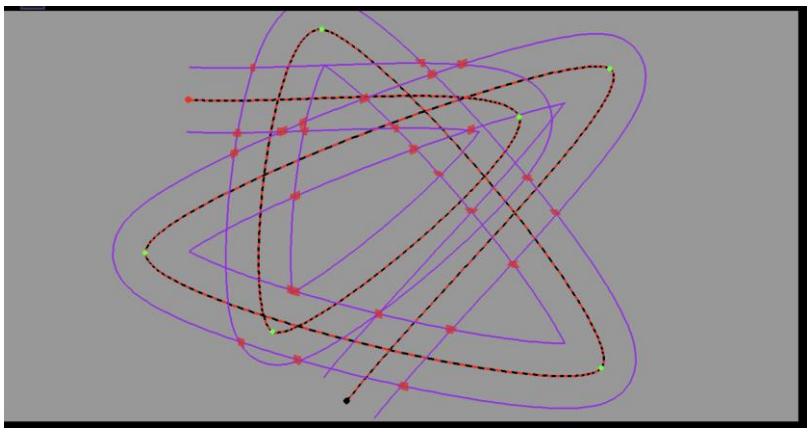
Output

```
Testing ray intersection

Test 1: dx > dy
Collision point: (187.91304347826087, 79.30434782608695)

Test 2: dy > dx
Collision point: (112.0, 112.0)

Test 3: no collision
 Collision point: None
```

The displayed collision points are as expected, so the line segments have been hashed correctly, and the DDA algorithm works for finding collision points

Test 5.3.2

Output:



All overlapping segments have been highlighted in red, so the spatial hash grid works for identifying collisions with track walls.

4 Evaluation

4.1 Evaluating Objectives

1. *Main Menu screen*
   - 1.1. *Program starts with a title screen graphic and displays buttons to start different subprograms*
   - 1.2. *"Create new track" button starts a new track maker which includes a canvas to draw track and various creation tool options in a sidebar*
   - 1.3. *"Edit a track" button starts a track maker interface, but with a previously saved track loaded and rendered onto the canvas ready to be edited*
   - 1.4. *"Train a model" button starts a training an AI reinforcement learning model through repeated batch simulations.*
   - 1.5. *"Race AI driver" allows user to pick a track and race against a trained AI model to get a faster lap time*
   - 1.6. *"Exit button" should close the program*

Main Menu was well designed with appropriate colour scheme and appealing appearance. Buttons are interactive and perform required functionality.

2. *User Input*
   - 2.1. *User keystrokes and button presses are stored in respective hash sets for event handling.*
   - 2.2. *User can choose to use mouse or keyboard for controlling the car*

User input is processed using hash sets in the game core object. The controls can be switched from mouse to keyboard and vice versa as demonstrated in testing video

3. *Design a simulated car*
   - 3.1. *Car movement*
     - 3.1.1. *Use vector-based representations of acceleration, velocity and displacement to update the car parameters.*
     - 3.1.2. *Calculate the acceleration of the car from the throttle input. Apply braking/ negative acceleration for negative throttle input*
     - 3.1.3. *Use trigonometry to calculate vector of velocity and displacement based on scalar speed and angle orientation*
     - 3.1.4. *Simulate friction by adding grip to the movement and slowing down when no throttle is applied*

Car movement functions smoothly for the purpose of this program. By differentiating acceleration w.r.t time twice, the displacement can be calculated each frame. The use of heading direction to find the vector of displacement works well. However, this model does not accurately simulate reality, since the simulation of engine forces has been omitted. Additionally, tire grip is simulated by interpolating velocity instead of simulating forces on tires. This means this model cannot be used to simulate oversteer and understeer (drifting).

   - 3.2. *Car Turning*
     - 3.2.1. *Car turns w.r.t the current steering input*
     - 3.2.2. *Apply basic version of kinematic bicycle model to handle car rotation:*
       - 3.2.2.1. *Ensure rate of turning is related to the speed, steering and wheelbase*
       - 3.2.2.2. *Compute the angular velocity and update the orientation of the car w.r.t this angular velocity*

The kinematic bicycle model allows for dynamic turning radius depending on the angular velocity of the car. This allows for semi-realistic model of how steering influences heading direction of a car. However, this models the car as a 2-wheeled object, when cars have 4 wheels.

3.3. Car Rendering
3.3.1. Rotate the image of the car with respect to its original image and the current orientation.
3.3.2. Rotated image of the car and the relative position of the image is stored to be rendered.
3.4. Controlling car
3.4.1. Input sets a target parameter value, and the values are smoothly incremented to approach this target value
3.4.2. Design throttle and braking input mechanism
3.4.2.1. Throttle and braking are incremented by separate factors where braking increments slower than throttle
3.4.2.2. Positive throttle values lead to increasing acceleration and negative braking values lead to deceleration.
3.4.3. Design steering input mechanism
3.4.3.1. Steering is updated by a constant factor each time step until it reaches the target steer value.

Car sprite moves and rotates so that it is always facing the heading direction of movement. Car controls are designed to model actuator delay when accelerating, braking and steering which adds realism.

4. Player-controlled car
4.1. User input
4.1.1. User can press the 'W' key to increment the throttle and 'S' to apply braking
4.1.2. User can use 'A' and 'D' keys for steering
4.1.3. Alternatively, the user has the option to use an on-screen slider which they can control with the cursor to steer the car.

Player controlled car words as indented with both keyboard controls from steering, as well as using mouse and on-screen slider. The slider allows to hold a partial steering angle, which is more similar to have a steering wheel works. Keyboard controls just increment the steering to the max steering value, so partial steering angle cannot be held.

5. Track
5.1. Use data of user defined points and widths to generate a track with inner and outer walls.
5.2. Render track with a checkered line at the start/finish. Render walls as pattern of white and red
5.3. Implement optimised collision detection methods
5.4. Implement ray casting for AI car sensor data.

Track shapes are created by the user in the track maker, and the track data is saved as a JSON file. When a track object is created, the data from the corresponding track shape file is fetched and interpreted to create a track. The track renders with red and white walls, a black and white striped start line and grey road. The track object implements a spatial hash grid with DDA traversal for optimised collision detection. This collision detection system is used for both hitboxes and for ray casting to get sensor data.

6. Agentic AI cars

6.1. Input
6.1.1.Input values are set directly from the output of the reinforcement learning model
6.2. Create sensors for AI car to observe the environment
6.2.1.Start point of ray is centre of car and a max ray length is set
6.2.2.Ray is set to an angle from the direction the car is facing
6.2.3.First intersection with a wall along the ray is set to the hit point
6.2.4.Measure distance along the ray up till hit point to get sensor value

The AICar object overrides the update parameter so that the output of the RL model can be used to control the acceleration and steering of the car. Sensors are set to be at fixed angles from the heading direction and are equally balanced to the left and right. Sensors are used as input for RL model, and also useful determining cars position on track by calculating imbalance of right and left sensors.

6.3. Design reward function
6.3.1.Reward efficiency (calculated from the speed and distance moved)
6.3.2.Reward steering in the direction of more open space
6.3.3.Reward steering if approaching corner
6.3.4.Reward survival
6.3.5.Reward lap completion
6.3.6.Punish crashing
6.3.7.Punish not steering when moving into a wall
6.3.8.Punish driving very close to walls
6.3.9.Punish staying static/steering without moving

Reward function is designed to provide signals that promote good and fast driving. Reward function has time step level rewards (efficiency, steering, survival) which accumulate over all timesteps of the episode, and has terminal rewards (crashing, lap completion, static) that only occur when specific events take place. Both type of rewards signals different type of behaviour and together enable the agent to effectively learn to drive.

Designing the reward function was a difficult iterative process, since the reward signals must be carefully balanced to not overwhelm training. Due to my lack of expertise, I was only able to design the reward function by manually tweaking the reward component coefficient over hundreds of iterations to design a function that enables some basic standard of performance. Since the capability of a model is closely linked to how well the reward function models the environment, limitations of a reward function correspond to limitations in model training. The reward function I created is able to model the environment sufficiently well to be able to train models that complete tracks within a tractable ab

In this project, the limitations of the model include very slow convergence, noisy updates and limitations of complexity of tack that can be trained on (tracks with sharp turns or narrow walls cause model to get stuck in local minima).This also means that the quality and speed of training vary highly between training attempts. In some iterations, model can complete laps consistently in under 200 episodes, in others, even after 500 episodes the model still does not consistently finish laps on the same track. However, if it were possible to design a better function, then the model would show more consistent, faster and optimal training patterns.

7. Creating racing environment

7.1. Start and finish line
7.1.1. Car(s) starts at a designated start line being oriented in the correct direction to drive
7.1.2. Timer records the time taken for car to finish lap
7.2. Crash reset
7.2.1. When a car crashes, it is reset to the start line
7.2.2. The progress of the car is reset to the start of the running lap. Completed laps are not reset
7.3. End of Simulation
7.3.1. When cars in the racing environment complete the number of laps set, simulation ends
7.3.2. Ends screen displays lap time statistics. The car with the best average lap time/fastest lap time is shown as winner

Racing environment is created using the CarSimulation class, and specific environments (SoloSimulation, RacingAI and AITrainingEnvironment) inherit from this. This modular design works effectively. Car resets are prompt, independent and reset to current lap while keeping record of completed laps. At the end of simulation, end screen is displayed with required statistics. In Solo simulation, this is the lap times and total race time. In Racing AI, this is the Player lap times, AI lap times and the overall winner.

8. Track maker interface
8.1. Curve tool for make track shape
8.1.1. Create an anchor point of when left mouse button is clicked and connect all anchor points with a smooth continuous curve.
8.1.2. Anchor points can be dragged to different positions using the cursor
8.1.3. Clicking on the track spine curve inserts a new anchor in between
8.1.4. Curve can be made into a smooth closed loop
8.1.5. Anchors can be deleted

Curve tool has smooth intuitive design. Track renders as smooth curves without lagging or glitching. Points can be easily selected, deleted and moved.

8.2. Design tools to allow the user to modify features of the track.
8.2.1. User can clear interface and start over.
8.2.2. Create curvature handles that the user can edit to modify the curvature of the track
8.2.3. User can click at a point on the track and use a slider to modify width at the point

Curvature handles are generated using Bezier curve control points. When one point is moved, the connected point also moves in opposite direction which give it a “handle” appearance. Width points can be created and modified using a slider.

8.3. Enforcing track criteria
8.3.1. Check track forms a full complete loop
8.3.2. Track must have a minimum length
8.3.3. Check there are no overlapping sections
8.3.4. Ensure there are no kinks in the shape of the track

Track criteria ensures that track shapes that cannot be raced on are not saved and loaded into simulations. For ever validity violation, the user is shown a unique error message to inform them of what the issue with their track is, which allows for greater clarity on how to fix the track.

8.4. Save track

8.4.1. User is prompted to enter a unique track name that names the save file
8.4.2. All track properties are stored into a json file
8.4.3. Saving the track updates the previously saved values to the new values
8.5. Load track
8.5.1. User is prompted to enter the name of saved track
8.5.2. Track data is loaded in from the selected file
8.5.3. Implement algorithm to regenerate smooth curves for track walls from the track data loaded from file

Tracks are saved and loaded by reading/writing data of all anchor, control and width points to JSON files. Tracks can be regenerated from this data.

9. Multi-Layer Perceptron (MLP) Neural network
9.1. Create "layers" of neurons
9.1.1. Each layer has a vector of weights and biases where each weight and biases is associated with a single neuron
9.1.2. Every neuron of one layer is connected to all the neurons in the next layer
9.1.3. Includes an 'input layer', an 'output layer', and a certain number of 'hidden layers.
9.1.4. The weight and biases parameters can be updated during training
9.2. Get actions from network from feedforward
9.2.1. Input is passed into the input layer
9.2.2. Data is passed from one layer to the next through the network and the final output is taken from the output layer
9.2.3. Implement options for non-linear activation functions that wrap the output of each layer
9.2.3.1. Tanh
9.2.3.2. Elu
9.2.3.3. ReLu
9.3. Update network parameters with backpropagation
9.3.1. Compute gradients of objective function w.r.t weights and biases
9.3.2. Implement Adam optimiser to optimally update weights and biases to converge on global minimum
9.3.2.1. Compute Adam moments of Momentum and uncentered Variance
9.3.2.2. Update parameters by factors of the Adam moments
9.4. Design network to compute batch inputs of sensor data to produce batch output where each output corresponds to a single simulated agent

The Layer and NeuralNetwork classes use numpy arrays and lists to create custom neural network implementation. Neural network is designed to be able to process batch data, which allows for simultaneous simulation of many cars. The non-linear activations can be set at time of model creation, and each perform differently with different tasks and different network sizes as demonstrated in testing. The activation functions display expected behaviour, with tanh showing smooth patterns, Elu showing semi smooth patterns, and ReLu showing abrupt non continuous patterns. Adam optimiser effectively speeds up convergence.

10. Create base Policy Gradient Reinforcement Learning model
10.1. Plugs the input vector into the neural network to get the output
10.2. Computes actions stochastically from neural network output
10.3. Stores trajectory of recent episodes
10.4. Computes total return at each time step
10.5. Implements gradient ascent to update neural network

RL model implements basic Policy gradient model functions and the actor neural network (responsible for outputting actions). Input vector of sensor data is plugged into network to produce output and the actions are sampled stochastically based on this output. This works correctly all cars being simulated are controlled by the same neural network output, but each car acts slightly differently from each other. Trajectory stores the states, actions and rewards for all cars for all time steps of an episode. Monte carlo returns are computed correctly with discounted sum.

11. Creating Specific RL models

11.1. REINFORCE model

11.1.1. Implement a single neural network that outputs the actions each cycle

11.1.2. Computes Monte-Carlo returns

11.1.3. Uses returns to compute advantage value for network parameter updates

REINFORCE model does not implement any separate neural network, just the actor network. The parameter updates are carried out by directly calculating the advantage from the monte carlo returns. Advantage value is normalised to reduce variance

The REINFORCE model trains quite slowly due to high variance. In early training, the model rolls back several times due to catastrophic forgetting and therefore takes longer to converge. In late training, once the model has learnt a behaviour for many episodes, it doesn't change that behaviour easily since it limits exploration and focusses on exploitation. This can be good in situations where the agent has learnt good behaviour and doesn't forget it. However, at times this can cause the agent to get stuck in a local minimum. The general patterns is that trained REINFORCE models tend to take longer to train, drive slower and can get stuck on complex tracks, but if the model learns to finish laps, it learns to exploit the strategy quite consistently.

11.2. Monte Carlo Actor Critic (MCAC) model

11.2.1. Implement an actor network that outputs action each cycle

11.2.2. Implement a critic network that computes a Value estimate at end of each episode

11.2.3. Use predicted critic value and returns to compute the advantage value for network parameter updates.

MCAC model implements a second critic neural network that trains to estimate the value of the states that the agent experiences. The difference between the predicted value and the actual monte carlo returns is the advantage, and gradients are calculated w.r.t this advantage.
The second neural network adds stability to training, and so the trained models undergo fewer rollbacks during training due to lesser forgetting. It also in general tends to converge quicker. However, the models can show inconsistencies between episodes especially when critic loss is high. This is because even though the actor network has converged on optimal actions, if the critic value is still far off from the returns, the parameters still undergo large updates and causes depreciation.

In my project, I was able to moderate the critic loss to some degree by scaling down rewards. However, during late training on complex tracks, the effects of this depreciation are still evident. This is shown by the model performance plateau, where it exhibits mostly good behaviour, but is not consistent across episodes. In general, these models train faster and learn to drive faster with greater efficiency then REINFORCE models, however they can be less consistent on complex tracks.

12. Model training environment

12.1. Batch simulation management

12.1.1. Allow user to configure number of cars to train simultaneously

12.1.2. Spawn all configured cars at track start line with independent state tracking

12.1.3. Process all cars in parallel each timestep, collecting individual trajectories

12.1.4. Handle individual car crashes and resets without affecting other cars in batch

Batch simulation allows for many trajectories to be sampled simultaneously. Cars are simulated independent of each other. Sampling more trajectories stabilises training.

12.2. Performance tracking and metrics

12.2.1. Track average and maximum progress across all cars in recent

12.2.2. Track average and fastest lap times when cars complete laps

12.2.3. Update performance metrics in real-time

12.2.4. Log episode number, duration, average progress, and average reward for each episode

Performance metrics include episode logs (average progress, episode duration and if rollbacks occurred), performance statistics (max progress, min lap time etc.) and training statistics (exploration rate, average returns etc). This allows user to monitor training.

12.3. Model rollback system

12.3.1. Save best model parameters when average performance improves

12.3.2. Detect performance degradation when recent average drops significantly below best historical average

12.3.3. Revert to best parameters if performance degradation is detected.

Stochastic policy gradient models can experience catastrophic forgetting where good behaviours are forgotten since the model has no memory of previous episodes. Model rollbacks save best model parameters and if performance depreciates, the model is rolled back to its best parameters. This stops the training from collapsing.

12.4. Training speed controls

12.4.1. Implement tick speedup

12.4.2. Process multiple simulation frames per render frame based on speedup value

This speeds up the physics engine and training by rendering fewer times per second which reduces overheads.

12.5. Model configuration interface

12.5.1. Allow user to configure neural network architecture (hidden layer sizes and activation functions.

12.5.2. Allow user to configure training hyperparameters (learning rate, entropy bonus, Adam betas, L2 regularisation)

12.5.3. Allow user to configure critic network architecture for MCAC model.

12.5.4. Display appropriate error messages for invalid configurations.

A pop screen allows the user to con figure all hyperparameters before starting AI training. Measures taken to ensure hypermeters can only be set to valid set of values.

13. Racing Modes

13.1. Initialise Cars at start line

13.2. Create control panel for user to drive car, along with HUD for speedometer and throttle meter
13.3. Track Completion of laps and record individual lap times
13.4. Reset crashed car to start of current lap
13.5. Implement solo racing and racing against AI models
13.5.1. Solo racing
13.5.1.1. Only player car drives around track and player lap times are recorded
13.5.1.2. End screen displays individual lap times and total race time
13.5.2. Race AI
13.5.2.1. 1 Player and 1 AI car race on the track
13.5.2.2. Lap times for each car recorded separately
13.5.2.3. Car with the lowest total race time is displayed as the winner.

During racing simulations, the cars are simulated independently so that lap times and car resets upon crashes are

14. File selection
14.1. Create a global file selection system for saving and loading tracks and AI models
14.1.1. Displays all relevant existing files within the required directory in a selection list
14.1.2. Allows user to enter a file name into a text box
14.1.3. Loading
14.1.3.1. User must select a relevant existing file in the directory.
14.1.3.2. User is displayed an appropriate error message if try to select a non-existing file
14.1.3.3. User is displayed an appropriate error message if there is an error in loading the selected file
14.1.4. Saving
14.1.4.1. User is allowed to name their file a unique or existing name
14.1.4.2. If file name already exists, the user is displayed a warning message and asked to confirm their choice of overwriting the file
14.1.4.3. User is displayed an appropriate error message if the file name they have entered is invalid.

The user is presented with an interactive interface to save and load files. The interface is consistent for both saving and loading files, with a selection list displaying all other files in the relevant directory. There in input protection which prevents the user from saving files with invalid files names. Corrupted files or invalid files names are handled smoothly by presenting the user with suitable error messages.

4.2 End user feedback

I presented the project to my end user base, which included my friends George Keylock, Nigel Wong, Joseph Thompson and William Appleby, as well as my specialist users D. Davis and C. McCormack. The overall feedback from both general and specialist users was highly positive, with the main highlights being the breadth of functionality and the high degree of technical sophistication in the implemented frameworks. Nigel and William provided feedback that feature of being able to create their custom tracks was a great addition that makes the system more customisable and interactive. Feedback on the track maker interface and the curve tool was that it is easy to make non-standard curve shapes with varying widths. Nigel did point out that due to the nature of the curve tool, creating long straight can be

less intuitive, so it would have been very convenient if there was some way to toggle the curve tool rather than have it on constantly. Additionally, George provided feedback that he would like to see the feature of modifying track boundaries so that there aren't walls all the way around.

C. McCormack's feedback on the program was focussed on the technical implementation of reinforcement learning. He commented that the neural network implementation with NumPy arrays and Adam optimiser is highly effective at generalising problems that aren't limited to racing (for example, predicting Fourier functions as demonstrated in the testing vide). He did suggest that it would have been better practice to make Adam optimiser optional, and maybe even implement other optimisers so that we can test in isolation whether the optimiser speeds up training. He especially liked the ability to design unique neural network configurations and evaluate their performances. This includes comparing large vs small layers, deep vs shallow networks and different activation functions. With reference to the implementation of Reinforcement learning using stochastic policy gradient, his feedback was that all in all, the model and training environment design is effective and that trained models to perform significantly well. However, he suggested that the reward function design is not entirely optimal, since it does not have a component that indicates to the model that minimising lap times is good. He pointed out that the scaling of rewards is not completely optimal, with some terminal rewards not being well scaled. He also commented on the training environment, saying that my design of computing actions once per frame at 50 fps is too aggressive and leads to overly noisy actions. His feedback was that lowering the rate at which actions were computed to only a few times per second would potentially improve training.

Major positive feedback is the quality of reinforcement learning. Models can be trained from no previous knowledge to be able to proficiently complete complex racetracks. The efficiency of training both in terms of time and quality is substantially better than what was expected from a fully hand coded implementation. While there are improvements that could be made, the system can be effectively used to test the efficacy of using reinforcement learning to drive race cars.

D. Davis' feedback on the racing simulation was that the behaviour and mechanics of the car are quite realistic in nature. He especially loved the modelling of actuator delay in the throttle and steering controls. While the abstracted bicycle model of kinematics works well, he did agree that modelling of additional physics like drifting would have been a good touch. His main criticism is that there is limited ability to customise cars. While features like the driving force, steering and throttle sensitivity and so on are determined by constants, there is no way for the user to directly manipulate these constants to change the driving experience.

Further feedback on the Racing environment was provided by Joseph and William. They were positive about the lap timing and the competitive nature of racing against trained AI models. Their criticism was that the controls of the car can take some time

to get used to since the orientation of the car is independent of the controls. Joseph commented that it may be more intuitive if the camera orientation rotated along with the car as it turned.

The overall user feedback on the GUI was highly positive, with highlights being that the colour scheme is aesthetically pleasing, buttons are interactive and instructions are clear. Users highlighted that the system is suitable both for technical and non-technical purposes.

4.3 Possible Improvements

Although the system successfully meets all the core objectives, there are several improvements that could be made in terms of improving functionality, as well as in improving technical aspects:

Functionality:

- Track maker Features: The current track maker lets users create custom tracks using Bézier curves with adjustable widths. However, based on the feedback provided by Nigel and George, adding additional design tools would enable even more customisation. A kerb-generation tool can be added to allow users place kerbs along corner edges instead of the entire track boundary being walls. This would allow creation of tracks that are closer to real world racing tracks and enables cars to cut corners by driving over the kerbs.

- Variable Track conditions and Car customisation: D. Davis' feedback on Car customisation highlighted that the simulation would be more versatile if there was more customisation in the actual car configuration. Real world racing, track conditions play a very significant role is racing mechanics and strategy. Adding the options to simulate dynamic conditions like rain would improve the simulation's realism. Additionally, allowing the user to customise features of their car/ of the AI car further enables experimenting with different racing strategies on the same track. These changes would also allow to test how well reinforcement learning models perform under unfamiliar conditions.

- Improved Camera System: Joseph's feedback on unintuitive driving due to no camera movement and rotation highlighted the possible improvements in camera system. Adding a dynamic camera system that allows zooming and movement would greatly improve user experience. In the track maker, this would allow creation of tracks of any size, while still maintaining fine detail. In racing simulations, a dynamic camera that follows the car, with adjustable zoom or rotation based on speed and direction, would enhance the user experience.

Technical improvements:

- Force based car physics: force-based model, that accounts for tire grip, friction, air resistance, and weight transfer would make the simulation more realistic and dynamic. Complex driving mechanics will be made possible, including drifting, oversteer, and understeer. This form of modelling would also

be a good way to test how reinforcement learning models perform when driving cars that follow this more realistic physics.

- Car to car interactions: Introducing car to car interaction mechanics in racing environment would create more realistic dynamics and enable events like overtaking, blocking, and collisions. This could involve modelling air flow disruptions, collision forces, momentum transfer to determine how cars behave when driving close by and when they come in contact. By incorporating these interactions into the reinforcement learning environment, the AI could learn more advanced behaviours, including collision avoidance, position defence, and overtaking. For this, the AI training environment must be reconfigured to train on tracks when racing against other cars (potentially other already trained AI), and the reward function would also become significantly more complex.

- Multithreading AI training environment: Training multiple cars simultaneously leads to significant limitations in batch sizes and simulation speed due to bottleneck. Introducing multithreading would allow the simulation of multiple cars or environments at the same time with significantly lesser bottleneck. This would allow the sampling of more trajectories within a given amount of time, which in turn speeds up the training process.

- Implementing Advantage actor critic (A2C): The REINFORCE and MCAC models depend on Monte Carlo returns, which create high variance and slow down learning since it is averaged over a whole episode. C. McCormack said it would be better to adopt a full Advantage Actor-Critic (A2C) algorithm with temporal-difference (TD) bootstrapping. This approach would let the critic network estimate value functions at ‘n’ timesteps instead of waiting until an episode ends, which would meaningfully reduce variance and speed up convergence. This would require redesigning the training environment to allow for updates to be carried out while episodes are still running.
- Reward function: The current reward function progress, speed, steering alignment, wall proximity penalties, survival reward crash penalty and lap completion bonus, which enable the agent to learn basic racing. The function was developed through manual tuning, which can result in imbalanced or conflicting reward signals. The reward function can be improved by simplifying and normalizing reward components and introducing more targeted rewards to reward more advanced behaviours like optimal racing lines, fast lap times and efficient cornering.